Python Lecture Note

A. Python Functions, Closures, and Decorators

Dong Hyun KIM

A Python Functions, Closures, and Decorators

A.1 함수 객체, 이름 바인딩, callable, 호출과 반환

A.1.1 클래스 객체 모델과의 연결

클래스 객체 모델에서 이런 코드를 살펴볼 수 있다.

```
class Person:
    def introduce(self):
        return "Hello"


p = Person()

method = p.introduce

print(method.__self__ is p)
print(method.__func__ is Person.introduce)
```

실행 결과:

```
True
True
```

우리는 이 결과를 디스크립터 프로토콜(Descriptor Protocol)을 통해 설명한다.

`Person.__dict__["introduce"]`에는 함수 객체가 저장되고,

`p.introduce`에 접근하면 함수가 인스턴스에 바인딩된 메서드 객체를 얻는다.

그런데 여기에는 아직 독립적으로 설명하지 않은 전제가 있다.

클래스 네임스페이스에 저장된 함수 객체는 애초에 어떻게 생성되었으며, Python은 그 객체를 어떻게 호출하는가?

또한 다음 표현식을 이미 사용하고 있었다.

```
Person.introduce
Person.introduce(p)
```

두 표현식은 괄호의 유무만 다른 것처럼 보이지만, 실제로는 서로 다른 객체 연산을 나타낸다.

첫 번째는 속성 탐색으로 얻은 함수 객체를 평가하고, 두 번째는 그 함수 객체를 호출한다.

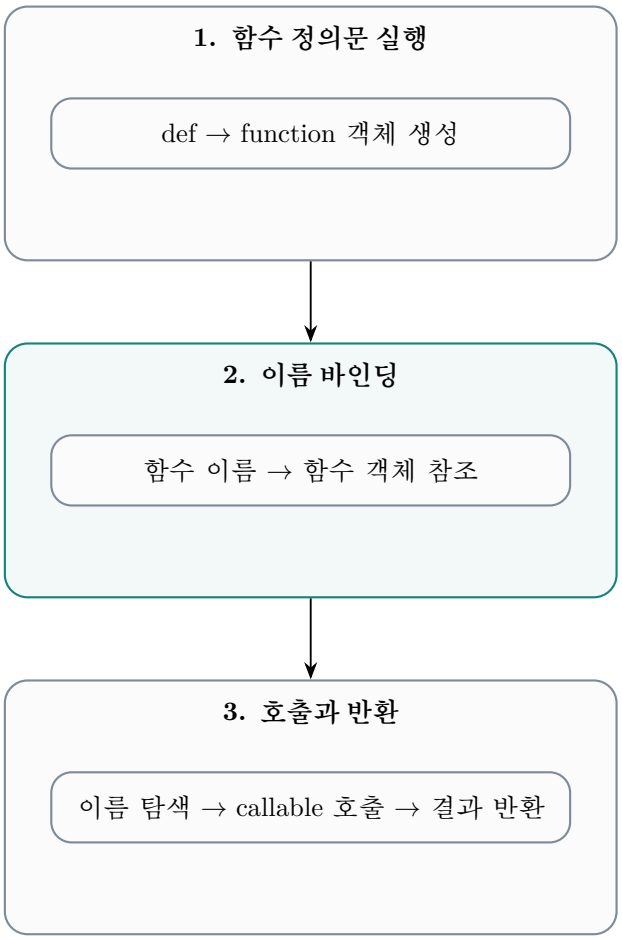
이번 학습에서는 이 차이를 출발점으로 삼을 것이다.

A.1.2 이번 학습의 핵심 질문

이번 학습에서 설명할 수 있어야 하는 핵심 질문은 다음과 같다.

- `def`는 함수 본문을 실행하는가, 함수 객체를 생성하는가?
- 함수의 이름과 함수 객체는 어떤 관계인가?
- `func`와 `func()`는 각각 무엇을 평가하는가?
- 함수 객체가 아닌데도 `()`로 호출할 수 있는 객체는 무엇인가?
- 호출할 때 인수는 어떻게 전달되며 `return`은 무엇을 반환하는가?
- 이름을 재바인딩하면 기존 함수 객체에도 영향을 주는가?

이 질문들은 독립적으로 보이지만 모두 하나의 실행 모델로 연결된다.



함수 정의, 이름 바인딩, 호출은 서로 연결되지만 별개로 분석해야 하는 실행 단계다.

이제 각 단계를 실제 Python 객체 모델에 대응시켜 보자.

A.1.3 함수 객체(Function Object)의 정확한 정의

Python에서 함수(Function)는 단순한 코드 블록이 아니다.

함수 객체(Function Object)는 호출 가능한 객체로, 실행할 코드와 함수 실행에 필요한 메타데이터 및 관련 네임스페이스 정보를 가진 객체다.

다음 코드를 보자.

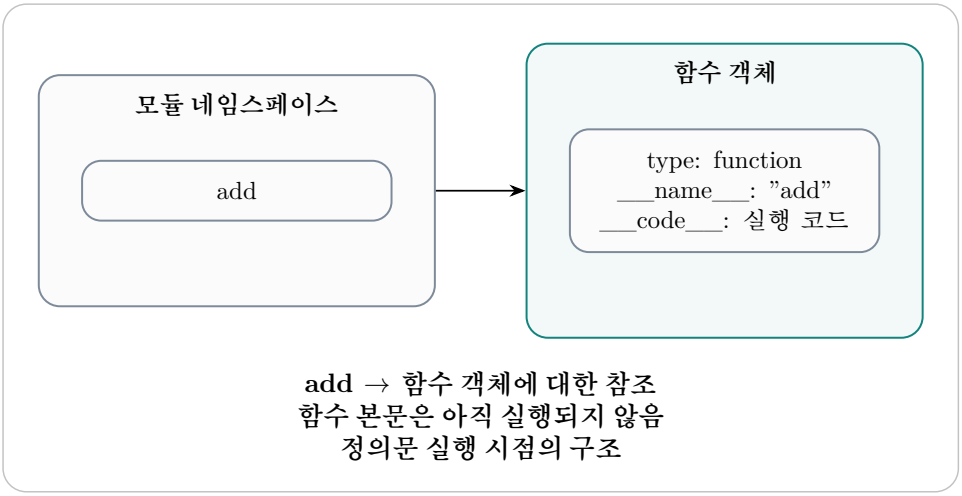
```
def add(a,b):
```

```
    return a + b
```

이 코드를 실행하면 Python은 add()의 본문을 즉시 실행하지 않는다.

대신 def 정의문을 실행하면서 새로운 함수 객체를 만들고, 현재 네임스페이스의 add라는 이름을 그 객체에 바인딩한다.

이 과정을 구분하면 다음과 같다.



여기서 구분해야 할 것은 세 가지다.

구성 요소	의미
add	함수 객체를 참조하는 이름(Name)
함수 객체	실제로 생성된 function 타입의 객체
함수 본문	해당 함수가 호출될 때 실행되는 코드

함수 객체와 그 객체를 가리키는 이름은 동일한 것이 아니다.

이것은 클래스 객체 모델에서 학습하는 다음 구조와 본질적으로 같다.

```
p = Person("Kim")
```

p라는 이름이 Person 인스턴스를 참조하는 것처럼, add라는 이름도 함수 객체를 참조하는 것이다.

다만 생성되는 객체의 타입과 기능이 다르다.

함수 객체의 타입 확인

```
from types import FunctionType

def add(a, b):
    return a + b

print(type(add))
print(isinstance(add, FunctionType))
print(callable(add))
```

예상 실행 결과:

```
<class 'function'>
True
True
```

여기에서 확인할 수 있는 사실은 다음과 같다.

add는 이름이면서 표현식으로 사용될 때 함수 객체를 참조한다.

type(add)는 그 객체의 실제 타입을 반환하고, callable(add)는 그 객체가 호출 가능한지를 검사한다.

또한 FunctionType은 Python에서 사용자 정의 함수 객체를 나타내므로,

isinstance(add, FunctionType)이 True가 되는 것이다.

중요한 것은 함수 정의문을 실행한 것만으로도 호출 가능한 객체가 만들어진다는 점이다.

A.1.4 def는 실행문이다 - 함수 생성과 함수 실행의 구분

다음 코드를 살펴보자.

```
def add(a, b):
    print("함수 본문 실행")
    return a + b


print("함수 정의 완료")

result = add(3, 5)
```

```
print(result)
```

예상 실행 결과:

```
함수 정의 완료
함수 본문 실행
8
```

왜 이런 순서가 발생할까?

1단계. 함수 정의문 실행

Python이 다음 문장을 만났다.

```
def add(a, b):
    print("함수 본문 실행")
    return a + b
```

이때 함수 객체가 생성되고 add 이름에 바인딩된다.

그러나 본문에 있는 print()와 return은 아직 실행되지 않는다.

2단계. 다음 문장 실행

```
print("함수 정의 완료")
```

이 문장은 함수 바깥의 모듈 수준 코드다.

따라서 즉시 실행되어 첫 번째 문자열이 출력된다.

3단계. 함수 객체 호출

```
result = add(3,5)
```

여기서 처음으로 add가 참조하는 함수 객체를 호출한다.

호출 과정에서 3과 5에 해당하는 인수 객체가 각각 매개변수 a와 b에 바인딩되고, 함수 본문이 실행된다.

4단계. 반환값에 이름 바인딩

```
return a + b
```

정수 3과 5의 덧셈 결과인 8이 반환된다.

그 결과가 result라는 이름에 바인딩된다.

5단계. 반환된 객체 사용

```
print(result)
```

이제 result는 함수 객체가 아니라 정수 객체 8을 참조하고 있다.

따라서 8이 출력된다.

여기서 얻을 결론은 명확하다.

def 문장이 실행되는 시점과 함수 본문이 실행되는 시점은 다르다.

그렇다면 def를 실행할 때 아무런 계산도 발생하지 않을까?

그렇지는 않다.

def 문장은 함수 본문을 실행하지 않을 뿐, 함수 정의에 포함된 다른 표현식은 평가할 수 있다.

대표적인 사례가 기본 인수(Default Argument)다.

```
def create_default():
    print("기본값 생성")
    return []

def record(value, history=create_default()):
    history.append(value)
    return history

print("함수 정의 완료")

print(record(10))
print(record(20))
```

예상 실행 결과:

```
기본값 생성
함수 정의 완료
[10]
[10, 20]
```

여기서는 record()가 호출되기도 전에 "기본값 생성"이 출력된다.

왜냐하면 `history=create_default()`의 기본값 표현식은 `record` 함수의 정의문이 실행될 때 평가되기 때문이다.

그리고 평가 결과인 리스트 객체 하나가 함수의 기본 인수로 유지된다.

따라서 기본 인수를 생략한 두 호출은 같은 리스트 객체를 사용하고, 첫 번째 호출의 변경이 두 번째 호출에서도 관찰되는 것이다.

이 원리는 Python 공식 언어 참조에도 명시되어 있다.

따라서 다음 두 명제는 구별해야 한다.

명제	판단
함수 정의문 실행은 함수 본문을 실행하지 않는다.	참
함수 정의문 실행 중에는 어떤 표현식도 평가하지 않는다.	거짓

데코레이터 표현식 역시 함수 정의 시점에 평가될 수 있다.

다만 그 평가·적용 순서는 이후에 별도로 분석할 것이다.

A.1.5 이름 바인딩 - 하나의 함수 객체를 여러 이름이 참조할 수 있는가?

가능하다.

그리고 이것이 일급 함수(First-Class Function)를 이해하기 위한 첫 번째 조건이다.

다음 코드를 보자.

```
def add(a, b):
    return a + b

operation = add

print(operation is add)
print(operation(3, 5))
```

예상 실행 결과:

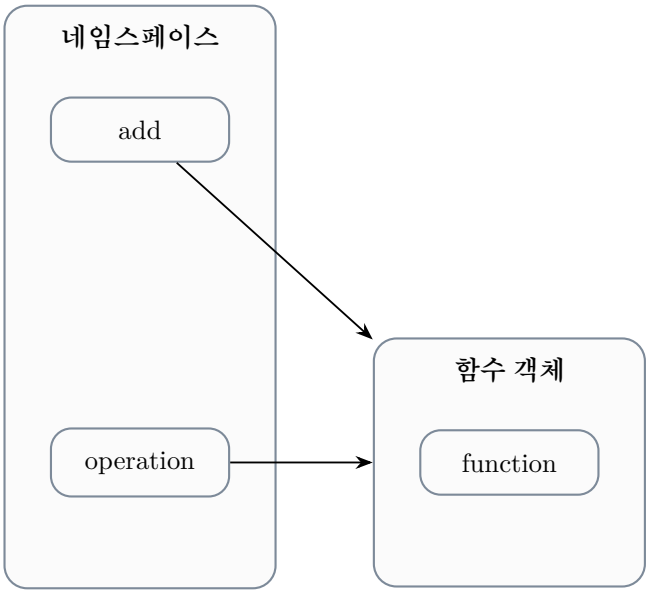
```
True
8
```

여기서 중요한 문장은 다음 문장이다.

```
operation = add
```

이 문장은 add 함수 객체를 복사하거나 새로 생성하지 않는다.

add가 참조하는 기존 함수 객체를 operation이라는 이름도 참조하도록 바인딩하는 것이다.



operation is add → True
두 이름이 동일한 객체를 참조함

이름이 바뀌면 함수 객체도 바뀔까?

이번에는 다음 코드를 살펴보자.

```
def double(x):
    return x * 2

saved = double

double = "replaced"

print(callable(double))
print(callable(saved))
print(saved(5))
```

```
print(saved.__name__)
```

예상 실행 결과:

```
False
True
10
double
```

이 과정은 객체 생성, 참조, 이름 재바인딩으로 나눠서 분석할 수 있다.

첫 번째 단계 - 함수 객체 생성

```
def double(x):
    return x * 2
```

함수 객체가 생성되고 `double`이라는 이름에 바인딩된다.

두 번째 단계 - 참조 공유

```
saved = double
```

`saved`도 동일한 함수 객체를 참조하게 된다.

세 번째 단계 - 이름 재바인딩

```
double = "replaced"
```

이 문장은 원래의 함수 객체를 수정하는 것이 아니다.

`double`이라는 이름을 새로운 문자열 객체에 다시 바인딩한다.

기존 함수 객체는 여전히 `saved`가 참조하고 있다.

네 번째 단계 - 기존 함수 객체 호출

```
saved(5)
```

`saved`는 원래의 함수 객체를 가리키므로 정상적으로 실행된다.

또한 `saved.__name__`이 `"double"`이라는 사실도 중요하다.

__name__은 함수 객체에 있는 메타데이터다.

함수 객체를 참조하는 현재 변수 이름과 반드시 일치할 필요는 없다.

따라서 다음 두 가지를 구분해야 한다.

- 이름 재바인딩(Rebinding) : double = "replaced"
- 객체 상태 변경(Mutation) : 객체 자체가 가진 정보를 변경

위 예제는 이름 재바인딩이지 함수 객체의 변경이 아니다.

이 구조는 나중에 다음 데코레이터 변환식을 이해하는 핵심이 된다.

```
add = decorator(add)
```

여기서 add라는 이름에 새로운 반환 객체를 바인딩하더라도, 기존 함수 객체를 참조하는 다른 이름이나 클로저가 있다면 기존 함수 객체는 계속 사용될 수 있다.

A.1.6 func와 func() - 함수 참조와 호출의 차이

이번에는 호출 표현식(Call Expression)을 살펴보자.

```
def greet():
    print("Hello")
    return 100
```

다음 두 표현식은 의미가 다르다.

```
greet
```

이것은 이름을 탐색하여 함수 객체를 얻는 표현식이다.

반면

```
greet()
```

이것은 greet이 참조하는 호출 가능한 객체를 호출하는 표현식이다.

실제 코드를 비교해 보자.

```python
def greet():
    print("Hello")
    return 100

a = greet
b = greet()

print(type(a).__name__)
print(b)
```

예상 실행 결과:

```
Hello
function
100
```

`a = greet`에서는 함수 본문이 실행되지 않는다.

그 대신 `a`가 함수 객체를 참조하게 된다.

반면 `b = greet()`에서는 함수 객체를 호출하여 `"Hello"`를 출력하고, 100이라는 정수 객체를 반환받아 `b`에 바인딩한다.

`()`의 의미

여기에서 하나 더 정확하게 이해할 부분이 있다.

`()`는 단순히 함수만 실행하는 문법이 아니다.

`()`는 앞의 표현식을 평가하여 얻은 호출 가능한 객체(Callable)를 호출하는 문법이다.

따라서 다음 표현식들은 모두 호출 표현식이다.

```python
func()
obj.method()
SomeClass()
callable_instance()
```

각각 호출하는 객체의 종류는 다를 수 있다.

표현식	평가하여 호출하는 대상
`func()`	일반적으로 함수 객체
`obj.method()`	일반적으로 바운드 메서드 객체
`SomeClass()`	클래스 객체
`callable_instance()`	`__call__`을 지원하는 인스턴스

또한 함수 이름으로 시작하지 않는 표현식도 호출할 수 있다.

```
def get_function():
    return print

get_function()("Hello")
```

예상 실행 결과:

```
Hello
```

실행 흐름을 따라가 보자.

먼저 `get_function()`이 호출된다.

이 함수는 `print`가 참조하는 내장 함수 객체를 반환한다.

그러면 바깥쪽 호출 표현식은 그 반환된 함수 객체를 `"Hello"`라는 인수로 다시 호출한다.

개념적으로 다음 과정과 같다.

```
f = get_function()
f("Hello")
```

이 원리는 추후 인수를 받는 데코레이터의 다음 구조를 분석할 때 다시 등장한다.

```
decorator_factory(config)(func)
```

여기서 첫 번째 호출의 반환값이 또 다른 호출 가능한 객체라는 사실이 중요하다.

A.1.7 callable() - 호출 가능한 객체는 함수뿐일까?

아니다.

Python은 함수 객체뿐 아니라 클래스 객체, 메서드 객체, 호출 가능하도록 설계된 인스턴스도 호출할 수 있다.

callable(obj)는 어떤 객체가 호출 가능한지 검사하는 내장 함수다.

다만 True가 나왔다고 해서 모든 인수 조합으로 호출이 성공한다는 뜻은 아니다.

사용자 정의 인스턴스를 호출 가능하게 만들기

클래스 객체 모델에서 살펴보는 __call__()을 연결해 보자.

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):
        return self.factor * value

multiply = Multiplier(3)

print(type(multiply).__name__)
print(callable(multiply))
print(multiply(4))
print(multiply.__call__.__self__ is multiply)
```

예상 실행 결과:

```
Multiplier
True
12
True
```

여기서 multiply는 함수 객체가 아니다.

Multiplier 인스턴스다.

그런데도 multiply(4)라는 표현식이 가능하다.

왜 그럴까?

호출 과정 분석

먼저 다음 표현식을 실행한다.

```
multiply = Multiplier(3)
```

클래스 객체 `Multiplier`가 호출되고 새로운 인스턴스가 생성된다.

`__init__()`이 실행되면서 해당 인스턴스의 `factor` 속성에 3이 바인딩된다.

다음 표현식은

```
multiply(4)
```

`multiply` 인스턴스를 호출하는 표현식이다.

Python의 특수 메서드 호출 메커니즘은 그 인스턴스의 타입에서 `__call__` 구현을 찾아 사용한다.

이 예제에서는 개념적으로 다음과 같은 효과를 가진다.

```
Multiplier.__call__(multiply, 4)
```

`self`는 `multiply` 인스턴스를 참조하고, `value`는 정수 4를 참조한다.

따라서 다음 연산이 수행된다.

```
self.factor * value
```

결과는 `3 * 4`, 즉 12이다.

일반적인 사용자 정의 클래스의 특수 메서드는 인스턴스 딕셔너리에서 단순히 같은 이름을 찾는 방식과 달리, 타입을 통한 특별한 탐색 규칙이 적용된다는 점도 기억한다.

중요한 비교

구분	add	multiply
객체 타입	function	Multiplier
callable()	True	True
호출 표현식	add(3, 4)	multiply(4)
호출 동작	함수 본문 실행	클래스에 정의된 __call__ 실행
상태 보관	함수 속성·기본값·클로저 등	일반적으로 인스턴스 속성

이로부터 다음 관계가 성립한다.

모든 일반적인 사용자 정의 함수 객체는 호출 가능하지만, 모든 호출 가능한 객체가 함수 객체인 것은 아니다.

여기서 기존 클래스 객체 모델과 연결되는 중요한 통찰이 하나 있다.

PyTorch의 model(x) 역시 함수 객체를 호출하는 것이 아니다.

nn.Module을 상속한 모델 인스턴스를 호출하는 표현식이다.

반면 model.forward(x)는 속성 탐색을 통해 얻은 바운드 메서드를 호출한다.

두 표현식 모두 ()를 사용하지만 호출 대상과 실행 경로는 다르다.

callable()이 True여도 오류가 발생하는 경우

```
def add(a, b):
    return a + b

print(callable(add))

add(10)
```

첫 번째 출력은 다음과 같다.

```
True
```

하지만 두 번째 호출에서는 TypeError가 발생한다.

add는 분명히 호출 가능한 함수 객체이지만 필수 매개변수 b에 대응하는 인수를 전달하지 않았기 때문이다.

이 사례는 세 가지 판단을 분리해야 한다는 의미다.

판단	질문
호출 가능성	이 객체를 호출할 수 있는가?
인수 적합성	전달한 인수가 함수의 매개변수 규약에 맞는가?
실행 성공	함수 본문이 예외 없이 정상 종료되는가?

`callable()`은 첫 번째 판단을 위한 기능이다.

나머지 두 가지까지 보장하지는 않는다.

A.1.8 함수 호출 시 인수는 어떻게 전달되는가?

클래스 객체 모델에서는 인스턴스를 생성하거나 메서드를 호출할 때 인수를 전달한다.

```
Person("Kim")
p.introduce()
```

이제 이 원리를 함수 호출에 적용해 보자.

다음 코드를 살펴보자.

```
def update(values):
    values.append("B")

    values = ["C"]

    return values

original = ["A"]

result = update(original)

print(original)
print(result)
print(original is result)
```

예상 실행 결과:

```
['A', 'B']
['C']
False
```

왜 이 결과가 발생할까?

호출 전 상태

```
original = ["A"]
```

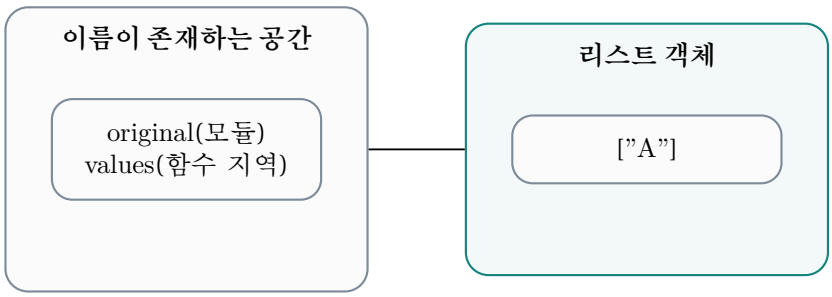
리스트 객체 하나가 생성되고 original이라는 이름에 바인딩된다.

이후 다음 표현식이 실행된다.

```
update(original)
```

Python은 original을 평가하여 리스트 객체를 얻고, 함수의 매개변수 values에 같은 객체를 바인딩한다.

이 시점의 구조는 다음과 같다.



두 이름이 동일한 리스트 객체를 참조

이후 함수 본문에서 다음 문장이 실행된다.

```
values.append("B")
```

values가 참조하는 리스트 객체 자체의 상태가 변경된다.

따라서 original이 참조하는 리스트에도 "B"가 추가된 것이 관찰되는 것이다.

다음 문장은 다르게 동작한다.

```
values = ["C"]
```

이 문장은 새로운 리스트 객체를 만들고 함수의 지역 이름 `values`를 그 객체에 다시 바인딩한다.

기존 리스트 객체는 여전히 `original`이 참조하고 있다.

마지막으로

```
return values
```

새로운 리스트 객체가 반환되고, 호출한 쪽의 `result` 이름에 바인딩된다.

따라서 최종 관계는 다음과 같다.

`original` → `["A", "B"]`

`result` → `["C"]`

이 예제에서 반드시 구분해야 할 연산은 다음 세 가지다.

표현식	발생한 연산
`update(original)`	인수 객체를 매개변수 이름에 바인딩
`values.append("B")`	기존 리스트 객체의 상태 변경
`values = ["C"]`	새로운 리스트 객체 생성 및 지역 이름 바인딩

이 원리는 데코레이터의 Wrapper에서도 동일하게 적용된다.

예를 들어 나중에 다음 코드를 보면

```
def wrapper(*args, **kwargs):
    return func(*args, **kwargs)
```

`args`와 `kwargs`가 어떤 객체를 담고 있고, 그 객체들이 원본 함수의 매개변수에 어떻게 전달되는지 분석할 수 있어야 한다.

A.1.9 `return` - 무엇을 반환하는가?

함수는 반드시 숫자나 문자열을 반환해야 하는 것이 아니다.

return은 함수 실행을 종료하고, 지정된 표현식을 평가하여 얻은 객체를 호출한 쪽에 반환하는 문장이다.

첫 번째 경우 - 정수 객체 반환

```
def compute():
    return 10

result = compute()
```

compute()가 호출되고 정수 객체 10이 반환된다.

그 결과 result가 정수 객체를 참조하게 된다.

두 번째 경우 - 함수 객체 반환

```
def double(x):
    return x * 2

def get_operation():
    return double

operation = get_operation()

print(operation is double)
print(operation(5))
```

예상 실행 결과:

```
True
10
```

이 코드에서 get_operation()은 정수나 문자열이 아니라 함수 객체를 반환한다.

따라서 반환된 함수 객체를 operation이라는 이름에 바인딩한 다음, 호출할 수 있다.

여기서 get_operation() 자체는 중첩 함수를 정의하지 않았다.

이미 존재하는 함수 객체를 반환했을 뿐이다.

그러므로 이 예제에는 클로저가 필요하지 않다.

세 번째 경우 - 함수 호출 결과 반환

이번에는 다음 코드를 살펴보자.

```
def double(x):
    return x * 2

def run_operation():
    return double(5)

result = run_operation()

print(result)
print(type(result).__name__)
```

예상 실행 결과:

```
10
int
```

이번에는 `double`이라는 함수 객체를 반환하는 것이 아니라, `double(5)`를 먼저 호출하고, 그 실행 결과인 정수 객체를 반환한다.

`return func`와 `return func()` 비교

표현식	실행 의미	반환 대상
`return func`	함수 객체를 평가한 뒤 반환	함수 객체
`return func()`	함수 객체를 호출한 뒤 결과 반환	호출 결과 객체

이 차이는 A.6과 A.7에서 매우 중요해진다.

예를 들어 Wrapper 함수 객체를 반환해야 하는 데코레이터에서

```
return wrapper
```

대신

```
return wrapper()
```

를 사용하면 어떻게 될까?

후자의 경우 Wrapper를 반환하기 전에 즉시 호출하려고 한다.

인수가 필요한 Wrapper라면 `TypeError`가 발생할 수도 있다.

인수가 필요 없더라도 Wrapper의 실행 결과가 반환되므로 의도했던 함수 객체가 반환되지 않을 수 있다.

물론 Wrapper의 호출 결과 자체가 다시 호출 가능한 객체라면 이후 호출이 가능할 수도 있지만, 그것은 별도로 설계한 동작이지 일반적인 데코레이터의 기본 구조는 아니다.

`return이 없으면 무엇이 반환되는가?`

다음 코드를 보자.

```
def show(value):
    print(value)

result = show(10)

print(result is None)
```

예상 실행 결과:

```
10
True
```

`show()`에는 명시적인 `return`이 없다.

함수 본문이 정상적으로 끝나면 Python은 `None`을 반환한다.

여기서 `print(value)`가 화면에 10을 출력했다는 사실과 `show()`가 10을 반환했다는 것은 서로 다른 이야기이다.

`print()`의 출력은 관찰 가능한 부작용(Side Effect)이고, 함수의 반환값은 호출 표현식이 평가된 결과다.

또한 함수 안에서 인수 없는 `return`을 실행하는 경우에도 `None`이 반환된다.

이는 일반적인 `def` 함수의 실행을 설명한 것이며, 제너레이터 함수나 코루틴 함수에는 추가적인 호출·실행 규칙이 적용된다.

그 차이는 `A.13`에서 따로 다룰 것이다.

A.1.10 일급 함수(First-Class Function) - 함수를 데이터처럼 다루기

지금까지 함수 객체를 다른 이름에 바인딩할 수도 있고, 반환할 수도 있다는 사실을 확인했다.

이러한 성질은 Python에서 함수가 일급 객체(First-Class Object)로 취급되기 때문에 가능하다.

일급 객체라는 표현은 특정 객체를 일반적인 값처럼 변수에 바인딩하고, 자료구조에 저장하고, 함수에 전달하고, 반환할 수 있다는 의미로 사용된다.

다음 예제를 보자.

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def apply(operation, a, b):
    return operation(a, b)

operations = {
    "sum": add,
    "product": multiply,
}

print(apply(operations["sum"], 3, 4))
print(apply(operations["product"], 3, 4))
```

예상 실행 결과:

```
7
12
```

이 코드는 단순하지만 중요한 구조를 포함하고 있다.

객체 생성 및 저장

먼저 `add`, `multiply`, `apply`라는 세 개의 함수 객체가 생성된다.

그리고 다음 딕셔너리를 만든다.

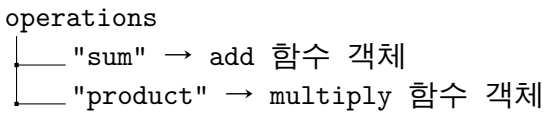
```
operations = {
    "sum": add,
    "product": multiply,
```

```
}
```

이 딕셔너리의 값은 함수 실행 결과가 아니다.

함수 객체에 대한 참조다.

따라서 다음 구조가 형성된다.



함수 객체를 인수로 전달

다음 표현식을 실행하면

```
apply(operations["sum"], 3, 4)
```

먼저 딕셔너리에서 `"sum"`에 대응하는 함수 객체를 조회한다.

그 함수 객체가 `apply`의 첫 번째 매개변수 `operation`에 전달된다.

그리고 3, 4가 각각 `a`, `b`에 전달된다.

함수 본문에서는 다음 호출이 실행된다.

```
operation(a, b)
```

`operation`은 이 호출에서 `add` 함수 객체를 참조하고 있으므로 `add(3, 4)`가 실행되는 것이다.

고차 함수(Higher-Order Function)와의 관계

여기서 `apply`는 고차 함수다.

고차 함수는 다른 함수를 인수로 받거나 함수를 반환하는 함수를 의미한다.

따라서 일급 함수와 고차 함수는 관계가 있지만 동일한 개념은 아니다.

일급 함수는 함수 객체를 어떤 값으로 취급할 수 있는가에 관한 성질이고, 고차 함수는 그 함수 객체를 인수나 반환값으로 사용하는 함수의 형태를 설명하는 개념이다.

이 구분은 데코레이터를 학습할 때 다시 연결된다.

데코레이터 함수는 보통 다른 함수 객체를 인수로 받으므로 고차 함수의 한 사례가 될 수 있다.

A.1.11 함수 객체의 내부 정보 - __name__, __code__, __globals__

이제 함수가 정말 하나의 객체라면 어떤 속성을 가지고 있는지도 확인해 보자.

다음 코드를 살펴보자.

```
def add(a, b=10):
    return a + b

print(add.__name__)
print(add.__defaults__)
print(add.__globals__ is globals())
print(add.__closure__)
print(type(add.__code__).__name__)
```

예상 실행 결과:

```
add
(10,)
True
None
code
```

이 결과는 함수 객체가 단순히 실행 코드 하나만 가진 구조가 아니라는 사실을 보여준다.

속성	의미
`__name__`	함수의 이름을 나타내는 메타데이터
`__qualname__`	함수가 정의된 렉시컬 구조를 반영하는 정규화된 이름
`__code__`	함수의 실행 코드 정보를 담는 코드 객체
`__globals__`	함수가 정의된 모듈의 전역 네임스페이스
`__defaults__`	위치 매개변수 기본값을 담는 튜플 또는 `None`
`__kwdefaults__`	키워드 전용 매개변수 기본값 정보
`__closure__`	캡처한 자유 변수와 연결된 셀 객체들의 튜플 또는 `None`
`__dict__`	일반 사용자 정의 함수 객체에 추가한 속성을 저장하는 딕셔너리

이러한 속성은 Python의 함수 데이터 모델에 정의되어 있다.

`__code__`와 함수 객체는 동일한가?

아니다.

`__code__`는 코드 객체(Code Object)를 가리킨다.

반면 함수 객체는 코드 객체 이외에도 기본 인수, 네임스페이스 참조, 클로저 등의 정보를 가지고 있다.

따라서 함수 객체를 코드 객체 하나로 환원해서 이해하면 안 된다.

`__globals__`는 무엇을 참조하는가?

다음 코드를 보자.

```
RATE = 2

def calculate(x):
    return x * RATE

print(calculate(4))

RATE = 3

print(calculate(4))
```

예상 실행 결과:

```
8
12
```

여기서 calculate 함수 객체는 처음 정의되었을 때 전역 변수 RATE의 값 2를 복사해 저장한 것이 아니다.

함수가 정의된 모듈의 전역 네임스페이스를 참조하고 있고, 호출 시 RATE를 조회할 때 그 네임스페이스에 현재 바인딩된 객체를 사용하는 것이다.

따라서 RATE = 3으로 전역 이름을 재바인딩한 뒤 함수를 다시 호출하면 결과가 달라진다.

이것은 A.3의 이름 탐색과 연결되는 중요한 예제다.

__globals__와 __closure__의 차이

위 예제의 RATE는 전역 이름이다.

따라서 이 예제에서는 calculate.__closure__가 None이더라도 RATE에 접근할 수 있다.

클로저가 외부 함수의 지역 바인딩을 어떻게 보존하는지는 A.4에서 학습할 것이다.

이때는 __closure__, 코드 객체의 co_freevars, 셀 객체(Cell Object)를 연결해 살펴본다.

A.1.12 클래스 객체 모델과 다시 연결하기

이제 A.1에서 학습한 함수 객체를 클래스 내부에 배치해 보자.

```python
class Calculator:
    def add(self, a, b):
        return a + b

calc = Calculator()

func = Calculator.__dict__["add"]

method = calc.add

print(type(func).__name__)
print(type(method).__name__)
```

```
print(method.__func__ is func)
print(method.__self__ is calc)

print(func(calc, 3, 4))
print(method(3, 4))
```

예상 실행 결과:

```
function
method
True
True
7
7
```

여기에서는 같은 함수를 서로 다른 경로로 호출하고 있다.

첫 번째 경로를 살펴보자.

```
func = Calculator.__dict__["add"]
```

클래스 네임스페이스에서 add에 대응하는 함수 객체를 직접 얻는다.

따라서 func의 타입은 function이다.

다음 호출은

```
func(calc, 3, 4)
```

함수 객체를 직접 호출하면서 첫 번째 인수로 calc를 전달하는 구조다.

반면 두 번째 경로에서는

```
method = calc.add
```

calc.add 속성에 접근할 때 함수 객체의 디스크립터 메커니즘을 통해 calc에 바인딩된 메서드 객체를 얻는다.

따라서 다음 호출에서는

```
method(3, 4)
```

이미 바인딩된 calc가 메서드 호출 과정에서 첫 번째 인수로 전달되는 것이다.

이 구조를 비교하면 다음과 같다.

구분	func	method
객체 타입	`function`	`method`
참조 대상	클래스에 저장된 함수 객체	인스턴스에 바인딩된 메서드 객체
`self` 전달	명시적으로 전달	바인딩된 인스턴스가 전달
호출	`func(calc, 3, 4)`	`method(3, 4)`

이렇게 보면 A.1의 함수 객체는 클래스 객체 모델과 분리된 별개의 시스템이 아니다.

클래스 내부의 일반적인 인스턴스 메서드 역시 함수 객체를 바탕으로 디스크립터와 메서드 바인딩 규칙이 결합된 결과라는 사실을 확인할 수 있다.

이 원리는 나중에 데코레이터로 함수 객체를 교체하거나, Wrapper를 클래스 메서드로 사용할 때 특히 중요해진다.

A.1.13 실제 응용 - 함수 객체를 이용한 명령 처리기

함수 객체를 이름에 바인딩하고, 딕셔너리에 저장하며, 나중에 호출하는 구조는 실제 프로그램에서도 유용하다.

예를 들어 다음과 같은 간단한 명령 처리 시스템을 만들 수 있다.

```
def greet(name):
    return f"Hello, {name}"

def farewell(name):
    return f"Goodbye, {name}"

handlers = {}

handlers["greet"] = greet
handlers["farewell"] = farewell

def dispatch(command, name):
    handler = handlers[command]

    return handler(name)

print(dispatch("greet", "Kim"))
```

```
print(dispatch("farewell", "Lee"))
```

예상 실행 결과:

```
Hello, Kim
Goodbye, Lee
```

이 프로그램에서는 명령 문자열에 따라 다른 함수 객체를 선택해 호출한다.

구조적으로 중요한 부분은 다음이다.

```
handlers["greet"] = greet
```

여기서는 `greet()`을 실행하지 않는다.

대신 `greet` 함수 객체를 딕셔너리에 등록해 둔다.

나중에 다음 호출이 발생하면

```
dispatch("greet", "Kim")
```

딕셔너리에 등록된 함수 객체를 얻는다.

```
handler = handlers[command]
```

그 다음에 해당 함수 객체를 호출한다.

```
return handler(name)
```

즉, 함수의 등록 시점과 실행 시점이 분리되어 있다.

이것이 추후 등록형 데코레이터를 이해하기 위한 첫 번째 기반이다.

A.12에서는 이러한 명령 처리 구조를 더 발전시켜, `self.routes` 같은 인스턴스 저장소에 함수 객체를 등록하고 요청 시 호출하는 메커니즘을 직접 구성한 뒤 FastAPI의 경로 등록 구조와 비교할 것이다.

그때는 기존에 배운 클래스 객체, `self`, 메서드 바인딩과 이번 과정의 클로저까지 모두 연결할 수 있게 된다.

A.1.14 경계 조건과 흔한 오해

이번 단원에서 반드시 구별해야 할 표현식들을 표로 정리해 보자.

표현식	정확한 의미
`def f(...): ...`	함수 정의문을 실행하여 함수 객체를 생성하고 이름을 바인딩
`f`	이름을 탐색하여 현재 바인딩된 객체를 얻음
`f()`	f가 참조하는 객체를 호출하고 결과를 얻음
`g = f`	기존 함수 객체에 새로운 이름을 바인딩
`g = f()`	함수 호출의 반환 객체를 g에 바인딩
`return f`	함수 객체를 반환
`return f()`	함수 객체를 호출한 결과를 반환
`callable(f)`	해당 객체가 호출 가능한지 검사
`f.__name__`	함수 객체의 이름 메타데이터 조회
`f.__globals__`	함수가 참조하는 전역 네임스페이스 조회
`f.__closure__`	캡처된 자유 변수의 셀 객체 정보 조회
`type(f)`	f가 참조하는 객체의 실제 타입 조회

한 가지 더 기억할 점이 있다.

함수 객체를 호출하는 행위가 항상 정상적인 반환으로 이어지는 것은 아니다.

호출 과정에서 인수 바인딩 오류가 발생할 수도 있고, 본문 실행 중 예외가 전파될 수도 있으며, 함수가 다른 호출 가능한 객체를 반환할 수도 있다.

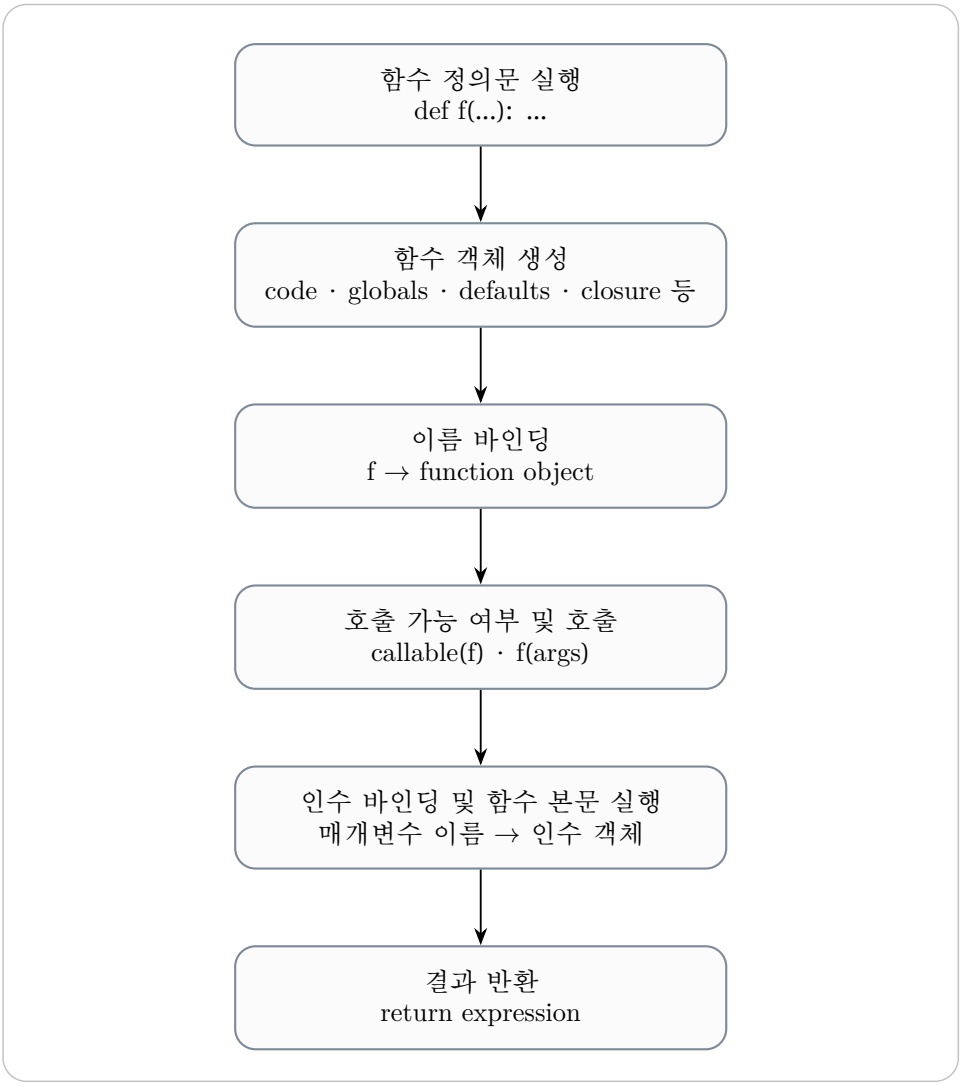
또한 함수 객체가 반환되었다고 해서 그것이 반드시 클로저라는 의미는 아니다.

우리가 A.1에서 살펴본 `get_operation()`의 반환 함수는 외부 함수의 지역 상태를 캡처하지 않았다.

이러한 차이를 유지해야 A.2에서 중첩 함수를 학습하더라도 함수 반환과 클로저 형성을 혼동하지 않게 된다.

A.1.15 핵심 정리와 판단 기준

이번 단원의 핵심은 다음 실행 모델이다.



이제 코드를 분석할 때는 다음 판단을 구분하면 된다.

첫째, 새로운 함수 객체가 생성되는 것인지, 이미 존재하는 함수 객체를 참조하는 것인지 판단해야 한다.

둘째, 이름에 객체를 바인딩하는 것인지, 기존 이름을 재바인딩하는 것인지 구분해야 한다.

셋째, ()가 호출하는 대상이 함수 객체인지, 클래스 객체인지, 바운드 메서드인지, 아니면 다른 호출 가능한 인스턴스인지 확인해야 한다.

넷째, 호출 시 어떤 객체가 매개변수에 바인딩되고, 함수 본문에서 실제로 어떤 객체가 변경되는지 추적해야 한다.

다섯째, 함수 객체 자체를 반환하는 것과 함수 호출 결과를 반환하는 것을 구별해야 한다.

이 다섯 가지가 A.1의 판단 기준이다.

A.2 중첩 함수와 Lexical Scope

Nested Functions · Lexical Scoping · Function Definition, Execution

A.2.1 이전 내용과의 연결

A.1에서는 Python의 함수가 단순한 코드 블록이 아니라

실제로 생성되고 참조되는 함수 객체(Function Object)라는 사실을 학습했다.

특히 다음 네 가지를 구분하였다.

1. def 문장의 실행은 함수 객체를 생성하고 이름을 바인딩한다.
2. func은 함수 객체를 참조하지만, func()은 그 객체를 호출한다.
3. 함수 객체는 다른 함수의 인수로 전달하거나 반환할 수 있다.
4. 함수 호출 시 인수 객체가 매개변수에 바인딩되며,

   return은 호출 결과를 반환한다.

이번에는 이 원리들을 함수 내부에서 또 다른 함수를 정의하는 경우로 확장할 것이다.

다음 코드를 살펴보자.

```
def outer():
    x = 10
```

```
    def inner():
        return x

    return inner

f = outer()

print(f())
```

예상 실행 결과:

```
10
```

여기에는 중요한 질문이 숨어 있다.

`outer()`의 호출은 이미 끝났는데, 왜 나중에 호출한 `inner()`는 `outer()` 내부에서 정의한 `x`에 접근할 수 있을까?

이 현상을 설명하려면 다음 개념을 구분해야 한다.

- 함수가 어디에서 정의되었는가?
- 함수 객체는 언제 생성되는가?
- 함수 본문은 언제 실행되는가?
- 함수 내부의 이름은 어떤 스코프를 기준으로 탐색되는가?
- 외부 함수의 실행이 종료되면 그 지역 변수의 바인딩은 어떻게 되는가?

마지막 질문은 `A.4`의 클로저와 직접 연결된다.

이번 `A.2`에서는 그에 앞서 중첩 함수의 정의 구조와 Lexical Scope의 원리를 확립할 것이다.

A.2.2 중첩 함수(Nested Function)의 정의

중첩 함수란 다른 함수의 본문 내부에서 정의되는 함수를 말한다.

가장 기본적인 구조는 다음과 같다.

```
def outer():
```

```
    def inner():
        return "Hello"

    return inner()
```

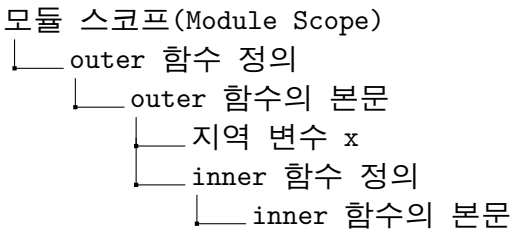
여기서 `outer`는 외부 함수(Outer Function), `inner`는 내부 함수(Inner Function)이다.

하지만 이 용어가 두 함수 사이에 특별한 상속 관계가 있다는 의미는 아니다.

`outer`와 `inner`는 각각 독립적인 함수 객체로 생성될 수 있다.

두 함수 사이의 중요한 관계는 정의된 위치에 따른 렉시컬 스코프의 중첩이다.

중첩 함수의 기본 구조



여기서 주의할 점이 있다.

위 그림은 함수의 소스 코드상 정의 위치를 나타내는 것이지, 함수 객체가 모두 동시에 생성되어 있다는 뜻은 아니다.

이 둘을 구별하는 것이 이번 단원의 첫 번째 핵심이다.

A.2.3 중첩 함수의 생성 시점

다음 코드를 살펴보자.

```
def outer():
    print("outer 실행 시작")

    def inner():
        print("inner 실행")
        return 10

    print("inner 정의 완료")
```

```
    return inner

print("모듈 코드 실행")

f = outer()

print("outer 호출 완료")

result = f()

print(result)
```

예상 실행 결과:

```
모듈 코드 실행
outer 실행 시작
inner 정의 완료
outer 호출 완료
inner 실행
10
```

이 결과를 정확히 이해하려면 함수 객체 생성과 함수 본문 실행을 순서대로 추적해야 한다.

1. outer 함수 객체 생성

Python이 다음 정의문을 실행한다.

```
def outer():
    ...
```

이때 outer 함수 객체가 생성되고 모듈 네임스페이스의 outer라는 이름에 바인딩된다.

그러나 outer 본문은 실행되지 않는다.

따라서 이 시점에는 outer 본문에 있는 def inner 문장도 실행되지 않았다.

여기서 중요한 구분이 있다.

Python은 소스 코드를 컴파일하는 과정에서 중첩 함수의 코드 객체를 준비할 수 있다.

하지만 그것이 곧 실행 시점의 inner 함수 객체가 생성되었다는 의미는 아니다.

코드 객체(Code Object)와 함수 객체(Function Object)는 서로 다른 객체다.

2. outer() 호출

```
f = outer()
```

이제 outer 함수 객체가 호출된다.

일반적인 Python 함수 호출에서는 해당 호출을 위한 실행 프레임(Frame)이 생성되고, 매개변수와 지역 이름을 관리하는 실행 환경이 마련된다.

이 프레임에서 다음 문장이 실행된다.

```
print("outer 실행 시작")
```

첫 번째 함수 내부 출력이 발생한다.

3. inner 함수 객체 생성

이어서 다음 정의문에 도달한다.

```
def inner():
    print("inner 실행")
    return 10
```

이때 새로운 함수 객체가 생성된다.

그리고 outer() 호출의 지역 스코프에서 inner라는 이름이 그 객체에 바인딩된다.

그러나 inner 함수 본문은 아직 실행되지 않았다.

즉, 현재 상태는 다음과 같다.

```
모듈 네임스페이스
└── outer → outer 함수 객체

outer() 실행 환경
└── inner → inner 함수 객체
```

여기서 두 함수 객체는 서로 다른 객체다.

```
type(outer)
type(inner)
```

두 객체 모두 일반적인 사용자 정의 함수라면 타입은 function이지만, inner라는 지역 이름은 현재 outer() 실행 환경에서만 직접 접근할 수 있다.

따라서 모듈 수준에서 곧바로 inner를 참조하려고 하면

일반적으로 NameError가 발생한다.

4. inner 함수 객체 반환

```
return inner
```

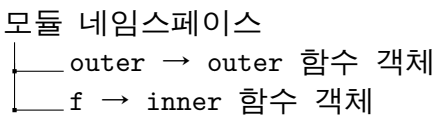
여기서 inner()가 아니라 inner를 반환한다는 점에 주목해야 한다.

inner가 참조하는 함수 객체 자체를 반환하는 것이다.

따라서 다음 문장의 실행이 완료되면

```
f = outer()
```

모듈 네임스페이스의 f가 반환된 함수 객체를 참조하게 된다.



이후 outer() 호출은 종료된다.

그러나 반환된 inner 함수 객체는 f가 참조하고 있으므로 계속 존재할 수 있다.

5. 반환된 함수 객체 호출

마지막으로 다음 문장이 실행된다.

```
result = f()
```

f는 inner 함수 객체를 참조하고 있다.

따라서 inner 함수 본문이 실행되고 10을 반환한다.

이 결과가 result에 바인딩되는 것이다.

핵심 구분

사건	발생 시점
outer 함수 객체 생성	모듈에서 def outer 실행 시
inner 함수 객체 생성	outer() 실행 중 def inner에 도달할 때
outer 함수 본문 실행	outer() 호출 시
inner 함수 본문 실행	inner() 또는 f() 호출 시
inner 함수 객체 반환	outer()의 return inner 실행 시

여기까지가 중첩 함수의 기본 실행 구조다.

A.2.4 중첩 함수는 호출할 때마다 새로 생성되는가?

이제 다음 코드를 살펴보자.

```
def outer():

    def inner():
        return 10

    return inner


f1 = outer()
f2 = outer()

print(f1 is f2)

print(f1.__code__ is f2.__code__)
```

일반적인 CPython 실행 결과:

```
False
True
```

왜 이런 차이가 발생할까?

함수 객체의 식별성

먼저 다음 두 호출을 살펴보자.

```
f1 = outer()
f2 = outer()
```

첫 번째 `outer()` 호출에서 `def inner` 문장이 실행되면서 함수 객체가 생성된다.

두 번째 `outer()` 호출에서도 같은 정의문이 다시 실행되므로 새로운 함수 객체가 생성된다.

따라서 두 함수 객체는 동일한 객체가 아니다.

```
f1 is f2
```

결과는 `False`가 된다.

그러데 두 함수의 코드 객체는 어떨까?

```
f1.__code__ is f2.__code__
```

일반적인 CPython에서는 두 함수 객체가 동일한 코드 객체를 참조한다.

왜냐하면 같은 함수 정의에 대응하는 컴파일된 코드 객체를 재사용하면서 서로 다른 함수 객체를 생성할 수 있기 때문이다.

다만 코드 객체의 `is` 동일성은 Python 언어 차원에서 모든 구현에 보장되는 일반 규칙으로 취급하면 안 된다.

여기서 확실하게 구분해야 하는 것은 다음이다.

동일한 함수 정의에서 생성된 함수 객체라고 해서 반드시 동일한 함수 객체인 것은 아니다.

반면 같은 함수 객체를 여러 이름에 바인딩하는 것은 새로운 함수 객체 생성과 다르다.

```
f1 = outer()
f2 = f1

print(f1 is f2)
```

출력:

```
True
```

이 경우에는 함수 객체를 새로 생성하지 않고 기존 객체에 이름을 추가로 바인딩했기 때문이다.

이 차이는 A.4에서 여러 클로저 인스턴스가 서로 독립적인 상태를 유지하는 원리를 이해할 때 중요해진다.

A.2.5 Lexical Scope란 무엇인가?

이제 이번 학습의 중심 개념으로 넘어가 보자.

Lexical Scope(렉시컬 스코프)란 함수 내부에서 사용한 이름이 어떤 외부 스코프와 연결되는지가 소스 코드상 함수의 정의 위치에 따라 결정되는 규칙이다.

이를 정적 스코프(Static Scope)라고도 부른다.

여기서 중요한 것은 함수가 어디에서 호출되었는가가 아니라 어디에서 정의되었는가이다.

다음 코드를 보자.

```
x = "Global"

def outer():
    x = "Outer"

    def inner():
        print(x)

    return inner

def caller(func):
    x = "Caller"
    func()

f = outer()

caller(f)
```

예상 실행 결과:

```
Outer
```

이 결과는 매우 중요하다.

왜 "Caller"가 아니라 "Outer"가 출력될까?

실행 과정 분석

먼저 모듈 스코프에 다음 이름이 바인딩된다.

```
x = "Global"
```

그다음 outer와 caller 함수 객체가 생성된다.

이후 다음 표현식이 실행된다.

```
f = outer()
```

outer() 내부에서는 지역 이름 x가 문자열 "Outer"를 참조하게 된다.

그리고 inner 함수 객체가 생성된다.

이때 inner 함수는 소스 코드상 outer 내부에 정의되어 있다.

따라서 inner의 렉시컬 외부 스코프는 outer의 지역 스코프다.

이후 inner 함수 객체가 반환되어 f에 바인딩된다.

이제 다음 호출을 살펴보자.

```
caller(f)
```

caller()의 매개변수 func는 f가 참조하는 inner 함수 객체를 참조하게 된다.

caller() 내부에서는 다음 문장이 실행된다.

```
x = "Caller"
```

그러나 이 x는 caller() 호출의 지역 이름이다.

inner 함수가 정의된 렉시컬 구조에는 caller의 지역 스코프가 포함되어 있지 않다.

따라서 다음 호출에서

```
func()
```

inner가 실행되더라도 caller의 지역 변수 x를 탐색하지 않는다.

inner는 자신이 정의된 외부 함수 outer의 바인딩을 사용한다.

그래서 "Outer"가 출력되는 것이다.

Lexical Scope와 Dynamic Scope 비교

구분	Lexical Scope	Dynamic Scope
이름 탐색의 기준	함수가 정의된 코드 구조	함수 호출의 동적 경로
외부 이름의 연결	렉시컬 외부 스코프	호출자 측 실행 환경
Python의 일반적인 함수 이름 탐색	사용	사용하지 않음
위 예제의 x	"Outer"	가상의 동적 스코프라면 "Caller"가 될 수 있음

Python은 일반적인 함수 이름 탐색에 Lexical Scope를 사용한다.

따라서 함수 객체를 다른 함수에 전달하거나 다른 위치에서 호출한다고 해서 그 함수의 렉시컬 외부 스코프가 변경되지는 않는다.

이것이 Lexical Scope의 핵심이다.

A.2.6 정의 위치와 호출 위치는 왜 구분해야 할까?

앞의 예제를 조금 변형해 보자.

```
x = 100

def outer():
    x = 200

    def inner():
        return x

    return inner

f = outer()

def execute():
    x = 300
```

```
    return f()

print(execute())
```

예상 실행 결과:

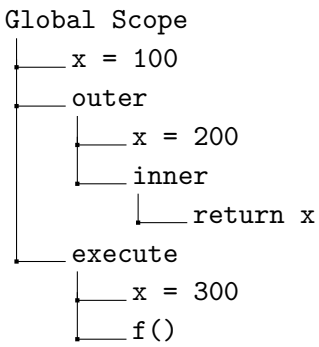
```
200
```

여기서 inner 함수는 execute() 내부에서 호출되고 있다.

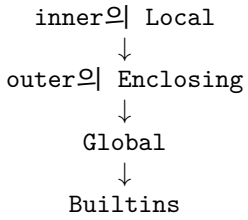
하지만 execute()의 지역 변수 x=300은 inner의 이름 탐색에 영향을 주지 않는다.

또한 모듈 스코프에는 x=100이 존재하지만, inner는 자신을 감싸는 outer의 x를 사용한다.

이 관계를 렉시컬 구조로 나타내면 다음과 같다.



inner가 참조하는 x의 탐색 관계는 다음과 같다.



여기서는 outer의 x를 찾으므로 그 값을 사용한다.

반면 execute는 inner의 렉시컬 외부 스코프에 포함되지 않는다.

함수 호출 관계(Call Relationship)와 렉시컬 스코프 관계(Lexical Scope Relationship)는 서로 다른 구조라는 사실을 기억해야 한다.

이것은 이후 클로저와 데코레이터에서 매우 중요해진다.

A.2.7 중첩 함수는 외부 함수의 모든 지역 변수를 사용할 수 있는가?

중첩 함수는 자신을 감싸는 외부 함수의 지역 이름을 참조할 수 있다.

하지만 모든 이름이 자동으로 클로저에 저장되는 것은 아니다.

다음 코드를 살펴보자.

```
def outer():
    x = 10
    y = 20

    def inner():
        return x

    return inner

f = outer()

print(f())
print(f.__code__.co_freevars)
```

예상 실행 결과:

```
10
('x',)
```

여기서 `inner`는 `x`만 참조하고 있다.

`y`는 `outer`의 지역 변수이지만 `inner`에서 사용되지 않는다.

따라서 `inner`의 자유 변수(Free Variable) 목록에는 `x`만 나타난다.

여기서 자유 변수란 현재 함수의 지역 스코프에서 바인딩되지 않았지만, 렉시컬 외부 함수 스코프의 바인딩을 사용하는 이름을 말한다.

`co_freevars`는 코드 객체가 어떤 자유 변수 이름을 사용하는지 확인하는 데 유용하다.

다만 지금은 자유 변수가 존재한다는 사실만 확인하면 충분하다.

자유 변수가 실제로 어떤 셀 객체와 연결되고, 외부 함수 호출이 종료된 뒤에도 어떻게 유지되는지는 A.4에서 자세히 분석할 것이다.

A.2.8 외부 함수의 지역 변수는 언제 결정되는가?

Lexical Scope를 학습할 때 자주 발생하는 오해가 있다.

함수의 정의 위치가 이름 탐색의 스코프를 결정한다는 사실과, 그 이름에 바인딩된 객체가 함수 정의 시점에 고정된다는 것은 서로 다른 이야기다.

다음 코드를 보자.

```
def outer():
    x = 10

    def inner():
        return x

    x = 20

    return inner

f = outer()

print(f())
```

예상 실행 결과:

```
20
```

왜 10이 아니라 20일까?

inner 함수 객체가 생성되는 시점에는 x가 10을 참조하고 있다.

하지만 inner는 단순히 그 시점의 정수 객체 10을 복사해서 보관하는 것이 아니다.

inner가 사용하는 외부 이름 x는 외부 함수의 바인딩과 연결된다.

따라서 이후 다음 문장이 실행되면

```
x = 20
```

외부 함수의 x 바인딩이 새로운 정수 객체 20을 참조하도록 변경된다.

그리고 inner를 호출하면 현재 연결된 바인딩을 통해 20을 얻는 것이다.

이것이 이후 클로저를 학습할 때 중요한 출발점이 된다.

더 극단적인 사례

```
def outer():

    def inner():
        return x

    x = 100

    return inner

f = outer()

print(f())
```

예상 실행 결과:

```
100
```

inner가 정의되는 순간에는 아직 x = 100이 실행되지 않았다.

그런데도 inner()는 정상적으로 100을 반환한다.

왜냐하면 이름의 렉시컬 연결과 실제 값의 조회 시점이 다르기 때문이다.

단, 이 예제에서 x = 100이 실행되기 전에 inner()를 호출하면 바인딩되지 않은 자유 변수를 읽으려 하므로 NameError가 발생할 수 있다.

이 오류는 A.3에서 지역 변수의 정적 판정과 함께 분석할 것이다.

A.2.9 중첩 함수와 일반 함수의 차이

지금까지의 내용을 비교해 보자.

```
x = 10

def normal():
    return x

def outer():
```

```
    x = 20

    def inner():
        return x

    return inner

f = outer()

print(normal())
print(f())
```

예상 실행 결과:

```
10
20
```

두 함수 모두 본문에서 x라는 이름을 사용한다.

하지만 normal은 모듈 스코프에서 정의되었고, inner는 outer 내부에서 정의되었다.

따라서 이름 탐색 경로가 다르다.

구분	normal	inner
정의 위치	모듈 스코프	outer 내부
x의 종류	전역 이름	자유 변수
외부 탐색 대상	모듈 전역 네임스페이스	outer의 렉시컬 스코프
반환값	10	20

이 차이는 단순히 중첩 함수가 특별한 문법을 사용하기 때문에 발생하는 것이 아니다.

함수가 정의된 렉시컬 환경에 따라 이름 탐색의 대상이 달라지기 때문이다.

A.2.10 중첩 함수와 클래스 메서드의 차이

이제 기존 클래스 객체 모델과 연결해 보자.

다음 코드를 살펴보자.

```
class Example:
    x = 100

    def method(self):
        return x

obj = Example()
```

여기서 method() 내부의 x는 클래스 네임스페이스에 있는 Example.x를 자동으로 참조할까?

그렇지 않다.

일반적인 메서드의 함수 본문에서 이름을 탐색할 때 클래스 네임스페이스가 자동으로 Enclosing 함수 스코프처럼 취급되는 것은 아니다.

따라서 모듈 수준에 x가 정의되어 있지 않다면 다음 호출은 NameError를 발생시킨다.

```
obj.method()
```

반면 다음 코드는 정상적으로 동작한다.

```
class Example:
    x = 100

    def method(self):
        return self.x

obj = Example()

print(obj.method())
```

예상 실행 결과:

```
100
```

여기서는 이름 탐색과 속성 탐색을 구분해야 한다.

```
return x
```

는 이름 탐색(Name Resolution)이다.

반면

```
return self.x
```

는 먼저 `self`라는 이름을 탐색한 뒤, 그 객체의 `x`속성을 탐색하는 표현식이다.

따라서 `self.x`는 기존에 배운 인스턴스·클래스 속성 탐색 규칙에 따라 처리된다.

이것은 Lexical Scope와 객체의 속성 탐색이 서로 다른 메커니즘이라는 사실을 보여준다.

단, 클래스 본문 안에서 정의된 모든 중첩 함수가 클래스 네임스페이스를 절대 참조할 수 없다는 뜻은 아니다.

특히 클래스 본문에서 직접 실행되는 표현식, 컴프리헨션, `__class__` 셀과 관련된 특수한 규칙은 별도로 존재한다.

이번 학습에서는 일반적인 인스턴스 메서드의 이름 탐색과 `self`를 통한 속성 탐색이 다르다는 점을 확립하면 된다.

A.2.11 실제 응용 - 함수 생성기(Function Factory)

중첩 함수의 실용적인 활용을 살펴보자.

```
def make_multiplier(factor):

    def multiply(value):
        return factor * value

    return multiply


double = make_multiplier(2)
triple = make_multiplier(3)

print(double(10))
print(triple(10))
```

예상 실행 결과:

```
20
30
```

이 코드는 함수 객체를 생성하여 반환하는 함수 생성기(Function Factory)의 예제다.

첫 번째 호출

```
double = make_multiplier(2)
```

make_multiplier()가 호출되면서 factor에 정수 객체 2가 바인딩된다.

그 호출 중 multiply 함수 객체가 생성된다.

이 함수 객체는 외부 함수의 factor 바인딩을 참조하는 구조를 가진다.

이후 multiply 함수 객체가 반환되고 double에 바인딩된다.

두 번째 호출

```
triple = make_mulitplier(3)
```

이번에는 별도의 make_muliplier() 호출이 발생한다.

새로운 실행 환경에서 factor에 3이 바인딩되고, 새로운 multiply 함수 객체가 생성된다.

따라서 double과 triple은 서로 다른 함수 객체다.

```
print(double is triple)
```

출력:

```
False
```

각 함수는 서로 다른 외부 함수 호출에서 형성된 바인딩을 참조하므로 서로 다른 결과를 반환할 수 있다.

여기까지 이해했다면 다음 질문이 자연스럽게 발생할 것이다.

외부 함수 호출이 끝났는데도 factor에 대한 바인딩이 어떻게 유지되는가?

이 질문이 바로 A.4에서 다룰 클로저의 핵심이다.

현재는 함수가 정의된 렉시컬 환경과 실제 호출 시점이 분리될 수 있다는 사실에 집중한다.

A.2.12 핵심 정리

이번 학습에서 가장 중요한 것은 다음 다섯 가지다.

> 1. 중첩 함수는 함수 내부에서 정의되는 함수다.

```
def outer():
    def inner():
        pass
```

두 함수는 서로 다른 함수 객체가 될 수 있다.

> 2. 함수 객체 생성과 함수 본문 실행은 다르다.

`outer()`가 호출되어 `def inner` 문장에 도달하면 `inner` 함수 객체가 생성된다.

그러나 `inner` 본문은 `inner()`가 호출될 때 실행된다.

> 3. Lexical Scope는 정의 위치를 기준으로 한다.

함수가 어디에서 호출되는지가 아니라 어디에서 정의되었는지가 렉시컬 외부 스코프를 결정한다.

> 4. 이름의 탐색 대상과 바인딩된 값은 구분해야 한다.

함수가 외부 이름을 참조한다고 해서 함수 정의 시점의 값을 반드시 복사하는 것은 아니다.

> 5. 함수 호출 관계와 스코프 관계는 다르다.

다른 함수가 중첩 함수를 호출하더라도 호출자의 지역ㄷ 스코프가 그 중첩 함수의 Enclosing Scope가 되는 것은 아니다.

A.3 LEGB와 이름 탐색

Name Resolution · Static Scope Analysis · UnboundLocalError · global

A.3.1 이전 내용과의 연결

A.1에서는 함수 객체의 생성, 이름 바인딩, 호출과 반환을 학습하였다.

A.2에서는 이를 중첩 함수(Nested Function)로 확장하면서 두 가지 사실을 확인하였다.

1. 함수 객체가 생성되는 시점과 함수 본문이 실행되는 시점은 다르다.
2. 함수 내부의 이름 탐색은 호출 위치가 아니라

   정의 위치에 따른 렉시컬 스코프를 기준으로 한다.

그런데 여기서 해결하지 않은 문제가 하나 있다.

다음 코드를 보자.

```
x = 10

def func():
    print(x)

func()
```

예상 출력:

```
10
```

func()의 지역 스코프에는 x가 없으므로 모듈의 전역 스코프에서 x를 찾은 것이다.

그렇다면 다음 코드는 어떨까?

```
x = 10

def func():
    print(x)
    x = 20

func()
```

위에서 아래로 실행된다고 생각하면 print(x) 시점에는 아직 x = 20이 실행되지 않았으므로 전역 변수 x = 10을 사용할 것 같다.

하지만 실제로는 다음 예외가 발생한다.

```
UnboundLocalError: cannot access local variable 'x'
where it is not associated with a value
```

왜 이런 결과가 발생할까?

Python은 이름을 사용할 때마다 무조건 LEGB 순서로 모든 네임스페이스를 검색하는 것이 아니기 때문이다.

함수 본문을 실행하기 전에 해당 이름이 지역 이름인지, 외부 함수의 이름인지, 전역 이름인지 등을 판정하는 과정이 존재한다.

이번 학습에서는 이 원리를 정확하게 분석할 것이다.

A.3.2 이번 학습의 핵심 질문

이번 학습에서는 다음 질문에 답할 수 있어야 한다.

1. LEGB의 각 영역은 정확히 무엇을 의미하는가?
2. 이름의 스코프는 언제 결정되고, 실제 객체는 언제 조회할까?
3. 함수 안에 대입문이 있다는 이유만으로
   왜 UnboundLocalError가 발생할까?
4. global은 어떤 이름 바인딩 규칙을 변경할까?
5. global과 nonlocal은 어떤 점에서 다를까?
6. 이름 탐색(Name Resolution)과 객체의 속성 탐색(Attribute Lookup)은
   왜 구별해야 할까?

특히 다음 구분을 중심으로 설명할 것이다.

> 이름의 스코프를 정적으로 판정하는 과정과,
> 실행 시점에 그 이름이 참조하는 객체를 조회하는 과정은 다르다.

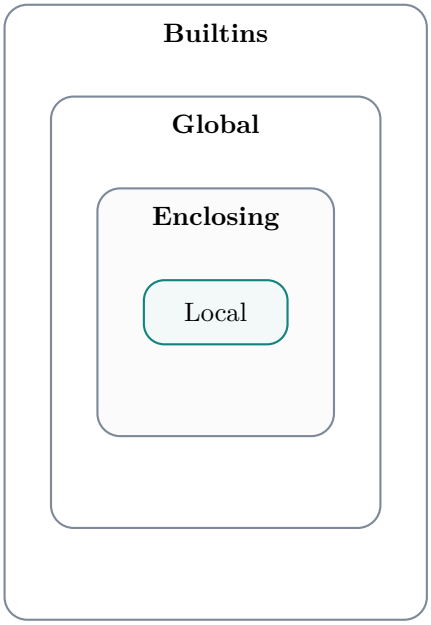
A.3.3 LEGB의 정확한 의미

LEGB는 Python에서 일반적인 이름 탐색을 설명하는 모델이다.

각 문자는 다음 영역을 의미한다.

구분	명칭	의미
L	Local	현재 함수 호출의 지역 스코프
E	Enclosing	렉시컬 구조상 현재 함수를 감싸는 외부 함수 스코프
G	Global	해당 함수가 속한 모듈의 전역 스코프
B	Builtins	Python의 내장 이름 스코프

기본적인 탐색 구조는 다음과 같다.



LEGB 네임스페이스의 포함 관계를 나타낸 개념도

이 그림은 개념적인 스코프 구조를 나타낸다.

실제 CPython이 모든 이름에 대해 네 개의 딕셔너리를 순서대로 검색한다는 의미는 아니다.

일반적인 함수의 지역 변수는 최적화된 실행 환경에서 관리될 수 있고, 자유 변수는 셀(Cell)을 통해 접근할 수 있다.

또한 `global`과 `nonlocal` 선언은 이름의 분류 자체를 변경한다.

따라서 LEGB는 이름 탐색의 의미론적 구조를 이해하기 위한 모델로 사용하는 것이 정확하다.

기본 예제

```
x = "Global"

def outer():
    x = "Enclosing"

    def inner():
        x = "Local"
        print(x)

    inner()

outer()
```

예상 출력:

```
Local
```

`inner()`에서 사용한 `x`는 현재 함수의 지역 이름이므로 외부 스코프를 탐색할 필요가 없다.

이번에는 지역 바인딩을 제거해 보자.

```
x = "Global"

def outer():
    x = "Enclosing"

    def inner():
        print(x)

    inner()

outer()
```

예상 출력:

```
Enclosing
```

`inner`에는 지역 이름 `x`가 없고, 렉시컬 외부 함수 `outer`의 `x`를 참조하므로 `"Enclosing"`이 출력된다.

다음으로 외부 함수의 x도 제거해 보자.

```
x = "Global"

def outer():

    def inner():
        print(x)

    inner()

outer()
```

예상 출력:

```
Global
```

이제 inner의 x는 전역 이름으로 분류되고, 모듈의 x를 조회한다.

마지막으로 내장 이름을 살펴보자.

```
def func():
    print(len([10, 20, 30]))

func()
```

예상 출력:

```
3
```

일반적인 실행 환경에서 len은 지역이나 전역 스코프에 별도로 정의되어 있지 않으므로 내장 이름을 사용한다.

여기까지는 익숙한 LEGB 설명이다.

하지만 이것으로는 UnboundLocalError를 설명할 수 없다.

A.3.4 핵심 원리 - 이름의 정적 판정(Static Scope Analysis)

Python은 함수 본문을 실행하기 전에 해당 함수 블록의 이름 사용과 바인딩을 분석한다.

이 과정에서 어떤 이름이 지역 이름인지, 자유 변수인지, 전역 이름인지 등이

결정된다.

이러한 분석은 함수 호출 시점에 처음 수행되는 것이 아니라, 일반적으로 소스 코드가 컴파일되는 과정에서 수행된다.

여기서 두 단계를 구분해야 한다.

1단계. 이름의 분류

컴파일 단계에서 함수 블록의 이름 사용을 분석한다.

예를 들어 다음과 같은 질문에 답하는 것이다.

- 현재 함수에서 x를 바인딩하는가?
- x에 global 선언이 있는가?
- x에 nonlocal 선언이 있는가?
- x가 외부 함수의 바인딩을 참조하는가?

2단계. 실행 시점의 객체 조회

실제로 함수가 호출되면 해당 이름에 연결된 바인딩을 통해 객체를 조회한다.

따라서 다음 두 문장은 서로 다른 의미를 가진다.

이름의 스코프가 결정되었다.

그 이름에 객체가 바인딩되었다.

이름이 지역 변수로 분류되었다고 해서 그 지역 변수에 반드시 객체가 바인딩되어 있는 것은 아니다.

바로 이 차이가 UnboundLocalError의 원인이다.

A.3.5 UnboundLocalError

는 왜 발생할까?

다음 코드를 다시 살펴보자.

```
x = 10
```

```
def func():
    print(x)
    x = 20

func()
```

예상 결과:

```
UnboundLocalError: cannot access local variable 'x'
where it is not associated with a value
```

예외 메시지의 구체적인 표현은 Python의 버전에 따라 달라질 수 있다.

1. 컴파일 단계

Python은 `func`의 본문을 분석한다.

```
def func():
    print(x)
    x = 20
```

여기서 다음 대입문을 발견한다.

```
x = 20
```

일반적인 함수 블록에서 별도의 `global` 또는 `nonlocal` 선언 없이 이름에 대입하면, 그 이름은 해당 함수의 지역 이름으로 분류된다.

따라서 `func` 내부의 `x`는 지역 변수로 판정된다.

중요한 점은 대입문이 함수 본문의 뒤쪽에 있다는 사실이 이 판정에 영향을 주지 않는다는 것이다.

2. 함수 호출

이제 `func()`가 호출된다.

현재 호출의 지역 실행 환경에는 `x`라는 지역 이름을 위한 자리가 있지만, 아직 객체가 바인딩되지 않았다.

3. print(x) 실행

다음 문장이 실행된다.

```
print(x)
```

Python은 이미 x를 지역 이름으로 판정했으므로 그 지역 바인딩에서 객체를 가져오려고 한다.

그런데 아직 x = 20이 실행되지 않았다.

따라서 x에는 객체가 바인딩되어 있지 않다.

그 결과 UnboundLocalError가 발생한다.

4. 전역 변수로 넘어가지 않는 이유

여기가 가장 중요하다.

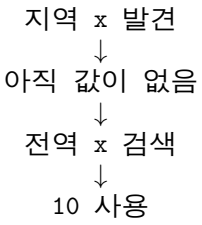
전역 스코프에는 분명히 다음 바인딩이 존재한다.

```
x = 10
```

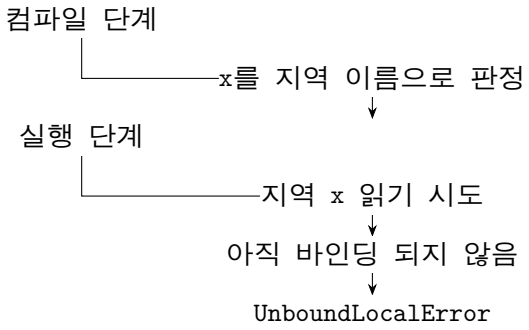
하지만 func 내부의 x는 이미 지역 이름으로 판정되었다.

따라서 지역 이름을 읽는 과정에서 바인딩되지 않은 상태가 발견되었다고 해서 전역 스코프로 다시 탐색을 진행하지 않는다.

즉, 다음처럼 동작하는 것이 아니다.



실제 의미론적 동작은 다음에 가깝다.



LEGB는 지역 이름이 바인딩되지 않았을 때 무조건 다음 스코프로 넘어간다는 규칙이 아니다.

이 차이를 정확하게 이해해야 한다.

A.3.6 조건문 안의 대입도 지역 이름을 결정한다

이제 조금 더 흥미로운 예제를 살펴보자.

```
x = 10

def func():
    if False:
        x = 20

    print(x)

func()
```

예상 결과:

```
UnboundLocalError
```

실제로 `if False` 내부의 대입문은 실행되지 않는다.

그런데도 `x`는 지역 이름으로 판정된다.

왜 그럴까?

함수 블록의 지역 이름 판정은 일반적으로 해당 대입문이 실제 실행되는지 여부와 독립적이기 때문이다.

컴파일러는 함수 블록에 있는 바인딩 연산을 분석한다.

따라서 다음 코드도 마찬가지다.

```
x = 10

def func(flag):
    if flag:
        x = 20

    return x
```

```
print(func(True))
print(func(False))
```

첫 번째 호출에서는 20이 출력된다.

두 번째 호출에서는 UnboundLocalError가 발생한다.

여기서 중요한 차이는 다음과 같다.

구분	func(True)	func(False)
x의 스코프	Local	Local
x = 20 실행 여부	실행	실행되지 않음
지역 x 바인딩	존재	존재하지 않음
결과	20	UnboundLocalError

같은 함수의 이름 분류는 호출할 때마다 달라지지 않지만, 실제 바인딩 상태는 실행 경로에 따라 달라질 수 있다.

이것이 정적 스코프 판정과 동적 실행 상태의 차이다.

A.3.7 대입 연산만 지역 이름을 만드는 것은 아니다

지금까지는 x = 20을 중심으로 설명하였다.

하지만 지역 이름을 만드는 바인딩 연산은 대입문만 있는 것이 아니다.

예를 들어 함수 내부의 다음 구문도 이름을 바인딩할 수 있다.

```
def func(x):
    ...
```

매개변수 x는 함수의 지역 이름이다.

```
def func():
    import math
```

여기서 math도 지역 이름이다.

```
def func():
    for item in [1,2,3]:
        pass
```

item 역시 지역 이름으로 분류된다.

또한 함수 내부의 def, class, with ... as,

예외 처리의 except ... as 등도 바인딩에 관여할 수 있다.

다음 예제를 살펴보자.

```
value = 100

def func():
    print(value)

    for value in []:
        pass

func()
```

예상 결과:

```
UnboundLocalError
```

반복 대상이 빈 리스트이므로 for문의 본문은 실행되지 않는다.

하지만 for value in ...의 value는 함수 블록에서 바인딩 대상이 되는 이름이다.

따라서 value는 지역 이름으로 분류된다.

이름 분류는 단순히 = 문자의 존재만으로 결정되는 것이 아니라, Python 문법에서 어떤 구문이 이름을 바인딩하는지에 따라 결정된다는 점을 기억해야 한다.

A.3.8 global의 정확한 역할

이제 global을 살펴보자.

다음 코드를 실행해 보자.

```
x = 10

def func():
    global x
```

```
    print(x)
    x = 20

func()

print(x)
```

예상 출력:

```
10
20
```

여기서 `global x`는 무엇을 하는 걸까?

많이 하는 오해는 다음과 같다.

> `global`은 지역 변수를 전역 변수로 이동시킨다.

하지만 정확하지 않다.

`global x`는 현재 코드 블록에서 `x`를 지역 이름으로 취급하지 않고, 모듈의 전역 이름으로 취급하도록 지정하는 선언문이다.

즉, `global`은 객체를 이동시키는 연산이 아니다.

이름의 스코프 판정에 영향을 주는 선언이다.

실행 과정 분석

먼저 모듈에서 다음 문장이 실행된다.

```
x = 10
```

모듈 네임스페이스의 `x`가 정수 객체 10을 참조한다.

이후 `func`가 정의된다.

컴파일 과정에서 다음 선언을 분석한다.

```
global x
```

따라서 `func` 내부의 `x`는 전역 이름으로 분류된다.

이제 `func()`가 호출된다.

```
print(x)
```

전역 이름 x를 조회하므로 10이 출력된다.

다음 문장이 실행된다.

```
x = 20
```

이 대입은 지역 이름을 새로 바인딩하는 것이 아니다.

모듈 전역 네임스페이스의 x를 정수 객체 20에 재바인딩한다.

따라서 함수 호출이 끝난 뒤에도

```
print(x)
```

는 20을 출력하는 것이다.

바인딩 구조

호출 전:

```
Global Namespace
x → int(10)
```

호출 후:

```
Global Namespace
x → int(20)
```

기존 정수 객체 10이 20으로 변경된 것이 아니다.

전역 이름 x가 새로운 정수 객체를 참조하도록 재바인딩된 것이다.

이 차이는 A.1에서 학습한 이름 바인딩과 동일한 원리다.

A.3.9 global은 반드시 전역 변수가 이미 존재해야 사용할 수 있을까?

그렇지는 않다.

다음 코드를 보자.

```
def create_global():
    global message
    message = "Hello"

create_global()

print(message)
```

예상 출력:

```
Hello
```

`global message` 선언 자체가 전역 이름에 객체를 바인딩하는 것은 아니다.

실제로 다음 대입문이 실행될 때

```
message = "Hello"
```

모듈의 전역 네임스페이스에 `message`가 바인딩되는 것이다.

따라서 다음 코드는 다르게 동작한다.

```
def func():
    global message
    print(message)

func()
```

현재 모듈에 `message`가 없고 내장 이름으로도 존재하지 않는 일반적인 상황이라면 `NameError`가 발생한다.

즉, `global` 선언이 있다고 해서 그 이름에 자동으로 `None` 같은 객체가 바인딩되는 것은 아니다.

A.3.10 `global`의 범위는 어디까지일까?

다음 코드를 살펴보자.

```
x = 10

def outer():
```

```
    x = 20

    def inner():
        global x
        x = 30

    inner()

    print("outer x:", x)

outer()

print("global x:", x)
```

예상 출력:

```
outer x: 20
global x: 30
```

이 결과가 중요한 이유는 global이 가장 가까운 외부 함수의 이름을 의미하지 않기 때문이다.

inner에는 렉시컬 외부 함수인 outer가 존재한다.

그런데 global x는 outer의 x를 대상으로 하지 않는다.

모듈 전역의 x를 대상으로 한다.

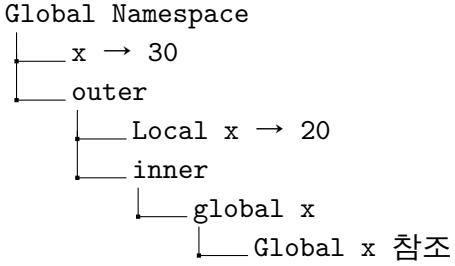
따라서 inner()에서 다음 대입문이 실행되면

```
x = 30
```

전역 x가 재바인딩 된다.

반면 outer의 지역 x는 여전히 20을 참조하고 있다.

구조를 나타내면 다음과 같다.



여기서 다음 원칙을 확립할 수 있다.

global은 가장 가까운 외부 스코프를 선택하는 선언이 아니라, 모듈 전역 스코프를 대상으로 하는 선언이다.

A.3.11 global과 nonlocal의 차이

global과 함께 반드시 비교해야 하는 선언이 nonlocal이다.

nonlocal의 내부 구조와 클로저 상태 변경은 A.5에서 자세히 다룰 예정이므로, 이번에는 스코프 선택 관점에서만 살펴본다.

```
x = 10

def outer():
    x = 20

    def inner():
        nonlocal x
        x = 30

    inner()

    print("outer x:", x)

outer()

print("global x:", x)
```

예상 출력:

```
outer x: 30
global x: 10
```

이번에는 inner에서 nonlocal x를 선언하였다.

이 선언은 현재 함수의 지역 스코프가 아닌, 렉시컬 외부 함수 스코프에 존재하는 x 바인딩을 사용하도록 지정한다.

따라서 inner() 내부의

```
x = 30
```

은 outer의 x를 재바인딩한다.

전역 x에는 영향을 주지 않는다.

global과 nonlocal 비교

구분	global	nonlocal
대상	모듈 전역 스코프	렉시컬 외부 함수 스코프
지역 이름 판정	현재 블록에서 지역 이름으로 취급하지 않음	현재 블록에서 지역 이름으로 취급하지 않음
기존 바인딩 필요 여부	없어도 선언 가능	외부 함수 스코프에 기존 바인딩이 필요
재바인딩 대상	전역 이름	외부 함수의 이름
주된 활용	모듈 수준 상태 변경	클로저 상태 변경

여기서 nonlocal의 중요한 제약이 있다.

```
def outer():

    def inner():
        nonlocal x
        x = 10

    inner()
```

이 코드는 SyntaxError를 발생시킨다.

왜냐하면 inner를 감싸는 외부 함수 스코프에 x라는 바인딩이 존재하지 않기 때문이다.

반면 global x는 기존 전역 바인딩이 없어도 사용할 수 있었다.

이 차이를 기억해야 한다.

A.3.12 global은 어디에 선언해야 할까?

다음 코드를 보자.

```
x = 10

def func():
    print(x)
    global x
```

이 코드는 일반적인 Python 컴파일 과정에서 SyntaxError를 발생시킨다.

같은 코드 블록에서 global x 선언보다 앞서 x를 사용했기 때문이다.

따라서 보통 다음처럼 작성해야 한다.

```
x = 10

def func():
    global x

    print(x)
```

여기서 global은 함수가 실행되는 동안 특정 시점부터 효력이 발생하는 일반적인 명령문이 아니다.

해당 코드 블록의 이름 해석 방식에 영향을 주는 컴파일러 지시문이다.

따라서 선언이 함수 본문 전체의 이름 분류와 연결된다는 점을 이해해야 한다.

또한 global 선언은 자신이 포함된 코드 블록에 적용된다.

외부 함수에 global x를 선언했다고 해서 그 내부에 정의된 별도 함수 블록까지 자동으로 같은 선언이 적용되는 것은 아니다.

A.3.13 이름의 스코프를 직접 확인하기 - symtable

지금까지 이름의 정적 판정을 설명하였다.

그런데 실제로 Python이 이름을 어떻게 분류하는지 확인할 방법은 없을까?

표준 라이브러리의 symtable 모듈을 활용할 수 있다.

symtable은 Python 소스 코드의 심볼 테이블(Symbol Table) 정보를 분석하는 기능을 제공한다.

다음 코드를 보자.

```
import symtable

source = """
x = 10
```

```
def func():
    print(x)
    x = 20
"""

table = symtable.symtable(source, "<string>", "exec")

func_table = next(
    child
    for child in table.get_children()
    if child.get_name() == "func"
)

x_symbol = func_table.lookup("x")

print(x_symbol.is_local())
print(x_symbol.is_global())
print(x_symbol.is_referenced())
print(x_symbol.is_assigned())
```

예상 출력:

```
True
False
True
True
```

여기서 확인할 수 있는 것은 다음과 같다.

x는 func 내부에서 지역 이름으로 분류된다.

또한 읽기 대상으로 참조되고, 대입 대상으로 사용되고 있다.

중요한 점은 이 코드를 실행할 때 분석 대상 문자열의 func()를 호출하지 않았다는 사실이다.

즉, 실제 함수 본문을 실행하지 않고도 이름의 정적 분류를 확인할 수 있다.

이번에는 global을 추가해 보자.

```
import symtable

source = """
x = 10

def func():
    global x
    print(x)
```

```
    x = 20
"""

table = symtable.symtable(source, "<string>", "exec")

func_table = next(
    child
    for child in table.get_children()
    if child.get_name() == "func"
)

x_symbol = func_table.lookup("x")

print(x_symbol.is_local())
print(x_symbol.is_global())
print(x_symbol.is_declared_global())
```

예상 출력:

```
False
True
True
```

이번에는 x가 지역 이름이 아니라 명시적으로 선언된 전역 이름으로 분류된다는 사실을 확인할 수 있다.

이것은 global이 실행 시점에 객체를 이동시키는 기능이 아니라 이름 분류를 변경하는 선언이라는 설명과 일치한다.

A.3.14 바이트 코드로 확인하기 - dis

한 단계 더 들어가 보자.

Python의 표준 라이브러리 dis를 사용하면 CPython이 함수의 이름 접근을 어떤 바이트코드 명령으로 구현하는지 관찰할 수 있다.

```
import dis

x = 10

def local_func():
    x = 20
    return x
```

```
def global_func():
    return x


def declared_global_func():
    global x
    x = 30
    return x


dis.dis(local_func)
dis.dis(global_func)
dis.dis(declared_global_func)
```

Python 버전에 따라 정확한 바이트코드 구성과 출력 형식은 달라질 수 있다.

다만 일반적인 CPython에서는 다음과 같은 명령의 차이를 관찰할 수 있다.

함수	주요 명령	의미
local_func	STORE_FAST, LOAD_FAST	지역 이름 바인딩 및 조회
global_func	LOAD_GLOBAL	전역·내장 이름 조회
declared_global_func	STORE_GLOBAL, LOAD_GLOBAL	전역 이름 바인딩 및 조회

이 차이는 중요한 사실을 보여준다.

Python이 함수 호출 때마다 모든 이름에 대해 동일한 방식으로 LEGB 검색을 수행하는 것은 아니다.

컴파일 단계에서 이름의 종류를 분석하고, 그 결과에 따라서 서로 다른 접근 방식을 사용할 수 있다.

단, 바이트코드는 CPython 버전의 바이트코드 구현을 구분해야 한다.

A.3.15 NameError와 UnboundLocalError의 차이

이번에는 두 예외를 비교해 보자.

사례 1. 이름을 찾을 수 없는 경우

```
def func():
    return unknown_name
```

```
func()
```

예상 결과:

```
NameError
```

unknown_name은 지역 이름으로 바인딩되지 않았고, 외부 함수 스코프나 전역·내장 스코프에서도 찾을 수 없는 일반적인 상황이다.

따라서 NameError가 발생한다.

사례 2. 지역 이름이지만 아직 바인딩되지 않은 경우

```
def func():
    print(x)
    x = 10

func()
```

예상 결과:

```
UnboundLocalError
```

여기서는 x가 지역 이름으로 판정되었지만 읽는 시점에 바인딩되어 있지 않다.

따라서 UnboundLocalError가 발생한다.

사례 3. 외부 함수의 자유 변수가 아직 바인딩되지 않은 경우

```
def outer():

    def inner():
        return x

    result = inner()

    x = 10

    return result


outer()
```

예상 결과:

```
NameError
```

이 경우는 앞의 두 사례와 구분해야 한다.

`inner` 내부의 `x`는 지역 이름이 아니다.

`outer`의 렉시컬 스코프에 존재하는 자유 변수다.

하지만 `inner()`를 호출하는 시점에는 `outer`의 `x = 10`이 아직 실행되지 않았다.

따라서 외부 스코프의 자유 변수 바인딩을 읽을 수 없어 `NameError`가 발생한다.

CPython에서는 일반적으로 다음과 유사한 메시지를 볼 수 있다.

```
NameError: cannot access free variable 'x'
where it is not associated with a value in enclosing scope
```

여기서 중요한 점은 외부 함수의 자유 변수가 바인딩되지 않은 경우와 현재 함수의 지역 변수가 바인딩되지 않은 경우의 예외 종류가 다를 수 있다는 것이다.

비교 정리

상황	이름 분류	대표적인 예외
현재 함수의 지역 이름이 아직 바인딩되지 않음	`Local`	`UnboundLocalError`
외부 함수의 자유 변수 바인딩이 아직 존재하지 않음	`Free Variable`	`NameError`

`UnboundLocalError`는 `NameError`의 하위 클래스다.

```
print(issubclass(UnboundLocalError, NameError))
```

출력:

```
True
```

따라서 `except NameError`는 `UnboundLocalError`도 포착할 수 있다.

하지만 두 예외가 발생하는 원인은 구분해야 한다.

A.3.16 이름 탐색과 속성 탐색은 다르다

기존 클래스 객체 모델과 연결해 보자.

```
class Example:
    x = 100

    def method(self):
        return x


obj = Example()
```

여기서 `method()` 내부의 `x`는 `Example.x`를 자동으로 참조하지 않는다.

일반적인 메서드의 이름 탐색에서 클래스 네임스페이스가 외부 함수 스코프처럼 포함되지 않기 때문이다.

반면 다음 코드는 정상적으로 동작한다.

```
class Example:
    x = 100

    def method(self):
        return self.x


obj = Example()

print(obj.method())
```

예상 출력:

```
100
```

왜 그럴까?

```
return self.x
```

는 두 단계로 평가된다.

먼저 self라는 이름을 현재 함수의 지역 스코프에서 조회한다.

일반적인 인스턴스 메서드 호출에서는 self가 인스턴스 객체를 참조한다.

그다음 그 인스턴스 객체에서 x 속성을 탐색한다.

이 속성 탐색은 LEGB가 아니라 객체의 속성 접근 규칙에 따라 처리된다.

일반적인 사용자 정의 인스턴스에서는 인스턴스 네임스페이스, 클래스와 상속 계층, 디스크립터 등이 관여할 수 있다.

따라서 다음 두 표현식은 전혀 다른 메커니즘이다.

표현식	메커니즘
x	이름 탐색(Name Resolution)
self.x	self 이름 탐색 후 속성 탐색 (Attribute Lookup)
Example.x	Example 이름 탐색 후 클래스 객체의 속성 탐색

이 구분은 PyTorch의 nn.Module을 이해할 때도 중요하다.

A.3.17 실전 응용 - PyTorch에서의 이름 탐색과 속성 탐색

다음 예제를 살펴보자.

```
import torch
from torch import nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(3, 2)

    def forward(self, x):
        return self.linear(x)


model = MyModel()

x = torch.randn(1, 3)
```

```
print(model(x).shape)
```

예상 출력:

```
torch.Size([1, 2])
```

이 코드는 기존에 학습한 nn.Module 객체 모델과 연결된다.

forward() 내부의 다음 표현식을 보자.

```
self.linear(x)
```

여기에는 여러 단계가 존재한다.

1. self 이름 탐색

self는 forward() 호출의 지역 이름이다.

따라서 Local 스코프에서 조회된다.

2. self.linear 속성 탐색

이제 self가 참조하는 MyModel 인스턴스에서 linear 속성을 탐색한다.

PyTorch의 nn.Module은 하위 모듈 등록과 조회를 위해 일반적인 Python 객체 모델을 확장하고 있다.

self.linear = nn.Linear(3,2)가 실행되면 nn.Module의 등록 메커니즘을 통해 하위 모듈이 관리된다.

이후 self.linear에 접근할 때는 nn.Module이 제공하는 속성 조회 동작이 관여할 수 있다.

3. self.linear(x) 호출

조회된 nn.Linear 인스턴스는 호출 가능한 객체다.

따라서 __call__을 통한 모듈 호출 과정이 진행되고, 최종적으로 선형 변환이 수행된다.

여기서 LEGB가 담당하는 것은 self라는 이름을 조회하는 부분이다.

linear를 찾는 과정은 객체의 속성 탐색이다.

이 둘을 구별해야 Python의 일반적인 스코프 규칙과 PyTorch가 확장한 객체 동작을 혼동하지 않게 된다.

A.3.18 경계 조건 - LEGB만으로 모든 이름 접근을 설명할 수 있을까?

LEGB는 일반적인 함수 이름 탐색을 이해하는 데 유용하지만, Python의 모든 이름 접근을 단순한 네 단계 검색으로 환원할 수는 없다.

대표적인 예외와 주의점을 정리해 보자.

1. 클래스 본문은 일반적인 함수 스코프와 다르다

```
x = 10

class Example:
    x = 20

    def method(self):
        return x
```

`method()`의 `x`는 일반적인 상황에서 클래스 본문의 `x = 20`을 렉시컬 외부 함수 변수처럼 사용하지 않는다.

모듈 전역의 `x = 10`을 조회한다.

클래스 본문에는 별도의 이름 해석 규칙이 존재하기 때문이다.

2. 컴프리헨션은 별도의 스코프 규칙을 갖는다

```
x = 100

values = [x for x in range(3)]

print(values)
print(x)
```

예상 출력:

```
[0, 1, 2]
100
```

Python 3에서 리스트 컴프리헨션의 반복 변수 `x`는 일반적으로 바깥 스코프의 `x`를 덮어쓰지 않는다.

다만 컴프리헨션의 종류와 Python 버전에 따라 내부 구현 및 세부 규칙이 달라질 수 있다.

3. exec()는 일반적인 정적 이름 바인딩과 구분해야 한다

```
def func():
    exec("x = 10")
```

exec()이 문자열을 실행한다고 해서 컴파일된 func의 지역 이름 분류가 그 문자열에 맞춰 다시 결정되는 것은 아니다.

특히 최적화된 함수 지역 스코프에서 exec()로 생성한 이름을 일반적인 지역 변수처럼 접근할 수 있다고 가정하면 안 된다.

4. del도 바인딩 상태에 영향을 준다

```
def func():
    x = 10
    del x
    return x


func()
```

예상 결과:

```
UnboundLocalError
```

x는 지역 이름으로 판정되어 있고, 처음에는 정수 객체 10에 바인딩된다.

하지만 del x가 실행되면서 그 지역 이름의 바인딩이 제거된다.

이후 return x에서 다시 조회하려고 하면 바인딩된 객체가 없으므로 예외가 발생한다.

여기서 del x는 정수 객체 10을 직접 파괴하는 명령이 아니라 이름의 바인딩을 제거하는 연산이라는 점도 기억해야 한다.

A.3.19 종합 예제 - 이름 분류와 실행 상태 추적

이번 학습의 핵심 개념을 하나의 코드에 통합해 보자.

```
x = 10
```

```
def outer():
    x = 20
    y = 30

    def inner():
        global x
        nonlocal y

        x = 100
        y = 200

        return x, y

    result = inner()

    print("outer x:", x)
    print("outer y:", y)

    return result


result = outer()

print("global x:", x)
print("result:", result)
```

예상 출력:

```
outer x: 20
outer y: 200
global x: 100
result: (100, 200)
```

이제 실행 과정을 순서대로 분석해 보자.

1. 모듈 전역 이름 바인딩

```
x = 10
```

전역 이름 x가 정수 객체 10을 참조한다.

2. outer() 호출

```
x = 20
y = 30
```

현재 outer() 호출의 지역 스코프에 x와 y가 바인딩된다.

3. inner 함수 객체 생성

inner 함수 객체가 생성된다.

이 함수의 본문에는 다음 선언이 있다.

```
global x
nonlocal y
```

따라서 inner의 x는 전역 이름으로, y는 외부 함수 스코프의 이름으로 분류된다.

4. inner() 호출

```
x = 100
```

전역 x가 정수 객체 100에 재바인딩된다.

다음으로

```
y = 200
```

외부 함수 outer의 y가 정수 객체 200에 재바인딩된다.

여기서 outer의 지역 x는 변경되지 않는다.

5. 반환

```
return x, y
```

현재 전역 x와 외부 함수의 y를 조회한다.

각각 100, 200을 얻고 튜플 객체를 구성해 반환한다.

6. outer()의 나머지 분문 실행

```
print("outer x:", x)
```

여기서 x는 outer의 지역 이름으로 20이 출력된다.

```
print("outer y:", y)
```

inner()에서 nonlocal y를 통해 외부 함수의 바인딩을 변경했으므로 200이 출력된다.

7. 모듈 코드 실행

마지막으로 전역 x는 100을 참조하고 있다.

그리고 result에는 (100, 200) 튜플 객체가 바인딩된다.

이 예제는 다음 네 가지를 동시에 보여준다.

- Local 이름과 Global 이름은 같은 철자를 사용해도
  서로 다른 바인딩일 수 있다.
- global은 모듈 전역 이름을 대상으로 한다.
- nonlocal은 렉시컬 외부 함수의 이름을 대상으로 한다.
- 함수의 실행 순서와 이름의 스코프 판정은 구별해야 한다.

A.3.20 핵심 정리와 판단 기준

이번 학습에서 반드시 확립해야 하는 판단 기준은 다음과 같다.

첫째, 이름을 읽는다고 해서 무조건 LEGB 전체를
순차 검색하는 것은 아니다.

Python은 컴파일 과정에서 함수 블록의 이름 사용을 분석하고, 이름 종류를 판정한다.

따라서 지역 이름으로 판정된 이름이 바인딩되지 않았다고 해서 자동으로 전역 이름을 사용하는 것은 아니다.

둘째, 이름의 분류와 객체의 바인딩 상태는 서로 다르다.

```
def func():
    print(x)
    x = 10
```

여기서 x는 지역 이름이지만, print(x) 실행 시점에는 아직 객체가 바인딩되어 있지 않다.

그래서 UnboundLocalError가 발생한다.

셋째, global은 객체를 이동시키지 않는다.

```
global x
```

는 해당 코드 블록에서 x를 전역 이름으로 취급하도록 지정하는 선언이다.

실제 객체 바인딩은 대입문 등이 실행될 때 발생한다.

넷째, global과 nonlocal은 대상 스코프가 다르다.

global은 모듈 전역 스코프를 대상으로 하고, nonlocal은 렉시컬 외부 함수 스코프의 기존 바인딩을 대상으로 한다.

다섯째, 이름 탐색과 속성 탐색을 구분해야 한다.

```
x
```

와

```
self.x
```

는 서로 다른 메커니즘으로 처리된다.

전자는 이름 탐색이고, 후자는 self 이름을 조회한 뒤 객체의 속성을 탐색하는 표현식이다.

A.4 클로저(Closure)의 생성과 자유 변수(Free Variable)

Closure · Free Variable · Cell Object · Lexical Environment · Variable Lifetime

A.4.1 이전 학습과의 연결

지금까지 함수 객체를 세 단계에 걸쳐 학습하였다.

단원	핵심 개념	확립한 원리
A.1	함수 객체와 이름 바인딩	def는 함수 객체를 생성하고 이름에 바인딩한다.
A.2	중첩 함수와 Lexical Scope	함수의 렉시컬 외부 스코프는 정의 위치에 따라 결정된다.
A.3	LEGB와 이름의 정적 판정	이름의 스코프 분류와 실행 시점의 객체 조회는 구별된다.
A.4	클로저와 자유 변수	함수 객체가 외부 바인딩에 접근하는 환경을 어떻게 유지하는지 분석한다.

A.2와 A.3에서는 다음 코드를 여러 차례 살펴보았다.

```
def outer():
    x = 10

    def inner():
        return x

    return inner

f = outer()

print(f())
```

예상 출력:

```
10
```

이 결과를 지금까지 배운 개념으로 설명하면 다음과 같다.

1. outer() 호출 중 x가 정수 객체 10에 바인딩된다.
2. def inner가 실행되면서 새로운 함수 객체가 생성된다.
3. inner 내부의 x는 outer의 지역 스코프를 참조하는 자유 변수로 판정된다.
4. return inner를 통해 함수 객체가 반환된다.
5. 이후 f()를 호출하면 inner가 실행되고 10을 반환한다.

하지만 아직 설명하지 않은 부분이 있다.

outer()의 실행이 종료되었는데, inner()는 어떻게 outer의 지역 변수 x에 접근할 수 있을까?

여기서 두 가지 가능성을 생각해 볼 수 있다.

```
- inner 함수 객체가 생성될 때
  x의 값인 10을 복사해서 보관하는가?

- 아니면 x라는 이름의 바인딩에 계속 접근할 수 있도록
  별도의 구조를 유지하는 것인가?
```

Python의 클로저는 두 번째 방식에 해당한다.

이번 학습에서는 이 구조를 함수 객체, 코드 객체, 자유 변수, 셀 객체의 관계로 분석할 것이다.

A.4.2 이번 단원의 핵심 질문

이번 단원을 마치면 다음 질문에 답할 수 있어야 한다.

1. 클로저(Closure)란 정확히 무엇인가?
2. 자유 변수(Free Variable)는 일반적인 지역 변수와 어떻게 다른가?
3. inner 함수 객체는 외부 변수의 값을 복사하는가,
   바인딩을 참조하는가?
4. __closure__와 셀 객체(Cell Object)는 어떤 관계인가?
5. co_freevars와 co_cellvars는 무엇을 나타내는가?
6. 외부 함수의 실행이 종료되어도 클로저가 유지되는 이유는 무엇인가?
7. 서로 다른 클로저는 상태를 공유하는가?
8. 클로저와 전역 변수 참조는 어떻게 다른가?

여기서 가장 중요한 구분은 다음이다.

클로저는 외부 함수의 실행 프레임 전체를 보존하는 구조가 아니라,

함수가 필요로 하는 렉시컬 외부 바인딩에 접근할 수 있도록 하는 구조다.

이 문장을 이해하는 것이 이번 학습의 목표다.

A.4.3 클로저의 정확한 정의

1. 클로저란 무엇인가?

클로저(Closure)는 일반적으로 다음 두 요소가 결합된 구조를 의미한다.

1. 실행할 함수의 코드

2. 그 코드가 참조하는 렉시컬 외부 변수의 바인딩에 접근할 수 있는 환경

이를 개념적으로 나타내면 다음과 같다.

$$\text{Closure} = \text{Function Code} + \text{Captured Lexical Environment}$$

다만 이 수식은 개념적인 표현이지 Python 내부 객체의 실제 덧셈이나 결합 연산을 의미하지는 않는다.

Python에서는 이러한 구조가 함수 객체와 클로저 셀을 통해 구현될 수 있다.

다음 코드를 보자.

```
def outer():
    x = 10

    def inner():
        return x

    return inner


f = outer()
```

f는 일반적인 Python 함수 객체를 참조한다.

```
print(type(f))
```

출력:

```
<class 'function'>
```

즉, 클로저를 형성했다고 해서 별도의 Closure 타입 인스턴스가 생성되는 것은 아니다.

inner는 여전히 함수 객체다.

다만 이 함수 객체는 외부 변수 x에 접근하기 위한 클로저 환경을 가지고 있다.

그 환경은 다음 속성으로 확인할 수 있다.

```
print(f.__closure__)
```

일반적인 CPython에서는 다음과 유사한 결과가 출력된다.

```
(<cell at 0x...: int object at 0x...>,)
```

메모리 주소는 실행 환경에 따라 달라진다.

여기서 중요한 것은 출력에 cell이라는 객체가 등장한다는 사실이다.

이 셀 객체가 이번 학습의 핵심이다.

A.4.4 자유 변수(Free Variable)란 무엇인가?

클로저를 이해하려면 먼저 자유 변수의 의미를 정확하게 알아야 한다.

다음 코드를 살펴보자.

```
def outer():
    x = 10
    y = 20

    def inner():
        z = 30
        return x + z

    return inner
```

여기에는 세 개의 이름이 등장한다.

- x
- y
- z

그런데 이 세 이름은 inner를 기준으로 서로 다른 성격을 가진다.

1. z는 지역 변수(Local Variable)

```
def inner():
    z = 30
    return x + z
```

z는 inner 내부에서 바인딩된다.

따라서 inner의 지역 이름이다.

2. x는 자유 변수(Free Variable)

inner 내부에서는 x를 읽지만, x를 지역 이름으로 바인딩하지 않는다.

대신 렉시컬 외부 함수 outer에서 정의한 x를 사용한다.

따라서 inner를 기준으로 x는 자유 변수다.

3. y는 외부 함수의 지역 변수지만 inner의 자유 변수는 아니다

y는 outer의 지역 변수다.

그러나 inner에서 사용하지 않는다.

따라서 inner가 접근해야 하는 외부 바인딩 목록에 포함될 필요가 없다.

이를 정리하면 다음과 같다.

이름	outer 기준	inner 기준
x	지역 변수	자유 변수
y	지역 변수	사용하지 않는 이름
z	정의되지 않은 이름	지역 변수

여기서 중요한 사실이 있다.

하나의 이름은 어떤 함수 블록을 기준으로 분석하는지에 따라 서로 다른 분류를 가질 수 있다.

x는 outer에서는 지역 이름이지만, inner에서는 자유 변수다.

이것은 모순이 아니다.

스코프 분석의 기준이 서로 다르기 때문이다.

A.4.5 co_freevars와 co_cellvars

이제 Python이 이러한 이름 관계를 어떻게 기록하는지 확인해 보자.

함수 객체에는 __code__ 속성이 있다.

이 속성은 해당 함수의 코드 객체(Code Object)를 참조한다.

코드 객체에는 자유 변수와 셀 변수에 관한 정보가 포함되어 있다.

대표적인 속성이 다음 두 가지다.

- co_freevars
- co_cellvars

먼저 코드를 실행해 보자.

```
def outer():
    x = 10
    y = 20

    def inner():
        z = 30
        return x + z

    return inner


f = outer()

print(outer.__code__.co_cellvars)
print(f.__code__.co_freevars)
print(f.__code__.co_cellvars)
```

예상 출력:

```
('x',)
('x',)
()
```

왜 이런 결과가 발생할까?

```
1. outer.__code__.co_cellvars
```

```
outer.__code__.co_cellvars
```

결과는 다음과 같다.

```
('x',)
```

outer의 지역 변수 x는 중첩 함수 inner에서 참조된다.

따라서 outer에서는 이 바인딩을 셀을 통해 관리해야 한다.

이러한 지역 이름을 셀 변수(Cell Variable)라고 한다.

반면 y는 중첩 함수에서 사용되지 않으므로 co_cellvars에 포함되지 않는다.

```
2. f.__code__.co_freevars
```

```
f.__code__.co_freevars
```

결과는 다음과 같다.

```
('x',)
```

f는 inner 함수 객체를 참조하고 있다.

inner의 코드에서는 외부 함수의 x를 사용하고 있으므로 x가 자유 변수로 기록된다.

```
3. f.__code__.co_cellvars
```

```
f.__code__.co_cellvars
```

결과는 빈 튜플이다.

```
()
```

inner 내부의 지역 변수 z는 더 깊이 중첩된 함수에서 참조되지 않는다.

따라서 inner에는 셀 변수로 관리할 지역 이름이 없는 것이다.

정리

속성	기준	의미
co_freevars	현재 코드 객체	외부 함수 스코프에서 가져와야 하는 자유 변수 이름
co_cellvars	현재 코드 객체	중첩 함수에서 접근할 수 있도록 셀로 관리하는 현재 함수의 지역 변수 이름

여기서 주의해야 할 점이 있다.

co_freevars와 co_cellvars는 이름에 관한 정보를 제공한다.

실제 외부 객체를 직접 저장하는 속성은 아니다.

실제 바인딩에 접근하기 위한 구조는 다음에 살펴볼 __closure__와 연결된다.

A.4.6 셀 객체(Cell Object)의 역할

이제 클로저의 실제 구조를 살펴보자.

다음 코드를 실행해 보자.

```
def outer():
    x = 10

    def inner():
        return x

    return inner


f = outer()

print(f.__code__.co_freevars)
print(len(f.__closure__))
print(f.__closure__[0].cell_contents)
```

예상 출력:

```
('x',)
1
10
```

하나씩 분석해 보자.

1. co_freevars

```
f.__code__.co_freevars
```

결과:

```
('x',)
```

inner가 외부 함수의 x를 사용한다는 사실을 나타낸다.

2. __closure__

```
f.__closure__
```

__closure__는 함수 객체가 참조하는 클로저 셀들의 튜플이다.

클로저 셀이 없다면 None이 될 수 있다.

여기서는 자유 변수 x에 대응하는 셀 객체 하나가 존재하므로 튜플의 길이가 1이다.

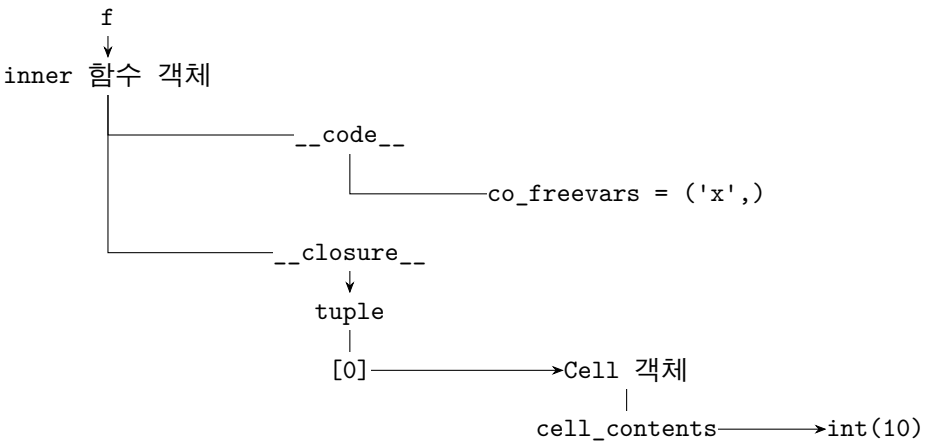
3. cell_contents

```
f.__closure__[0].cell_contents
```

cell_contents는 셀에 현재 바인딩된 객체를 가져오는 속성이다.

따라서 결과는 10이 된다.

여기서 중요한 관계를 정리해 보자.



이 구조에서 중요한 것은 x라는 이름과 10이라는 객체가 서로 다른 역할을 한다는 사실이다.

- co_freevars는 어떤 자유 변수 이름이 필요한지 나타낸다.
- __closure__는 그 자유 변수에 대응하는 셀 객체를 제공한다.
- 셀 객체는 현재 바인딩된 객체를 참조한다.

따라서 클로저가 단순히 정수 값 10을 함수 객체에 복사해 넣었다고 이해하면 정확하지 않다.

함수 객체는 셀을 통해 외부 바인딩에 접근하는 구조를 유지하는 것이다.

A.4.7 자유 변수가 여러 개라면?

이번에는 외부 변수 두 개를 사용하는 함수를 만들어 보자.

```
def outer():
    x = 10
    y = 20

    def inner():
        return x + y

    return inner


f = outer()
```

```
print(f.__code__.co_freevars)

for name, cell in zip(
    f.__code__.co_freevars,
    f.__closure__
):
    print(name, cell.cell_contents)
```

예상 출력:

```
('x', 'y')
x 10
y 20
```

이 코드에서 co_freevars와 __closure__는 서로 대응하는 순서를 가지고 있다.

따라서 다음과 같이 해석할 수 있다.

```
co_freevars          __closure__
('x','y')            (cell_x, cell_y)

     (                      (

    'x'———————————————cell_x

     ,                      ,

    'y'———————————————cell_y

     )                      )
```

정확히는 동일한 인덱스끼리 대응한다.

```
co_freevars[0] = 'x'  ↔  __closure__[0] = cell_x

co_freevars[1] = 'y'  ↔  __closure__[1] = cell_y
```

그리고 각 셀에는 현재 객체가 바인딩되어 있다.

```
cell_x → int(10)

cell_y → int(20)
```

이 대응 관계는 클로저를 디버깅하거나 함수 객체의 내부 구조를 확인할 때 유용하다.

다만 이 이름의 순서를 소스 코드에서 등장한 순서라고 일반화하면 안 된다.

중요한 것은 co_freevars의 순서와 __closure__ 셀의 순서가 대응한다는 사실이다.

A.4.8 클로저는 값을 복사하는가, 바인딩을 유지하는가?

이번 학습에서 가장 중요한 실험이다.

다음 코드를 살펴보자.

```
def outer():
    x = 10

    def inner():
        return x

    x = 20

    return inner


f = outer()

print(f())
print(f.__closure__[0].cell_contents)
```

예상 출력:

```
20
20
```

왜 10이 아니라 20일까?

이 코드를 실행 순서대로 분석해 보자.

```
1. outer() 호출
```

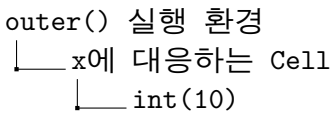
```
x = 10
```

outer의 지역 이름 x가 정수 객체 10에 바인딩된다.

그런데 x는 중첩 함수 inner에서 참조되는 이름이다.

따라서 이 바인딩은 클로저 셀을 통해 접근할 수 있도록 관리된다.

개념적으로 다음과 같다.

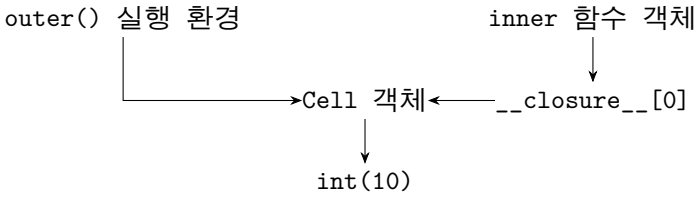


2. inner 함수 객체 생성

```
def inner():
    return x
```

inner 함수 객체가 생성된다.

이 함수 객체는 x에 대응하는 셀 객체를 참조한다.



3. x = 20 실행

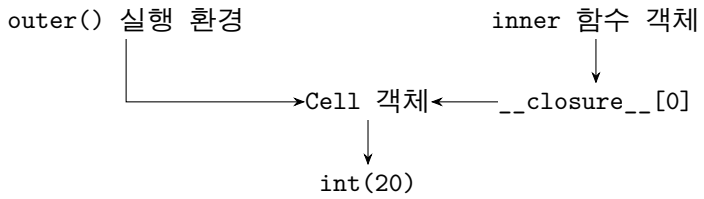
이제 다음 문장이 실행된다.

```
x = 20
```

정수 객체 10이 20으로 변경되는 것은 아니다.

x에 대응하는 바인딩이 새로운 정수 객체 20을 참조하도록 변경되는 것이다.

클로저 셀 관점에서는 다음과 같이 이해할 수 있다.



여기서 inner 함수 객체를 다시 생성한 것은 아니다.

기존 함수 객체가 참조하는 셀의 바인딩 대상이 변경된 것이다.

4. inner 함수 객체 반환

```
return inner
```

inner 함수 객체가 반환되고 f에 바인딩된다.

이후 outer() 호출이 종료된다.

5. f() 호출

f()를 호출하면 inner 본문이 실행된다.

```
return x
```

x는 자유 변수이므로 대응하는 클로저 셀을 통해 현재 객체를 조회한다.

현재 셀은 정수 객체 20을 참조하고 있다.

따라서 20이 반환된다.

핵심 결론

클로저는 자유 변수의 값을 함수 정의 시점에 고정하는 구조가 아니다.

클로저는 자유 변수의 바인딩에 접근할 수 있도록 하는 구조다.

이 차이는 이후 데코레이터에서 상태를 유지하거나 변경하는 원리를 이해하는 데 중요하다.

A.4.9 외부 함수가 종료되어도 클로저가 유지되는 이유

이제 가장 근본적인 질문으로 돌아가 보자.

```
def outer():
    x = 10

    def inner():
        return x

    return inner


f = outer()
```

outer()의 호출은 종료되었다.

그런데도 f()는 10을 반환한다.

여기서 흔히 다음과 같이 이해하기 쉽다.

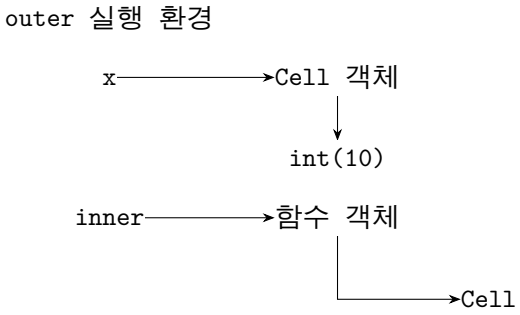
외부 함수가 종료되어도 지역 변수 전체가 메모리에 남아 있기 때문이다.

하지만 이 설명은 지나치게 단순화되어 있다.

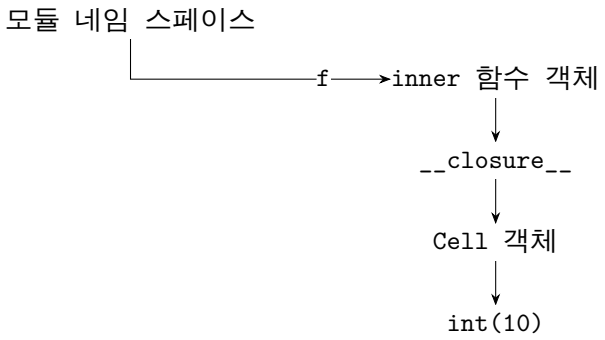
정확하게는 반환된 함수 객체가 필요한 클로저 셀을 참조하고 있기 때문에 해당 바인딩에 접근할 수 있는 것이다.

함수 호출의 실행 프레임 전체가 반드시 유지되어야 하는 것은 아니다.

outer() 실행 중의 구조



outer() 종료 후의 구조



여기서 `outer()` 호출의 일반적인 실행은 끝났다.

하지만 `f`가 `inner` 함수 객체를 참조하고 있고, 그 함수 객체가 셀 객체를 참조하고 있다.

따라서 셀과 그 셀이 참조하는 객체는 계속 접근 가능한 상태로 유지될 수 있다.

이 구조를 이해할 때 다음 세 가지를 구별해야 한다.

대상	**outer() 종료 후**
`outer()` 호출의 실행	종료됨
반환된 `inner` 함수 객체	`f`가 참조하므로 유지됨
`inner`가 참조하는 클로저 셀	함수 객체를 통해 유지됨

CPython에서는 일반적인 참조 관리와 가비지 컬렉션 메커니즘이 객체의 수명에 관여한다.

다만 특정 객체가 언제 메모리에서 해제되는지는 Python 구현과 참조 관계에 따라 달라질 수 있으므로, 이를 단순히 함수 종료 시점과 동일시하면 안 된다.

여기서 확립해야 하는 원리는 다음이다.

함수의 실행 수명(Lifetime)과 함수가 생성한 객체의 수명은 서로 다를 수 있다.

이것은 `A.1`에서 학습한 함수 객체와 이름 바인딩의 원리가 클로저로 확장된 결과다.

A.4.10 외부 함수의 모든 지역 변수가 유지되는 것은 아니다

다음 코드를 살펴보자.

```
def outer():
    x = 10
    y = 20
    z = 30

    def inner():
        return x

    return inner


f = outer()

print(outer.__code__.co_cellvars)
print(f.__code__.co_freevars)
print(len(f.__closure__))
```

예상 출력:

```
('x',)
('x',)
1
```

`outer()`에는 세 개의 지역 이름이 존재한다.

하지만 `inner`에서 참조하는 것은 `x`뿐이다.

따라서 이 예제에서 클로저 셀을 통해 유지해야 하는 외부 바인딩은 `x`다.

`y`와 `z`는 `inner`의 자유 변수가 아니다.

이것은 클로저가 외부 함수의 지역 네임스페이스 전체를 그대로 복사하는 구조가 아니라는 사실을 보여준다.

물론 클로저 셀이 참조하는 객체가 다른 객체들을 다시 참조하고 있다면, 그 참조 관계를 통해 추가적인 객체들이 유지될 수는 있다.

예를 들어 셀이 리스트 객체를 참조하고, 그 리스트가 여러 객체를 포함하고 있다면 해당 객체들도 참조 관계의 영향을 받는다.

하지만 그것은 클로저가 모든 지역 변수를 자동으로 보관하기 때문이 아니다.

클로저가 유지하는 바인딩과, 그 바인딩을 통해 도달할 수 있는 객체들의 참조 관계를 구별해야 한다.

A.4.11 서로 다른 클로저는 같은 셀을 공유할까?

이제 클로저를 여러 번 생성해 보자.

```
def outer(value):

    def inner():
        return value

    return inner


f1 = outer(10)
f2 = outer(20)

print(f1())
print(f2())

print(f1 is f2)

print(
    f1.__closure__[0]
    is f2.__closure__[0]
)
```

예상 출력:

```
10
20
False
False
```

왜 이런 결과가 발생할까?

`outer(10)`과 `outer(20)`은 서로 다른 함수 호출이다.

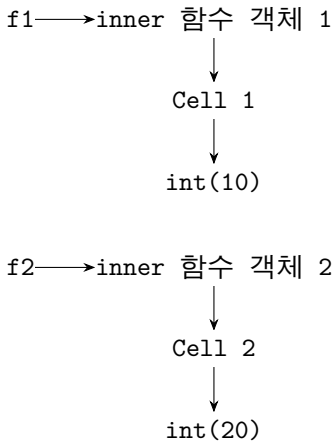
각 호출에는 독립적인 지역 바인딩 환경이 형성된다.

첫 번째 호출에서는 `value`가 10을 참조하고, 두 번째 호출에서는 `value`가 20을 참조한다.

각 호출에서 `inner` 함수 객체도 새로 생성된다.

그리고 각 함수 객체는 자신이 생성된 호출의 value 바인딩에 대응하는 셀을 참조한다.

구조를 나타내면 다음과 같다.



따라서 다음 비교는 False다.

```
f1.__closure__[0] is f2.__closure__[0]
```

서로 다른 셀 객체이기 때문이다.

여기서 중요한 점은 같은 함수 정의문에서 생성된 함수 객체라고 해서 클로저 환경까지 공유하는 것은 아니라는 사실이다.

각 외부 함수 호출에서 생성된 클로저는 독립적인 바인딩 환경을 가질 수 있다.

이것이 함수 생성기(Function Factory)가 서로 다른 설정을 유지할 수 있는 이유다.

A.4.12 하나의 외부 함수에서 생성된 여러 클로저는?

앞에서는 서로 다른 outer() 호출에서 생성된 클로저를 비교했다.

이번에는 하나의 outer() 호출에서 여러 내부 함수를 생성해 보자.

```
def outer():
    x = 10

    def first():
```

```
        return x

    def second():
        return x * 2

    return first, second


f1, f2 = outer()

print(f1())
print(f2())

print(
    f1.__closure__[0]
    is f2.__closure__[0]
)
```

예상 출력:

```
10
20
True
```

이번에는 셀 객체의 동일성 비교가 True다.

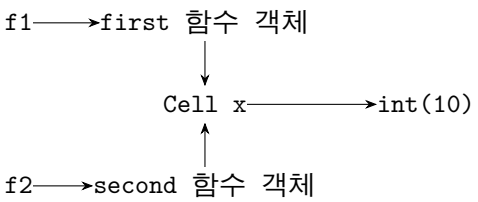
왜 그럴까?

first와 second는 같은 outer() 호출에서 생성되었다.

그리고 두 함수 모두 동일한 외부 지역 변수 x를 참조한다.

따라서 두 함수 객체는 같은 셀 객체를 참조할 수 있다.

구조를 나타내면 다음과 같다.



여기서 다음 관계가 성립한다.

```
f1.__closure__[0] is f2.__closure__[0]
```

결과는 True다.

이것은 매우 중요한 차이다.

생성 방식	셀 공유 여부
서로 다른 outer() 호출에서 생성	일반적으로 독립적인 셀
같은 outer() 호출에서 동일한 외부 바인딩을 참조	동일한 셀 공유

이 차이는 이후 nonlocal을 사용하여 여러 내부 함수가 하나의 상태를 읽고 변경하는 구조를 만들 때 핵심이 된다.

A.4.13 클로저와 가변 객체(Mutable Object)

지금까지는 정수 객체를 사용했다.

이번에는 리스트 객체를 사용해 본다.

```
def outer():
    values = []

    def add(value):
        values.append(value)
        return values

    return add


f = outer()

print(f(10))
print(f(20))
print(f(30))
```

출력 결과:

```
[10]
[10, 20]
[10, 20, 30]
```

이 코드는 앞에서 배운 객체 변경과 이름 재바인딩의 차이를 보여준다.

1. outer() 호출

```
values = []
```

새로운 리스트 객체가 생성된다.

values는 그 리스트 객체를 참조한다.

2. add 함수 객체 생성

```
def add(value):
    values.append(value)
    return values
```

add는 외부 함수의 values를 참조한다.

따라서 values는 add의 자유 변수다.

3. f(10) 호출

```
values.append(10)
```

여기서 values라는 이름을 새로운 객체에 재바인딩하지 않는다.

먼저 자유 변수 values를 통해 리스트 객체를 조회하고, 그 객체의 append 메서드를 호출한다.

따라서 기존 리스트 객체의 내부 상태가 변경된다.

4. f(20) 호출

같은 클로저 셀을 통해 동일한 리스트 객체에 접근한다.

그리고 그 리스트에 20을 추가한다.

따라서 결과는 다음과 같다.

```
[10, 20]
```

이 과정에서 새로운 리스트를 매번 생성하는 것은 아니다.

기존 리스트 객체를 계속 변경하는 것이다.

객체 동일성 확인

```
def outer():
    values = []

    def add(value):
        values.append(value)
        return values

    return add


f = outer()

a = f(10)
b = f(20)

print(a is b)
print(a)
print(b)
```

예상 출력:

```
True
[10, 20]
[10, 20]
```

a와 b는 같은 리스트 객체를 참조한다.

따라서 a is b는 True다.

또한 첫 번째 호출이 반환한 리스트 객체가 두 번째 호출에서 변경되었으므로, 마지막에 a를 출력해도 변경된 상태가 나타난다.

여기서 중요한 것은 다음 두 연산을 구별하는 것이다.

```
values.append(10)
```

이것은 리스트 객체의 상태 변경이다.

반면

```
values = [10]
```

이것은 values라는 이름의 재바인딩이다.

두 연산은 서로 다르다.

그리고 이 차이는 다음 A.5의 nonlocal과 직접 연결된다.

A.4.14 클로저에서 재바인딩을 시도하면?

다음 코드를 살펴보자.

```
def outer():
    x = 10

    def inner():
        x = 20
        return x

    return inner


f = outer()

print(f())
print(f.__closure__)
```

예상 출력:

```
20
None
```

여기서 inner 내부의 x는 자유 변수가 아니다.

왜냐하면 inner 내부에 다음 대입문이 있기 때문이다.

```
x = 20
```

A.3에서 배운 이름의 정적 판정 규칙에 따라 x는 inner의 지역 이름으로 분류된다.

따라서 inner는 외부 함수의 x를 참조하지 않는다.

그 결과 inner에는 클로저 셀이 필요하지 않고, __closure__는 None이다.

이제 다음 코드를 보자.

```
def outer():
```

```
    x = 10

    def inner():
        x = x + 1
        return x

    return inner


f = outer()

f()
```

예상 결과:

```
UnboundLocalError
```

왜 이런 결과가 발생할까?

inner 내부의 대입문 때문에 x는 지역 이름으로 판정된다.

그런데 오른쪽 표현식에서 먼저 x를 읽으려고 한다.

```
x + 1
```

현재 inner 호출의 지역 x에는 아직 객체가 바인딩되어 있지 않다.

따라서 UnboundLocalError가 발생한다.

이것을 해결하려면 외부 함수의 x를 재바인딩하겠다는 의도를 명시해야 한다.

그때 사용하는 것이 nonlocal이다.

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = x + 1
        return x

    return inner


f = outer()

print(f())
```

```
print(f())
print(f())
```

예상 출력:

```
11
12
13
```

이제 inner의 x는 외부 함수의 바인딩을 참조한다.

그리고 nonlocal x 선언에 따라 그 바인딩을 재설정할 수 있다.

다만 nonlocal을 통한 상태 변경은 다음 A.5에서 셀 객체와 함께 더 깊이 분석할 것이다.

이번 학습에서는 다음 차이만 확립하면 된다.

가변 객체의 내부 상태를 변경하는 것과

외부 이름의 바인딩을 변경하는 것은 서로 다른 연산이다.

A.4.15 클로저의 Late Binding

클로저를 학습할 때 반드시 알아야 하는 대표적인 사례가 있다.

바로 Late Binding이다.

다음 코드를 보자.

```
def make_functions():
    functions = []

    for i in range(3):

        def func():
            return i

        functions.append(func)

    return functions


f0, f1, f2 = make_functions()
```

```
print(f0())
print(f1())
print(f2())
```

예상 출력:

```
2
2
2
```

처음 보면 다음과 같은 결과를 예상하기 쉽다.

```
0
1
2
```

왜냐하면 반복문에서 `func`가 세 번 생성되기 때문이다.

하지만 실제 결과는 모두 2이다.

왜 이런 결과가 발생할까?

1. 첫 번째 반복

```
i = 0
```

첫 번째 `func` 함수 객체가 생성된다.

이 함수는 외부 함수의 `i`를 참조한다.

2. 두 번째 반복

```
i = 1
```

두 번째 `func` 함수 객체가 생성된다.

그런데 새로운 `i` 바인딩 환경이 만들어지는 것이 아니다.

Python의 일반적인 `for`문은 반복마다 새로운 함수 지역 스코프를 생성하지 않는다.

따라서 같은 `make_function()` 호출의 `i` 바인딩이 변경된다.

3. 세 번째 반복

```
i = 2
```

세 번째 func 함수 객체가 생성된다.

이 시점에서 i는 2를 참조한다.

함수 객체 반환

세 함수 객체가 리스트에 저장되어 반환된다.

이후 각 함수를 호출하면 어떻게 될까?

세 함수 모두 동일한 외부 함수 호출의 i 바인딩을 참조한다.

따라서 모두 2를 반환한다.

셀 동일성 확인

```
def make_functions():
    functions = []

    for i in range(3):

        def func():
            return i

        functions.append(func)

    return functions


f0, f1, f2 = make_functions()

print(
    f0.__closure__[0]
    is f1.__closure__[0]
)

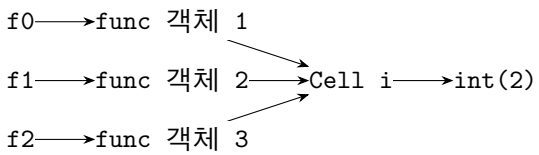
print(
    f1.__closure__[0]
    is f2.__closure__[0]
)
```

예상 출력:

```
True
True
```

세 함수 객체는 서로 다른 함수 객체지만 동일한 셀을 참조한다.

구조를 나타내면 다음과 같다.



이것이 Late Binding 문제의 핵심이다.

함수가 생성될 때 자유 변수의 값이 복사되는 것이 아니라, 나중에 함수가 실행될 때 해당 바인딩의 현재 객체를 조회하기 때문이다.

A.4.16 Late Binding을 해결하는 방법

앞의 문제를 해결하는 대표적인 방법은 두 가지다.

방법 1. 함수 생성기를 사용한다

```python
def make_function(value):

    def func():
        return value

    return func


functions = []

for i in range(3):
    functions.append(make_function(i))


for func in functions:
    print(func())
```

예상 출력:

```
0
```

```
1
2
```

왜 이번에는 원하는 결과가 나올까?

`make_function(i)`를 호출할 때마다 별도의 함수 호출 환경이 만들어지기 때문이다.

각 호출에서 매개변수 `value`는 서로 다른 바인딩 환경을 가진다.

따라서 생성된 `func` 함수 객체들은 서로 다른 셀을 참조한다.

`func 객체 1 → Cell value 1 → int(0)`

`func 객체 2 → Cell value 2 → int(1)`

`func 객체 3 → Cell value 3 → int(2)`

이 방법은 클로저의 원리를 직접 활용하는 방식이다.

방법 2. 기본 인수(Default Argument)를 사용한다

다음 코드를 보자.

```
def make_functions():
    functions = []

    for i in range(3):

        def func(value=i):
            return value

        functions.append(func)

    return functions


f0, f1, f2 = make_functions()

print(f0())
print(f1())
print(f2())
```

예상 출력:

```
0
```

```
1
2
```

이번에는 다음 정의문에 주목해야 한다.

```
def func(value=i):
    return value
```

A.1에서 배운 것처럼 기본 인수 표현식은 함수 정의문이 실행될 때 평가된다.

따라서 반복문이 실행되는 각 시점에 i가 참조하는 객체가 기본 인수로 저장된다.

첫 번째 함수 객체의 기본 인수는 0, 두 번째는 1, 세 번째는 2가 되는 것이다.

클로저 방식과 기본 인수 방식의 차이

```
def make_functions():
    functions = []

    for i in range(3):

        def func(value=i):
            return value

        functions.append(func)

    return functions


f0, f1, f2 = make_functions()

print(f0.__defaults__)
print(f1.__defaults__)
print(f2.__defaults__)

print(f0.__closure__)
```

예상 출력:

```
(0,)
(1,)
(2,)
None
```

여기서 중요한 차이가 드러난다.

기본 인수 방식에서는 각 함수 객체의 `__defaults__`에 해당 객체가 저장된다.

반면 `func` 본문은 외부 함수의 `i`를 참조하지 않는다.

따라서 자유 변수가 없고 `__closure__`도 `None`이다.

구분	클로저 방식	기본 인수 방식
외부 값의 사용	자유 변수 바인딩을 통해 접근	기본 인수 객체를 통해 접근
주요 저장 구조	`__closure__`	`__defaults__`
외부 바인딩의 후속 재바인딩 반영	가능	일반적으로 반영되지 않음
함수 정의 시점의 객체 보관	바인딩 자체에 접근	기본 인수 객체 참조 저장

여기서 기본 인수가 객체를 복사한다는 뜻은 아니다.

기본 인수 역시 객체 참조를 저장한다.

따라서 가변 객체를 기본 인수로 사용하면 그 객체의 상태 변경이 이후 호출에도 영향을 줄 수 있다.

기본 인수의 정의 시점 평가와 클로저의 자유 변수 조회는 서로 다른 메커니즘이라는 사실을 기억해야 한다.

A.4.17 클로저와 전역 변수 참조의 사이

이제 클로저와 전역 변수 참조를 비교해 보자.

```
x = 10


def global_func():
    return x


def outer():
    x = 20

    def closure_func():
        return x

    return closure_func
```

```
f = outer()

print(global_func())
print(f())
```

예상 출력:

```
10
20
```

이 차이는 A.3의 LEGB와 연결된다.

global_func의 x는 전역 이름이다.

반면 closure_func의 x는 외부 함수의 자유 변수다.

두 함수의 내부 정보를 확인해 보자.

```
print(global_func.__code__.co_freevars)
print(global_func.__closure__)

print(f.__code__.co_freevars)
print(f.__closure__[0].cell_contents)
```

예상 출력:

```
()
None
('x',)
20
```

전역 이름을 참조하는 함수는 그 이름에 대응하는 클로저 셀을 필요로 하지 않는다.

일반적인 Python 함수 객체는 __globals__를 통해 자신의 전역 네임스페이스를 참조할 수 있다.

반면 렉시컬 외부 함수의 바인딩은 클로저 셀을 통해 유지될 수 있다.

따라서 다음 두 구조는 다르다.

```
global_func
└── __globals__
    └── x → int(10)

closure_func
└── __closure__
    └── Cell x → int(20)
```

이 차이를 이해하면 함수 객체의 `__globals__`와 `__closure__`가 왜 별도로 존재하는지도 설명할 수 있다.

A.4.18 클로저 셀의 바인딩이 제거되면

이번에는 조금 더 깊은 경계 조건을 살펴보자.

다음 코드를 실행해 보자.

```
def outer():
    x = 10

    def inner():
        return x

    del x

    return inner


f = outer()

print(f.__code__.co_freevars)
print(f.__closure__)
```

예상 결과는 다음과 같은 형태다.

```
('x',)
(<cell at 0x...: empty>,)
```

메모리 주소는 실행 환경에 따라 달라진다.

여기서 중요한 것은 셀 객체가 여전히 존재하지만 비어 있다는 사실이다.

왜 이런 상태가 발생할까?

inner는 외부 함수의 x를 참조한다.

따라서 x에 대응하는 셀이 필요하다.

그런데 outer() 내부에서 다음 문장이 실행된다.

```
del x
```

이 문장은 x의 바인딩을 제거한다.

따라서 셀 객체 자체는 존재하지만 더 이상 참조하는 객체가 없는 상태가 될 수 있다.

이후 다음 호출을 수행하면

```
f()
```

일반적인 Python에서는 NameError가 발생한다.

왜냐하면 inner가 자유 변수 x의 바인딩을 조회하려고 하지만, 해당 셀이 비어 있기 때문이다.

이것은 중요한 사실을 보여준다.

클로저 셀이 존재한다는 사실과 그 셀에 객체가 바인딩되어 있다는 사실은 서로 다르다.

A.3에서 학습한 이름의 분류와 실행 시점의 바인딩 상태 구분이 클로저에도 그대로 적용되는 것이다.

참고로 비어 있는 셀의 cell_contents를 직접 읽으려고 하면 ValueError가 발생할 수 있다.

반면 그 셀을 자유 변수로 사용하는 함수 본문에서 이름을 읽으면 NameError가 발생한다.

같은 바인딩 상태라도 어떤 연산을 수행하는지에 따라 예외가 달라질 수 있다는 점에 주의해야 한다.

A.4.19 클로저와 객체 상태 - 실전 응용

이제 클로저가 실제 프로그램에서 어떻게 활용되는지 살펴보자.

다음 예제는 호출 횟수를 기록하는 함수 생성기다.

```
def make_counter():
    state = {"count": 0}

    def counter():
        state["count"] += 1
        return state["count"]

    return counter


counter_a = make_counter()
counter_b = make_counter()

print(counter_a())
print(counter_a())
print(counter_a())

print(counter_b())
print(counter_b())
```

예상 출력:

```
1
2
3
1
2
```

이 코드를 객체 모델 관점에서 분석해 보자.

1. 첫 번째 make_counter() 호출

```
state = {"count": 0}
```

새로운 딕셔너리 객체가 생성된다.

그리고 state가 그 딕셔너리를 참조한다.

counter 함수 객체는 자유 변수 state를 참조한다.

따라서 반환된 함수 객체는 해당 바인딩에 접근할 수 있는 클로저를 가진다.

2. counter_a() 호출

```
state["count"] += 1
```

이 연산은 state라는 이름을 새로운 딕셔너리 객체에 재바인딩하는 것이 아니다.

먼저 state가 참조하는 딕셔너리 객체를 조회하고, "count" 키에 대응하는 값을 갱신한다.

따라서 같은 딕셔너리 객체의 상태가 변경된다.

3. 두 번째 make_counter() 호출

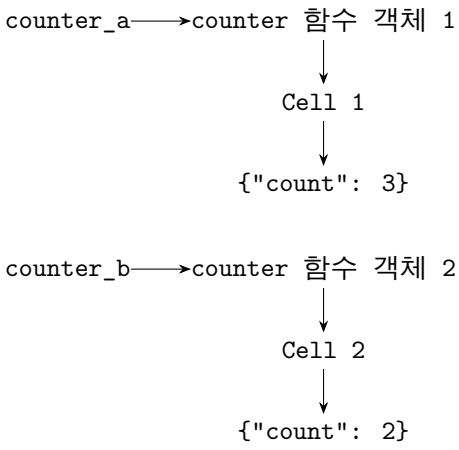
```
counter_b = make_counter()
```

이번에는 새로운 딕셔너리 객체가 생성된다.

그리고 별도의 클로저 셀을 참조하는 새로운 counter 함수 객체가 생성된다.

따라서 counter_a와 counter_b는 독립적인 상태를 유지할 수 있다.

구조를 나타내면 다음과 같다.



여기서 state["count"] += 1은 딕셔너리 내부 상태를 변경하는 연산이므로 nonlocal state가 필요하지 않다.

반면 다음처럼 state라는 이름 자체를 재바인딩하려 한다면 이야기가 달라진다.

```
state = {"count": 10}
```

이 경우에는 외부 함수의 state를 재바인딩하려는 의도를 nonlocal로 명시해야 한다.

이 차이는 다음 A.5에서 더 깊이 다룰 것이다.

A.4.20 클래스 인스턴스와 클로저의 상태 관리 비교

지금까지 학습한 클래스 객체 모델과 클로저를 연결해 보자.

앞의 카운터는 클래스로도 구현할 수 있다.

```
class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count


counter = Counter()

print(counter())
print(counter())
print(counter())
```

예상 출력:

```
1
2
3
```

이 코드와 앞의 클로저 구현은 외부에서 호출 가능한 객체를 통해 상태를 유지한다는 공통점이 있다.

하지만 내부 구조는 다르다.

클로저 방식

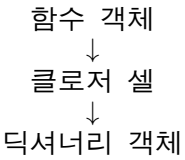
```
def make_counter():
    state = {"count": 0}

    def counter():
        state["count"] += 1
```

```
        return state["count"]

    return counter
```

상태는 외부 함수의 바인딩을 통해 접근한다.

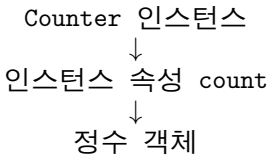


클래스 방식

```
class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count
```

상태는 인스턴스의 속성으로 관리된다.



비교

구분	클로저	클래스 인스턴스
호출 대상	함수 객체	`__call__`을 구현한 인스턴스
상태 접근	자유 변수와 클로저 셀	인스턴스 속성
상태 저장 구조	셀과 참조 객체	인스턴스 네임스페이스 등
상태 변경	객체 변경 또는 `nonlocal` 재바인딩	속성 대입 및 객체 변경
상태 확인	`__closure__` 등	`__dict__`, 속성 접근 등
주요 활용	함수 생성기, 데코레이터, 간단한 상태 유지	복잡한 상태·행동의 캡슐화

여기서 어느 방식이 무조건 우수하다고 할 수는 없다.

상태가 단순하고 함수 자체를 반환하는 구조가 자연스럽다면 클로저가 유용할 수 있다.

반면 상태가 많고 여러 메서드가 필요하거나 명시적인 객체 인터페이스가 중요하다면 클래스가 더 적합할 수 있다.

이 차이는 데코레이터를 함수 기반으로 궁녀할지 클래스 기반으로 구현할지 판단할 때도 연결된다.

A.4.21 클로저와 데코레이터의 연결

이제 이번 학습 주제의 최종 목표인 데코레이터와 연결해 보자.

다음 코드를 살펴보자.

```
def make_wrapper(func):

    def wrapper():
        print("호출 전")

        result = func()

        print("호출 후")

        return result

    return wrapper
```

```
def greet():
    print("Hello")
    return 10


wrapped = make_wrapper(greet)

print(wrapped())
```

예상 출력:

```
호출 전
Hello
호출 후
10
```

여기서 make_wrapper는 함수 객체를 인수로 전달받아 새로운 함수 객체를 반환한다.

이 구조를 분석해 보자.

1. greet 함수 객체 생성

```
def greet():
    print("Hello")
    return 10
```

greet 함수 객체가 생성되고 모듈 이름 greet에 바인딩된다.

2. make_wrapper(greet) 호출

```
wrapped = make_wrapper(greet)
```

greet이 참조하는 함수 객체가 make_wrapper의 매개변수 func에 바인딩된다.

3. wrapper 함수 객체 생성

```
def wrapper():
    print("호출 전")

    result = func()
```

```
    print("호출 후")

    return result
```

wrapper 함수 객체가 생성된다.

이 함수는 외부 함수의 매개변수 func를 참조한다.

따라서 func는 wrapper의 자유 변수다.

4. wrapper 함수 객체 반환

```
return wrapper
```

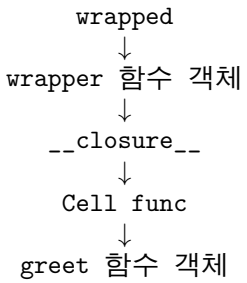
wrapper 함수 객체가 반환되고 wrapped에 바인딩된다.

이후 make_wrapper() 호출은 종료된다.

하지만 반환된 wrapper 함수 객체는 func에 대응하는 클로저 셀을 참조하고 있다.

그 셀은 원본 greet 함수 객체를 참조한다.

구조를 나타내면 다음과 같다.



5. wrapped() 호출

wrapper 함수 본문이 실행된다.

먼저 "호출 전"이 출력된다.

이후 다음 문장이 실행된다.

```
result = func()
```

자유 변수 func를 통해 원본 greet 함수 객체를 조회하고 호출한다.

그 결과 10이 반환된다.

이후 "호출 후"가 출력되고 wrapper는 10을 반환한다.

이것이 함수 기반 데코레이터의 핵심 구조다.

내부 구조 확인

```
print(wrapped.__code__.co_freevars)

print(
    wrapped.__closure__[0].cell_contents
    is greet
)
```

예상 출력:

```
('func',)
True
```

이 결과는 매우 중요하다.

데코레이터가 원본 함수 객체를 감싸는 구조는 클로저를 통해 구현할 수 있다.

wrapper가 원본 함수의 이름을 기억하는 것이 아니라, 외부 매개변수 func의 바인딩에 접근할 수 있는 클로저를 유지하는 것이다.

이 원리를 이해하면 다음과 같은 데코레이터 변환식도 자연스럽게 설명할 수 있다.

```
@make_wrapper
def greet():
    print("Hello")
    return 10
```

이는 개념적으로 다음 변환과 대응한다.

```
def greet():
    print("Hello")
    return 10


greet = make_wrapper(greet)
```

여기서 오른쪽의 greet은 원본 함수 객체를 참조한다.

make_wrapper(greet)은 원본 함수를 인수로 전달받아 wrapper 함수 객체를 반환한다.

그리고 왼쪽의 greet은 반환된 wrapper 함수 객체에 재바인딩된다.

이때 원본 함수 객체는 사라지는 것이 아니라 wrapper의 클로저를 통해 계속 참조될 수 있다.

이 구조를 A.6에서 정식으로 분석할 것이다.

A.4.22 자주 발생하는 오해와 경계 조건

이번 학습에서 특히 주의해야 할 개념을 정리해 보자.

> 오해 1. 중첩 함수는 모두 클로저 셀을 가진다

그렇지 않다.

```
def outer():

    def inner():
        return 10

    return inner


f = outer()

print(f.__closure__)
```

출력:

```
None
```

inner는 중첩 함수이지만 외부 함수의 바인딩을 참조하지 않는다.

따라서 클로저 셀이 필요하지 않다.

중첩 함수라는 사실과 실제로 클로저 환경을 가진다는 사실은 구별해야 한다.

> 오해 2. 클로저는 외부 변수의 값을 복사한다

그렇지 않다.

클로저는 외부 바인딩에 접근하는 구조를 유지한다.

따라서 외부 바인딩이 변경되면 이후 호출에서 변경된 객체를 조회할 수 있다.

> 오해 3. 외부 함수가 종료되면 지역 변수에 접근할 수 없다

일반적인 지역 변수는 호출 종료 후 직접 접근할 수 없지만, 중첩 함수가 필요한 바인딩을 클로저로 유지할 수 있다.

따라서 함수 호출 종료와 바인딩의 수명을 동일시하면 안 된다.

> 오해 4. 클로저는 외부 함수의 모든 지역 변수를 저장한다

그렇지 않다.

중첩 함수가 사용하는 자유 변수에 대응하는 바인딩이 클로저 환경에 포함된다.

외부 함수의 모든 지역 변수가 자동으로 저장되는 것은 아니다.

> 오해 5. 서로 다른 함수 객체는 반드시
> 서로 다른 클로저 상태를 가진다

그렇지 않다.

같은 외부 함수 호출에서 생성된 여러 내부 함수는 동일한 셀을 공유할 수 있다.

따라서 함수 객체의 동일성과 클로저 셀의 동일성은 별도로 판단해야 한다.

> 오해 6. 클로저를 사용하면 항상 nonlocal이 필요하다

그렇지 않다.

자유 변수로 참조하는 가변 객체의 내부 상태를 변경하는 경우에는 `nonlocal`이 필요하지 않을 수 있다.

반면 외부 함수의 이름 자체를 재바인딩하려면 `nonlocal`이 필요하다.

A.4.23 핵심 정리와 판단 기준

이번 학습의 내용을 하나의 실행 흐름으로 정리해 보자.

```
def outer():
    x = 10
```

```
    def inner():
        return x

    return inner


f = outer()

print(f())
```

이 코드가 실행되는 과정을 정확하게 설명하면 다음과 같다.

1. 컴파일 단계

inner에서 사용한 x는 외부 함수의 바인딩을 참조하는 자유 변수로 분류된다.

outer의 x는 중첩 함수에서 접근해야 하는 셀 변수로 분류된다.

2. outer() 호출

현재 호출을 위한 실행 환경이 마련된다.

x에 정수 객체 10이 바인딩된다.

3. inner 함수 객체 생성

def inner가 실행되면서 새로운 함수 객체가 생성된다.

이 함수 객체는 x에 대응하는 클로저 셀을 참조한다.

4. inner 함수 객체 반환

return inner가 실행되면서 함수 객체가 반환된다.

모듈 이름 f가 반환된 함수 객체를 참조하게 된다.

5. outer() 실행 종료

외부 함수의 실행은 종료된다.

그러나 f가 참조하는 함수 객체는 계속 존재하며, 그 함수 객체를 통해 클로저 셀에 접근할 수 있다.

6. f() 호출

inner 함수 본문이 실행된다.

자유 변수 x에 대응하는 셀을 통해 정수 객체 10을 조회한다.

그 객체를 반환한다.

이 흐름을 이해했다면 클로저의 핵심 원리르 설명할 수 있다.

최종 판단 기준

질문	판단 기준
어떤 이름이 자유 변수인가?	현재 함수에서 지역 바인딩되지 않고 렉시컬 외부 함수의 바인딩을 사용하는가?
어떤 이름이 셀 변수인가?	현재 함수의 지역 바인딩을 중첩 함수에서 접근해야 하는가?
클로저 셀이 존재하는가?	함수 객체의 `__closure__`를 확인한다.
셀에 어떤 객체가 바인딩되어 있는가?	셀의 `cell_contents`를 확인한다.
서로 다른 함수가 상태를 공유하는가?	대응하는 셀 객체 또는 셀이 참조하는 객체의 동일성을 확인한다.
함수 정의 시점의 값이 고정되는가?	자유 변수 바인딩을 사용하는지, 기본 인수 객체를 저장하는지 구별한다.
외부 함수 종료 후에도 접근 가능한가?	반환된 함수 객체와 클로저 셀의 참조 관계를 확인한다.
외부 객체의 상태 변경인가, 이름 재바인딩인가?	메서드 호출·항목 변경과 이름 대입을 구별한다.

A.5 `nonlocal`과 클로저의 상태 변경

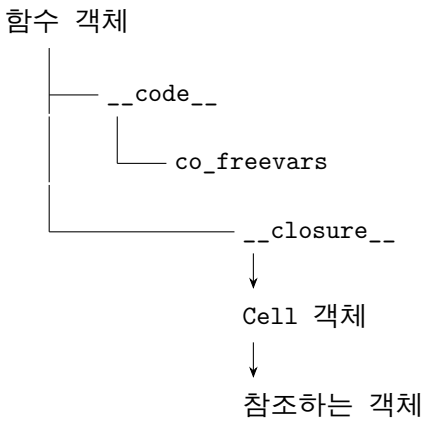
Nonlocal Binding · Shared Closure Cells · Rebinding · Mutation · State Management

A.5.1 이전 학습과의 연결

A.4에서는 클로저(Closure)가 단순히 외부 변수의 값을 복사하는 구조가 아니라,

렉시컬 외부 함수의 바인딩에 접근할 수 있는 환경을 유지하는 구조라는 사실을 학습했다.

특히 다음 객체 관계를 확인했다.



그리고 하나의 외부 함수 호출에서 생성된 여러 내부 함수가 동일한 자유 변수를 참조한다면, 같은 셀 객체를 공유할 수 있다는 사실도 확인했다.

그런데 아직 중요한 문제가 남아 있다.

외부 함수의 바인딩을 읽는 것과 그 바인딩을 변경하는 것은 어떻게 다를까?

다음 코드를 살펴보자.

```
def outer():
    x = 10

    def get():
        return x

    def set_value(value):
        nonlocal x
        x = value

    return get, set_value


get, set_value = outer()

print(get())

set_value(100)

print(get())
```

예상 출력:

```
10
```

여기서 get과 set_value는 서로 다른 함수 객체다.

그런데 set_value()를 호출하면 get()의 반환값까지 변경된다.

왜 이런 결과가 발생할까?

두 함수가 같은 셀 객체를 공유하고, set_value()의 nonlocal x 선언이 그 셀에 연결된 바인딩을 변경하기 때문이다.

이번 학습에서는 이 과정을 Python의 객체 모델에 따라 분석할 것이다.

A.5.2 이번 학습의 핵심 질문

이번 학습에서 해결할 질문은 다음과 같다.

1. nonlocal은 정확히 어떤 이름을 대상으로 하는가?
2. nonlocal 선언은 셀 객체를 생성하거나 값을 변경하는가?
3. 여러 클로저가 동일한 셀을 공유하면 어떤 일이 발생하는가?
4. 셀 객체의 동일성과 셀이 참조하는 객체의 동일성은 어떻게 다른가?
5. 이름 재바인딩과 가변 객체의 상태 변경은 어떻게 구별하는가?
6. nonlocal 없이도 클로저의 상태를 변경할 수 있는가?
7. nonlocal은 왜 특정 상황에서 SyntaxError 또는 UnboundLocalError와 연결되는가?
8. 클로저를 이용한 상태 관리는 클래스 인스턴스의 상태 관리와 어떻게 다른가?

이번 학습 전체를 관통하는 핵심 원리는 다음과 같다.

> nonlocal은 외부 객체를 변경하는 명령이 아니라, 현재 함수 블록의 이름 바인딩 대상을 렉시컬 외부 함수 스코프로 지정하는 선언이다.

이 차이를 정확하게 이해해야 한다.

A.5.3 nonlocal의 정확한 정의

nonlocal은 현재 함수 블록에서 사용하는 이름이 현재 함수의 지역 이름이 아니라, 렉시컬 구조상 바깥에 있는 함수 스코프의 기존 바인딩을 사용하도록 지정하는 선언문이다.

가장 기본적인 형태는 다음과 같다.

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = 20

    inner()

    return x


print(outer())
```

예상 출력 결과:

```
20
```

여기서 nonlocal x가 수행하는 역할을 정확하게 구분해 보자.

> nonlocal x 자체가 수행하지 않는 일

nonlocal x는 다음 작업을 직접 수행하지 않는다.

- 새로운 정수 객체를 생성하지 않는다.
- 새로운 셀 객체를 매번 생성하지 않는다.
- 외부 함수의 x를 즉시 변경하지 않는다.
- 외부 함수의 지역 변수를 현재 함수로 복사하지 않는다.

대신 현재 함수에서 x라는 이름을 어떻게 해석하고 바인딩할지 지정한다.

실제 바인딩 변경은 다음 문장이 실행될 때 발생한다.

```
x = 20
```

이때 x가 현재 함수의 지역 이름이 아니라 외부 함수의 바인딩을 대상으로 한다는 점이 중요하다.

A.5.4 nonlocal이 없는 경우와 있는 경우

먼저 두 코드를 비교해 보자.

사례 1. nonlocal이 없는 경우

```
def outer():
    x = 10

    def inner():
        x = 20
        return x

    result = inner()

    print("inner:", result)
    print("outer:", x)


outer()
```

예상 출력:

```
inner: 20
outer: 10
```

왜 외부 함수의 x는 변경되지 않았을까?

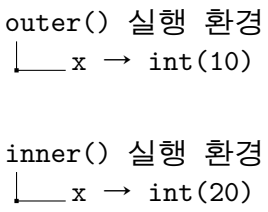
A.3에서 학습한 이름의 정적 판정 규칙 때문이다.

inner 내부에는 다음 대입문이 있다.

```
x = 20
```

별도의 global 또는 nonlocal 선언이 없으므로 x는 inner의 지역 이름으로 판정된다.

따라서 두 함수에는 서로 다른 바인딩이 존재한다.



두 이름의 철자가 같더라도 동일한 바인딩을 의미하지 않는다.

따라서 inner()가 종료된 뒤에도 outer의 x는 10을 참조한다.

사례 2. nonlocal이 있는 경우

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = 20
        return x

    result = inner()

    print("inner:", result)
    print("outer:", x)


outer()
```

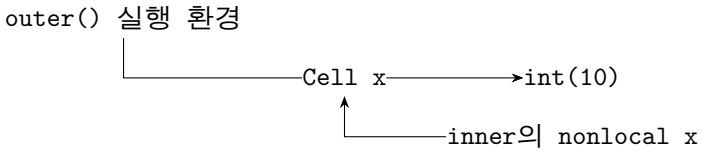
예상 출력:

```
inner: 20
outer: 20
```

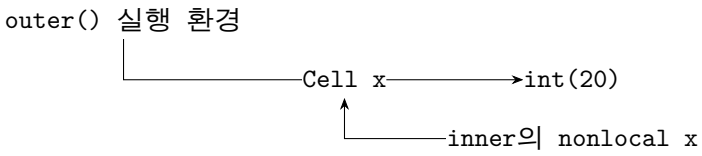
이번에는 inner의 x가 외부 함수의 바인딩을 대상으로 한다.

따라서 x = 20은 inner에 새로운 지역 바인딩을 만드는 것이 아니라 outer의 x 바인딩을 변경한다.

개념적으로 다음과 같은 구조다.



inner()에서 대입이 실행되면 다음과 같이 바뀐다.



여기서 셀 객체 자체가 교체되는 것은 아니다.

기존 셀이 참조하는 객체가 변경되는 것이다.

이 차이를 다음 절에서 직접 확인해 보자.

A.5.5 셀 객체의 동일성과 바인딩 변경

이번 학습에서 가장 중요한 실험이다.

다음 코드를 살펴보자.

```
def outer():
    x = 10

    def get():
        return x

    def set_value(value):
        nonlocal x
        x = value

    return get, set_value


get, set_value = outer()

get_cell = get.__closure__[0]
set_cell = set_value.__closure__[0]

print(get_cell is set_cell)

print(get_cell.cell_contents)

set_value(100)

print(get_cell.cell_contents)

print(get_cell is set_cell)
```

예상 출력:

```
True
10
100
True
```

이 결과에는 서로 다른 두 종류의 동일성 관계가 포함되어 있다.

첫 번째 동일성 - 셀 객체의 동일성

```
get_cell is set_cell
```

결과는 True다.

get과 set_value는 같은 outer() 호출에서 생성되었고, 동일한 외부 변수 x를 참조하기 때문이다.

두 함수는 하나의 셀 객체를 공유한다.

두 번째 관계 - 셀이 참조하는 객체

처음에는 다음과 같다.

```
get_cell.cell_contents
```

결과:

```
10
```

그런데 다음 호출을 실행하면

```
set_value(100)
```

set_value 내부에서 다음 문장이 실행된다.

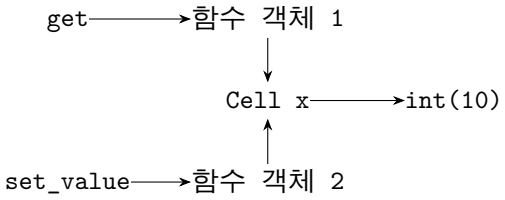
```
x = value
```

nonlocal x가 선언되어 있으므로 외부 함수의 x 바인딩이 변경된다.

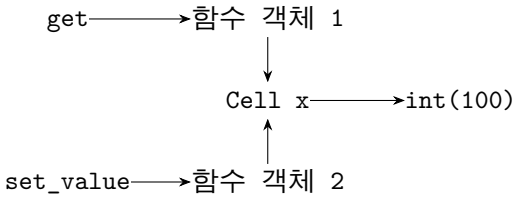
따라서 셀이 참조하는 객체가 정수 객체 10에서 정수 객체 100으로 바뀐다.

하지만 셀 객체는 그대로다.

실행 전



실행 후



두 그림에서 바뀐 것은 `Cell x`가 참조하는 객체다.

`get` 함수 객체도 그대로이고, `set_value` 함수 객체도 그대로이며, 셀 객체 역시 동일하다.

이를 다음처럼 정리할 수 있다.

비교 대상	set_value(100) 실행 전후
`get` 함수 객체	동일
`set_value` 함수 객체	동일
공유하는 셀 객체	동일
셀이 참조하는 정수 객체	변경
`get()` 반환값	10 → 100

`nonlocal`에 의한 재바인딩은 셀 객체를 교체하는 것이 아니라, 기존 셀의 바인딩 대상을 변경하는 연산이다.

A.5.6 co_freevars, co_cellvars, __closure__로 검증하기

앞의 예제를 조금 확장해 보자.

```
def outer():
```

```
    x = 10

    def get():
        return x

    def set_value(value):
        nonlocal x
        x = value

    return get, set_value


get, set_value = outer()

print("outer cellvars:", outer.__code__.co_cellvars)

print("get freevars:", get.__code__.co_freevars)

print("set freevars:", set_value.__code__.co_freevars)

print(
    "same cell:",
    get.__closure__[0] is set_value.__closure__[0]
)
```

예상 출력:

```
outer cellvars: ('x',)
get freevars: ('x',)
set freevars: ('x',)
same cell: True
```

이 결과를 순서대로 해석해 보자.

```
1. outer.__code__.co_cellvars
```

```
('x',)
```

outer의 지역 변수 x는 중첩 함수에서 접근해야 하는 바인딩이다.

따라서 셀 변수로 분류된다.

```
2. get.__code__.co_freevars
```

```
('x',)
```

get은 외부 함수의 x를 읽어야 한다.

따라서 x는 get의 자유 변수다.

3. __closure__ 비교

두 함수의 __closure__에 포함된 셀 객체를 비교하면 동일한 객체라는 사실을 확인할 수 있다.

여기서 중요한 결론은 다음과 같다.

nonlocal이 선언되었다고 해서 해당 이름이 현재 함수의 지역 변수가 되는 것이 아니다.

오히려 nonlocal은 그 이름을 현재 함수의 지역 바인딩 대상으로 취급하지 않도록 지정한다.

따라서 set_value의 x는 외부 함수의 바인딩을 사용하는 자유 변수로 분류된다.

A.5.7 객체 변경(Mutation)과 이름 재바인딩(Rebinding)

이제 nonlocal을 사용할 때 가장 자주 혼동하는 문제를 살펴보자.

다음 두 표현식은 서로 다른 연산이다.

```
values.append(10)
```

```
values = [10]
```

첫 번째는 values가 참조하는 리스트 객체의 내부 상태를 변경한다.

두 번째는 values라는 이름을 새로운 리스트 객체에 바인딩한다.

이 차이를 클로저에 적용해 보자.

사례 1. 리스트 객체의 상태 변경

```
def outer():
    values = []

    def add(value):
        values.append(value)
        return values
```

```
    return add

add = outer()

print(add(10))
print(add(20))
print(add(30))
```

예상 출력:

```
[10]
[10, 20]
[10, 20, 30]
```

여기에는 nonlocal 선언이 없다.

그런데도 외부 함수에서 생성한 리스트의 상태가 변경되고 있다.

왜 가능할까?

add() 내부에서는 values라는 이름에 새로운 객체를 대입하지 않기 때문이다.

다음 표현식을 분석해 보자.

```
values.append(value)
```

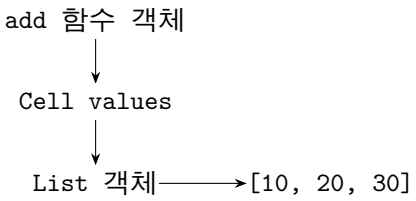
실행 과정은 다음과 같다.

1. 자유 변수 values를 통해 리스트 객체를 조회한다.
2. 그 리스트 객체에서 append 메서드를 조회한다.
3. 현재 호출의 매개변수 value가 참조하는 객체를 인수로 전달한다.
4. append 메서드가 기존 리스트 객체를 변경한다.

이 과정에서 외부 이름 values의 바인딩 자체는 변경되지 않는다.

따라서 nonlocal values가 필요하지 않은 것이다.

객체 구조



리스트 객체는 계속 동일하다.

그 내부 원소 구성이 변경되는 것이다.

A.5.8 리스트를 재바인딩하려면?

이번에는 다음 코드를 살펴보자.

```
def outer():
    values = []

    def reset():
        values = []
        return values

    def get():
        return values

    return reset, get


reset, get = outer()

print(get())
print(reset())
print(get())
```

예상 출력:

```
[]
[]
[]
```

세 출력이 모두 빈 리스트이므로 얼핏 보면 `reset()`이 외부 리스트를 초기화한 것처럼 보일 수 있다.

하지만 실제로는 그렇지 않다.

reset() 내부의 다음 문장에 주목해 보자.

```
values = []
```

nonlocal 선언이 없으므로 values는 reset의 지역 이름이다.

따라서 reset()은 새로운 리스트 객체를 생성하고 자신의 지역 이름에 바인딩할 뿐이다.

외부 함수의 values에는 영향을 주지 않는다.

이를 명확하게 확인해 보자.

```
def outer():
    values = [10, 20]

    def reset():
        values = []
        return values

    def get():
        return values

    return reset, get


reset, get = outer()

print("before:", get())
print("reset result:", reset())
print("after:", get())
```

예상 출력:

```
before: [10, 20]
reset result: []
after: [10, 20]
```

이제 차이가 분명해졌다.

reset()에서 새로운 리스트 객체를 생성했지만, 외부 함수의 values는 변경되지 않았다.

nonlocal을 사용한 경우

```
def outer():
```

```
    values = [10, 20]

    def reset():
        nonlocal values
        values = []
        return values

    def get():
        return values

    return reset, get


reset, get = outer()

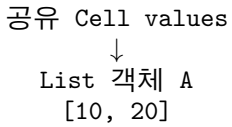
print("before:", get())
print("reset result:", reset())
print("after:", get())
```

예상 출력:

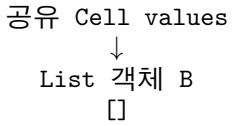
```
before: [10, 20]
reset result: []
after: []
```

이번에는 `reset()`이 외부 함수의 `values` 바인딩을 새로운 리스트 객체로 변경한다.

실행 전



실행 후



여기서 기존 리스트 객체 A가 반드시 즉시 소멸한다는 뜻은 아니다.

다른 이름이나 객체가 리스트 A를 참조하고 있다면 계속 존재할 수 있다.

핵심은 셀의 참조 대상이 리스트 A에서 리스트 B로 변경되었다는 사실이다.

A.5.9 같은 결과처럼 보여도 객체 관계는 다를 수 있다

이번에는 `clear()`와 재바인딩을 비교해 보자.

```
def make_state():
    values = [10, 20]

    def get():
        return values

    def clear_in_place():
        values.clear()

    def replace():
        nonlocal values
        values = []

    return get, clear_in_place, replace


get, clear_in_place, replace = make_state()

original = get()

print("initial:", original)

clear_in_place()

print("after clear:", original)

replace()

print("after replace:", original)
print("current:", get())

print(original is get())
```

예상 출력:

```
initial: [10, 20]
after clear: []
after replace: []
```

```
current: []
False
```

이 결과는 특히 중요하다.

`clear_in_place()`와 `replace()`를 실행한 뒤 모두 빈 리스트가 나타나지만, 두 함수가 수행한 연산은 다르다.

```
clear_in_place()
```

```
values.clear()
```

기존 리스트 객체의 내부 상태를 변경한다.

따라서 `original`이 참조하는 리스트 객체도 비워진다.

```
replace()
```

```
nonlocal values
values = []
```

새로운 리스트 객체를 생성하고 외부 바인딩을 변경한다.

따라서 `original`은 여전히 기존 리스트 객체를 참조하지만, `get()`은 새로운 리스트 객체를 반환한다.

마지막 비교가 이를 보여준다.

```
original is get()
```

결과는 `False`다.

이 차이는 클래스 인스턴스에서도 동일하게 나타난다.

```
self.values.clear()
```

와

```
self.values = []
```

는 서로 다른 연산이다.

전자는 기존 리스트 객체를 변경하고, 후자는 일반적인 속성 대입 규칙에 따라

`values` 속성의 참조 대상을 변경한다.

따라서 클로저의 상태 변경 원리는 기존에 학습한 객체 모델과 분리된 특별한 규칙이 아니다.

Python의 일반적인 객체 변경과 바인딩 변경 원리가 클로저 환경에서도 적용되는 것이다.

A.5.10 `+=` 연산은 왜 주의해야 할까?

이번에는 다음 코드를 살펴보자.

```
def outer():
    count = 0

    def increment():
        count += 1
        return count

    return increment


increment = outer()

print(increment())
```

예상 결과:

```
UnboundLocalError
```

왜 이런 결과가 발생할까?

`count += 1`은 단순히 외부 정수 객체의 상태를 변경하는 연산이 아니다.

정수 객체는 불변 객체(Immutable Object)다.

또한 `count += 1`은 이름 `count`에 대한 대입을 포함하는 증강 대입문(Augmented Assignment)이다.

따라서 별도의 `nonlocal` 선언이 없으면 `count`는 `increment`의 지역 이름으로 분류된다.

그런데 증강 대입에서는 기존 값을 먼저 읽어야 한다.

현재 지역 count에는 아직 객체가 바인딩되지 않았으므로

UnboundLocalError가 발생하는 것이다.

해결 방법

```
def outer():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count

    return increment


increment = outer()

print(increment())
print(increment())
print(increment())
```

예상 출력:

```
1
2
3
```

이번에는 count가 외부 함수의 바인딩을 대상으로 한다.

따라서 기존 정수 객체를 읽고, 덧셈 결과를 다시 외부 이름에 바인딩할 수 있다.

실행 과정

첫 번째 호출:

```
    Cell count → int(0)
```

count += 1 실행 후:

```
    Cell count → int(1)
```

두 번째 호출 후:

```
Cell count → int(2)
```

세 번째 호출 후:

```
Cell count → int(3)
```

여기서 정수 객체 자체가 변경되는 것은 아니다.

매번 새로운 계산 결과에 바인딩되는 것이다.

A.5.11 가변 객체의 +=는 어떻게 다를까?

이제 정수 대신 리스트를 사용해 보자.

```
def outer():
    values = []

    def add(value):
        values += [value]
        return values

    return add


add = outer()

print(add(10))
```

이 코드는 어떻게 될까?

리스트는 가변 객체이므로 정상적으로 동작할 것 같지만,

실제로는 `UnboundLocalError`가 발생한다.

왜 그럴까?

여기서 중요한 것은 객체가 가변인지 여부와 이름의 스코프 판정은 별개의 문제라는 사실이다.

`values += [value]`는 증강 대입문이므로 `values`를 바인딩 대상으로 사용한다.

따라서 `nonlocal` 선언이 없다면 `values`는 현재 함수의 지역 이름으로 판정된다.

그런데 기존 지역 values가 아직 바인딩되지 않았으므로 읽는 과정에서 오류가 발생하는 것이다.

nonlocal을 추가하면?

```
def outer():
    values = []

    def add(value):
        nonlocal values
        values += [value]
        return values

    return add


add = outer()

print(add(10))
print(add(20))
```

예상 출력:

```
[10]
[10, 20]
```

이제 정상적으로 동작한다.

그런데 여기서 한 단계 더 생각해야 한다.

리스트의 +=는 일반적으로 기존 리스트 객체를 직접 변경하는

__iadd__ 동작을 사용할 수 있다.

따라서 nonlocal values가 있다고 해서 반드시 새로운 리스트 객체가 생성되는 것은 아니다.

다음 코드로 확인해 보자.

```
def outer():
    values = []

    def add(value):
        nonlocal values

        before = values
```

```
        values += [value]

        return before is values

    return add


add = outer()

print(add(10))
print(add(20))
```

일반적인 Python 리스트에서 예상 출력:

```
True
True
```

이 결과는 다음 두 가지를 동시에 보여준다.

1. nonlocal은 이름의 바인딩 대상을 지정한다.
2. 시제 증강 대입이 기존 객체를 변경할지 새로운 객체를 반환할지는 피연산자의 타입과 연산 구현에 따라 달라질 수 있다.

따라서 다음 세 연산을 동일하게 취급하면 안 된다.

```
values.append(10)

values += [10]

values = values + [10]
```

일반적인 리스트에서는 첫 번째와 두 번째가 기존 리스트 객체를 변경하지만, 세 번째는 새로운 리스트 객체를 만들어 이름을 재바인딩한다.

또한 두 번째와 세 번째는 이름 대입을 포함하므로 외부 함수의 values를 대상으로 하려면 nonlocal이 필요하다.

객체 변경 여부와 이름의 스코프 판정 여부는 서로 다른 기준으로 판단해야 한다.

A.5.12 nonlocal은 가장 가까운 외부 함수의 바인딩을 찾는다

지금까지는 외부 함수가 한 단계만 존재했다.

이번에는 세 단계의 중첩 함수를 살펴보자.

```
def level1():
    x = 10

    def level2():
        x = 20

        def level3():
            nonlocal x
            x = 30

        level3()

        print("level2:", x)

    level2()

    print("level1:", x)

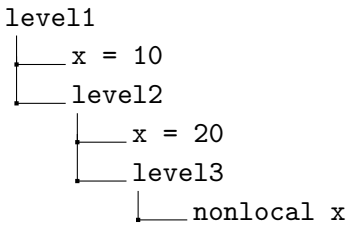

level1()
```

예상 출력:

```
level2: 30
level1: 10
```

왜 level1의 x는 변경되지 않았을까?

level3의 렉시컬 외부 함수 스코프를 살펴보자.



level3에서 nonlocal x를 선언하면 현재 함수의 지역 스코프를 제외하고,

렉시컬 외부 함수 스코프에서 x의 기존 바인딩을 찾는다.

가장 가까운 외부 함수인 level2에 x가 존재하므로 그 바인딩을 사용한다.

따라서 level2의 x가 30으로 변경된다.

반면 level1의 x는 그대로 10이다.

주의할 점

nonlocal은 특정 외부 함수를 이름으로 지정하는 문법을 제공하지 않는다.

따라서 다음처럼 작성할 수는 없다.

```
nonlocal level1.x
```

이는 유효한 Python 문법이 아니다.

nonlocal은 점 표기법을 통한 속성 탐색이 아니라 렉시컬 외부 함수 스코프의 이름 바인딩을 선택하는 선언이기 때문이다.

A.5.13 중간 함수에서 이름을 바인딩하지 않는 경우

다음 코드를 살펴보자.

```
def level1():
    x = 10

    def level2():

        def level3():
            nonlocal x
            x = 30

        level3()

    level2()

    return x


print(level1())
```

예상 출력:

30

이번에는 level2에 x 바인딩이 없다.

따라서 level3의 nonlocal x는 더 바깥쪽 함수인 level1의 바인딩을 대상으로 한다.

여기서 중요한 사실이 있다.

nonlocal은 반드시 바로 바깥 함수에만 접근하는 선언이 아니다.

렉시컬 외부 함수 스코프들을 탐색하여 가장 가까운 적절한 기존 바인딩을 사용한다.

이 원리는 A.3에서 학습한 Enclosing Scope와 연결된다.

A.5.14 nonlocal의 제약과 예외

nonlocal은 아무 이름에나 사용할 수 있는 선언이 아니다.

대표적인 제약을 살펴보자.

1. 외부 함수 스코프에 바인딩이 존재해야 한다

```
def outer():

    def inner():
        nonlocal x
        x = 10

    return inner
```

이 코드는 SyntaxError를 발생시킨다.

왜냐하면 inner를 감싸는 외부 함수 스코프에 x라는 바인딩이 존재하지 않기 때문이다.

nonlocal은 새로운 외부 바인딩을 생성하기 위한 선언이 아니다.

기존 렉시컬 외부 함수 바인딩을 대상으로 하는 선언이다.

2. 모듈 전역 이름만으로는 충분하지 않다

```
x = 10

def func():
    nonlocal x
    x = 20
```

이 코드 역시 SyntaxError를 발생시킨다.

모듈 전역의 x는 nonlocal의 대상이 아니기 때문이다.

전역 이름을 재바인딩하려면 global을 사용해야 한다.

```
x = 10

def func():
    global x
    x = 20


func()

print(x)
```

예상 출력:

```
20
```

3. 함수 내부에서 사용한 뒤 선언할 수 없다

```
def outer():
    x = 10

    def inner():
        print(x)
        nonlocal x
```

이 코드는 SyntaxError를 발생시킨다.

같은 코드 블록에서 nonlocal x 선언보다 앞서 x를 사용했기 때문이다.

nonlocal은 일반적인 실행 명령이 아니라 해당 코드 블록의 이름 분류에 영향을 주는 선언이므로, 이러한 제약이 존재한다.

4. 외부 바인딩이 아직 객체를 참조하지 않을 수도 있다

다음 코드를 보자.

```
def outer():

    def inner():
        nonlocal x
        return x

    result = inner()

    x = 10

    return result


outer()
```

이 코드는 컴파일 자체는 가능하다.

왜냐하면 `outer` 내부에 `x = 10`이라는 바인딩 구문이 존재하기 때문이다.

하지만 실제 실행에서는 `NameError`가 발생한다.

`inner()`를 호출하는 시점에 `outer`의 `x = 10`이 아직 실행되지 않았기 때문이다.

이 차이는 A.3에서 배운 내용과 동일하다.

이름의 정적 분류가 가능하다는 사실과 실행 시점에 객체가 바인딩되어 있다는 사실은 서로 다르다.

A.5.15 `global`과 `nonlocal`의 객체 모델 비교

이번에는 두 선언을 같은 코드에서 비교해 보자.

```
x = 10


def outer():
    x = 20

    def change_global():
        global x
        x = 100

    def change_nonlocal():
        nonlocal x
        x = 200
```

```
    change_global()
    change_nonlocal()

    return x


result = outer()

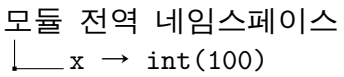
print("global:", x)
print("outer:", result)
```

예상 출력:

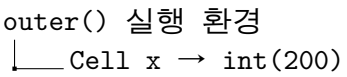
```
global: 100
outer: 200
```

두 선언은 서로 다른 바인딩을 대상으로 한다.

```
global x
```




```
nonlocal x
```



여기서 다음 구분이 중요하다.

비교 기준	global	nonlocal
대상	모듈 전역 바인딩	렉시컬 외부 함수 바인딩
기존 바인딩 필요	선언 시 불필요	외부 함수 스코프에 바인딩 구문 필요
대표적인 접근 구조	전역 네임스페이스	클로저 셀
선언의 목적	전역 이름의 바인딩 지정	외부 함수 이름의 바인딩 지정
객체 자체를 변경하는가?	선언 자체는 변경하지 않음	선언 자체는 변경하지 않음

또한 global과 nonlocal은 서로 대체 가능한 문법이 아니다.

어떤 스코프의 이름을 변경하려는지에 따라 선택해야 한다.

A.5.16 클로저를 이용한 상태 관리 - Counter

이제 지금까지 학습한 개념을 하나의 함수 생성기에 적용해 보자.

```
def make_counter(initial=0):
    count = initial

    def increment(step=1):
        nonlocal count

        count += step

        return count

    def get():
        return count

    def reset():
        nonlocal count

        count = initial

    return increment, get, reset


increment, get, reset = make_counter(10)

print(get())

print(increment())
print(increment(5))

reset()

print(get())
```

예상 출력:

```
10
11
16
10
```

이 코드를 객체 관계로 분석해 보자.

1. make_counter(10) 호출

매개변수 initial에 정수 객체 10이 바인딩된다.

다음 문장이 실행된다.

```
count = initial
```

count 역시 정수 객체 10을 참조한다.

여기서 initial과 count는 서로 다른 이름이지만 처음에는 동일한 정수 객체를 참조한다.

2. 내부 함수 객체 생성

세 개의 함수 객체가 생성된다.

```
increment
get
reset
```

각 함수는 필요한 외부 바인딩에 접근하기 위한 클로저 환경을 가진다.

여기서 중요한 점이 있다.

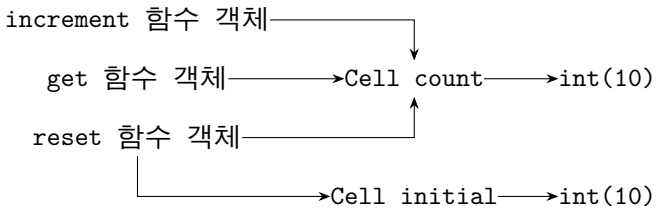
increment와 get은 count를 참조한다.

reset은 count와 initial을 참조한다.

따라서 count에 대응하는 셀은 세 함수가 공유한다.

반면 initial에 대응하는 셀은 reset에서만 필요하다.

이를 구조적으로 나타내면 다음과 같다.



3. increment() 호출

```
count += step
```

count는 외부 함수의 바인딩이다.

nonlocal count 선언에 따라 기존 값을 읽고 새로운 계산 결과를 다시 그 바인딩에 연결한다.

따라서 count는 11을 참조하게 된다.

4. increment(5) 호출

이번에는 step이 5를 참조한다.

기존 count는 11이므로 계산 결과는 16이다.

이후 count가 16에 재바인딩된다.

5. reset() 호출

```
count = initial
```

count는 nonlocal 선언에 의해 외부 바인딩을 대상으로 한다.

initial은 외부 함수의 자유 변수로 읽힌다.

따라서 count가 다시 정수 객체 10을 참조하게 된다.

여기서 새로운 make_counter() 호출이 발생하는 것은 아니다.

기존 클로저 환경의 바인딩을 변경하는 것이다.

A.5.17 __closure__를 이용한 상태 공유 검증

앞의 예제에서 실제로 같은 셀을 공유하는지 확인해 보자.

```
def make_counter(initial=0):
    count = initial

    def increment(step=1):
        nonlocal count
        count += step
        return count

    def get():
```

```
        return count

    def reset():
        nonlocal count
        count = initial

    return increment, get, reset


increment, get, reset = make_counter(10)


def closure_map(func):
    if func.__closure__ is None:
        return {}

    return dict(
        zip(
            func.__code__.co_freevars,
            func.__closure__
        )
    )


increment_cells = closure_map(increment)
get_cells = closure_map(get)
reset_cells = closure_map(reset)

print(increment_cells.keys())
print(get_cells.keys())
print(reset_cells.keys())

print(
    increment_cells["count"]
    is get_cells["count"]
)

print(
    get_cells["count"]
    is reset_cells["count"]
)

print(
    reset_cells["initial"].cell_contents
)
```

예상 출력:

```
dict_keys(['count'])
```

```
dict_keys(['count'])
dict_keys(['count', 'initial'])
True
True
10
```

이 예제에서 `closure_map()`은 함수 객체의 자유 변수 이름과 셀 객체를 대응시키는 보조 함수다.

여기서 다음 사실을 직접 확인할 수 있다.

- `increment`, `get`, `reset`은 모두 `count` 셀을 공유한다.
- `reset`은 추가로 `initial` 셀을 참조한다.
- `initial`과 `count`는 처음에 같은 정수 객체를 참조하더라도 서로 다른 바인딩이다.

이제 다음 비교를 생각해 보자.

```
reset_cells["initial"] is reset_cells["count"]
```

예상 결과:

```
False
```

왜냐하면 두 이름은 서로 다른 바인딩이기 때문이다.

처음에 동일한 정수 객체를 참조한다는 사실만으로 같은 셀 객체를 사용해야 하는 것은 아니다.

이것은 A.1에서 배운 객체의 동일성과 이름 바인딩의 동일성을 구분해야 한다는 원리가 클로저에도 적용된 사례다.

A.5.18 서로 다른 Counter 인스턴스의 상태 독립성

이번에는 함수 생성기를 두 번 호출해 보자.

```
def make_counter():
    count = 0

    def increment():
```

```
        nonlocal count
        count += 1
        return count

    return increment


counter_a = make_counter()
counter_b = make_counter()

print(counter_a())
print(counter_a())

print(counter_b())

print(
    counter_a.__closure__[0]
    is counter_b.__closure__[0]
)
```

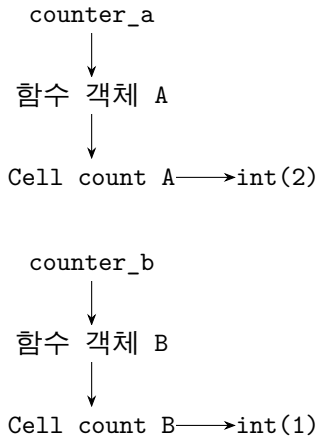
예상 출력:

```
1
2
1
False
```

여기서 counter_a와 counter_b는 서로 다른 함수 객체다.

그리고 각각 다른 make_counter() 호출에서 생성되었다.

따라서 서로 독립적인 count 바인딩을 가진다.



이 구조는 클래스의 서로 다른 인스턴스가 독립적인 상태를 가지는 것과 유사하다.

다만 상태가 저장되는 위치는 다르다.

클래스에서는 일반적으로 인스턴스 속성을 통해 상태를 관리하지만, 여기서는 함수 객체가 참조하는 클로저 셀을 통해 상태를 관리하는 것이다.

A.5.19 클래스 인스턴스와 클로저의 상태 관리 비교

다음 두 구현을 비교해 보자.

클로저 기반 구현

```
def make_counter():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count

    return increment
```

클래스 기반 구현

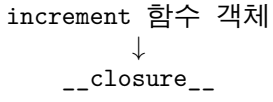
```
class Counter:
    def __init__(self):
        self.count = 0

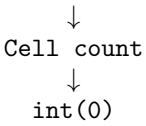
    def increment(self):
        self.count += 1
        return self.count
```

두 구현 모두 호출 사이에 상태를 유지할 수 있다.

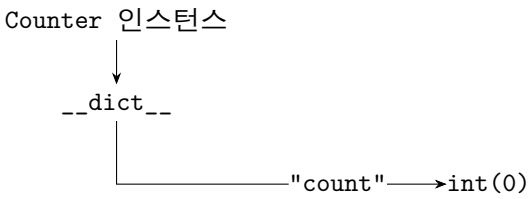
하지만 상태에 접근하는 메커니즘은 다르다.

클로저 기반





클래스 기반



일반적인 사용자 정의 클래스에서 `self.count`는 인스턴스 속성 탐색과 대입을 통해 관리된다.

반면 클로저의 `count`는 자유 변수 바인딩을 통해 접근한다.

비교 기준	클로저	클래스
상태 접근	자유 변수	인스턴스 속성
상태 저장	셀과 참조 객체	인스턴스 네임스페이스 등
상태 재바인딩	`nonlocal`	`self.attribute = ...`
여러 동작의 상태 공유	같은 셀을 공유하는 함수들	같은 인스턴스를 참조하는 메서드들
독립적인 상태 생성	외부 함수 재호출	인스턴스 생성
대표적인 활용	함수 생성기, 데코레이터	명시적인 상태 객체, 복잡한 인터페이스

여기서 중요한 것은 클로저가 클래스를 대체하는 문법이라는 뜻이 아니다.

둘은 서로 다른 객체 구조를 통해 상태를 관리하는 방법이다.

문제의 규모와 필요한 인터페이스에 따라 적합한 방식이 달라진다.

A.5.20 실전 응용 - 데코레이터의 호출 횟수 관리

이제 클로저 상태 변경을 데코레이터와 연결해 보자.

다음 코드를 살펴보자.

```
def count_calls(func):
    count = 0

    def wrapper(*args, **kwargs):
        nonlocal count

        count += 1

        print(f"호출 횟수: {count}")

        return func(*args, **kwargs)

    return wrapper


def add(a, b):
    return a + b


add = count_calls(add)

print(add(10, 20))
print(add(30, 40))
```

예상 출력:

```
호출 횟수: 1
30
호출 횟수: 2
70
```

이 코드는 이후 A.6과 A.7에서 자세히 학습할 함수 기반 데코레이터의 기본 구조다.

이번에는 클로저 상태에 집중해서 분석해 보자.

1. 원본 함수 객체 생성

```
def add(a, b):
    return a + b
```

원본 add 함수 객체가 생성된다.

2. count_calls(add) 호출

```
add = count_calls(add)
```

원본 함수 객체가 count_calls의 매개변수 func에 바인딩된다.

그리고 다음 문장이 실행된다.

```
count = 0
```

호출 횟수를 나타내는 지역 바인딩이 생성된다.

3. wrapper 함수 객체 생성

```
def wrapper(*args, **kwargs):
    nonlocal count

    count += 1

    print(f"호출 횟수: {count}")

    return func(*args, **kwargs)
```

wrapper는 외부 함수의 두 이름을 사용한다.

- count

- func

따라서 이 두 이름은 wrapper의 자유 변수다.

count는 nonlocal 선언을 통해 재바인딩 대상이 되고,

func는 원본 함수 객체를 호출하기 위해 읽는 이름이다.

4. wrapper 함수 객체 반환

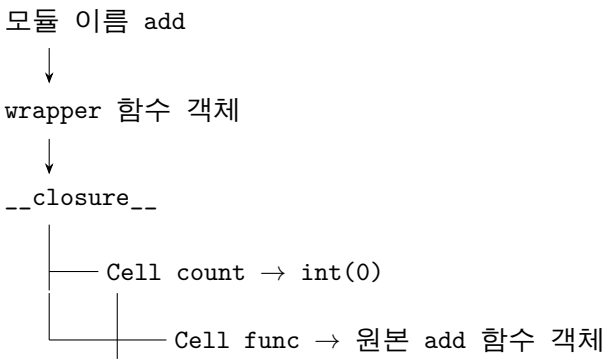
count_calls()는 wrapper 함수 객체를 반환한다.

그리고 모듈 이름 add가 반환된 함수 객체에 재바인딩된다.

이후 count_calls() 호출은 종료된다.

하지만 반환된 wrapper 함수 객체는 클로저를 통해 count와 func의 바인딩에 접근할 수 있다.

객체 구조



5. 첫 번째 add(10, 20) 호출

현재 모듈 이름 add는 wrapper 함수 객체를 참조한다.

따라서 wrapper가 실행된다.

```
count += 1
```

count가 1로 재바인딩된다.

이후 원본 함수 객체를 호출한다.

```
func(*args, **kwargs)
```

원본 add는 30을 반환한다.

그 결과 wrapper도 30을 반환한다.

6. 두 번째 add(30, 40) 호출

이번에도 같은 wrapper 함수 객체가 호출된다.

따라서 같은 클로저 셀을 사용한다.

기존 count가 1이므로 count += 1 실행 후 2가 된다.

이후 원본 함수 객체를 호출하고 70을 반환한다.

여기서 핵심은 다음과 같다.

데코레이터가 호출 횟수를 유지하는 이유는 함수 객체가

호출 사이에 동일한 클로저 셀을 계속 참조하기 때문이다.

A.5.21 nonlocal 상태 관리에서 주의할 점

클로저를 사용하면 상태를 간단하게 캡슐화할 수 있다.

하지만 몇 가지 경계 조건을 이해해야 한다.

1. 클로저 상태는 자동으로 불변이 되지 않는다

함수 객체의 `__closure__`에 포함된 셀은 외부 바인딩에 접근하기 위한 구조다.

그 셀이 참조하는 객체가 가변 객체라면 내부 상태를 변경할 수 있다.

또한 `nonlocal`을 사용하면 바인딩 자체를 변경할 수도 있다.

따라서 클로저를 사용한다는 사실만으로 상태가 불변이거나 변경으로부터 보호된다고 볼 수는 없다.

2. 상태 변경은 자동으로 스레드 안전하지 않다

다음 연산을 생각해 보자.

```
count += 1
```

이 연산은 기존 값을 읽고, 계산하고, 결과를 다시 바인딩하는 과정을 포함한다.

여러 실행 흐름이 동일한 클로저 상태를 동시에 변경한다면 경쟁 상태(Race Condition)가 발생할 수 있다.

따라서 클로저를 사용한다는 이유만으로 동시성 안전성이 보장되는 것은 아니다.

3. 외부 바인딩을 유지하면 객체의 수명도 영향을 받을 수 있다

클로저 셀이 큰 객체를 참조하고 있다면, 함수 객체가 유지되는 동안 그 객체도 계속 참조될 수 있다.

따라서 클로저에 불필요하게 큰 데이터를 캡처하면 메모리 사용에 영향을 줄 수 있다.

이 문제는 장기간 유지되는 콜백 함수나 데코레이터를 설계할 때 고려해야 한다.

`nonlocal`은 객체의 속성 탐색과 무관하다

다음 표현식을 비교해 보자.

```
nonlocal state
state = new_state


state.value = 10
```

첫 번째는 외부 이름 state의 바인딩을 변경한다.

두 번째는 state가 참조하는 객체의 속성에 대입한다.

따라서 두 연산은 서로 다른 메커니즘이다.

객체의 속성 변경에는 일반적으로 nonlocal이 필요하지 않다.

A.5.22 종합 예제 - 여러 클로저가 하나의 상태를 관리하는 구조

이번 학습의 내용을 하나의 프로그램으로 통합해 보자.

```
def make_account(initial_balance):
    balance = initial_balance

    def deposit(amount):
        nonlocal balance

        balance += amount

        return balance

    def withdraw(amount):
        nonlocal balance

        if amount > balance:
            raise ValueError("잔액이 부족합니다.")

        balance -= amount

        return balance

    def get_balance():
        return balance

    return deposit, withdraw, get_balance
```

```
deposit, withdraw, get_balance = make_account(1000)

print(get_balance())

print(deposit(500))

print(withdraw(300))

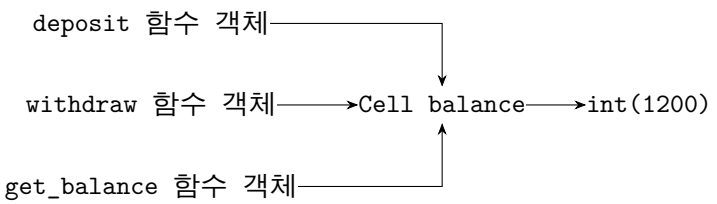
print(get_balance())
```

예상 출력:

```
1000
1500
1200
1200
```

이 코드는 세 개의 내부 함수가 하나의 상태를 공유하는 구조다.

객체 관계



여기서 `balance`는 세 함수의 자유 변수다.

`deposit`과 `withdraw`는 `nonlocal balance`를 선언했으므로 외부 바인딩을 변경할 수 있다.

반면 `get_balance`는 값을 읽기만 하므로 `nonlocal`이 필요하지 않다.

이 구조를 보면 다음 사실을 알 수 있다.

여러 함수 객체가 동일한 셀을 공유하면, 하나의 함수가 변경한 바인딩을 다른 함수도 관찰할 수 있다.

이는 클래스 인스턴스에서 여러 메서드가 동일한 인스턴스 상태를 읽고 변경하는 구조와 대응된다.

다만 이 예제는 클로저 상태 관리의 원리를 보여주기 위한 것이며, 실제 금융 거래 시스템에 필요한 동시성 제어와 영속성 등을 구현한 것은 아니다.

A.5.23 핵심 정리와 판단 기준

이번 학습에서 확립해야 할 핵심 개념을 정리해 보자.

1. nonlocal은 이름의 바인딩 대상을 지정한다

```
nonlocal x
```

는 현재 함수에서 x를 지역 이름으로 취급하지 않고 렉시컬 외부 함수의 기존 바인딩을 사용하도록 지정한다.

2. 셀 객체와 셀이 참조하는 객체는 구별해야 한다

```
cell is other_cell
```

은 셀 객체의 동일성을 비교한다.

반면

```
cell.cell_contents
```

는 셀에 현재 바인딩된 객체를 조회한다.

따라서 같은 셀 객체가 시간에 따라 서로 다른 객체를 참조할 수 있다.

3. 객체 변경과 이름 재바인딩은 다르다

```
values.append(10)
```

은 리스트 객체의 상태를 변경한다.

반면

```
values = [10]
```

은 이름의 바인딩을 변경한다.

외부 함수의 바인딩을 변경하려면 nonlocal이 필요하다.

4. 증강 대입은 이름의 스코프 판정에 영향을 준다

```
count += 1
```

은 이름에 대한 대입을 포함한다.

따라서 외부 함수의 count를 대상으로 하려면 nonlocal count가 필요하다.

가변 객체에 대한 +=도 이 이름 분류 규칙에서 예외가 아니다.

5. 여러 클로저는 동일한 셀을 공유할 수 있다

같은 외부 함수 호출에서 생성된 여러 내부 함수가 동일한 외부 바인딩을 참조한다면 같은 셀을 공유할 수 있다.

따라서 한 함수의 재바인딩이 다른 함수의 반환값에 영향을 줄 수 있다.

6. 서로 다른 외부 함수 호출은 독립적인 상태를 만들 수 있다

```
counter_a = make_counter()
counter_b = make_counter()
```

두 호출에서 생성된 클로저는 서로 다른 바인딩 환경을 가질 수 있다.

따라서 독립적인 상태 관리가 가능하다.

최종 판단 기준

질문	판단 기준
nonlocal이 필요한가?	외부 함수의 이름 자체를 재바인딩하는가?
가변 객체 변경에 nonlocal이 필요한가?	객체 내부만 변경한다면 일반적으로 필요하지 않음
+=에 nonlocal이 필요한가?	외부 이름을 대상으로 하는 증강 대입이라면 필요
여러 함수가 상태를 공유하는가?	동일한 셀 또는 동일한 가변 객체를 참조하는지 확인
상태 변경 후 셀 객체도 바뀌는가?	일반적인 nonlocal 재바인딩에서는 기존 셀을 유지
global과 nonlocal 중 무엇을 사용하는가?	모듈 전역 바인딩인지 렉시컬 외부 함수 바인딩인지 구분
UnboundLocalError가 발생하는가?	지역 이름으로 판정되었지만 읽는 시점에 바인딩되지 않았는지 확인
SyntaxError가 발생하는가?	nonlocal의 대상 바인딩이 외부 함수 스코프에 존재하는지 확인

A.6 데코레이터의 정의와 @decorator 변환식

Decorator · Function Object · Name Rebinding · Closure · Definition Time · Call Time

A.6.1 이전 단원과의 연결

A.1부터 A.5까지 학습한 내용은 데코레이터(Decorator)를 이해하기 위한 기반이었다.

단원	학습한 원리	데코레이터와의 연결
A.1	함수도 객체이며 이름에 바인딩된다.	함수 객체를 인수로 전달하고 다른 함수 객체를 반환할 수 있다.
A.2	중첩 함수는 정의 위치에 따른 렉시컬 스코프를 가진다.	데코레이터 내부에서 새로운 함수를 정의할 수 있다.
A.3	이름의 스코프는 정적으로 판정된다.	내부 함수는 외부 매개변수를 자유 변수로 참조할 수 있다.
A.4	클로저는 외부 바인딩에 접근하는 환경을 유지한다.	반환된 함수가 원본 함수 객체를 계속 참조할 수 있다.
A.5	nonlocal은 외부 바인딩의 재바인딩을 허용한다.	데코레이터가 호출 횟수 등의 상태를 유지할 수 있다.

이제 이 개념들을 하나의 실행 구조로 통합할 것이다.

다음 코드를 살펴보자.

```
def decorator(func):

    def wrapper():
        print("Before")
        result = func()
        print("After")
        return result

    return wrapper


@decorator
```

```
def greet():
    print("Hello")
    return 10


print(greet())
```

예상 출력:

```
Before
Hello
After
10
```

표면적으로는 @decorator라는 문법이 greet()의 실행 전후에 기능을 추가한 것처럼 보인다.

하지만 Python의 객체 모델에서는 훨씬 구체적인 과정이 발생한다.

1. 원본 greet 함수 객체가 생성된다.
2. 그 함수 객체가 decorator에 전달된다.
3. decorator가 새로운 wrapper 함수를 생성한다.
4. wrapper는 클로저를 통해 원본 함수 객체를 참조한다.
5. decorator가 반환한 wrapper 함수 객체가
   greet 이라는 이름에 바인딩된다.
6. 이후 greet()을 호출하면 원본 함수가 아니라
   wrapper가 먼저 실행된다.

이번 학습에서 가장 중요한 질문은 다음과 같다.

> @decorator는 기존 함수 객체를 수정하는 문법일까, 아니면
>
> 함수 객체를 전달하고 반환값으로 이름을 다시 바인딩하는 문법일까?

정답은 일반적으로 두 번째다.

다만 데코레이터가 반드시 새로운 함수를 반환해야 하는 것은 아니라는 점까지 정확하게 이해해야 한다.

A.6.2 데코레이터의 정확한 정의

Python에서 데코레이터는 함수나 클래스를 정의할 때 적용하는 호출 가능한 객체(Callable)다.

함수 데코레이터의 기본 구조는 다음과 같다.

```
def decorator(func):
    # func를 이용한 처리
    return replacement
```

여기서 `func`는 데코레이션 대상인 함수 객체를 참조하고,

`replacement`는 그 이름에 최종적으로 바인딩될 객체다.

일반적인 함수 기반 데코레이터에서는 `replacement`가 새로운 함수 객체인 경우가 많다.

하지만 Python 문법이 반드시 새로운 함수 객체를 반환하도록 강제하지는 않는다.

핵심은 다음 변환식이다.

데코레이터의 의미론적 변환 - 데코레이터 문법

```
@decorator
def greet():
    return 10
```

데코레이터의 의미론적 변환 - 대응하는 일반 함수 호출

```
def greet():
    return 10

greet = decorator(greet)
```

두 코드는 일반적인 단일 데코레이터 상황에서 같은 의미를 가진다.

여기서 다음 표현식을 정확하게 분석해야 한다.

```
greet = decorator(greet)
```

오른쪽의 `greet`은 함수 객체를 참조하는 이름이다.

`decorator(greet)`은 그 함수 객체를 인수로 전달하는 호출 표현식이다.

그리고 호출 결과가 왼쪽 이름 `greet`에 다시 바인딩된다.

따라서 데코레이터의 본질은 다음과 같이 정리할 수 있다.

> 데코레이터는 대상 함수 객체를 받아 어떤 객체를 반환하며,
>
> 데코레이터 문법은 그 반환 객체를
>
> 원래 정의 대상의 이름에 바인딩하는 구조다.

여기서 데코레이터의 기능과 `@` 문법의 역할을 구분해야 한다.

- 데코레이터가 무엇을 반환할지는 데코레이터 구현에 따라 결정된다.
- `@` 문법은 데코레이터를 적용하고 그 결과를 정의 대상의 이름에 바인딩하는 역할을 한다.

A.6.3 데코레이터를 사용하지 않고 먼저 구현해 보자

`@` 문법을 잠시 제외하고, 일반적인 함수 객체 전달과 반환만으로 동일한 구조를 만들어 보자.

```
def decorator(func):
    print("decorator 실행")

    def wrapper():
        print("Before")

        result = func()

        print("After")

        return result

    return wrapper


def greet():
    print("Hello")
    return 10
```

```
original = greet

greet = decorator(greet)

print("정의 및 데코레이션 완료")

result = greet()

print("반환값:", result)
print("같은 함수 객체인가?", original is greet)
```

예상 결과:

```
decorator 실행
정의 및 데코레이션 완료
Before
Hello
After
반환값: 10
같은 함수 객체인가? False
```

여기서 original이라는 이름을 추가한 이유는 원본 함수 객체와 데코레이션 후 함수 객체를 비교하기 위해서다.

이 코드를 단계별로 분석해 보자.

1. decorator 함수 객체 생성

```
def decorator(func):
    ...
```

모듈 코드에서 def decorator 문장이 실행되면 함수 객체가 생성되고

decorator라는 이름에 바인딩된다.

이 시점에는 decorator의 본문이 실행되지 않는다.

2. 원본 greet 함수 객체 생성

```
def greet():
    print("Hello")
    return 10
```

새로운 함수 객체가 생성되고 greet에 바인딩된다.

아직 greet()을 호출하지 않았으므로 "Hello"는 출력되지 않는다.

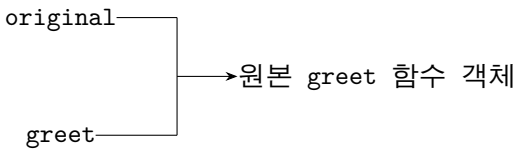
3. 원본 함수 객체의 참조 보존

```
original = greet
```

original이라는 이름이 원본 함수 객체를 참조하게 된다.

새로운 함수 객체가 생성되는 것은 아니다.

이 시점에는 다음 관계가 성립한다.



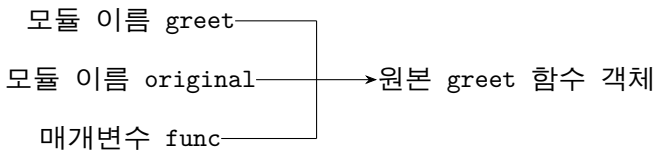
4. decorator(greet) 호출

```
greet = decorator(greet)
```

먼저 오른쪽 표현식을 평가한다.

decorator가 참조하는 함수 객체를 호출하고, 원본 greet 함수 객체를 인수로 전달한다.

따라서 decorator 호출의 지역 매개변수 func는 원본 함수 객체를 참조한다.



이제 decorator 본문이 실행되므로 다음 문장이 출력된다.

```
decorator 실행
```

5. wrapper 함수 객체 생성

다음 중첩 함수 정의문이 실행된다.

```
def wrapper():
```

```
    print("Before")

    result = func()

    print("After")

    return result
```

새로운 wrapper 함수 객체가 생성된다.

여기서 wrapper는 외부 함수 decorator의 매개변수 func를 참조한다.

따라서 func는 wrapper의 자유 변수(Free Variable)다.

이것이 A.4에서 학습한 클로저의 형성 조건과 연결된다.

6. wrapper 함수 객체 반환

```
return wrapper
```

decorator는 wrapper 함수 객체를 반환한다.

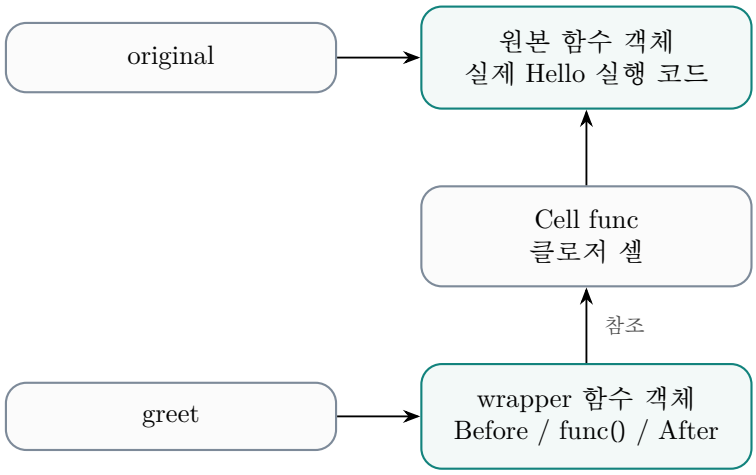
여기서 wrapper()를 호출한 것이 아니라, 함수 객체 자체를 반환했다는 점이 중요하다.

따라서 아직 "Before"와 "After"는 출력되지 않는다.

7. greet 이름 재바인딩

decorator(greet) 호출 결과로 반환된 wrapper 함수 객체가 왼쪽의 greet에 바인딩된다.

이제 객체 관계는 다음과 같다.



데코레이션 이후 객체 관계

greet은 wrapper를 참조하지만, 원본 함수 객체는 original과 wrapper의 클로저를 통해 계속 참조된다.

따라서 다음 비교는 `False`다.

```
original is greet
```

두 이름이 서로 다른 함수 객체를 참조하기 때문이다.

8. greet() 호출

마지막으로 다음 문장이 실행된다.

```
result = greet()
```

현재 `greet`은 `wrapper` 함수 객체를 참조하고 있다.

따라서 `wrapper`가 먼저 실행된다.

`wrapper` 내부에서 `func()`를 호출할 때 비로소 원본 함수 객체가 실행되는 것이다.

이것이 데코레이터의 기본 호출 구조다.

A.6.4 @decorator는 언제 실행될까?

이제 일반 함수 호출을 @ 문법으로 변경해 보자.

```
def decorator(func):
    print("decorating:", func.__name__)

    def wrapper():
        print("Before")
        result = func()
        print("After")
        return result

    return wrapper


@decorator
def greet():
    print("Hello")
    return 10


print("정의문 실행 완료")

print(greet())
```

예상 출력:

```
decorating: greet
정의문 실행 완료
Before
Hello
After
10
```

이 출력 순서는 데코레이터의 실행 시점을 설명한다.

여기에는 세 가지 시점이 존재한다.

1. 함수 정의문 실행 시점
   원본 함수 객체가 생성되고 데코레이터가 적용된다.

2. 이름 재바인딩 시점
   데코레이터 반환값이 greet에 바인딩된다.

3. greet() 호출 시점
   현재 greet에 바인딩된 wrapper가 실행되고 원본 함수를 호출한다.

여기서 다음 두 문장을 구별해야 한다.

`@decorator`가 적용되는 시점은 함수 정의문이 실행되는 시점이다.

`wrapper` 함수 본문이 실행되는 시점은 일반적으로 데코레이션 이후 해당 함수 객체가 호출되는 시점이다.

두 시점은 다르다.

또한 여기서 말하는 함수 정의 시점은 반드시 모듈을 처음 불러오는 시점만 의미하지 않는다.

예를 들어 중첩 함수에 데코레이터가 붙어 있다면, 그 중첩 함수의 `def` 문장이 실행될 때마다 데코레이터가 적용될 수 있다.

```
def decorator(func):
    print("decorating")

    def wrapper():
        return func()

    return wrapper


def outer():

    @decorator
    def inner():
        return 10

    return inner


f1 = outer()
f2 = outer()
```

예상 출력:

```
decorating
decorating
```

`outer()`를 두 번 호출했기 때문에 내부의 `def inner` 문장도 두 번 실행된다.

따라서 서로 다른 `inner` 함수 객체가 생성되고, 데코레이터도 각각 적용되는 것이다.

여기서 정확한 기준은 소스 코드가 어디에 적혀 있는가가 아니라 해당 정의문

이 언제 실행되는가라는 사실이다.

A.6.5 함수 정의, 데코레이션, 함수 호출을 구별하기

세 시점을 더 명확하게 확인해 보자.

```
events = []


def decorator(func):
    events.append("2. decorator 호출")

    def wrapper():
        events.append("4. wrapper 호출")

        result = func()

        events.append("6. wrapper 반환")

        return result

    return wrapper


events.append("1. 정의문 실행 전")


@decorator
def greet():
    events.append("5. 원본 greet 호출")
    return "Hello"


events.append("3. 정의문 실행 후")

print(events)

result = greet()

print(result)
print(events)
```

예상 출력:

```
['1. 정의문 실행 전', '2. decorator 호출', '3. 정의문 실행 후']
Hello
['1. 정의문 실행 전', '2. decorator 호출', '3. 정의문 실행 후',
```

```
'4. wrapper 호출', '5. 원본 greet 호출', '6. wrapper 반환']
```

이 코드는 다음 사실을 확인해 준다.

greet()을 호출하기 전에도 decorator는 이미 실행되었다.

하지만 wrapper 본문과 원본 greet 본문은 아직 실행되지 않았다.

이것은 다음 세 호출을 혼동하면 안 된다는 의미다.

표현식	실제 의미
decorator(greet)	원본 함수 객체를 인수로 전달하여 데코레이터 호출
greet()	현재 greet에 바인딩된 객체 호출
func()	wrapper 내부에서 자유 변수 func가 참조하는 원본 함수 호출

데코레이터가 함수 객체를 인수로 받는 것과 그 함수 객체를 호출하는 것은 전혀 다른 연산이다.

한 가지 경계 조건도 기억해야 한다.

데코레이터가 원본 함수를 자동으로 호출하는 것은 아니지만, 데코레이터 구현 안에 명시적으로 func()가 있다면 데코레이션 도중 원본 함수가 실행될 수도 있다.

따라서 원본 함수가 언제 실행되는지는 실제 데코레이터 구현을 확인해야 한다.

A.6.6 func와 func() - 가장 중요한 구분

다음 두 코드는 같은 의미가 아니다.

```
greet = decorator(greet)
```

```
greet = decorator(greet())
```

첫 번째 코드에서는 오른쪽의 greet이 함수 객체를 참조한다.

따라서 데코레이터의 매개변수 func에는 함수 객체가 전달된다.

반면 두 번째 코드에서는 먼저 greet()이 실행된다.

그다음 그 반환값이 decorator에 전달된다.

실제 차이를 확인해 보자.

```
def decorator(func):
    print("전달받은 타입:", type(func).__name__)

    def wrapper():
        return func()

    return wrapper


def greet():
    return 10


greet = decorator(greet())
```

예상 출력:

```
전달받은 타입: int
```

여기서 중요한 점이 있다.

이 코드가 반드시 decorator(greet()) 호출 시점에 오류를 발생시키는 것은 아니다.

위 예제의 decorator는 전달받은 객체가 호출 가능한지 즉시 검사하지 않는다.

단순히 그 객체를 클로저에 보관하는 wrapper를 생성해서 반환한다.

따라서 이후 다음 문장을 실행할 때

```
greet()
```

wrapper 내부의 func()에서 오류가 발생한다.

```
TypeError: 'int' object is not callable
```

왜냐하면 func가 함수 객체가 아니라 정수 객체 10을 참조하고 있기 때문이다.

이를 표로 정리하면 다음과 같다.

표현식	데코레이터에 전달되는 객체
`decorator(greet)`	`greet`가 참조하는 함수 객체
`decorator(greet())`	`greet()` 호출의 반환 객체
`decorator(lambda: 10)`	새로 생성된 함수 객체
`decorator(10)`	정수 객체 `10`

데코레이터의 매개변수 이름이 `func`라고 해서 Python이 자동으로 함수 타입을 보장하는 것은 아니다.

전달받은 객체를 어떻게 사용할지는 데코레이터 구현에 달려 있다.

A.6.7 원본 함수 객체는 어디에 보관될까?

이제 A.4와 A.5에서 학습한 클로저 구조를 직접 연결해 보자.

다음 코드를 실행해 보자.

```
def decorator(func):

    def wrapper():
        return func()

    return wrapper


@decorator
def greet():
    return 10


print(greet.__name__)
print(greet.__code__.co_freevars)
print(greet.__closure__[0].cell_contents.__name__)
```

예상 출력:

```
wrapper
('func',)
greet
```

이 출력은 세 가지 사실을 보여준다.

1. 현재 greet은 wrapper 함수 객체를 참조한다

```
print(greet.__name__)
```

결과는 "wrapper"다.

왜냐하면 greet이라는 이름이 데코레이터의 반환값인 wrapper 함수 객체에 재바인딩되었기 때문이다.

greet이라는 이름이 붙어 있다고 해서 그 이름이 원본 함수 객체를 참조한다는 뜻은 아니다.

2. wrapper는 func를 자유 변수로 사용한다

```
print(greet.__code__.co_freevars)
```

결과는 다음과 같다.

```
('func',)
```

wrapper 본문은 외부 함수 decorator의 매개변수 func를 참조한다.

따라서 func는 자유 변수다.

3. 클로저 셀은 원본 함수 객체를 참조한다

```
greet.__closure__[0].cell_contents
```

이 표현식은 wrapper의 클로저 셀에 바인딩된 객체를 반환한다.

이 예제에서는 그 객체가 원본 greet 함수 객체다.

따라서 다음 코드는 False를 출력한다.

```
original = greet.__closure__[0].cell_contents

print(original is greet)
print(original.__name__)
```

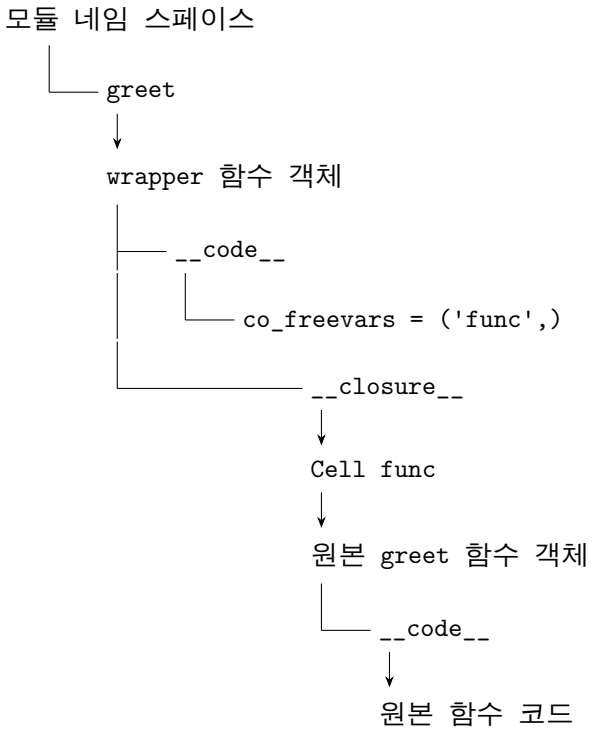
예상 출력:

```
False
```

greet

현재 greet이 참조하는 함수 객체와 original이 참조하는 함수 객체가 서로 다르기 때문이다.

객체 관계 정리



여기서 A.4에서 학습한 원리가 그대로 적용된다.

decorator() 호출은 종료되었지만, 반환된 wrapper 함수 객체가 클로저 셀을 참조하고 있기 때문에 원본 함수 객체에 계속 접근할 수 있는 것이다.

단, 모든 데코레이터가 원본 함수를 클로저에 보관하는 것은 아니다.

이것은 지금 작성한 wrapper 기반 데코레이터의 구조다.

A.6.8 데코레이터는 반드시 wrapper를 만들어야 할까?

그렇지 않다.

데코레이터는 대상 함수 객체를 전달받아 다른 객체를 반환하는 구조이지, 반드시 중첩 함수를 생성해야 하는 것은 아니다.

다음 예제를 살펴보자.

```
def identity(func):
    print("데코레이터 적용")
    return func


@identity
def greet():
    return "Hello"


print(greet())
print(greet.__name__)
print(greet.__closure__)
```

예상 출력:

```
데코레이터 적용
Hello
greet
None
```

이 데코레이터는 새로운 `wrapper`를 생성하지 않는다.

전달받은 원본 함수 객체를 그대로 반환한다.

따라서 데코레이션 후에도 `greet`은 원본 함수 객체를 참조한다.

이 예제에서는 외부 함수의 바인딩을 캡처하지 않으므로 `__closure__`도 `None`이다.

즉, 데코레이터 적용과 클로저 생성은 반드시 함께 발생하는 것이 아니다.

반환값이 함수가 아닌 경우

다음 코드도 문법적으로 가능하다.

```
def replace(func):
    return 100


@replace
def greet():
```

```
    return "Hello"


print(greet)
print(type(greet))
```

예상 출력:

```
100
<class 'int'>
```

여기서는 `greet`이 정수 객체 100에 바인딩된다.

따라서 이후 `greet()`을 호출하면 `TypeError`가 발생한다.

이 예제가 보여주는 원리는 중요하다.

데코레이터의 반환값이 반드시 함수 객체여야 한다는 언어 차원의 제약은 없다.

다만 데코레이션 이후에도 함수처럼 호출하려면 반환 객체가 호출 가능해야 한다.

실무에서 일반적인 함수 데코레이터가 `wrapper` 함수 객체를 반환하는 이유도 기존 함수의 호출 인터페이스를 유지하면서 동작을 확장하기 위해서다.

A.6.9 이름 재바인딩은 언제 완료될까?

앞에서는 데코레이터의 반환값이 원래 이름에 다시 바인딩된다고 설명했다.

그런데 그 바인딩은 데코레이터가 호출되기 전에 완료될까, 아니면 호출이 끝난 뒤 완료될까?

다음 코드를 살펴보자.

```
def greet():
    return "Old"


def decorator(func):
    print("데코레이션 중:", greet())
    return func
```

```
@decorator
def greet():
    return "New"


print("데코레이션 후:", greet())
```

예상 출력:

```
데코레이션 중: Old
데코레이션 후: New
```

왜 이런 결과가 발생할까?

처음에는 greet이라는 이름은 "Old"를 반환하는 함수 객체를 참조한다.

그 다음 새로운 greet 정의문이 실행된다.

이때 새로운 함수 객체가 생성되고 decorator에 전달된다.

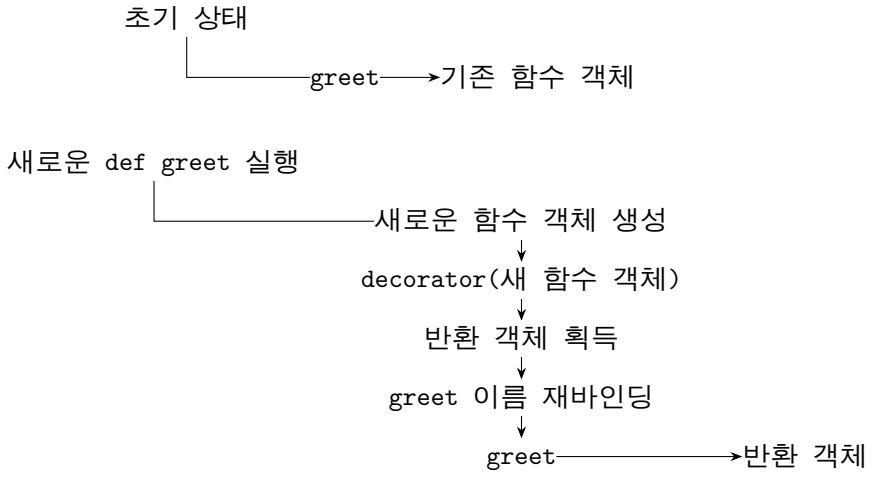
하지만 아직 새로운 함수 객체가 모듈 이름 greet에 최종 바인딩되기 전이다.

따라서 decorator 내부에서 전역 이름 greet을 조회하면 기존 함수 객체를 찾게 된다.

이후 decorator가 새로운 함수 객체를 반환하고, 그 반환값이 greet에 바인딩된다.

그래서 마지막 호출에서는 "New"가 반환되는 것이다.

이 과정을 나타내면 다음과 같다.



여기서 중요한 판단 기준은 다음이다.

데코레이터가 적용되는 동안 새로 정의한 함수 객체가 반드시 기존 이름에 먼저 바인딩되어 있는 것은 아니다.

최종 이름 바인딩은 데코레이터 적용 결과가 결정된 뒤 이루어진다.

A.6.10 데코레이터 적용과 원본 함수의 내부 이름 탐색

여기서 조금 더 깊이 생각해 보자.

데코레이션 후 원본 함수 객체는 클로저를 통해 유지될 수 있다.

그런데 원본 함수 본문에서 자기 자신의 이름을 참조한다면 어떤 일이 발생할까?

```
def decorator(func):

    def wrapper(n):
        print("wrapper:", n)
        return func(n)

    return wrapper


@decorator
def countdown(n):
    if n == 0:
        return 0

    return countdown(n - 1)


print(countdown(2))
```

예상 출력:

```
wrapper: 2
wrapper: 1
wrapper: 0
0
```

왜 `wrapper`가 세 번 실행될까?

원본 `countdown` 함수 본문에 다음 표현식이 있다.

```
countdown(n - 1)
```

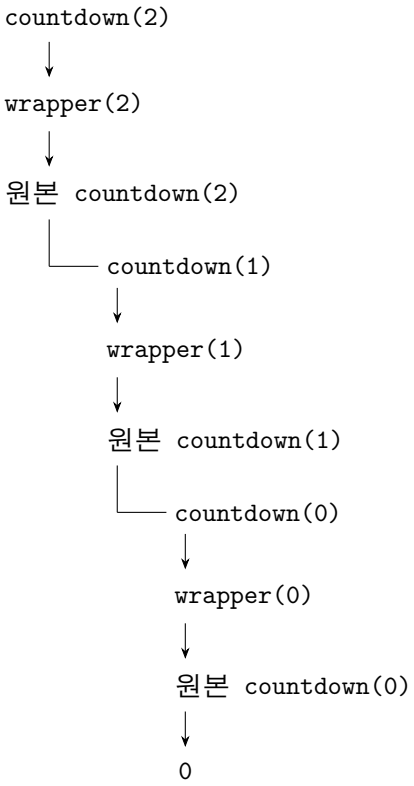
이 countdown은 함수 객체 자체를 직접 가리키는 특별한 자기 참조가 아니다.

일반적인 Python 이름 탐색 규칙에 따라 조회되는 전역 이름이다.

그런데 데코레이션 이후 모듈의 countdown은 원본 함수가 아니라 wrapper를 참조하고 있다.

따라서 원본 함수가 재귀 호출을 수행할 때도 wrapper를 다시 호출하게 되는 것이다.

실행 흐름을 살펴보자.



이 예제는 두 가지 원리를 연결한다.

첫째, 원본 함수 객체는 클로저에 의해 유지될 수 있다.

둘째, 원본 함수 본문에 작성된 전역 이름은 함수 정의 당시의 객체를 영구적으로 가리키는 것이 아니라, 실행 시점의 전역 바인딩을 조회한다.

따라서 데코레이터에 의한 이름 재바인딩은 원본 함수 내부의 재귀 호출 경로에도 영향을 줄 수 있다.

이것은 A.3의 이름 탐색 원리가 데코레이터와 결합된 사례다.

A.6.11 @decorator와 @decorator()는 다르다

다음 두 문법을 비교해 보자.

```
@decorator
def greet():
    return 10

@decorator()
def greet():
    return 10
```

두 표현식은 같은 의미가 아니다.

첫 번째는 decorator가 참조하는 호출 가능한 객체를 데코레이터로 사용한다.

따라서 개념적으로 다음과 대응한다.

```
greet = decorator(greet)
```

반면 두 번째는 먼저 decorator()를 호출한다.

그리고 그 호출 결과로 얻은 객체를 실제 데코레이터로 사용한다.

개념적으로 다음과 대응한다.

```
greet = decorator()(greet)
```

따라서 decorator()가 반환하는 객체도 호출 가능해야 한다.

이러한 구조는 인수를 받는 데코레이터에서 자주 사용된다.

예를 들어 다음과 같은 형태다.

```
@repeat(3)
def greet():
    print("Hello")
```

이때 `repeat(3)`은 실제 데코레이터를 반환하는 데코레이터 생성기(Decorator Factory) 역할을 할 수 있다.

이 구조는 A.9에서 상세하게 학습할 것이다.

지금은 다음 구분만 정확하게 확립하면 된다.

문법	적용 방식
`@decorator`	`decorator` 자체를 데코레이터로 사용
`@decorator()`	`decorator()` 호출 결과를 데코레이터로 사용
`@decorator(arg)`	`decorator(arg)` 호출 결과를 데코레이터로 사용

특히 `@decorator`를 `@decorator()`로 임의로 변경하면

함수 정의 시점에 `TypeError`가 발생하거나 전혀 다른 동작이 나타날 수 있다.

A.6.12 wrapper의 인수 전달 문제

지금까지 사용한 wrapper는 다음과 같았다.

```
def wrapper():
    return func()
```

이 구조에는 한 가지 제한이 있다.

원본 함수가 인수를 받는다면 `wrapper`도 그 호출을 처리할 수 있어야 한다.

다음 코드를 살펴보자.

```
def decorator(func):

    def wrapper():
        return func()

    return wrapper


@decorator
def add(a, b):
    return a + b
```

```
print(add(10, 20))
```

예상 결과:

```
TypeError: ... wrapper() takes 0 positional arguments
but 2 were given
```

왜 오류가 발생할까?

데코레이션 후 add는 원본 함수가 아니라 wrapper를 참조한다.

따라서 다음 호출은

```
add(10, 20)
```

실제로는 wrapper(10, 20) 호출에 해당한다.

그런데 wrapper에는 매개변수가 없다.

따라서 원본 add에 도달하기 전에 wrapper 호출 단계에서 오류가 발생한다.

이 문제를 해결하는 대표적인 방법은 다음과 같다.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper
```

여기서 *args와 **kwargs를 통해 wrapper가 받은 인수를 원본 함수에 전달할 수 있다.

하지만 단순히 인수를 전달하는 것만으로 모든 함수 호출 계약이 자동으로 보존되는 것은 아니다.

원본 함수의 반환값, 예외, 시그니처(Signature), 동기·비동기 동작 등을 어떻게 처리할지도 고려해야 한다.

이 부분은 다음 A.7에서 자세히 분석할 것이다.

A.6.13 데코레이터 적용 후 함수 메타데이터는 어떻게 될까?

앞에서 다음 결과를 확인했다.

```
print(greet.__name__)
```

데코레이션 후 greet이 wrapper 함수 객체를 참조한다면 결과는 일반적으로 다음과 같다.

```
wrapper
```

왜 그럴까?

함수 객체의 __name__은 그 함수 객체에 관한 메타데이터다.

모듈 이름 greet이 wrapper를 참조하도록 바뀌었다고 해서 wrapper함수 객체의 __name__이 자동으로 "greet"으로 변경되는 것은 아니다.

다음 코드를 살펴보자.

```
def decorator(func):

    def wrapper():
        return func()

    return wrapper


@decorator
def greet():
    """인사말을 반환한다."""
    return "Hello"


print(greet.__name__)
print(greet.__doc__)
```

예상 출력:

```
wrapper
None
```

원본 함수의 __name__은 "greet"이고, __doc__에는 독스트링이 존재한다.

하지만 데코레이션 후 greet은 새로운 wrapper 함수 객체를 참조하고 있다.

따라서 wrapper의 메타데이터가 나타나는 것이다.

이 문제를 해결하기 위해 일반적으로 functools.wraps를 사용한다.

```
from functools import wraps


def decorator(func):

    @wraps(func)
    def wrapper():
        return func()

    return wrapper
```

다만 functools.wraps가 어떤 속성을 복사하고, __wrapped__를 어떻게 설정하며, inspect.signature()가 이를 어떻게 활용하는지는 A.8에서 다룰 것이다.

이번 학습에서는 이름 재바인딩이 원본 함수의 메타데이터를 자동으로 새 함수 객체에 복사하지 않는다는 점을 이해하면 충분하다.

A.6.14 데코레이터의 내부 구조를 종합적으로 검증하기

이제 지금까지의 내용을 하나의 독립적인 코드로 통합해 보자.

```
def decorator(func):
    print("[1] decorator 호출")

    def wrapper():
        print("[3] wrapper 호출")

        result = func()

        print("[5] wrapper 종료")

        return result

    return wrapper


@decorator
def greet():
    print("[4] 원본 greet 호출")
    return 10
```

```
print("[2] 데코레이션 완료")

wrapped = greet

freevars = wrapped.__code__.co_freevars
cells = wrapped.__closure__

closure_map = dict(zip(freevars, cells))

original = closure_map["func"].cell_contents

print("현재 이름의 함수:", wrapped.__name__)
print("원본 함수:", original.__name__)
print("함수 객체 동일성:", wrapped is original)

print("반환값:", greet())
```

예상 실행 결과:

```
[1] decorator 호출
[2] 데코레이션 완료
현재 이름의 함수: wrapper
원본 함수: greet
함수 객체 동일성: False
[3] wrapper 호출
[4] 원본 greet 호출
[5] wrapper 종료
반환값: 10
```

이 코드는 다음 다섯 가지 관계를 직접 확인한다.

> 첫째, 데코레이터 호출은 함수 정의문 실행 중 발생한다

따라서 [1]이 [2]보다 먼저 출력된다.

> 둘째, 데코레이션 이후 greet은 wrapper를 참조한다

따라서 `wrapped.__name__`은 `"wrapper"`다.

> 셋째, 원본 함수 객체는 wrapper의 클로저를 통해 접근할 수 있다

`closure_map["func"].cell_contents`는 원본 함수 객체를 반환한다.

> 넷째, wrapper와 원본 함수는 서로 다른 객체다

따라서 `wrapped is original`은 `False`다.

다섯째, wrapper가 원본 함수를 호출하고 반환값을 전달한다

따라서 [3], [4], [5] 순서로 출력되고 최종적으로 10이 반환된다.

여기서 closure_map을 사용한 이유도 중요하다.

__closure__[0]이 항상 원본 함수 객체를 참조한다고 일반화하면 안 된다.

wrapper가 여러 자유 변수를 사용한다면 셀이 여러 개 존재할 수 있다.

따라서 내부 구조를 분석할 때는 co_freevars의 이름과 __closure__의 셀을 대응시키는 것이 더 정확하다.

A.6.15 기존 Python 개념과의 연결

이번 학습에서 등장한 데코레이터는 Python의 별도 객체 모델을 새로 도입한 것이 아니다.

일반적인 객체 모델을 결합한 구조다.

기존 개념	데코레이터에서의 역할
First-class Function	원본 함수 객체를 다른 함수의 인수로 전달
함수 객체 생성	wrapper 함수 객체 생성
이름 바인딩	데코레이터 반환 객체를 원래 이름에 바인딩
Lexical Scope	wrapper가 외부 함수의 매개변수를 참조
Free Variable	wrapper의 func
Closure Cell	원본 함수 객체에 접근하는 바인딩 유지
Callable	wrapper를 함수처럼 호출
반환	wrapper가 원본 함수 호출 결과를 반환
Identity	원본 함수와 wrapper가 서로 다른 객체인지 비교

특히 클래스의 메서드 바인딩과도 구별해야 한다.

obj.method

에서 발생하는 메서드 바인딩은 디스크립터(Descriptor)와 속성 탐색 규칙에 연결된다.

반면

```
@decorator
def method(self):
    ...
```

에서의 데코레이터 적용은 클래스 본문에서 함수 정의문이 실행될 때 발생하는 객체 변환 과정이다.

이후 그 객체가 인스턴스 속성으로 접근될 때는 다시 디스크립터와 메서드 바인딩 규칙이 관여할 수 있다.

두 메커니즘은 서로 다른 단계에서 작동한다.

이 관계는 A.11에서 자세히 분석할 것이다.

A.6.16 실전 응용 - 함수 실행 기록

데코레이터의 가장 대표적인 활용 중 하나는 원본 함수의 핵심 로직을 변경하지 않고 부가 기능을 추가하는 것이다.

예를 들어 다음과 같은 실행 기록 기능을 생각해 볼 수 있다.

```
def log_call(func):

    def wrapper(*args, **kwargs):
        print(f"{func.__name__} 호출 시작")

        result = func(*args, **kwargs)

        print(f"{func.__name__} 호출 종료")

        return result

    return wrapper


@log_call
def multiply(a, b):
    return a * b
```

```
print(multiply(3, 4))
```

예상 출력:

```
multiply 호출 시작
multiply 호출 종료
12
```

여기서 원본 `multiply` 함수는 곱셈 연산만 담당한다.

실행 기록 기능은 `wrapper`가 담당한다.

이 구조가 유용한 이유는 기능의 책임을 분리할 수 있기 때문이다.

원본 함수는 자신의 계산을 수행하고, 데코레이터는 호출 전후의 공통 처리를 수행한다.

실제 프로그램에서는 이러한 구조를 이용해 로깅(Logging), 권한 검사, 호출 횟수 기록, 실행 시간 측정, 캐싱(Caching) 등을 구현할 수 있다.

하지만 각각의 기능은 단순히 `@`를 붙이는 것만으로 자동 궇녀되는 것이 아니다.

데코레이터가 반환하는 객체가 실제로 어떤 동작을 수행하도록 작성되어 있는지가 핵심이다.

A.6.17 자주 발생하는 오해와 경계 조건

오해	정확한 설명
`@decorator`는 원본 함수 객체를 직접 수정한다.	일반적으로 원본 객체를 전달하고 반환 객체를 이름에 다시 바인딩해.
`@decorator`는 `decorator()`와 같다.	전자는 데코레이터 객체를 사용하고, 후자는 먼저 호출 결과를 얻어 사용해.
데코레이터에는 함수 호출 결과가 전달된다.	일반적인 함수 데코레이터에는 함수 객체 자체가 전달돼.
`wrapper`는 데코레이션 시점에 실행된다.	`wrapper` 함수 객체는 생성되지만 본문은 별도 호출이 있어야 실행돼.
데코레이터는 반드시 새로운 함수를 반환한다.	원본 함수를 그대로 반환하거나 다른 종류의 객체를 반환할 수도 있어.
원본 함수 객체는 데코레이션 후 사라진다.	`wrapper`의 클로저 등 다른 참조가 있다면 계속 유지될 수 있어.
데코레이션 후 함수 이름은 원본 함수 객체를 가리킨다.	그 이름은 데코레이터가 반환한 객체를 가리켜.
`wrapper`가 원본 함수의 인수와 메타데이터를 자동으로 보존한다.	인수 전달과 메타데이터 보존은 별도로 구현해야 해.

여기서 특히 강조할 점은 데코레이터 문법 자체와 특정 데코레이터의 구현을 구분해야 한다는 것이다.

`@decorator`가 적용되었다는 사실만으로 원본 함수의 호출 횟수, 반환값, 예외 처리 방식, 메타데이터가 어떻게 달라지는지 단정할 수 없다.

실제 동작은 데코레이터가 반환하는 객체와 그 객체의 구현에 따라 결정된다.

A.6.18 핵심 정리와 판단 기준

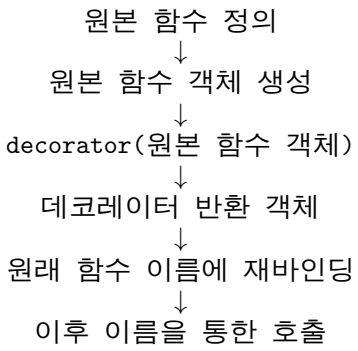
이번 학습에서 가장 중요한 변환식은 다음과 같다.

$$\texttt{@decorator} \quad \Longleftrightarrow \quad \texttt{func = decorator(func)}$$

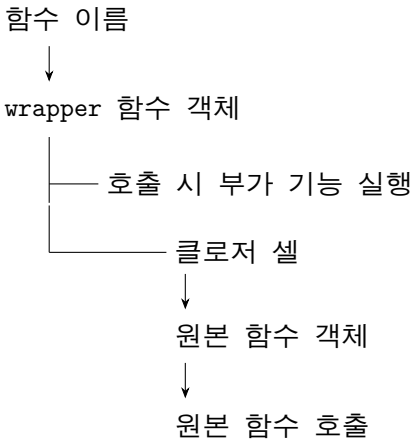
이 표현은 단일 함수 데코레이터의 일반적인 의미를 나타낸 것이다.

실제 소스 코드가 내부적으로 반드시 이 대입문으로 문자 그대로 치환한다는 의미는 아니다.

이를 객체 관계로 해석하면 다음과 같다.



wrapper 기반 데코레이터라면 다음 구조가 추가된다.



이제 다음 질문에 스스로 답할 수 있다면 A.6의 핵심 구조를 이해한 것이다.

- @decorator는 언제 적용되는가?
- 데코레이터에는 어떤 객체가 인수로 전달되는가?
- 데코레이터가 반환한 객체는 어디에 바인딩되는가?
- 원본 함수 객체와 wrapper 함수 객체는 동일한가?
- wrapper가 원본 함수를 참조한다면 그 바인딩은 어디에 유지되는가?
- 데코레이션 이후 func()를 호출하면 실제로 어떤 객체가 호출되는가?

가장 중요한 판단 기준은 하나다.

데코레이터를 분석할 때는 함수 이름만 따라가지 말고,

각 시점에 그 이름이 어떤 객체를 참조하는지 추적해야 한다.

A.7 Wrapper의 생성, 인수 전달과 반환

Wrapper Function · Argument Binding · *args · **kwargs · Return Value · Exception Propagation

A.7.1 이전 학습과의 연결

A.6에서는 데코레이터(Decorator)의 본질이 특별한 함수 수정 기능이 아니라, 함수 객체를 전달하고 데코레이터의 반환 객체를 원래 이름에 다시 바인딩하는 과정이라는 사실을 학습했다.

다음 두 코드는 단일 함수 데코레이터의 일반적인 의미에서 대응한다.

```
@decorator
def add(a, b):
    return a + b


def add(a, b):
    return a + b

add = decorator(add)
```

그리고 decorator가 새로운 wrapper 함수 객체를 반환한다면, 데코레이션 이후 add라는 이름은 원본 함수가 아니라 wrapper를 참조하게 된다.

그렇다면 다음 문제가 발생한다.

원본 함수 add(a, b)처럼 두 인수를 받도록 정의되어 있는데, wrapper는 어떤 인수를 받아야 할까?

원본 함수가 반환한 객체는 어떻게 호출자에게 전달될까?

원본 함수에서 예외가 발생하면 wrapper의 실행은 어떻게 될까?

이번 학습은 이러한 문제를 Python의 일반적인 함수 호출과 인수 바인딩 규칙으로 설명하는 것이 목표다.

이전 단원	이번 단원
데코레이터가 언제 적용되는가?	wrapper는 언제 호출되는가?
원본 함수 객체는 어디에 유지되는가?	wrapper는 원본 함수를 어떻게 호출하는가?
@decorator는 어떤 객체를 반환하는가?	반환값과 예외는 호출자에게 어떻게 전달되는가?
함수 이름이 어떤 객체에 재바인딩되는가?	각 호출에서 어떤 지역 네임스페이스가 만들어지는가?

여기서 가장 중요한 원리는 다음이다.

데코레이터가 적용되더라도

Python의 일반적인 함수 호출 규칙이 바뀌는 것은 아니다.

단지 호출 경로에 새로운 함수 객체가 추가되는 것이다.

A.7.2 Wrapper의 정확한 정의와 역할

Wrapper는 다른 함수나 호출 가능한 객체의 호출을 감싸는 역할을 수행하는 함수다.

Python에 별도의 wrapper 타입이 존재하는 것은 아니다.

일반적인 함수 객체가 다른 함수를 호출하는 구조로 작성되었을 때 그 역할을 wrapper라고 부르는 것이다.

대표적인 구조는 다음과 같다.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        # 원본 함수 호출 전 처리

        result = func(*args, **kwargs)

        # 원본 함수 호출 후 처리

        return result

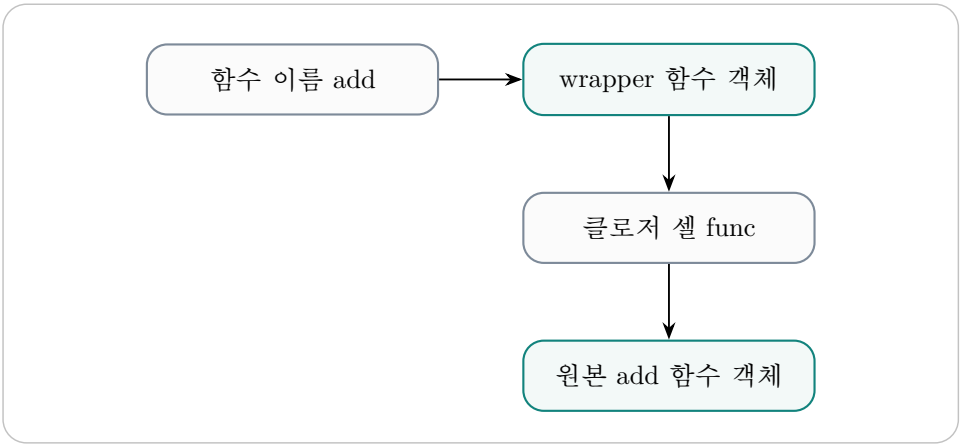
    return wrapper
```

이 코드를 객체 모델 관점에서 구분하면 다음과 같다.

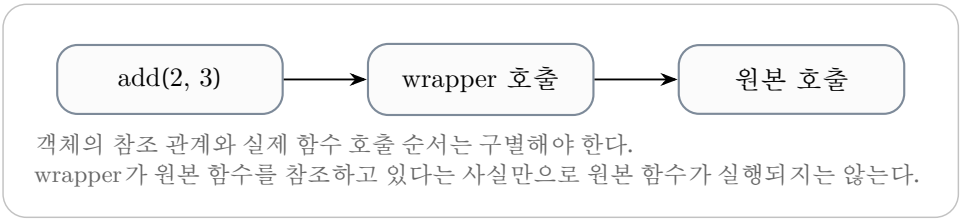
- `decorator`는 원본 함수 객체를 전달받는 함수다.
- `func`는 원본 함수 객체를 참조하는 외부 함수의 매개변수다.
- `wrapper`는 데코레이터 호출 중 새로 생성되는 함수 객체다.
- `args, kwargs`는 `wrapper`가 호출될 때 바인딩되는 지역 매개변수다.
- `func(*args, **kwargs)`는 원본 함수 객체를 실제로 호출하는 표현식이다.
- `result`는 원본 함수가 반환한 객체를 참조하는 지역 이름이다.

이 관계를 시점별로 구별해야 한다.

정의 시점의 객체 관계



호출 시점의 실행 흐름



객체의 참조 관계와 실제 함수 호출 순서는 구별해야 한다.
wrapper가 원본 함수를 참조하고 있다는 사실만으로 원본 함수가 실행되지는 않는다.

A.7.3 최소 실행 예제 - 정의 시점과 호출 시점

먼저 하나의 완전한 예제를 살펴보자.

```
def trace(func):
    print("데코레이터 적용:", func.__name__)

    def wrapper(*args, **kwargs):
        print("호출 전:", args, kwargs)

        result = func(*args, **kwargs)

        print("호출 후:", result)

        return result

    return wrapper


@trace
def calculate(a, b, *, scale=1):
    return (a + b) * scale


print("함수 정의 완료")

answer = calculate(2, 3, scale=4)

print("최종 결과:", answer)
```

예상 실행 결과:

```
데코레이터 적용: calculate
함수 정의 완료
호출 전: (2, 3) {'scale': 4}
호출 후: 20
최종 결과: 20
```

이 코드는 두 단계로 나누어 분석해야 한다.

1. 함수 정의문이 실행될 때

```
@trace
def calculate(a, b, *, scale=1):
    return (a + b) * scale
```

원본 `calculate` 함수 객체가 생성된다.

그 함수 객체가 `trace`에 전달되고, `trace` 내부에서 `wrapper` 함수 객체가 생성된다.

trace가 반환한 wrapper가 모듈 이름 calculate에 바인딩된다.

이 시점에는 wrapper의 본문이 실행되지 않았다.

따라서 출력은 다음 한 줄뿐이다.

데코레이터 적용: calculate

2. calculate(2, 3, scale=4)가 호출될 때

이제 모듈 이름 calculate는 wrapper 함수 객체를 참조하고 있다.

따라서 호출 표현식은 실제로 다음 함수의 호출이 된다.

```
def wrapper(*args, **kwargs):
    ...
```

호출 과정에서 wrapper의 지역 매개변수는 다음 객체에 바인딩된다.

```
args = (2, 3)
kwargs = {"scale": 4}
```

이것은 호출 결과를 설명하기 위한 표현이다.

Python이 실제로 이 두 대입문을 소스 코드처럼 실행한다는 뜻은 아니다.

이후 다음 표현식이 평가된다.

```
func(*args, **kwargs)
```

func은 클로저 셀을 통해 원본 calculate 함수 객체를 참조한다.

따라서 원본 함수가 호출되고 다음과 같이 인수가 바인딩된다.

원본 매개변수	바인딩된 객체	전달 방식
a	2	위치 인수
b	3	위치 인수
scale	4	키워드 인수

원본 함수는 (2 + 3) * 4를 계산해 정수 객체 20을 반환한다.

그 객체가 wrapper의 지역 이름 result에 바인딩되고,

다시 `return result`를 통해 최종 호출자에게 전달되는 것이다.

이제 이 과정을 인수 바인딩, 반환, 예외의 세 부분으로 나누어 더 깊이 분석해 보자.

A.7.4 *args와 **kwargs의 정확한 역할

다음 함수 정의를 살펴보자.

```
def wrapper(*args, **kwargs):
    return func(*args, **kwargs)
```

여기서 *args와 **kwargs는 함수 정의 위치와 함수 호출 위치에서 서로 다른 역할을 한다.

위치	표현식	의미
함수 정의	def wrapper(*args, **kwargs)	전달받은 위치 인수와 키워드 인수를 수집
함수 호출	func(*args, **kwargs)	저장된 객체들을 위치 인수와 키워드 인수로 펼쳐 전달

같은 기호를 사용하지만 동작은 반대 방향이다.

1. Wrapper에서 인수 수집

```
def inspect_call(func):

    def wrapper(*args, **kwargs):
        print("args:", args)
        print("kwargs:", kwargs)

        return func(*args, **kwargs)

    return wrapper


@inspect_call
def add(a, b):
    return a + b


print(add(10, b=20))
```

예상 출력:

```
args: (10,)
kwargs: {'b': 20}
30
```

`add(10, b=20)`을 호출하면 실제로는 `wrapper`가 호출된다.

`wrapper`의 매개변수 바인딩 결과는 다음과 같다.

```
args = (10,)
kwargs = {"b": 20}
```

`args`는 튜플 객체를, `kwargs`는 딕셔너리 객체를 참조한다.

이때 원본 `add`의 매개변수 `a`, `b`에 아직 인수가 바인딩된 것은 아니다.

현재 실행중인 함수는 `wrapper`이므로, 먼저 `wrapper`의 매개변수 바인딩이 발생하는 것이다.

2. 원본 함수 호출에서 인수 펼치기

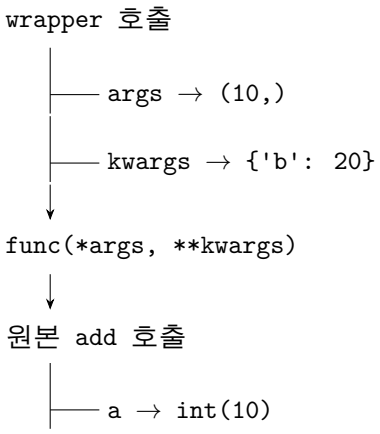
이후 다음 문장이 실행된다.

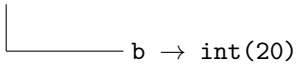
```
return func(*args, **kwargs)
```

이 호출은 현재 예제에서 다음과 같은 호출과 대응한다.

```
func(10, b=20)
```

이제 원본 `add`의 매개변수 바인딩이 수행된다.





여기서 인수 전달이 두 번 발생한다는 사실이 중요하다.

첫 번째는 호출자에서 `wrapper`로, 두 번째는 `wrapper`에서 원본 함수로 전달되는 과정이다.

다만 이것이 인수 객체 자체를 두 번 복사한다는 뜻은 아니다.

각 함수 호출에서 새로운 매개변수 바인딩이 형성되는 것이다.

A.7.5 원본 함수의 매개변수 규칙은 그대로 적용된다

`*args`와 `**kwargs`를 사용하면 `wrapper`가 다양한 형태의 호출을 받아들일 수 있다.

그렇다고 원본 함수의 매개변수 규칙까지 사라지는 것은 아니다.

다음 코드를 살펴보자.

```
def relay(func):

    def wrapper(*args, **kwargs):
        print("wrapper:", args, kwargs)
        return func(*args, **kwargs)

    return wrapper


@relay
def formula(a, /, b, *, c=0):
    return a + b + c


print(formula(1, b=2, c=3))

try:
    formula(a=1, b=2, c=3)
except TypeError as error:
    print(type(error).__name__)
```

예상 출력:

```
wrapper: (1,) {'b': 2, 'c': 3}
```

```
6
wrapper: () {'a': 1, 'b': 2, 'c': 3}
TypeError
```

왜 첫 번째 호출은 성공하고 두 번째 호출은 실패할까?

원본 함수의 정의를 다시 보자.

```
def formula(a, /, b, *, c=0):
    return a + b + c
```

a는 위치 전용 매개변수(Positional-only Parameter)다.

b는 위치 또는 키워드로 전달할 수 있는 매개변수이고, c는 키워드 전용 매개변수(Keyword-only Parameter)다.

첫 번째 호출은 원본 함수의 규칙을 만족한다.

```
formula(1, b=2, c=3)
```

반면 두 번째 호출은 다음과 같다.

```
formula(a=1, b=2, c=3)
```

wrapper는 이 호출을 받아들일 수 있다.

모든 키워드 인수를 kwargs에 수집하면 되기 때문이다.

하지만 원본 함수에 전달하는 순간 문제가 발생한다.

```
func(*args, **kwargs)
```

원본 formula는 a를 키워드 인수로 받는 것을 허용하지 않는다.

따라서 원본 함수의 인수 바인딩 단계에서 TypeError가 발생한다.

이 결과로 다음 원리를 확립할 수 있다.

Wrapper가 호출을 수용할 수 있다는 사실과

원본 함수가 그 호출을 수용할 수 있다는 사실은 서로 다르다.

*args, **kwargs는 인수를 유연하게 전달하는 수단이지, 원본 함수의 매개변수 규칙을 무효화하는 장치가 아니다.

A.7.6 기본 인수(Default Argument)

는 어디에서 적용될까?

다음 코드를 보자.

```
def relay(func):

    def wrapper(*args, **kwargs):
        print("wrapper:", args, kwargs)
        return func(*args, **kwargs)

    return wrapper


@relay
def greet(name="Kim"):
    return f"Hello, {name}"


print(greet())
```

예상 출력:

```
wrapper: () {}
Hello, Kim
```

여기서 흥미로운 점이 있다.

wrapper의 args와 kwargs는 모두 비어 있다.

그런데 원본 함수에서는 name이 "Kim"에 바인딩된다.

왜 이런 결과가 발생할까?

기본 인수는 원본 함수 객체의 매개변수 정의에 속하기 때문이다.

원본 greet 함수 객체는 정의될 때 기본 인수 객체를 보관한다.

wrapper가 다음 호출을 수행하면

```
func()
```

원본 함수의 인수 바인딩 과정에서 생략된 name에 기본 인수 객체가 사용된다.

따라서 기본 인수를 wrapper가 미리 채워서 전달할 필요는 없다.

다만 wrapper가 인수를 수정하거나 기본값을 별도로 적용하는 구현이라면 실제 동작은 달라질 수 있다.

인수 객체의 동일성

또 하나 중요한 점이 있다.

wrapper가 인수를 수집하고 다시 펼친다고 해서 인수 객체가 자동으로 복사되는 것은 아니다.

```
def relay(func):

    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@relay
def modify(values):
    values.append(10)
    return values


data = []

result = modify(data)

print(data)
print(result is data)
```

예상 출력:

```
[10]
True
```

호출자, wrapper, 원본 함수는 서로 다른 지역 이름을 사용할 수 있지만, 이 예제에서 모두 같은 리스트 객체를 참조한다.

따라서 원본 함수가 리스트를 변경하면 호출자의 data에서도 변경된 상태를 관찰할 수 있다.

이것은 데코레이터의 특별한 규칙이 아니라 A.1에서 학습한 Python의 일반적인 객체 전달 원리다.

A.7.7 Wrapper의 시그니처(Signature)와 실제 호출 계약

다음 코드를 살펴보자.

```
def decorator(func):

    def wrapper():
        return func()

    return wrapper


@decorator
def add(a, b):
    return a + b


add(10, 20)
```

이 코드는 TypeError를 발생시킨다.

원본 add에 인수를 전달하기도 전에 오류가 발생하는 것이다.

왜냐하면 현재 add라는 이름이 참조하는 함수 객체는 다음과 같기 때문이다.

```
def wrapper():
    return func()
```

wrapper는 위치 인수를 받을 매개변수를 정의하지 않았다.

따라서 add(10, 20) 호출에서 wrapper의 인수 바인딩이 실패한다.

여기서 중요한 점은 호출 대상 객체의 실제 시그니처가 먼저 적용된다는 사실이다.

다음과 같이 작성하면 이 문제를 해결할 수 있다.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper
```

그러나 이 방법에도 한계가 있다.

원본 함수의 시그니처가

```
def add(a: int, b: int) -> int:
    ...
```

라고 하더라도 `wrapper`의 실제 정의는

```
def wrapper(*args, **kwargs):
    ...
```

이다.

따라서 데코레이션 후 함수 객체의 메타데이터와 시그니처를 어떻게 보존할지 별도로 고려해야 한다.

이 문제는 다음 A.8의 `functools.wraps`와

`inspect.signature()`에서 다룰 것이다.

A.7.8 반환값 전달 - `return func()`와 `func()`의 차이

이번 학습에서 인수 전달만큼 중요한 것이 반환값이다.

다음 세 표현식을 비교해 보자.

```
return func
```

```
return func()
```

```
func()
```

세 표현식은 서로 다른 동작을 수행한다.

사례 1. return func

```
def original():
    return 42


def wrapper():
    return original
```

```
result = wrapper()

print(result is original)
```

예상 출력:

```
True
```

`wrapper()`는 원본 함수 객체를 반환한다.

원본 함수의 본문은 실행되지 않는다.

사례 2. return func()

```
def original():
    return 42


def wrapper():
    return original()


result = wrapper()

print(result)
```

예상 출력:

```
42
```

이번에는 원본 함수가 호출된다.

그 호출 결과인 정수 객체 42가 `wrapper`의 반환값으로 전달된다.

사례 3. func()만 실행

```
def original():
    return 42


def wrapper():
    original()


result = wrapper()
```

```
print(result)
```

예상 출력:

```
None
```

이번에는 원본 함수가 실행되고 42를 반환한다.

하지만 `wrapper`는 그 반환값을 다시 반환하지 않는다.

따라서 `wrapper` 본문의 실행이 끝나면 `None`이 반환된다.

이를 비교하면 다음과 같다.

표현식	원본 함수 호출	wrapper의 반환값
`return func`	호출하지 않음	원본 함수 객체
`return func()`	호출함	원본 함수의 반환 객체
`func()`	호출함	명시적 반환이 없다면 `None`

여기서 중요한 결론은 다음과 같다.

Wrapper가 원본 함수를 호출했다는 사실만으로 원본 함수의 반환값이 호출자에게 전달되는 것은 아니다.

Wrapper는 독립적인 함수이므로 자신의 `return` 규칙을 따르기 때문이다.

A.7.9 반환값스이 객체 동일성은 유지될까?

이번에는 원본 함수가 리스트 객체를 반환하는 경우를 살펴보자.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result

    return wrapper


@decorator
```

```
def create_list():
    return [10, 20]


result = create_list()

original = create_list.__closure__[0].cell_contents

print(type(result))
print(result)
```

예상 출력:

```
<class 'list'>
[10, 20]
```

여기서 `result`는 원본 함수가 생성한 리스트 객체를 참조한다.

Wrapper의 `return result`는 그 객체를 다시 반환한다.

새로운 리스트 객체를 복사해서 생성하는 것이 아니다.

이 관계를 더 명확하게 확인하려면 `wrapper` 내부에서 반환 객체를 따로 보관해 볼 수 있다.

```
def decorator(func):
    observed = []

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)

        observed.append(result)

        return result

    return wrapper, observed


def create_list():
    return [10, 20]


wrapped, observed = decorator(create_list)

result = wrapped()

print(result is observed[0])
```

예상 출력:

```
True
```

원본 함수가 반환한 리스트 객체와 최종 호출자가 받은 리스트 객체가 동일하다는 사실을 확인할 수 있다.

이 원리는 PyTorch의 Tensor 객체를 반환하는 함수에도 적용된다.

예를 들어 `wrapper`가 원본 함수의 반환값을 그대로 전달한다면 새로운 Tensor를 자동으로 복사하는 것은 아니다.

반환 객체의 동일성, 내부 상태, 계산 그래프와 같은 특성은 실제 `wrapper`가 어떤 연산을 수행하는지에 따라 결정된다.

A.7.10 예외(Exception)가 발생하면 호출 흐름은 어떻게 될까?

지금까지는 원본 함수가 정상적으로 반환하는 경우만 살펴봤다.

이번에는 예외가 발생하는 상황을 분석해 보자.

```
def trace(func):

    def wrapper(*args, **kwargs):
        print("Before")

        result = func(*args, **kwargs)

        print("After")

        return result

    return wrapper


@trace
def divide(a, b):
    return a / b


try:
    divide(10, 0)
except ZeroDivisionError:
    print("ZeroDivisionError 처리")
```

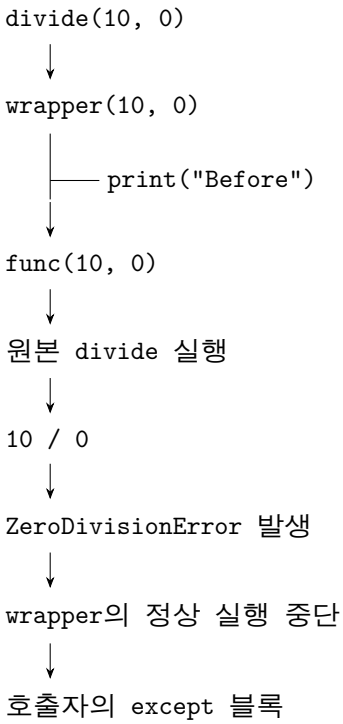
예상 출력:

```
Before
ZeroDivisionError 처리
```

여기서 "After"는 출력되지 않는다.

왜 그럴까?

실행 흐름을 살펴보자.



예외가 발생하면 다음 문장의 정상적인 평가가 완료되지 않는다.

```
result = func(*args, **kwargs)
```

따라서 result에 반환 객체가 바인딩되지 않고, 뒤에 있는 "After" 출력문에도 도달하지 않는다.

예외는 wrapper에서 처리되지 않았으므로 호출자 쪽으로 전파된다.

여기서 중요한 사실이 있다.

원본 함수에서 예외가 발생하더라도 `wrapper`가 자동으로 예외를 처리하지는 않는다.

Wrapper 역시 일반적인 함수이므로 Python의 예외 전파 규칙을 그대로 따르기 때문이다.

A.7.11 `try`, `except`, `finally`를 이용한 예외 처리

실전 데코레이터에서는 원본 함수가 성공하든 실패하든 특정 후처리를 실행해야 하는 경우가 있다.

예를 들어 실행 기록, 자원 정리, 처리 시간 측정 등이 해당된다.

이때 `finally`를 사용할 수 있다.

```
def trace(func):

    def wrapper(*args, **kwargs):
        print("Before")

        try:
            result = func(*args, **kwargs)
        finally:
            print("Cleanup")

        print("After")

        return result

    return wrapper


@trace
def divide(a, b):
    return a / b


print(divide(10, 2))

try:
    divide(10, 0)
except ZeroDivisionError:
    print("Exception caught")
```

예상 출력:

```
Before
Cleanup
After
5.0
Before
Cleanup
Exception caught
```

여기서 두 호출의 차이를 분석해 보자.

정상 반환하는 경우

`divide(10, 2)`는 5.0을 반환한다.

`try` 블록의 실행이 정상적으로 완료되면 `finally` 블록이 실행된다.

그다음 `"After"`가 출력되고 `result`가 반환된다.

예외가 발생하는 경우

`divide(10, 0)`에서는 `ZeroDivisionError`가 발생한다.

이때도 `finally` 블록이 실행된다.

따라서 `"Cleanup"`은 출력된다.

하지만 예외가 처리된 것은 아니다.

`finally` 블록이 정상적으로 종료되면 기존 예외가 계속 전파된다.

따라서 `"After"`는 출력되지 않고 호출자의 `except` 블록이 실행된다.

주의할 경계 조건

`finally`가 항상 기존 예외를 보존한다고 이해하면 안 된다.

예를 들어 `finally` 블록 안에서 새로운 예외를 발생시키거나

`return`을 실행하면 기존 예외 또는 반환 흐름이 달라질 수 있다.

따라서 예외를 보존하면서 후처리하려는 데코레이터에서는 `finally` 내부의 제어 흐름을 신중하게 설계해야 한다.

예외를 기록하고 다시 전파하기

다음 방식도 자주 사용된다.

```
def trace(func):

    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)

        except Exception as error:
            print("실행 실패:", type(error).__name__)
            raise

    return wrapper
```

여기서 raise는 현재 처리 중인 예외를 다시 전파한다.

이렇게 하면 데코레이터가 예외를 기록하면서도 호출자가 원래의 실패를 처리할 수 있다.

A.7.12 Wrapper는 원본 함수의 동작을 변경할 수도 있다

지금까지는 wrapper가 원본 함수의 인수와 반환값을 그대로 전달하는 구조를 살펴보았다.

하지만 wrapper는 반드시 그렇게 동작해야 하는 것은 아니다.

다음 코드를 보자.

```
def double_result(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)

        return result * 2

    return wrapper


@double_result
def add(a, b):
    return a + b


print(add(10, 20))
```

예상 출력:

```
60
```

원본 함수는 30을 반환하지만 `wrapper`는 그 결과에 2를 곱해 60을 반환한다.

따라서 데코레이터가 적용된 함수의 관찰 가능한 동작이 달라진 것이다.

또한 다음과 같이 원본 함수를 호출하지 않는 데코레이터도 만들 수 있다.

```
def skip_call(func):

    def wrapper(*args, **kwargs):
        return "Skipped"

    return wrapper


@skip_call
def greet():
    print("Hello")
    return 10


print(greet())
```

예상 출력:

```
Skipped
```

이 예제에서는 원본 `greet` 함수의 본문이 실행되지 않는다.

따라서 `"Hello"`도 출력되지 않는다.

이 사실은 중요한 설계 원리와 연결된다.

> 데코레이터는 호출을 중개하는 구조이므로 원본함수의
>
> 실행 여부, 인수, 반환값, 예외를 변경할 수 있다.

따라서 데코레이터를 분석할 때 단순히 원본 함수의 코드만 확인해서는 전체 동작을 판단할 수 없다.

실제 호출 대상인 `wrapper`의 구현까지 함께 살펴봐야 한다.

A.7.13 클로저와 호출 프레임의 관계

이제 A.4에서 학습한 클로저와 이번 단원의 호출 과정을 연결해 보자.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result

    return wrapper

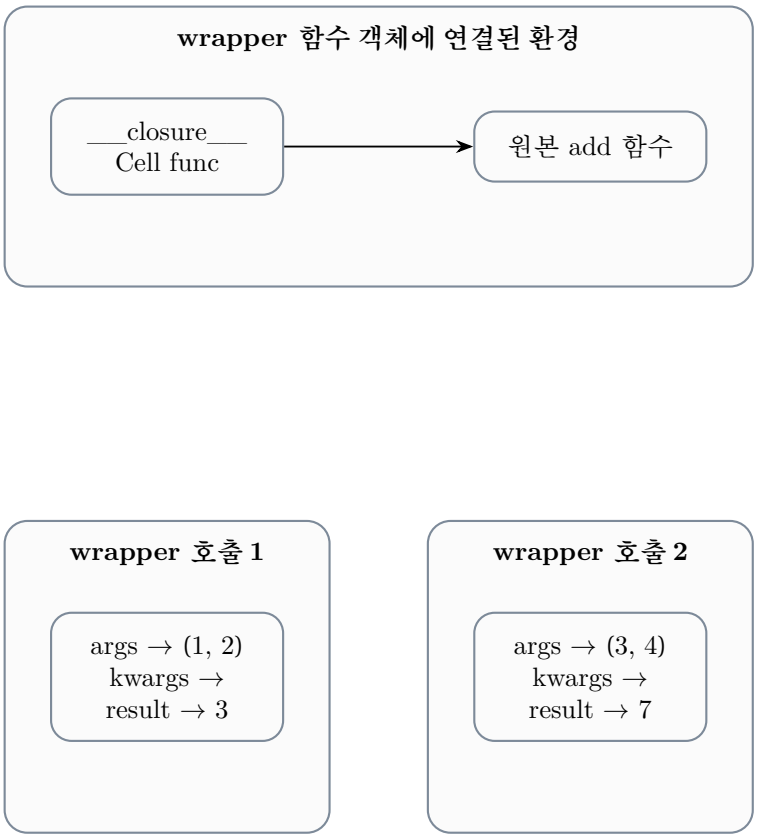

@decorator
def add(a, b):
    return a + b
```

이 코드에는 서로 다른 두 종류의 환경이 관여한다.

첫째는 wrapper 함수 객체가 유지하는 **클로저 환경**이다.

둘째는 wrapper를 호출할 때마다 만들어지는 **함수 호출의 지역 실행 환경**이다.

이 둘을 구별해야 한다.



개념도: 서로 다른 wrapper 호출은 각자의 지역 매개변수와 결과를 가지지만, 동일한 wrapper 함수 객체에 연결된 클로저 환경을 사용할 수 있다.

다음 코드를 살펴보자.

```
print(add(1, 2))
print(add(3, 4))
```

예상 출력:

```
3
7
```

첫 번째 호출에서는 `args`가 `(1, 2)`를 참조하고 `result`가 3을 참조한다.

두 번째 호출에서는 새로운 호출의 지역 바인딩 환경이 형성되고 `args`가 `(3, 4)`를 참조하며 `result`가 7을 참조한다.

하지만 두 호출 모두 같은 wrapper 함수 객체를 실행하므로, 그 함수 객체의 클로저를 통해 동일한 원본 함수 객체에 접근할 수 있다.

이것이 다음 두 개념의 차이다.

구분	클로저 환경	호출의 지역 실행 환경
생성과 연결	wrapper 함수 객체 생성 시 연결	wrapper 호출 시마다 형성
대표적인 이름	func	args, kwargs, result
여러 호출 사이의 관계	동일한 함수 객체의 클로저를 계속 사용 가능	각 호출의 지역 바인딩은 독립적
주요 역할	외부 바인딩 유지	현재 호출의 인수와 계산 결과 관리

이 구분은 A.5에서 학습한 호출 횟수 데코레이터를 이해하는 데도 중요하다.

count가 클로저 셀을 통해 유지된다면 호출마다 상태가 이어질 수 있지만, wrapper의 지역 이름 result는 호출마다 새롭게 바인딩되는 것이다.

A.7.14 Generator Function을 감쌀 때의 경계 조건

여기서 Generator Function과 Generator Object의 구분을 연결해 본다.

다음 코드를 살펴보자.

```
def trace(func):

    def wrapper(*args, **kwargs):
        print("wrapper before")

        result = func(*args, **kwargs)

        print("wrapper after")

        return result

    return wrapper


@trace
def numbers():
    print("generator body")
    yield 1
```

```
generator = numbers()

print("generator created")

print(next(generator))
```

예상 출력:

```
wrapper before
wrapper after
generator created
generator body
1
```

왜 `"wrapper after"`가 `"generator body"`보다 먼저 출력될까?

원본 `numbers`는 Generator Function이다.

Generator Function을 호출하면 일반적으로 함수 본문이 즉시 실행되는 것이 아니라 Generator Object가 반환된다.

따라서 `wrapper`의 다음 문장은

```
result = func(*args, **kwargs)
```

Generator Object를 생성하며 반환받는 과정이다.

아직 `yield`까지 본문이 실행된 것은 아니다.

이후 `wrapper`는 `"wrapper after"`를 출력하고 Generator Object를 반환한다.

마지막으로 다음 문장이 실행될 때

```
next(generator)
```

Generator Object의 실행이 시작되어 `"generator body"`가 출력되는 것이다.

이 예제는 매우 중요한 경계 조건을 보여준다.

일반 함수용 `wrapper`의 호출 전후 처리가 항상 원본 함수 본문의 실제 실행 전후를 의미하는 것은 아니다.

Generator Function에서는 Generator Object의 생성 시점과 실행·재개 시점

을 구별해야 한다.

비슷하게 `async def`로 정의된 코루틴 함수(Coroutine Function)를 감쌀 때도 주의가 필요하다.

일반적인 동기 `wrapper`가 코루틴 함수를 호출하면 코루틴 객체를 반환받을 수 있지만, 그 자체로 코루틴 본문의 실행 완료를 의미하지는 않는다.

코루틴의 실행과 반환을 올바르게 중개하려면 일반적으로 비동기 `wrapper`에서 `await`을 사용하는 구조를 고려해야 한다.

이것은 Python의 함수 종류에 따라 호출 결과의 성격이 달라질 수 있다는 사실과 연결된다.

A.7.15 실전 응용 - 호출 횟수와 예외를 함께 기록하는 데코레이터

이제 A.5의 `nonlocal`과 A.7의 인수 전달·반환·예외 전파를 하나의 예제로 통합해 보자.

```
def monitor(func):
    count = 0

    def wrapper(*args, **kwargs):
        nonlocal count

        count += 1

        print(f"호출 #{count}")

        try:
            return func(*args, **kwargs)

        except Exception as error:
            print("실행 실패:", type(error).__name__)
            raise

    return wrapper


@monitor
def calculate_cost(quantity, unit_price):
    if quantity < 0:
        raise ValueError("수량은 음수일 수 없습니다.")

    return quantity * unit_price
```

```
print(calculate_cost(3, 100))

try:
    calculate_cost(-1, 100)
except ValueError:
    print("호출자에서 예외 처리")

print(calculate_cost(2, 200))
```

예상 결과:

```
호출 #1
300
호출 #2
실행 실패: ValueError
호출자에서 예외 처리
호출 #3
400
```

이 코드를 객체와 바인딩 관점에서 분석해 보자.

1. 데코레이션 시점

`calculate_cost` 원본 함수 객체가 생성된다.

그 객체가 `monitor`에 전달되고, `monitor`의 매개변수 `func`에 바인딩된다.

다음 문장이 실행된다.

```
count = 0
```

그리고 `wrapper` 함수 객체가 생성된다.

`wrapper`는 외부 함수의 `func`와 `count`를 참조하므로 두 이름이 자유 변수로 사용된다.

`monitor`는 `wrapper`를 반환하고, 모듈 이름 `calculate_cost`가 `wrapper`에 재바인딩된다.

2. 첫 번째 호출

```
calculate_cost(3, 100)
```

`wrapper`가 호출되고 `args`가 `(3, 100)`을 참조한다.

nonlocal count에 따라 count += 1이 외부 바인딩을 변경한다.

그다음 원본 함수가 호출되고 300을 반환한다.

return func(*args, **kwargs)는 원본 함수의 반환 객체를 그대로 호출자에게 전달한다.

3. 두 번째 호출

```
calculate_cost(-1, 100)
```

wrapper가 다시 호출된다.

동일한 클로저 셀을 사용하므로 기존 count는 1이다.

count += 1 이후 count는 2를 참조한다.

원본 함수에서 ValueError가 발생하고 wrapper의 except 블록이 실행된다.

예외를 기록한 다음 raise로 다시 전파한다.

따라서 호출자의 except ValueError 블록이 실행된다.

4. 세 번째 호출

```
calculate_cost(2, 200)
```

앞선 호출에서 예외가 발생했지만 count는 이미 2로 변경되었다.

따라서 이번 호출에서 count는 3이 된다.

여기서 중요한 점은 호출 횟수가 성공한 호출의 수가 아니라 wrapper에 진입한 호출의 수라는 사실이다.

이것은 데코레이터의 상태 변경 시점에 따라 결정된다.

성공한 호출만 세고 싶다면 원본 함수가 정상적으로 반환된 이후에 카운트를 증가시키도록 구현해야 한다.

이처럼 데코레이터의 동작을 분석할 때는 단순히 어떤 기능을 추가했는지가 아니라, 그 기능이 호출 흐름의 어느 위치에서 실행되는지까지 확인해야 한다.

A.7.16 자주 발생하는 오해와 경계 조건

오해	정확한 설명
*args, **kwargs를 사용하면 원본 함수의 인수 제약이 사라진다.	원본 함수의 매개변수 바인딩 규칙은 그대로 적용된다.
Wrapper는 인수 객체를 자동으로 복사한다.	새로운 매개변수 바인딩이 형성될 뿐, 전달 객체가 자동 복사되는 것은 아니다.
원본 함수가 값을 반환하면 wrapper도 자동으로 같은 값을 반환한다.	Wrapper에 적절한 return이 있어야 한다.
return func와 return func()는 같다.	전자는 함수 객체를 반환하고, 후자는 호출 결과를 반환한다.
원본 함수에서 예외가 발생해도 wrapper의 나머지 문장이 실행된다.	예외를 처리하지 않으면 정상 실행이 중단되고 예외가 전파된다.
finally는 예외를 자동으로 처리한다.	일반적으로 후처리를 실행할 뿐, 예외를 자동으로 제거하지 않는다.
Wrapper는 반드시 원본 함수를 한 번 호출해야 한다.	원본 함수를 호출하지 않거나 여러 번 호출하도록 구현할 수도 있다.
Wrapper의 호출 완료는 Generator 본문의 실행 완료와 같다.	Generator Object 생성과 본문 실행은 서로 다른 시점이다.
Wrapper의 메타데이터는 원본 함수와 자동으로 동일해진다.	메타데이터 보존은 별도로 처리해야 한다.

A.7.17 핵심 정리와 판단 기준

이번 학습에서는 데코레이터가 적용된 함수의 실제 호출 과정을 분석하였다.

가장 중요한 구조는 다음과 같다.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result

    return wrapper
```

이 코드를 해석할 때는 다음 순서를 따르면 된다.

Wrapper 호출의 전체 흐름

1. 호출자가 데코레이션된 이름을 조회해 `wrapper` 함수 객체를 호출한다.

2. `Wrapper`의 매개변수 바인딩이 수행되고 `args`, `kwargs`가 해당 객체를 참조한다.

3. `Wrapper`가 클로저를 통해 원본 함수 객체 `func`를 조회한다.

4. `func(*args, **kwargs)`에서 인수를 펼쳐 원본 함수의 매개변수 바인딩을 수행한다.

5. 원본 함수가 정상적으로 반환하면 그 반환 객체가 `wrapper`의 `result`에 바인딩된다.

6. `Wrapper`의 `return result`가 반환 객체를 호출자에게 전달한다.

7. 중간에 예외가 발생하면 실제 예외 처리 구조에 따라 실행 경로가 달라진다.

여기서 최종적으로 구별해야 하는 것은 네 가지다.

첫째, 함수 객체의 생성과 호출은 다르다. 데코레이터 적용 시 wrapper 함수 객체가 생성되더라도 wrapper 본문이 즉시 실행되는 것은 아니다.

둘째, 함수 호출과 인수 바인딩은 호출 대상마다 별도로 수행된다. Wrapper와 원본 함수는 서로 다른 함수 객체이므로 각각 자신의 매개변수 규칙을 적용한다.

셋째, 원본 함수의 반환과 wrapper의 반환은 서로 다른 단계다. 원본 함수가 반환한 객체를 wrapper가 어떻게 처리하는지에 따라 최종 결과가 결정된다.

넷째, 예외는 반환값이 아니다. 원본 함수에서 예외가 발생하면 정상적인 반환 경로가 중단되며, wrapper의 예외 처리 구조에 따라 전파·변환·처리될 수 있다.

이 네 가지를 구별하면 대부분의 일반적인 함수 기반 데코레이터를 정확하게 추적할 수 있다.

A.8 `functools.wraps`와 함수 메타데이터

Function Metadata · `update_wrapper` · `__wrapped__` · Introspection · inspect.signature

A.8.1 이전 학습과의 연결

A.7에서는 데코레이터가 적용된 함수의 호출이 실제로 어떤 경로를 따라 실행되는지 분석했다.

```
def decorator(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result

    return wrapper


@decorator
def add(a, b):
    return a + b
```

이 코드에서 add(10, 20)을 호출하면 원본 add가 아니라 wrapper가 먼저 실행된다.

wrapper는 클로저를 통해 원본 함수 객체를 조회하고, 인수를 전달한 뒤 반환값을 호출자에게 다시 전달한다.

그런데 아직 해결하지 않은 문제가 있다.

데코레이션 이후 add라는 이름은 wrapper 함수 객체를 참조하는데, 외부에서는 이 객체를 원본 add 함수처럼 설명하고 다룰 수 있을까?

예를 들어 다음 정보들이 있다.

- 함수 이름: __name__
- 함수의 정규화된 이름: __qualname__
- 독스트링: __doc__
- 타입 주석: __annotations__
- 함수의 호출 시그니처(Signature)
- 원본 함수 객체에 대한 참조

이 정보들은 로깅, 문서 자동 생성, 디버깅, 테스트, 프레임워크의 함수 분석 등에 활용된다.

이번 학습에서는 실제 호출을 수행하는 함수 객체와 그 함수에 대해 외부 도구가 관찰하는 메타데이터를 구별하는 것이 핵심이다.

A.8.2 이번 학습의 핵심 질문

이번 학습에서는 다음 문제를 해결할 것이다.

1. 데코레이터를 적용하면 원본 함수의 메타데이터는 왜 사라진 것처럼 보일까?
2. functools.wraps는 어떤 객체를 생성하거나 변경할까?
3. @wraps(func)와 update_wrapper(wrapper, func)는 어떤 관계일까?
4. __wrapped__는 클로저의 __closure__와 어떻게 다를까?
5. inspect.signature()는 왜 wrapper의 실제 매개변수와 다른 시그니처를 반환할 수 있을까?
6. 여러 데코레이터가 적용되면 원본 함수 객체를 어떻게 추적할 수 있을까?
7. functools.wraps가 보존하지 못하는 것은 무엇일까?

이번 학습 전체를 관통하는 원리는 다음과 같다.

> functools.wraps는 wrapper를 원본 함수로 바꾸는 기능이 아니라, wrapper 함수 객체에 원본 함수의 메타데이터와 참조 정보를 설정한다.

A.8.3 데코레이터 적용 전후의 함수 메타데이터

먼저 functools.wraps를 사용하지 않은 예제를 살펴보자.

```
import inspect


def decorator(func):

    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
```

```
    return wrapper


@decorator
def add(a: int, b: int = 10) -> int:
    """두 정수를 더한다."""
    return a + b


print(add(5))

print(add.__name__)
print(add.__qualname__)
print(add.__doc__)
print(add.__annotations__)
print(inspect.signature(add))
```

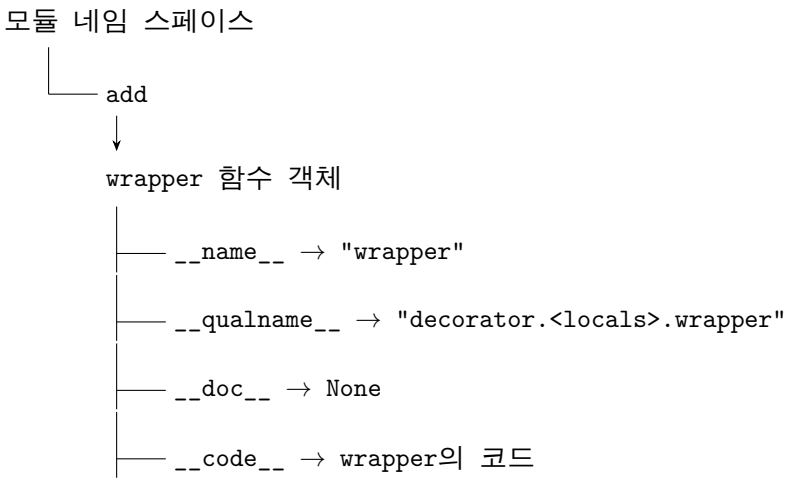
예상 출력:

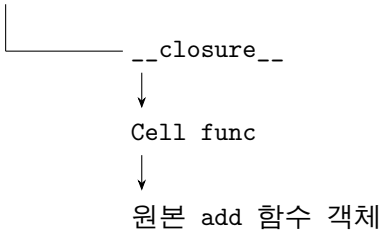
```
15
wrapper
decorator.<locals>.wrapper
None
{}
(*args, **kwargs)
```

왜 이런 결과가 발생할까?

데코레이션 이전에는 모듈 이름 add가 원본 함수 객체를 참조했다.

하지만 데코레이션 이후에는 다음 관계가 형성된다.





여기서 중요한 점은 원본 함수의 메타데이터가 실제로 삭제된 것이 아니라는 사실이다.

원본 함수 객체에는 여전히 다음 정보가 존재한다.

```
original = add.__closure__[0].cell_contents

print(original.__name__)
print(original.__doc__)
print(original.__annotations__)
print(inspect.signature(original))
```

예상 출력:

```
add
두 정수를 더한다.
{'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
(a: int, b: int = 10) -> int
```

즉, 원본 함수 객체와 wrapper 함수 객체는 각각 자신의 메타데이터를 가지고 있다.

문제는 데코레이션 이후 `add`라는 이름이 `wrapper`를 참조하므로, 일반적인 속성 조회에서 `wrapper`의 정보가 나타난다는 것이다.

따라서 해결해야 할 문제는 원본 함수 객체를 복원하는 것이 아니라, `wrapper`에 어떤 메타데이터를 설정할 것인지 결정하는 것이다.

A.8.4 `functools.wraps`를 적용하면 무엇이 달라질까?

이제 Python 표준 라이브러리의 `functools.wraps`를 사용해 본다.

```
import inspect
from functools import wraps
```

```
def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@decorator
def add(a: int, b: int = 10) -> int:
    """두 정수를 더한다."""
    return a + b


print(add(5))
print(add.__name__)
print(add.__qualname__)
print(add.__doc__)
print(add.__annotations__)
print(inspect.signature(add))
```

예상 출력:

```
15
add
add
두 정수를 더한다.
{'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
(a: int, b: int = 10) -> int
```

A.8.3과 비교하면 결과가 달라졌다.

조회 대상	@wraps 없음	@wraps(func) 사용
`__name__`	`wrapper`	`add`
`__qualname__`	`decorator.<locals>.wrapper`	`add`
`__doc__`	`None`	원본 독스트링
`__annotations__`	`{}`	원본 타입 주석
`inspect.signature()`	`(*args, **kwargs)`	`(a: int, b: int = 10) -> int`

그렇다면 `functools.wraps`가 `wrapper` 함수 객체를 원본 `add` 함수 객체로 바꾼 것일까?

그렇지 않다.

실제 함수 객체는 여전히 `wrapper`다.

`functools.wraps`는 `wrapper`에 원본 함수의 메타데이터를 설정하고, 원본 객체를 추적할 수 있도록 `__wrapped__` 속성을 추가한다.

그리고 `inspect.signature()`는 이 참조 정보를 활용해 원본 함수의 시그니처를 조회할 수 있다.

여기에는 서로 다른 두 동작이 포함된다.

1. `functools.wraps`가 함수 객체의 메타데이터를 설정하는 동작
2. `instpect.signature()`가 `__wrapped__`를 따라가며 시그니처를 조회하는 동작

이 둘을 구분해야 한다.

A.8.5 `functools.wraps`의 정확한 정의

`functools.wraps`는 `wrapper` 함수에 원본 함수의 메타데이터를 설정하기 위한 데코레이터 생성기(Decorator Factory)다.

기본적인 사용 형태는 다음과 같다.

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper
```

여기서 주목해야 할 표현식은 다음이다.

```
@wraps(func)
```

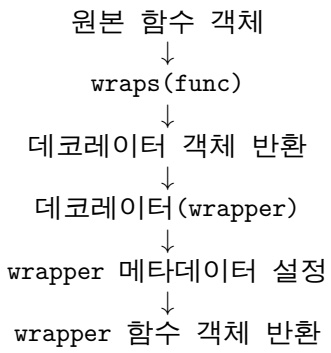
A.6에서 학습한 변환식을 적용하면, 이 표현식은 개념적으로 다음과 대응한다.

```
wrapper = wraps(func)(wrapper)
```

즉, wraps(func)는 실제로 wrapper를 꾸며 줄 데코레이터를 반환한다.

그다음 반환된 데코레이터가 wrapper 함수 객체에 적용되는 것이다.

이 과정을 나누면 다음과 같다.



여기서 중요한 사실이 있다.

@wraps(func)는 원본 함수 func를 호출하는 문법이 아니다.

wraps라는 데코레이터 생성기에 원본 함수 객체를 전달하고,

그 결과를 wrapper에 적용하는 문법이다.

A.8.6 wraps()와 update_wrapper()의 관계

functools.wraps를 정확하게 이해하려면 functools.update_wrapper를 알아야 한다.

두 함수의 관계는 다음과 같다.

- update_wrapper(wrapper, wrapped)는 wrapper 객체의 메타데이터를 설정하는 함수다.
- wraps(wrapped)는 update_wrapper를 데코레이터 형태로 사용하기 편하게 만든 함수다.

먼저 update_wrapper를 직접 사용해 보자.

```
from functools import update_wrapper

def original(a, b):
    """두 수를 더한다."""
    return a + b


def wrapper(*args, **kwargs):
    return original(*args, **kwargs)


before = wrapper

returned = update_wrapper(wrapper, original)

print(returned is before)
print(wrapper.__name__)
print(wrapper.__doc__)
print(wrapper.__wrapped__ is original)
```

예상 출력:

```
True
original
두 수를 더한다.
True
```

이 결과에서 중요한 부분은 다음이다.

```
returned is before
```

결과가 `True`라는 것은 `update_wrapper`가 새로운 `wrapper` 함수 객체를 만들어 반환한 것이 아니라는 뜻이다.

기존 `wrapper` 함수 객체의 메타데이터를 설정한 뒤 그 객체 자체를 반환한 것이다.

따라서 다음 두 코드는 일반적인 함수 기반 `wrapper`에서 같은 목적을 수행한다.

`@wraps` 사용

```
from functools import wraps

def decorator(func):
    @wraps(func)
```

```
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper
```

update_wrapper 직접 사용

```
from functools import update_wrapper

def decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    update_wrapper(wrapper, func)

    return wrapper
```

wraps는 내부적으로 update_wrapper를 사용한다.

표준 라이브러리에서는 functools.partial을 활용하여 update_wrapper의 wrapped 인수를 미리 지정한 데코레이터를 반환하는 방식으로 구현되어 있다.

따라서 A.6에서 배운 데코레이터 생성 구조가 여기에도 그대로 적용된다.

A.8.7 update_wrapper()는 어떤 속성을 설정할까?

기본 동작을 이해하려면 다음 두 상수를 살펴보면 된다.

```
from functools import WRAPPER_ASSIGNMENTS
from functools import WRAPPER_UPDATES
```

현대 Python의 기본 설정에서 update_wrapper는 크게 두 방식으로 메타데이터를 처리한다.

1. 속성 할당 - assigned

원본 객체의 특정 속성값을 wrapper에 할당한다.

속성	의미
`__module__`	함수가 정의된 모듈 이름
`__name__`	함수 이름
`__qualname__`	함수의 정규화된 이름
`__doc__`	독스트링
`__annotations__`	타입 주석 정보
`__type_params__`	제네릭 함수의 타입 매개변수 정보

`__type_params__`는 Python 3.12부터 기본 복사 대상에 포함되었다.

따라서 정확한 기본 속성 목록은 Python 버전에 따라 달라질 수 있다.

2. 네임스페이스 갱신 - updated

기본적으로 `wrapper`의 `__dict__`를 원본 객체의 `__dict__`에 있는 항목으로 갱신한다.

이는 원본 함수에 사용자가 추가한 속성도 `wrapper`에 전달할 수 있다는 뜻이다.

3. 원본 객체 참조 설정 - __wrapped__

마지막으로 다음 관계를 설정한다.

```
wrapper.__wrapped__ = wrapped
```

여기서 `wrapped`는 원본 함수 객체를 참조한다.

이 설정은 `assigned`와 `updated` 목록을 비워도 기본적으로 수행된다.

따라서 `update_wrapper`의 기본적인 동작을 개념적으로 단순화하면 다음과 같다.

```
# 개념적 수도 코드: 실제 구현 전체가 아님

for name in assigned:
    # 원본에 존재하는 메타데이터 속성을 wrapper에 할당
    ...

for name in updated:
    # wrapper의 해당 매핑을 원본의 매핑으로 갱신
    ...

wrapper.__wrapped__ = wrapped
```

```
return wrapper
```

이때 중요한 사실은 `__code__`, `__closure__`, `__default__` 등이 기본적으로 복사 대상이 아니라는 것이다.

따라서 `functools.wraps`를 사용해도 `wrapper`의 실제 코드와 호출 동작이 원본 함수로 바뀌지는 않는다.

A.8.8 함수 객체의 동일성과 메타데이터의 동일성

이제 실제 함수 객체가 여전히 다르다는 사실을 검증해 보자.

```
from functools import wraps


def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@decorator
def add(a, b):
    return a + b


original = add.__wrapped__

print(add is original)
print(add.__name__ == original.__name__)
print(add.__code__ is original.__code__)
print(add.__closure__ is original.__closure__)
```

예상 출력:

```
False
True
False
False
```

각 결과를 분석해 보자.

첫 번째 비교

```
add is original
```

False다.

두 이름이 서로 다른 함수 객체를 참조하기 때문이다.

두 번째 비교

```
add.__name__ == original.__name__
```

True다.

functools.wraps가 wrapper의 __name__을 원본 함수의 이름으로 설정했기 때문이다.

하지만 메타데이터 값이 같다는 사실은 함수 객체가 동일하다는 뜻이 아니다.

세 번째 비교

```
add.__code__ is original.__code__
```

False다.

wrapper와 원본 함수는 서로 다른 코드 객체를 사용한다.

functools.wraps가 wrapper의 코드를 원본 함수의 코드로 교체한 것이 아니기 때문이다.

네 번째 비교

```
add.__closure__ is original.__closure__
```

False다.

이 예제에서 wrapper는 func라는 자유 변수를 참조하므로 클로저를 가진다.

반면 원본 add는 외부 함수의 자유 변수를 사용하지 않으므로 __closure__가 None이다.

따라서 두 값은 동일하지 않다.

여기서 다음 네 가지를 구별해야 한다.

비교 대상	의미
함수 객체의 동일성	같은 함수 객체인가?
메타데이터의 값	이름, 독스트링 등이 같은가?
코드 객체의 동일성	같은 코드 객체를 사용하는가?
클로저 환경	어떤 외부 바인딩에 접근하는가?

`functools.wraps`는 이 네 가지를 모두 동일하게 만드는 기능이 아니다.

A.8.9 `__wrapped__`와 `__closure__`의 차이

이번 학습에서 가장 중요한 부분 중 하나다.

다음 두 속성은 모두 원본 함수 객체에 접근하는 데 사용될 수 있다.

```
wrapper.__wrapped__
```

```
wrapper.__closure__
```

하지만 두 속성의 의미는 전혀 다르다.

`__closure__`

클로저를 구성하는 셀 객체들의 튜플이다.

함수 본문에서 참조하는 자유 변수의 바인딩을 유지하기 위해 사용된다.

`__wrapped__`

`functools.update_wrapper`가 `wrapper`에 설정하는 속성이다.

어떤 객체를 감싸고 있는지 외부 도구가 추적할 수 있도록 제공하는 메타데이터다.

다음 코드를 살펴보자.

```
from functools import wraps

def decorator(func):

    @wraps(func)
```

```
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@decorator
def greet():
    return "Hello"


original_from_metadata = greet.__wrapped__

original_from_closure = greet.__closure__[0].cell_contents

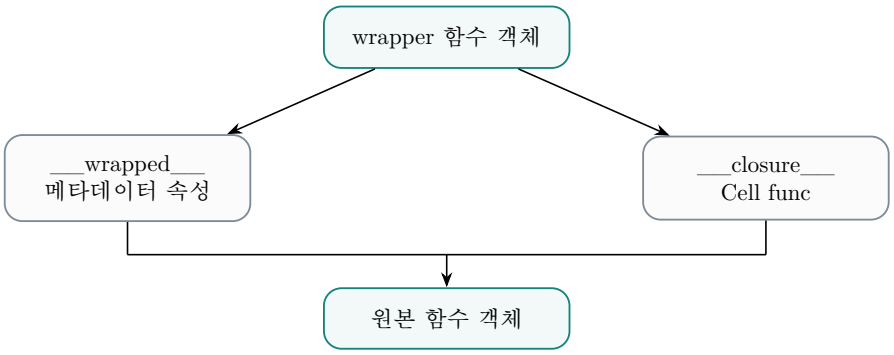
print(original_from_metadata is original_from_closure)
```

예상 출력:

```
True
```

두 경로가 같은 원본 함수 객체에 도달한다.

그렇다고 두 경로가 같은 메커니즘이라는 뜻은 아니다.



같은 원본 함수 객체를 참조하더라도, __wrapped__는 명시적인 속성이고 __closure__는 자유 변수 바인딩을 유지하는 셀 구조다.

클로저가 없어도 __wrapped__는 존재할 수 있다

이 차이를 명확하게 보여주는 예제를 살펴보자.

```
from functools import wraps
```

```
def original():
    return 10


@wraps(original)
def wrapper():
    return 0


print(wrapper.__closure__)
print(wrapper.__wrapped__ is original)
print(wrapper())
```

예상 출력:

```
None
True
0
```

왜 이런 결과가 발생할까?

`wrapper` 본문에서는 외부 함수의 자유 변수를 참조하지 않는다.

따라서 클로저가 필요하지 않다.

하지만 `@wraps(original)`은 `wrapper`의 `__wrapped__` 속성에 원본 함수 객체를 설정한다.

그러므로 `__wrapped__`는 존재하지만 `__closure__`는 `None`일 수 있다.

또한 `wrapper()`를 호출하면 0이 반환된다.

`__wrapped__`가 원본 함수 객체를 참조한다고 해서 Python이 자동으로 원본 함수를 호출하는 것은 아니기 때문이다.

`__wrapped__`는 호출을 위임하는 장치가 아니라 원본 객체를 가리키는 참조 정보다.

A.8.10 `inspect.signature()`는 무엇을 조회할까?

이제 함수 시그니처 문제를 살펴보자.

다음 코드를 실행해 보자.

```
import inspect
```

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@decorator
def power(base, exponent=2):
    return base ** exponent


print(inspect.signature(power))

print(
    inspect.signature(
        power,
        follow_wrapped=False
    )
)
```

예상 출력:

```
(base, exponent=2)
(*args, **kwargs)
```

두 결과가 다른 이유는 무엇일까?

기본적으로 `inspect.signature()`는 함수 객체에 `__wrapped__` 체인이 있으면 이를 따라가며 시그니처를 분석할 수 있다.

따라서 첫 번째 호출은 원본 `power`의 시그니처를 반환한다.

반면 다음 옵션을 사용하면

```
follow_wrapped=False
```

`wrapper`의 실제 함수 정의를 기준으로 시그니처를 조회한다.

이 예제에서 `wrapper`는 다음과 같이 정의되어 있다.

```
def wrapper(*args, **kwargs):
    return func(*args, **kwargs)
```

따라서 `(*args, **kwargs)`가 반환되는 것이다.

코드 객체를 통해 검증하기

```
print(power.__code__.co_varnames)

print(power.__wrapped__.__code__.co_varnames)
```

예상 출력:

```
('args', 'kwargs')
('base', 'exponent')
```

이 결과는 다음 사실을 보여준다.

`functools.wraps`를 적용했더라도 현재 `power`가 참조하는 함수 객체의 코드는 `wrapper`의 코드다.

반면 `power.__wrapped__`는 원본 함수 객체를 참조하므로 원본 매개변수 이름을 확인할 수 있다.

따라서 다음 두 표현식은 서로 다른 질문에 답하는 것이다.

```
inspect.signature(power)
```

기본 설정에서는 외부에 표현되는 원본 함수의 호출 인터페이스를 조회하려는 동작이다.

```
inspect.signature(power, follow_wrapped=False)
```

반면 이 표현식은 `__wrapped__`를 따라가지 않고 현재 함수 객체 자체의 시그니처를 조회하려는 동작이다.

단, `inspect.signature()`는 `__signature__` 같은 별도의 메타데이터도 고려할 수 있으므로, 모든 상황에서 두 번째 호출이 내부 구현의 모든 호출 제약을 완전히 설명한다고 단정할 수는 없다.

A.8.11 표시되는 시그니처와 실제 호출 규칙은 다를 수 있다

이제 조금 더 중요한 문제를 살펴보자.

`functools.wraps`를 적용하면 `inspect.signature()`는 원본 함수의 시그니

처를 반환할 수 있다.

그렇다면 Python이 실제로 `wrapper`를 호출할 때도 원본 함수의 시그니처를 사용해서 인수를 바인딩할까?

그렇지 않다.

다음 예제를 보자.

```
import inspect
from functools import wraps

def add_prefix(func):

    @wraps(func)
    def wrapper(prefix, *args, **kwargs):
        return prefix + func(*args, **kwargs)

    return wrapper


@add_prefix
def greet(name):
    return f"Hello, {name}"


print(inspect.signature(greet))

print(
    inspect.signature(
        greet,
        follow_wrapped=False
    )
)

print(greet(">> ", "Kim"))
```

예상 출력:

```
(name)
(prefix, *args, **kwargs)
>> Hello, Kim
```

여기서 외부에 표시되는 시그니처는 다음과 같다.

```
(name)
```

하지만 실제 wrapper의 정의는 다음과 같다.

```
def wrapper(prefix, *args, **kwargs):
    ...
```

따라서 다음 호출은

```
greet(">> ", "Kim")
```

실제로 wrapper의 매개변수에 다음과 같이 바인딩된다.

```
prefix → ">> "
args   → ("Kim",)
kwargs →
```

그다음 원본 함수가 호출된다.

```
func("Kim")
```

결과적으로 "Hello, Kim"이 반환되고 wrapper가 접두어를 추가한다.

여기서 중요한 문제가 드러난다.

functools.wraps가 원본 시그니처를 외부에 표현하도록 도와주더라도, 실제 wrapper의 인수 바인딩 규칙을 변경하지는 않는다.

따라서 원본 함수와 wrapper의 호출 인터페이스가 다르다면

inspect.signature()의 기본 결과가 실제 wrapper 호출 방법을 충분히 설명하지 못할 수 있다.

이것은 문서 자동 생성이나 함수 시그니처를 분석하는 프레임워크에서 중요한 문제다.

판단 기준

- Wrapper가 원본 함수의 인수와 반환값을 그대로 중개한다면 @wraps(func)는 대체로 자연스러운 선택이다.
- Wrapper가 호출 인터페이스를 변경한다면 원본 시그니처를 그대로 노출하는 것이 적절한지 검토해야 한다.
- 실제 호출 가능 여부는 최종적으로 호출 대상 객체의 동작과 인수 바인딩 규칙에 따라 결정된다.

A.8.12 `__wrapped__` 체인과 `inspect.unwrap()`

이번에는 데코레이터가 여러 번 적용되는 경우를 살펴보자.

A.10에서 데코레이터의 평가 순서와 적용 순서를 자세히 분석할 것이므로, 여기서는 원본 함수 참조 구조에만 집중할 것이다.

```
import inspect
from functools import wraps

def first(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


def second(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@first
@second
def calculate(x):
    return x * 2


outer = calculate
middle = calculate.__wrapped__
original = middle.__wrapped__

print(outer is middle)
print(middle is original)

print(inspect.unwrap(calculate) is original)

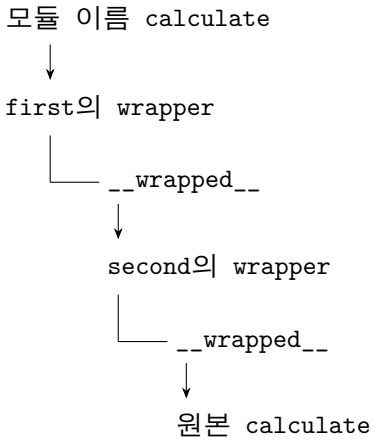
print(calculate(10))
```

예상 출력:

```
False
```

```
False
True
20
```

여기에는 서로 다른 세 함수 객체가 존재한다.



각 wrapper에 functools.wraps가 적용되었기 때문에

__wrapped__ 속성이 연결된다.

따라서 다음 표현식은

```
calculate.__wrapped__
```

바로 안쪽의 wrapper 함수 객체를 반환한다.

그리고 다음 표현식은

```
calculate.__wrapped__.__wrapped__
```

원본 함수 객체를 반환한다.

이러한 연결 구조를 따라 최종 원본 객체를 찾기 위해

inspect.unwrap()을 사용할 수 있다.

```
inspect.unwrap(calculate)
```

이 함수는 기본적으로 __wrapped__ 체인을 따라가 마지막 객체를 반환한다.

따라서 직접 여러 번 .__wrapped__를 조회하지 않아도 원본 함수 객체에 접근할 수 있다.

다만 inspect.unwrap()은 모든 경우에 무조건 마지막 함수까지 이동하는 단순 반복 함수만은 아니다.

사용자가 지정한 중단 조건이 있으면 중간에 탐색을 멈출 수 있고, 순환 참조가 발견되면 ValueError를 발생시킬 수 있다.

또한 inspect.signature()는 자체적인 시그니처 분석 규칙에 따라

__wrapped__ 체인을 탐색한다.

A.8.13 __dict__ 갱신과 얕은 복사(Shallow Copy)

이제 update_wrapper의 두 번째 주요 동작인 __dict__ 갱신을 살펴본다.

일반적인 Python 함수 객체는 사용자 정의 속성을 가질 수 있다.

```
def calculate(x):
    return x * 2

calculate.category = ["math"]
```

이때 category는 함수 객체에 추가된 속성이다.

functools.wraps는 기본적으로 원본 함수의 __dict__에 있는 항목을

wrapper의 __dict__에 갱신한다.

다음 코드를 살펴보자.

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


def calculate(x):
    return x * 2
```

```
calculate.category = ["math"]

wrapped = decorator(calculate)

print(wrapped.__dict__ is calculate.__dict__)

print(wrapped.category is calculate.category)

wrapped.category.append("python")

print(calculate.category)
```

예상 출력:

```
False
True
['math', 'python']
```

이 결과를 구분해서 해석해야 한다.

첫 번째 비교

```
wrapped.__dict__ is calculate.__dict__
```

`False`다.

`wrapper`와 원본 함수는 서로 다른 함수 객체이므로 일반적으로 서로 다른 속성 네임스페이스를 가진다.

`update_wrapper`가 두 함수의 `__dict__` 자체를 하나의 딕셔너리로 통합하는 것은 아니다.

두 번째 비교

```
wrapped.category is calculate.category
```

`True`다.

`__dict__`의 항목을 갱신할 때 리스트 객체를 깊은 복사(Deep Copy)하지 않기 때문이다.

따라서 두 함수 객체의 `category` 속성이 같은 리스트 객체를 참조할 수 있다.

세 번째 연산

```
wrapped.category.append("python")
```

기존 리스트 객체를 직접 변경한다.

원본 함수의 category도 같은 리스트 객체를 참조하므로 변경된 내용을 관찰할 수 있다.

이것은 A.5에서 학습한 객체 변경과 이름 재바인딩의 차이와 연결된다.

functools.wraps가 메타데이터를 복사한다는 설명을 모든 참조 객체에 대한 깊은 복사로 이해하면 안 된다.

참고로 __annotations__도 기본적으로 원본의 속성값이 wrapper에 할당된다.

따라서 일반적인 함수에서는 두 객체의 타입 주석 딕셔너리가 동일한 객체일 수 있다.

A.8.14 functools.wraps가 보존하지 않는 것

지금까지 functools.wraps가 설정하는 정보를 살펴보았다.

이제 반대로 무엇을 보존하지 않는지 정리해 본다.

항목	functools.wraps의 기본 동작
함수 이름	원본의 __name__을 설정
독스트링	원본의 __doc__를 설정
타입 주석	원본의 __annotations__를 설정
사용자 정의 속성	기본적으로 __dict__ 항목 갱신
원본 객체 참조	__wrapped__ 설정
실제 코드 객체	교체하지 않음
클로저 셀	원본의 셀로 교체하지 않음
실제 인수 바인딩 규칙	변경하지 않음
반환값 처리	변경하지 않음
예외 처리	변경하지 않음
동기·비동기 실행 방식	자동 변환하지 않음

여기서 중요한 원칙은 다음이다.

`functools.wraps`는 함수의 실행 의미를 보존하는 장치가 아니라, 메타데이터와 원본 참조를 설정하는 장치다.

따라서 다음과 같은 데코레이터를 작성할 수도 있다.

```
from functools import wraps

def skip(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return "Skipped"

    return wrapper


@skip
def calculate(x):
    return x * 2


print(calculate.__name__)
print(calculate(10))
print(calculate.__wrapped__(10))
```

예상 출력:

```
calculate
Skipped
20
```

`calculate.__name__`은 원본 함수의 이름으로 설정되어 있다.

하지만 `calculate(10)`은 원본 함수를 호출하지 않고 `"Skipped"`를 반환한다.

반면 `calculate.__wrapped__(10)`은 원본 함수 객체를 직접 호출하므로 20을 반환한다.

이 예제는 메타데이터와 실제 동작이 서로 독립적이라는 사실을 보여준다.

또한 `__wrapped__`를 통해 원본 함수에 접근할 수 있으므로, `wrapper`에 구현한 검사만으로 원본 함수의 직접 호출까지 차단된다고 가정하면 안 된다.

A.8.15 실전 응용 - 로깅 데코레이터와 함수 분석

이번에는 실제로 활용할 수 있는 로깅 데코레이터를 작성해 보자.

```
import inspect
from functools import wraps

def log_execution(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[START] {func.__name__}")

        try:
            return func(*args, **kwargs)

        finally:
            print(f"[END] {func.__name__}")

    return wrapper


@log_execution
def predict(features: list[float], scale: float = 1.0) -> float:
    """입력 특성값의 합계에 배율을 적용한다."""
    return sum(features) * scale


print(predict([1.0, 2.0, 3.0], scale=2.0))

print(predict.__name__)
print(predict.__doc__)
print(inspect.signature(predict))
```

이 코드는 세 가지 기능을 분리하고 있다.

> 첫째, 원본 함수의 계산 책임

predict는 입력 특성값을 합산하고 배율을 적용하는 계산만 담당한다.

> 둘째, wrapper의 실행 기록 책임

wrapper는 원본 함수의 호출 전후에 로그를 출력한다.

finally를 사용하므로 원본 함수에서 예외가 발생해도 종료 로그를 기록하려고 시도한다.

셋째, functools.wraps의 메타데이터 관리 책임

@wraps(func)는 wrapper에 원본 함수의 메타데이터를 설정한다.

따라서 함수 이름, 독스트링, 타입 주석 등의 정보를 외부 도구에서 활용할 수 있다.

이 구조는 PyTorch 모델의 추론 함수나 데이터 전처리 함수에 공통 로깅 기능을 추가할 때도 응용할 수 있다.

다만 functools.wraps 자체가 Tensor의 계산 그래프, 자동 미분, 장치 배치, 메모리 사용 등을 제어하는 것은 아니다.

그러한 동작은 원본 함수와 wrapper가 실제로 수행하는 연산에 따라 결정된다.

A.8.16 FastAPI와 함수 시그니처 분석의 연결

이제 이 개념이 왜 웹프레임워크에서 중요한지 살펴보자.

FastAPI와 같은 프레임워크는 함수의 시그니처를 분석하여 매개변수와 요청 데이터를 연결하는 기능을 제공한다.

예를 들어 다음과 같은 함수가 있다고 하자.

```
def get_item(item_id: int):
    return {"item_id": item_id}
```

함수의 시그니처에는 다음 정보가 포함된다.

```
(item_id: int)
```

그런데 이 함수에 일반적인 wrapper를 적용하면 실제 함수 객체의 정의는 다음과 같이 바뀔 수 있다.

```
def wrapper(*args, **kwargs):
    ...
```

이때 프레임워크가 어떤 객체의 시그니처를 분석하는지가 중요해진다.

functools.wraps는 __wrapped__를 설정하므로 inspect.signature()가 원본 함수의 시그니처를 추적하는 데 도움을 줄 수 있다.

하지만 여기서 다음과 같이 단정하면 안 된다.

> `@wraps`를 사용하면 모든 FastAPI 데코레이터가 정상적으로 동작한다.

실제 동작은 데코레이터의 적용 순서, 동기·비동기 함수 여부, `wrapper`의 인수 처리 방식, 프레임워크가 함수를 등록하고 분석하는 방식 등에 따라 달라질 수 있다.

따라서 `functools.wraps`는 함수 메타데이터 보존에 중요한 도구이지만, 프레임워크의 모든 호출 계약을 자동으로 보장하는 장치는 아니다.

이 부분은 A.12의 등록형 데코레이터와 FAstAPI에서 구체적인 실행 구조로 다시 분석할 것이다.

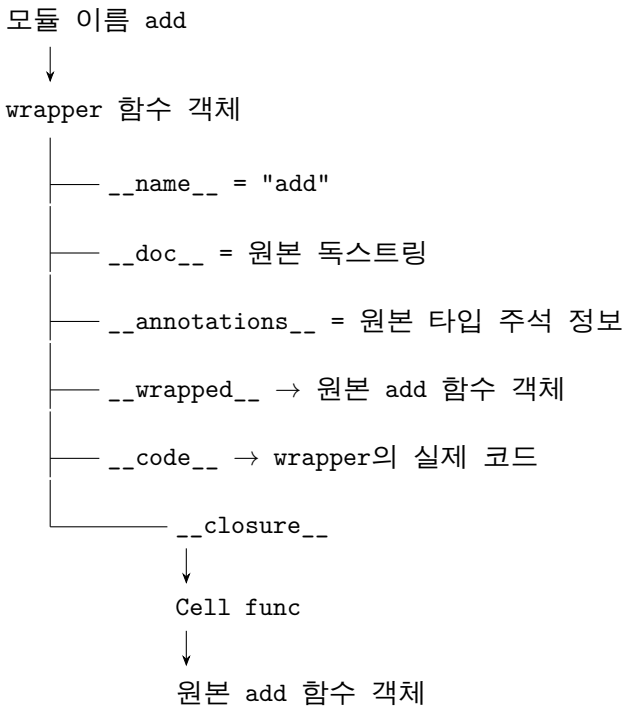
A.8.17 자주 발생하는 오해와 예외

이번 학습에서 주의해야 할 개념을 정리해 본다.

오해	정확한 설명
`@wraps(func)`는 원본 함수를 호출한다.	`wraps(func)`는 메타데이터 설정용 데코레이터를 생성한다.
`functools.wraps`는 새로운 함수 객체를 만든다.	기존 `wrapper` 함수 객체의 메타데이터를 설정하는 방식이다.
`__name__`이 같으면 같은 함수 객체다.	메타데이터 값과 객체 동일성은 별개다.
`__wrapped__`와 `__closure__`는 같은 역할이다.	전자는 원본 참조 메타데이터, 후자는 자유 변수 바인딩 환경이다.
`inspect.signature()`는 항상 실제 `wrapper`의 시그니처를 보여준다.	기본적으로 `__wrapped__` 체인을 따라 원본 시그니처를 조회할 수 있다.
`@wraps`를 사용하면 인수 전달이 자동으로 처리된다.	실제 인수 전달은 `wrapper` 구현에 달려 있다.
`@wraps`를 사용하면 원본의 실행 동작이 보존된다.	반환값, 예외, 실행 여부 등은 `wrapper`가 결정할 수 있다.
메타데이터 복사는 모든 객체의 깊은 복사다.	기본적으로 속성 할당과 매핑 갱신을 수행한다.
여러 데코레이터가 적용되면 원본 함수에 접근할 수 없다.	`__wrapped__` 체인과 `inspect.unwrap()`으로 추적할 수 있다.

A.8.18 핵심 정리와 판단 기준

이번 학습에서 가장 중요한 객체 관계를 다시 정리해 본다.



이 구조에서 중요한 사실은 다음과 같다.

함수 객체의 정체성은 메타데이터와 별개다.

functools.wraps가 __name__, __doc__, __annotations__ 등을 설정하더라도 wrapper와 원본 함수는 여전히 서로 다른 객체다.

__wrapped__와 __closure__는 서로 다른 참조 메커니즘이다.

__wrapped__는 원본 객체를 추적하기 위한 속성이고, __closure__는 자유 변수 바인딩을 유지하기 위한 구조다.

inspect.signature()는 실제 호출을 수행하는 함수 객체의 매개변수 정의와 다른 정보를 보여줄 수 있다.

기본적으로 __wrapped__를 따라가 원본 함수의 시그니처를 분석할 수 있기 때문이다.

functools.wraps는 호출 의미를 자동으로 보존하지 않는다.

Wrapper가 인수를 어떻게 전달하고, 원본 함수를 언제 호출하고, 반환값과 예외를 어떻게 처리하는지는 실제 구현에 따라 결정된다.

따라서 데코레이터를 분석할 때는 다음 네 가지 질문을 순서대로 확인하면 된다.

1. 현재 이름은 어떤 함수 객체를 참조하는가?
2. 그 함수 객체의 실제 코드와 호출 규칙은 무엇인가?
3. `__wrapped__`는 어떤 객체를 참조하는가?
4. 외부 도구가 표시하는 메타데이터는 어느 객체에서 유래했는가?

이 기준을 적용하면 `functools.wraps`를 단순한 이름 복사 도구가 아니라, 함수 객체의 실행 구조와 외부에 노출되는 인터페이스 정보를 연결하는 장치로 이해할 수 있다.

A.9 인수를 받는 데코레이터와 데코레이터 팩토리

Decorator Factory · Higher-Order Function · Nested Closure · Argument Binding · Configuration State

A.9.1 이전 학습과의 연결

A.6부터 A.8까지는 다음과 같은 데코레이터를 중심으로 학습했다.

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@decorator
def add(a, b):
    return a + b
```

여기서 @decorator는 일반적인 단일 데코레이터 상황에서 다음 변환식과 대응했다.

```
add = decorator(add)
```

이 구조에는 두 번의 함수 호출이 관여한다.

첫 번째는 함수 정의문이 실행될 때 발생하는 decorator(add) 호출이고, 두 번째는 이후 add(10, 20)을 실행할 때 발생하는 wrapper 호출이다.

A.8에서는 이 구조에 functools.wraps를 추가하여 원본 함수의 메타데이터와 __wrapped__ 참조를 유지하는 방법까지 학습했다.

그런데 다음과 같은 기능을 구현하려면 어떻게 해야 할까?

```
@repeat(3)
def greet():
    print("Hello")
```

이 코드는 greet()을 한 번 호출할 때 원본 함수의 본문을 세 번 실행하도록 만들려는 의도를 가지고 있다.

여기에는 두 종류의 인수가 존재한다.

- 3: 데코레이터의 동작을 설정하는 함수
- greet()에 전달하는 인수: 실제 원본 함수를 실행할 때 사용하는 인수

이 둘은 서로 다른 함수 호출에 속한다.

이번 학습에서는 이 차이를 바탕으로 데코레이터를 생성하는 함수, 원본 함수를 전달받는 함수, 실제 호출을 중개하는 함수를 각각 구별할 것이다.

A.9.2 이번 학습의 핵심 질문

이번 학습에서 해결할 질문은 다음과 같다.

1. @repeat(3)은 왜 @repeat와 다른가?
2. repeat(3)(greet)에는 어떤 함수 객체와 호출 결과가 관여하는가?
3. 세 단계로 중첩된 함수는 각각 어떤 네임스페이스를 가지는가?

4. 설정값 `times`와 원본 함수 객체 `func`는 어느 클로저 셀에 유지되는가?

5. `repeat(2)`와 `repeat(5)`로 만든 데코레이터는 상태를 공유하는가?

6. 데코레이터가 반환한 `wrapper`는 원본 함수의 인수와 반환값을 어떻게 처리하는가?

7. 데코레이터의 설정값을 변경하거나 검증하려면 어떤 조건을 고려해야 하는가?

가장 중요한 개념은 다음이다.

> 인수를 받는 데코레이터는 일반적으로 데코레이터를 반환하는 함수,
>
> 즉 데코레이터 팩토리(Decorator Factory)를 통해 구현한다.

여기서 팩토리는 새로운 종류의 Python 객체가 아니다.

다른 함수를 생성하거나 반환하는 역할을 수행하는 일반적인 호출 가능한 객체를 의미한다.

A.9.3 `@repeat(3)`의 기본 변환식

먼저 두 문법을 비교해 보자.

인수가 없는 데코레이터

```
@decorator
def greet():
    print("Hello")
```

대응 관계:

```
greet = decorator(greet)
```

인수를 받는 데코레이터

```
@repeat(3)
def greet():
    print("Hello")
```

대응 관계:

```
greet = repeat(3)(greet)
```

두 번째 변환식의 오른쪽을 분석해 보자.

```
repeat(3)(greet)
```

이 표현식은 하나의 함수에 인수 두 개를 전달하는 호출이 아니다.

다음과 같이 두 번의 호출이 연속으로 이루어지는 표현식이다.

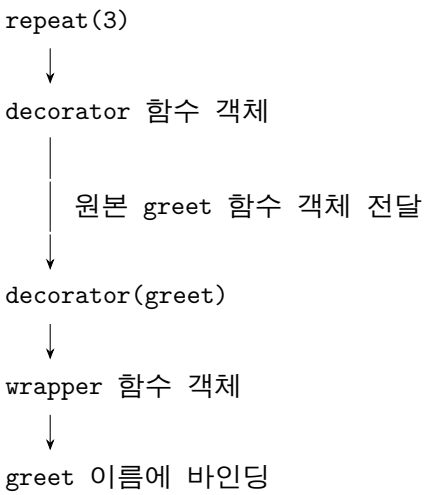
```
decorator = repeat(3)

replacement = decorator(greet)
```

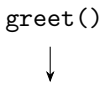
첫 번째 호출인 `repeat(3)`의 결과가 호출 가능한 객체여야 두 번째 호출인 `decorator(greet)`이 가능하다.

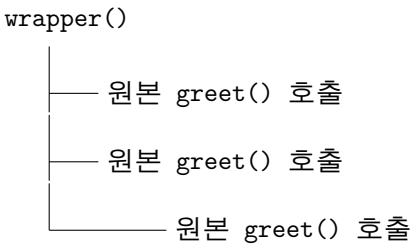
일반적으로 이 결과가 원본 함수를 전달받는 데코레이터 함수 객체다.

이후 데코레이터가 wrapper를 반환한다면 다음과 같은 관계가 성립한다.



그다음 사용자가 `greet()`을 호출하면 세 번째 함수 호출 단계가 시작된다.





따라서 `@repeat(3)` 구조에서는 다음 세 호출을 구별해야 한다.

단계	호출 표현식	목적	시점
1	`repeat(3)`	설정값을 받아 데코레이터 생성	정의문 실행 중
2	`decorator(greet)`	원본 함수를 받아 wrapper 생성	정의문 실행 중
3	`greet()`	wrapper를 통해 원본 함수 실행	이후 실제 호출 시

이 세 단계를 혼동하지 않는 것이 이번 학습의 출발점이다.

A.9.4 `repeat(3)`을 직접 구현해 보자

먼저 독립적으로 실행할 수 있는 전체 코드를 살펴본다.

```
from functools import wraps

def repeat(times):
    print("factory:", times)

    def decorator(func):
        print("decorator:", func.__name__)

        @wraps(func)
        def wrapper(*args, **kwargs):
            print("wrapper start")

            result = None

            for i in range(times):
                print("iteration:", i + 1)
                result = func(*args, **kwargs)

            print("wrapper end")
```

```
            return result

        return wrapper

    return decorator


@repeat(3)
def greet(name):
    print("Hello", name)
    return name.upper()


print("definition complete")

print("result:", greet("Kim"))
```

예상 출력:

```
factory: 3
decorator: greet
definition complete
wrapper start
iteration: 1
Hello Kim
iteration: 2
Hello Kim
iteration: 3
Hello Kim
wrapper end
result: KIM
```

이 출력은 세 함수의 실행 시점을 명확하게 보여준다.

`factory: 3`은 `repeat(3)`이 호출될 때 출력된다.

`decorator: greet`은 반환된 데코레이터가 원본 `greet` 함수 객체를 전달받을 때 출력된다.

반면 `wrapper start`부터 `wrapper end`까지는 실제로 `greet("Kim")`을 호출한 뒤에 출력된다.

즉, 데코레이터를 생성하는 것, 원본 함수를 데코레이션하는 것, 데코레이션된 함수를 실행하는 것은 서로 다른 단계다.

이제 각 단계에서 어떤 객체가 생성되고 어떤 이름이 바인딩되는지 분석해 보자.

A.9.5 첫 번째 단계 - repeat(3)과 설정값 바인딩

다음 표현식이 평가된다.

```
repeat(3)
```

이때 repeat 함수가 호출되고 매개변수 times가 정수 객체 3에 바인딩된다.

```
def repeat(times):
    ...
```

함수 본문에서 다음 중첩 함수 정의문이 실행된다.

```
def decorator(func):
    ...
```

여기서 새로운 decorator 함수 객체가 생성된다.

그다음 다음 문장을 만난다.

```
return decorator
```

이 문장은 decorator()를 호출하는 것이 아니다.

생성된 decorator 함수 객체 자체를 반환하는 문장이다.

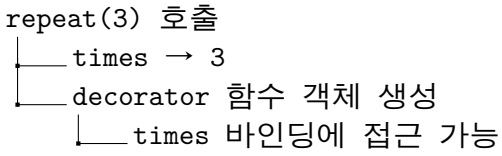
그렇다면 repeat() 호출이 종료된 뒤에도 decorator는 설정값 times를 사용할 수 있을까?

가능하다.

왜냐하면 decorator 내부에서 정의된 wrapper가 times를 참조하기 때문이다.

Python은 중첩된 함수들이 외부 함수의 바인딩에 접근할 수 있도록 클로저 환경을 구성한다.

여기서 A.4의 자유 변수와 셀 객체 개념이 다시 등장한다.



repeat() 호출이 끝났다고 해서 times를 참조하는 클로저 환경까지 즉시 소멸하는 것은 아니다.

반환된 함수 객체가 해당 바인딩에 접근할 수 있는 상태를 유지하는 것이다.

A.9.6 두 번째 단계 - decorator(greet)과 원본 함수 바인딩

이제 repeat(3)이 반환한 함수 객체가 원본 greet 함수 객체를 인수로 전달받아 호출된다.

개념적으로 다음과 같다.

```
configured_decorator = repeat(3)

replacement = configured_decorator(greet)
```

여기서 configured_decorator는 repeat(3)이 반환한 decorator 함수 객체를 참조한다.

그리고 configured_decorator(greet) 호출에서는 매개변수 func가 원본 greet 함수 객체에 바인딩된다.

```
def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        ...
```

이 단계에서 새로운 wrapper 함수 객체가 생성된다.

Wrapper는 두 가지 외부 이름을 사용한다.

```
times
func
```

times는 가장 바깥쪽 함수인 repeat의 매개변수이고, func는 바로 바깥쪽 함수인 decorator의 매개변수다.

따라서 wrapper는 서로 다른 렉시컬 스코프에 정의된 두 바인딩에 접근해야 한다.

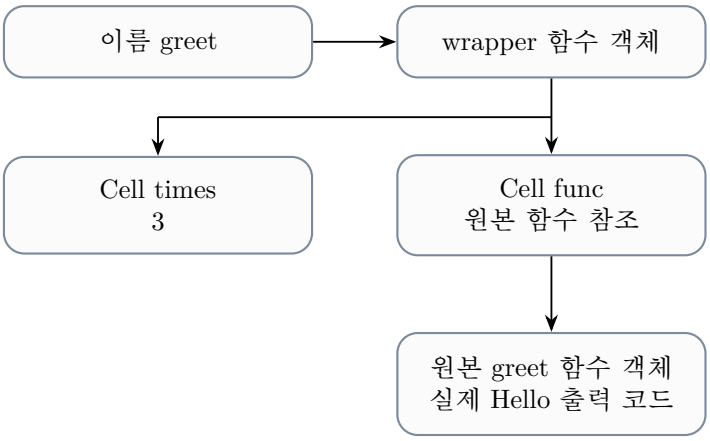
@wraps(func)도 이 단계에서 적용된다.

A.8에서 학습한 것처럼 wrapper의 메타데이터가 설정되고 `__wrapped__` 속성이 원본 함수 객체를 참조하게 된다.

마지막으로 다음 문장이 실행된다.

```
return wrapper
```

이제 데코레이터는 wrapper 함수 객체를 반환하고, 그 객체가 최종적으로 모듈 이름 `greet`에 바인딩된다.



데코레이션 이후

Wrapper는 설정값 times와 원본 함수 객체 func를 서로 다른 클로저 셀을 통해 참조한다. 그림은 핵심 바인딩 관계를 단순화한 것이다.

여기서 중요한 점은 `times`와 `func`가 같은 함수의 매개변수가 아니라는 사실이다.

`times`는 `repeat()` 호출에서 바인딩되고, `func`는 `decorator()` 호출에서 바인딩된다.

두 이름이 만들어지는 시점과 네임스페이스가 서로 다르다.

A.9.7 세 번째 단계 - Wrapper 호출과 반복 실행

이제 다음 문장을 실행한다고 하자.

```
greet("Kim")
```

현재 모듈 이름 greet은 wrapper 함수 객체를 참조한다.

따라서 wrapper가 호출되고 다음 바인딩이 형성된다.

```
args = ("Kim",)
kwargs = {}
```

Wrapper는 자신의 클로저를 통해 times가 참조하는 정수 객체 3을 조회한다.

그다음 다음 반복문을 실행한다.

```
for i in range(times):
    result = func(*args, **kwargs)
```

반복문은 세 번 실행된다.

각 반복에서 func가 참조하는 원본 greet 함수 객체가 호출된다.

원본 함수는 "Hello Kim"을 출력하고 문자열 "KIM"을 반환한다.

그 반환 객체가 wrapper의 지역 이름 result에 바인딩된다.

세 번의 호출이 완료되면 마지막 호출의 반환 객체가 result에 바인딩된 상태로 남아 있다.

따라서 다음 문장은

```
return result
```

마지막 원본 함수 호출의 반환 객체를 최종 호출자에게 전달한다.

여기서 중요한 점이 있다.

repeat(3)이 원본 함수의 반환값을 세 배로 만드는 것은 아니다. 원본 함수를 세 번 호출하는 것이다.

원본 함수가 상태를 변경한다면 그 상태 변경도 세 번 발생할 수 있다.

A.9.8 repeat(3)의 평가 시점과 함수 정의 시점

A.6에서는 데코레이터가 함수 정의문 실행 중 적용된다고 설명했다.

이번에는 한 단계 더 정확하게 구별한다.

데코레이터 표현식 repeat(3)은 원본 함수 객체가 생성되기 전에 평가된다.

반면 repeat(3)이 반환한 데코레이터는 원본 함수 객체가 생성된 다음 적용된다.

다음 코드로 확인해 보자.

```
def factory(times):
    print("factory:", times)

    def decorator(func):
        print("decorator:", func.__name__)
        return func

    return decorator

@factory(2)
def greet(value=print("default evaluated")):
    print("body")

print("definition complete")
greet()
```

예상 출력:

```
factory: 2
default evaluated
decorator: greet
definition complete
body
```

왜 이런 순서일까?

먼저 @factory(2)라는 데코레이터 표현식이 평가된다.

따라서 factory(2)가 호출되고 데코레이터 함수 객체가 반환된다.

그다음 함수 정의에 필요한 기본 인수 표현식이 평가된다.

```
value=print("default evaluated")
```

이 표현식은 "default evaluated"를 출력하고 None을 반환한다.

그 결과가 기본 인수 객체로 사용된다.

이후 원본 greet 함수 객체가 생성되고, 앞에서 준비한 데코레이터가 적용

된다.

마지막으로 데코레이터의 반환 객체가 greet이라는 이름에 바인딩된다.

따라서 A.9.3의 변환식

```
greet = repeat(3)(greet)
```

은 데코레이터 적용의 의미를 설명하는 데 유용하지만, 모든 부작용의 평가 순서까지 완전히 재현하는 문자 그대로의 소스 코드 치환은 아니다.

이 차이는 데코레이터 표현식이나 기본 인수 표현식에 부작용이 있을 때 중요해진다.

A.9.9 세 단계의 클로저를 직접 검증하기

이번에는 A.4와 A.5에서 사용한 co_cellvars, co_freevars, __closure__를 활용해 내부 구조를 확인해 보자.

```
from functools import wraps

def repeat(times):

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = func(*args, **kwargs)

            return result

        return wrapper

    return decorator


configured = repeat(2)


def square(x):
    return x * x
```

```
wrapped = configured(square)

print(repeat.__code__.co_cellvars)

print(configured.__code__.co_freevars)
print(configured.__code__.co_cellvars)

print(wrapped.__code__.co_freevars)

closure_map = dict(
    zip(wrapped.__code__.co_freevars, wrapped.__closure__)
)

print(closure_map["times"].cell_contents)
print(closure_map["func"].cell_contents is square)

print(wrapped.__wrapped__ is square)
```

예상 출력:

```
('times',)
('times',)
('func',)
('func', 'times')
2
True
True
```

각 결과를 분석해 보자.

1. repeat의 셀 변수

```
repeat.__code__.co_cellvars
```

결과는 다음과 같다.

```
('times',)
```

`times`는 `repeat`의 지역 매개변수이지만 중첩 함수에서도 참조된다.

따라서 클로저 셀을 통해 유지할 필요가 있다.

2. decorator의 자유 변수

```
configured.__code__.co_freevars
```

결과는 다음과 같다.

```
('times',)
```

decorator 함수는 repeat의 렉시컬 환경에서 정의되었다.

그 내부에 정의되는 wrapper가 times를 사용하므로, 이 바인딩을 중첩된 함수까지 전달할 수 있는 클로저 구조가 형성된다.

3. decorator의 셀 변수

```
configured.__code__.co_cellvars
```

결과는 다음과 같다.

```
('func',)
```

func는 decorator의 지역 매개변수다.

그런데 wrapper가 이 이름을 참조하므로 셀 변수로 관리된다.

4. wrapper의 자유 변수

```
wrapped.__code__.co_freevars
```

결과는 다음과 같다.

```
('func', 'times')
```

Wrapper는 원본 함수 객체아ㄴ 설정값에 모두 접근해야 하므로 두 자유 변수를 사용한다.

이때 __closure__의 셀 순서를 임의로 가정하면 안 된다.

따라서 다음과 같이 이름과 셀을 대응시키는 것이 정확하다.

```
closure_map = dict(
    zip(wrapped.__code__.co_freevars, wrapped.__closure__)
)
```

이후 다음 표현식으로 각 바인딩을 확인할 수 있다.

```
closure_map["times"].cell_contents
closure_map["func"].cell_contents
```

여기서 핵심은 다음이다.

데코레이터 팩토리는 중첩 함수가 세 단계라는 점이 특별한 것이 아니라,

서로 다른 단계에서 형성된 바인딩을 최종 wrapper가 함께 참조한다.

이 점이 중요하다.

A.9.10 서로 다른 팩토리 호출은 상태를 공유할까?

다음 코드를 살펴보자.

```
from functools import wraps

def repeat(times):

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = func(*args, **kwargs)

            return result

        return wrapper

    return decorator


@repeat(2)
def first():
    print("First")


@repeat(3)
def second():
    print("Second")
```

```
first()
second()
```

예상 출력:

```
First
First
Second
Second
Second
```

여기서 `repeat()`이 두 번 호출된다.

첫 번째 호출에서는 `times`가 2를 참조하고, 두 번째 호출에서는 `times`가 3을 참조한다.

각 호출에서 독립적인 클로저 환경이 형성된다.

따라서 두 wrapper는 서로 다른 `times` 셀을 참조하게 된다.

이를 직접 검증할 수 있다.

```
def cells_of(func):
    return dict(
        zip(func.__code__.co_freevars, func.__closure__)
    )


first_cells = cells_of(first)
second_cells = cells_of(second)

print(
    first_cells["times"] is second_cells["times"]
)
```

예상 출력:

```
False
```

두 wrapper가 서로 다른 팩토리 호출에서 만들어진 셀을 참조하기 때문이다.

> 반대로 하나의 데코레이터를 재사용한다면?

다음 구조를 생각해 보자.

```
configured = repeat(2)
```

```
@configured
def first():
    return 10


@configured
def second():
    return 20
```

여기서는 `repeat(2)`가 한 번만 호출된다.

반환된 동일한 `configured` 데코레이터 함수 객체를 두 함수에 적용한 것이다.

따라서 두 wrapper는 같은 `times` 셀을 공유할 수 있다.

하지만 각각 다른 원본 함수를 전달받았으므로 `func` 셀은 서로 다르다.

비교 대상	first와 second의 관계
`times` 셀	동일한 셀을 공유
`func` 셀	서로 다른 셀
`wrapper` 함수 객체	서로 다른 객체
원본 함수 객체	서로 다른 객체

이것은 팩토리 호출의 횟수와 데코레이터 적용 횟수가 서로 다르기 때문이다.

같은 팩토리 함수로 만들었다는 사실보다, 팩토리를 몇 번 호출했고, 어떤 반환 객체를 재사용했는지가 상태 공유 여부를 결정한다.

A.9.11 설정값을 읽을 때 nonlocal이 필요할까?

다음 코드를 보자.

```
def repeat(times):

    def decorator(func):

        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)

        return wrapper
```

```
    return decorator
```

Wrapper는 `times`를 조회하지만 재바인딩하지 않는다.

따라서 `nonlocal times`가 필요하지 않다.

하지만 다음과 같이 변경하면 상황이 달라진다.

```
def repeat(times):

    def decorator(func):

        def wrapper(*args, **kwargs):
            times += 1

            for _ in range(times):
                func(*args, **kwargs)

        return wrapper

    return decorator
```

이 코드는 wrapper를 호출할 때 `UnboundLocalError`가 발생할 수 있다.

왜냐하면 `times += 1`이 현재 함수 블록에서 일등르 할당하는 연산이기 때문이다.

Python은 `times`를 wrapper의 지역 이름으로 분류하지만, 증가 연산을 위해 값을 읽으려는 시점에는 아직 지역 바인딩이 존재하지 않는다.

외부 `times`를 재바인딩하려면 다음과 같이 작성해야 한다.

```
def wrapper(*args, **kwargs):
    nonlocal times

    times += 1
    ...
```

이 경우 `times`는 외부 `repeat` 호출의 바인딩을 변경하게 된다.

여기서 중요한 점은 `nonlocal`이 설정값을 읽기 위해 필요한 것이 아니라는 사실이다.

외부 바인딩을 재바인딩하려는 경우에 필요한 선언이다.

따라서 설정값을 읽기만 하는 일반적인 데코레이터 팩토리에서는 `nonlocal`

을 사용하지 않아도 된다.

A.9.12 설정값이 유효성 검사는 언제 수행해야 할까?

지금까지 작성한 repeat에는 한 가지 문제가 있다.

다음 호출이 가능하다는 점이다.

```
@repeat(0)
def greet():
    print("Hello")
```

기존 구현에서는 반복문이 한 번도 실행되지 않는다.

따라서 원본 함수도 호출되지 않고 result의 초기값인 None이 반환된다.

이것이 의도한 동작이라면 문제가 없지만, 반복 횟수가 반드시 양의 정수여야 한다는 계약이라면 유효성을 검사해야 한다.

```
from functools import wraps


def repeat(times):

    if (
        not isinstance(times, int)
        or isinstance(times, bool)
        or times < 1
    ):
        raise ValueError("times는 양의 정수여야 합니다.")

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = func(*args, **kwargs)

            return result

        return wrapper

    return decorator
```

이 검사는 `repeat(times)`가 호출되는 시점에 실행된다.

따라서 다음과 같은 코드는 함수 정의문 실행 중 오류가 발생한다.

```
@repeat(0)
def greet():
    print("Hello")
```

왜냐하면 `repeat(0)`이 데코레이터 표현식으로 평가되는 순간 `ValueError`가 발생하기 때문이다.

여기서 `bool`을 별도로 검사한 이유도 있다.

Python에서는 다음 표현식이 `True`다.

```
isinstance(True, int)
```

`bool`이 `int`의 하위 클래스이기 때문이다.

따라서 단순히 `isinstance(times, int)`만 검사하면 `True`도 정수로 받아들일 수 있다.

이처럼 설정값 검증은 데코레이터의 실행 의미를 명확하게 만드는 역할을 한다.

A.9.13 `@repeat`과 `@repeat(3)`을 혼동하면 어떻게 될까?

다음과 같이 정의된 팩토리를 생각해 보자.

```
def repeat(times):

    def decorator(func):
        ...

    return decorator
```

이 함수는 원본 함수 객체가 아니라 설정값 `times`를 먼저 전달받도록 설계되었다.

따라서 정상적인 사용법은 다음과 같다.

```
@repeat(3)
def greet():
```

```
    ...
```

반면 다음과 같이 작성하면 어떻게 될까?

```
@repeat
def greet():
    ...
```

이때 Python은 개념적으로 다음 호출을 수행한다.

```
greet = repeat(greet)
```

따라서 `times`에는 정수 3이 아니라 원본 `greet` 함수 객체가 전달된다.

이후 동작은 팩토리 구현에 따라 달라진다.

앞에서 작성한 유효성 검사가 있다면 함수 객체는 정수가 아니므로 정의 시점에 `ValueError`가 발생한다.

유효성 검사가 없다면 함수 객체가 `times`에 저장된 상태로 데코레이터 함수가 반환될 수 있고, 이후 호출에서 예상하지 못한 오류가 발생할 수 있다.

반대로 다음 코드도 주의해야 한다.

```
@repeat()
def greet():
    ...
```

`repeat`이 필수 매개변수 `times`를 요구한다면 인수를 전달하지 않았으므로 `TypeError`가 발생한다.

정리하면 다음과 같다.

문법	실제 적용 방식
`@repeat`	`repeat(원본 함수 객체)`
`@repeat()`	`repeat()`의 반환값을 원본 함수에 적용
`@repeat(3)`	`repeat(3)`의 반환값을 원본 함수에 적용
`@repeat(times=3)`	`repeat(times=3)`의 반환값을 원본 함수에 적용

하나의 데코레이터를 `@repeat`과 `@repeat(3)` 양쪽 형태로 사용하도록 구현

할 수도 있다.

하지만 이 경우에는 전달받은 객체가 설정값인지 원본 함수 객체인지 구별하는 추가 설계가 필요하다.

이번 학습에서는 두 형태를 명확하게 분리해서 이해하는 것이 우선이다.

A.9.14 `return decorator`, `return wrapper`, `return wrapper()`의 차이

데코레이터 팩토리를 처음 구현할 때 가장 자주 혼동하는 부분이다.

다음 구조를 다시 보자.

```
def repeat(times):

    def decorator(func):

        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = func(*args, **kwargs)

            return result

        return wrapper

    return decorator
```

여기에는 서로 다른 세 반환 위치가 존재한다.

위치	반환 표현식	반환 객체
`repeat`	`return decorator`	데코레이터 함수 객체
`decorator`	`return wrapper`	`Wrapper` 함수 객체
`wrapper`	`return result`	원본 함수 호출의 결과 객체

이 중 하나라도 잘못 작성하면 호출 구조가 달라진다.

예를 들어 다음과 같이 작성했다고 하자.

```
def decorator(func):
```

```
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper()
```

여기서는 wrapper 함수 객체를 반환하는 것이 아니라, wrapper를 즉시 호출하고 그 결과를 반환한다.

따라서 원본 함수가 인수를 요구한다면 데코레이션 시점에 오류가 발생할 수 있다.

반대로 인수가 필요 없는 원본 함수라면 데코레이션 시점에 원본 함수가 실행되고, 그 반환 객체가 원래 함수 이름에 바인딩될 수 있다.

이것은 일반적인 wrapper 데코레이터에서 의도한 구조와 다르다.

따라서 다음 구분을 기억해야 한다.

```
return decorator  # 데코레이터 함수 객체 반환
return wrapper    # wrapper 함수 객체 반환
return wrapper()  # wrapper를 호출한 결과 반환
```

A.1에서 학습한 함수 객체와 함수 호출 결과의 차이가 그대로 적용되는 것이다.

A.9.15 실전 응용 - 재시도 횟수를 설정하는 데코레이터

이번에는 단순 반복이 아니라 실제 프로그램에서 활용할 수 있는 재시도(Retry) 데코레이터를 구현해 보자.

함수 실행 중 일시적인 오류가 발생하면 지정된 횟수만큼 다시 호출하는 구조다.

```
from functools import wraps

class TemporaryError(Exception):
    pass

def retry(max_attempts):

    if (
        not isinstance(max_attempts, int)
        or isinstance(max_attempts, bool)
        or max_attempts < 1
```

```
    ):
        raise ValueError("max_attempts는 양의 정수여야 합니다.")

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):

            for attempt in range(1, max_attempts + 1):

                try:
                    return func(*args, **kwargs)

                except TemporaryError:

                    if attempt == max_attempts:
                        raise

                    print(
                        f"재시도: {attempt}/{max_attempts}"
                    )

        return wrapper

    return decorator


state = {"calls": 0}


@retry(3)
def unstable_operation():

    state["calls"] += 1

    if state["calls"] < 3:
        raise TemporaryError("일시적인 실패")

    return "Success"


print(unstable_operation())
```

예상 출력:

```
재시도: 1/3
재시도: 2/3
Success
```

이 코드를 순차적으로 분석해 보자.

1. 데코레이터 팩토리 호출

```
retry(3)
```

max_attempts가 정수 객체 3에 바인딩된다.

유효성 검사가 통과하면 decorator 함수 객체가 반환된다.

2. 원본 함수 데코레이션

반환된 데코레이터가 원본 unstable_operation 함수 객체를 전달받아 호출된다.

func가 원본 함수 객체에 바인딩되고 wrapper 함수 객체가 생성된다.

Wrapper는 max_attempts와 func를 클로저를 통해 참조할 수 있다.

3. 첫 번째 실제 호출

```
unstable_operation()
```

현재 이름은 wrapper를 참조하므로 wrapper가 실행된다.

반복문의 첫 번째 시도에서 원본 함수가 호출된다.

state["calls"]가 1이 되고 TemporaryError가 발생한다.

Wrapper는 해당 예외를 처리하고 다음 시도로 진행한다.

4. 두 번째 실제 호출

같은 wrapper 호출 안에서 원본 함수가 다시 호출된다.

state["calls"]가 2가 되고 TemporaryError가 발생한다.

아직 최대 시도 횟수에 도달하지 않았으므로 다음 반복으로 진행한다.

5. 세 번째 실제 호출

원본 함수가 세 번째로 호출된다.

이번에는 state["calls"]가 3이므로 예외가 발생하지 않고 "Success"를 반환한다.

따라서 다음 문장이 실행된다.

```
return func(*args, **kwargs)
```

원본 함수의 반환 객체가 wrapper의 반환 객체가 되고, wrapper 실행이 종료된다.

이 예제에서 중요한 설계 조건

재시도 데코레이터는 원본 함수를 여러 번 호출할 수 있다.

따라서 원본 함수가 외부 시스템의 상태를 변경하는 경우에는 주의해야 한다.

예를 들어 결제 요청, 데이터베이스 갱신, 파일 생성처럼 부작용이 있는 연산은 실패 후 재호출할 때 중복 처리가 발생할 수 있다.

이러한 경우에는 멱등성(Idempotency), 트랜잭션, 중복 요청 식별 등의 조건을 함께 고려해야 한다.

또한 모든 예외를 무조건 재시도하는 것이 적절하지는 않다.

위 예제에서는 의도적으로 `TemporaryError`만 처리하도록 설계했다.

즉, 데코레이터의 설정값은 단순한 반복 횟수가 아니라 호출 정책을 정의하는 구성 요소가 될 수 있다.

A.9.16 데코레이터 팩토리와 클래스 인스턴스의 상태 비교

이전 학습에서 클로저와 클래스 인스턴스의 상태 유지 방식을 비교했다.

이번에는 설정값을 가진 데코레이터 팩토리에도 같은 관점을 적용해 본다.

```
@repeat(3)
def greet():
    ...
```

여기서 반복 횟수 3은 클로저 셀을 통해 유지된다.

반면 클래스 기반으로 비슷한 긴으을 구현한다면 다음과 같은 구조를 생각할 수 있다.

```
class Repeat:

    def __init__(self, times):
```

```
        self.times = times

    def __call__(self, func):

        def wrapper(*args, **kwargs):
            result = None

            for _ in range(self.times):
                result = func(*args, **kwargs)

            return result

        return wrapper
```

이 클래스의 인스턴스는 호출 가능한 객체다.

따라서 다음과 같이 사용할 수 있다.

```
@Repeat(3)
def greet():
    print("Hello")
```

여기서 `@Repeat(3)`은 `Repeat` 인스턴스를 생성한다.

그리고 생성된 인스턴스가 데코레이터로 호출되면서 원본 함수 객체를 전달받는다.

함수 기반 팩토리와 클래스 기반 팩토리를 비교하면 다음과 같다.

비교 기준	함수 기반 팩토리	클래스 기반 팩토리
설정값 저장	클로저 셀	인스턴스 속성
설정값 접근	자유 변수 `times`	`self.times`
데코레이터 역할	반환된 함수 객체	`__call__`을 가진 인스턴스
원본 함수 보존	`wrapper`의 클로저	`wrapper`의 클로저 등
설정값 변경	외부 바인딩 재바인딩 또는 가변 객체 변경	인스턴스 속성 재바인딩

여기서 두 방식의 목적은 유사하지만 내부 상태 저장 구조는 다르다.

함수 기반 팩토리는 클로저 환경을 이용하고, 클래스 기반 팩토리는 인스턴스 네임스페이스를 이용한다.

또한 클래스 기반 구현에서 `self.times`를 변경하면 이미 생성된 `wrapper`의

동작에도 영향을 줄 수 있다.

왜냐하면 wrapper가 실행될 때 매마다 동일한 인스턴스의 `self.times` 속성을 조회하기 때문이다.

이것은 A.5에서 학습한 클로저의 상태 변경과 클래스의 속성 변경이 서로 다른 메커니즘으로 유사한 동작을 구현할 수 있다는 사례다.

A.9.17 Generator와 Coroutine에 적용할 때의 경계 조건

A.7에서 Generator Function에 일반적인 wrapper를 적용하면 Generator Object 생성과 실제 본문 실행 시점이 달라질 수 있다는 사실을 학습했다.

이번 학습의 `repeat`에도 같은 문제가 존재한다.

```
@repeat(3)
def numbers():
    yield 1
    yield 2
```

이 경우 wrapper가 원본 Generato Function을 세 번 호출하면 Generator Object가 세 개 생성될 수 있다.

하지만 각 Generator Object의 본문이 자동으로 실행되는 것은 아니다.

따라서 `repeat(3)`이 1, 2를 세 번 생성하는 반복 시퀀스를 자동으로 만들어 준다고 생각하면 안 된다.

마찬가지로 `async def`로 정의된 코루틴 함수에 동기 wrapper를 적용하면, 반복 호출에서 코루틴 객체만 여러 개 생성할 수 있다.

따라서 데코레이터의 설계는 원본 함수가 일반 함수인지, Generator Function인지, Coroutine Function인지에 따라 달라질 수 있다.

이 문제의 핵심은 함수 호출이 반환하는 객체와 함수 본문의 실제 실행 시점을 구별하는 것이다.

A.9.18 자주 발생하는 오해와 판단 기준

오해	정확한 설명
`@repeat(3)`은 원본 함수에 3을 전달한다.	3은 데코레이터 팩토리의 설정값이다.
`repeat(3)`이 실행되면 원본 함수도 세 번 실행된다.	일반적인 구현에서는 데코레이터가 생성될 뿐이다.
팩토리, 데코레이터, `wrapper`는 같은 함수 객체다.	일반적으로 서로 다른 함수 객체다.
`return decorator`와 `return decorator()`는 같다.	전자는 함수 객체 반환, 후자는 호출 결과 반환이다.
`Wrapper`가 `times`를 읽으려면 `nonlocal`이 필요하다.	외부 바인딩을 읽는 데는 필요하지 않다.
같은 팩토리 함수를 사용하면 모든 설정값을 공유한다.	각 팩토리 호출의 클로저 환경은 일반적으로 독립적이다.
`@wraps(func)`가 설정값을 저장한다.	설정값은 일반적으로 클로저에 유지되고, `wraps`는 메타데이터를 설정한다.
반복 호출의 반환값은 자동으로 모두 합쳐진다.	반환값 처리 방식은 `wrapper` 구현에 따라 결정된다.
재시도 데코레이터는 어떤 함수에도 안전하게 적용할 수 있다.	부작용, 예외 종류, 멱등성 등을 고려해야 한다.

A.9.19 핵심 정리

이번 학습의 핵심 구조는 다음과 같다.

```
from functools import wraps

def decorator_factory(config):

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):
            # config와 func를 활용
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

이 코드는 세 단계의 함수 객체와 서로 다른 바인딩 환경으로 구성된다.

데코레이터 팩토리의 전체 실행 구조

1. 팩토리 호출
   `decorator_factory(config)`가 실행되고 설정값이 외부 함수의
   매개변수에 바인딩된다. 데코레이터 함수 객체가 생성되어 반환된다.

2. 데코레이터 적용
   반환된 데코레이터가 원본 함수 객체를 전달받아 호출된다. `func`가
   원본 함수 객체에 바인딩되고 `wrapper` 함수 객체가 생성된다.

3. 이름 재바인딩
   데코레이터가 반환한 `wrapper` 함수 객체가 원래 함수 이름에 바인딩된다.

4. 실제 함수 호출
   이후 사용자가 데코레이션된 이름을 호출하면 `wrapper`가 실행된다.
   `Wrapper`는 클로저를 통해 설정값과 원본 함수 객체에 접근한다.

5. 반환 또는 예외 전파
   `Wrapper`의 구현에 따라 원본 함수의 호출 횟수, 인수 전달,
   반환값 처리, 예외 전파가 결정된다.

이번 학습에서 가장 중요한 판단 기준은 다음이다.

> `@factory(config)`를 분석할 때는 함수가 세 단계로 중첩되어 있다는
>
> 표면적인 구조보다, 각 함수가 언제 호출되고 어떤 객체를 반환하며
>
> 어떤 바인딩을 클로저로 유지하는지를 추적해야 한다.

이 기준을 적용하면 반복, 재시도, 캐싱 정책, 로깅 수준 설정 등 다양한 데코레이터 팩토리를 동일한 객체 모델로 이해할 수 있다.

A.10 여러 데코레이터의 평가·적용 순서

Decorator Stacking · Evaluation Order · Application Order · Wrapper Call Chain · Function Composition

A.10.1 이전 학습과의 연결

A.9에서는 인수를 받는 데코레이터를 다음 세 단계로 분석했다.

```
@repeat(3)
def greet():
    print("Hello")
```

1. repeat(3)을 호출해 데코레이터 함수 객체를 생성한다.

2. 반환된 데코레이터에 원본 greet 함수 객체를 전달해 wrapper를 생성한다.

3. 이후 greet()을 호출하면 wrapper가 실행된다.

이 구조를 이해했다면 이제 여러 데코레이터를 하나의 함수에 적용할 수 있다.

```
@first
@second
def greet():
    print("Hello")
```

이 코드는 단일 데코레이터의 변환식을 확장한 것으로, 개념적으로 다음과 대응한다.

```
greet = first(second(greet))
```

그런데 여기서 주의할 점이 있다.

first가 위에 작성되어 있으므로 먼저 적용될 것이라고 생각하기 쉽지만, 실제로는 그렇지 않다.

Python에서는 데코레이터 표현식의 평가 순서, 함수 객체에 데코레이터를 적용하는 순서, 최종 함수 호출 시 wrapper가 실행되는 순서를 구별해야 한다.

이 세 순서는 서로 관련되어 있지만 동일하지 않다.

A.10.2 이번 학습의 핵심 질문

이번 학습에서는 다음 문제를 해결할 것이다.

1. 여러 데코레이터 표현식은 어떤 순서로 평가되는가?

2. 평가된 데코레이터는 원본 함수 객체에 어떤 순서로 적용되는가?

3. 데코레이션 이후 함수 이름은 어떤 wrapper 객체를 참조하는가?

4. 실제 함수를 호출하면 ᄌrapper는 어떤 순서로 실행되는가?

5. 반환값과 예외는 wrapper 사이에서 어떤 경로로 전달되는가?

6. __wrapped__와 __closure__를 이용하면 중첩 구조를 어떻게 추적할 수 있는가?

7. 데코레이터의 적용 순서를 바꾸면 프로그램의 동작이 달라질 수 있는가?

핵심 원리는 다음과 같다.

데코레이터 표현식은 위에서 아래로 평가되고,

그 결과는 아래에서 위로 적용된다.

이후 실제 호출은 가장 바깥쪽 wrapper부터 안쪽으로 진행하며,

정상 반환은 반대 방향으로 전달된다.

다만 모든 데코레이터가 반드시 wrapper를 반환하는 것은 아니므로, 마지막의 호출·반환 순서는 함수 기반 wrapper를 중첩한 경우에 성립하는 구조다.

A.10.3 세 가지 순서를 먼저 구별하자

다음 코드를 기준으로 생각해 보자.

```
@A()
@B()
def func():
    ...
```

A()와 B()가 각각 데코레이터 함수 객체를 반환한다고 가정한다.

그러면 세 단계는 다음과 같이 진행 된다.

1. 데코레이터 표현식 평가 - 위에서 아래

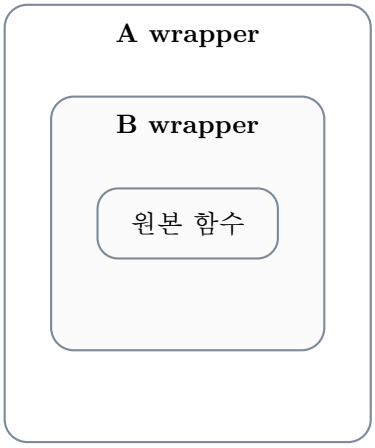
(1) A() : 데코레이터 A 생성

(2) B() : 데코레이터 B 생성

2. 원본 함수에 적용 - 아래에서 위

original → B(original) → A(B(original))

3. 실제 호출 - 바깥에서 안으로, 반환은 반대로



호출 진입은 A wrapper에서 원본 함수순으로,
반환 전달은 원본 함수에서 A wrapper 순으로

원본 함수에 가까운 wrapper가 먼저 생성되고, 마지막에 생성된 바깥쪽 wrapper가 최종 함수 이름에 바인딩된다.

이를 표로 정리하면 다음과 같다.

구분	순서	의미
표현식 평가	A() → B()	데코레이터 객체 준비
데코레이터 적용	B → A	원본 함수에서 바깥쪽으로 객체 변환
실제 호출 진입	A wrapper → B wrapper → 원본	바깥쪽에서 안쪽으로
정상 반환 전달	원본 → B wrapper → A wrapper	안쪽에서 바깥쪽으로

여기서 첫 번째와 두 번째 순서가 반대라는 점이 중요하다.

이것을 단순히 "데코레이터는 역순으로 실행된다"라고 암기하면 안 된다.

무엇을 평가하는지, 무엇을 적용하는지, 어떤 함수 객체를 실제로 호출하는지에 따라 순서가 달라지기 때문이다.

A.10.4 실행 결과로 세 가지 순서를 검증하기

먼저 하나의 독립적인 예제로 평가·적용·호출 순서를 모두 확인해 본다.

```
from functools import wraps

def layer(label):
    print("evaluate:", label)

    def decorator(func):
        print("apply:", label)

        @wraps(func)
        def wrapper(*args, **kwargs):
            print("enter:", label)

            result = func(*args, **kwargs)

            print("exit:", label)
            return result

        return wrapper

    return decorator


@layer("A")
@layer("B")
def calculate(x):
    print("body:", x)
    return x * 2


print("definition complete")
print("result:", calculate(3))
```

예상 출력:

```
evaluate: A
evaluate: B
apply: B
apply: A
definition complete
enter: A
enter: B
body: 3
exit: B
exit: A
```

```
result: 6
```

이 코드는 세 가지 시점으로 나누어 분석해야 한다.

1. 데코레이터 표현식 평가

Python이 함수 정의문을 실행하면서 먼저 다음 표현식을 평가한다.

```
layer("A")
```

layer의 매개변수 label이 문자열 객체 "A"에 바인딩된다.

layer는 중첩된 decorator 함수 객체를 생성하고 반환한다.

이때 다음 출력이 발생한다.

```
evaluate: A
```

이어서 두 번째 데코레이터 표현식을 평가한다.

```
layer("B")
```

마찬가지로 "B"를 참조하는 클로저 환경을 가진 새로운 decorator 함수 객체가 생성된다.

```
evaluate: B
```

여기서 두 데코레이터 함수 객체는 같은 decorator 정의문으로부터 생성되었지만 서로 다른 함수 객체다.

각각 다른 label 바인딩을 유지하고 있다.

2. 원본 함수 객체에 데코레이터 적용

데코레이터 표현식의 평가가 끝나면 원본 calculate 함수 객체가 생성된다.

이제 아래쪽 데코레이터부터 적용된다.

먼저 layer("B")가 반환한 데코레이터에 원본 함수 객체가 전달된다.

```
inner_wrapper = decorator_B(original_calculate)
```

이때 다음 출력이 발생한다.

```
apply: B
```

decorator_B는 새로운 wrapper 함수 객체를 생성하고 반환한다.

이 wrapper는 label="B"와 원본 calculate 함수 객체에 접근할 수 있다.

그다음 위쪽 데코레이터가 적용된다.

```
outer_wrapper = decorator_A(inner_wrapper)
```

여기서 decorator_A가 전달받는 func는

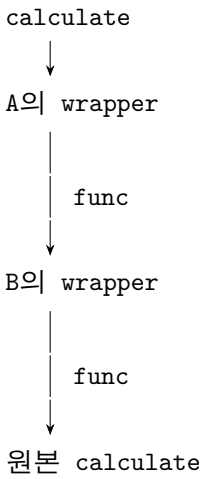
원본 calculate가 아니라 B의 wrapper 함수 객체다.

이때 다음 출력이 발생한다.

```
apply: A
```

마지막으로 outer_wrapper가 모듈 이름 calculate에 바인딩된다.

따라서 최종 관계는 다음과 같다.



3. 실제 함수 호출

이제 다음 문장이 실행된다.

```
calculate(3)
```

현재 calculate는 A의 wrapper 함수 객체를 참조하므로 A의 wrapper가 먼

저 호출된다.

```
enter: A
```

A의 wrapper는 자신의 클로저에 저장된 `func`를 호출한다.

그 객체는 B의 wrapper다.

```
enter: B
```

B의 wrapper 역시 자신의 클로저에 저장된 `func`를 호출한다.

이번에는 원본 `calculate` 함수 객체가 호출된다.

```
body: 3
```

원본 함수는 정수 객체 6을 반환한다.

그 반환값이 B의 wrapper로 전달되고, B의 wrapper는 `"exit: B"`를 출력한 뒤 다시 6을 반환한다.

이후 A의 wrapper가 `"exit: A"`를 출력하고 같은 반환 객체를 최종 호출자에게 전달한다.

따라서 최종 출력은 다음과 같다.

```
result: 6
```

여기서 중요한 점은 적용 순서가 B → A이기 때문에 최종 호출 순서가 A → B가 된다는 것이다.

마지막에 적용된 A가 가장 바깥쪽 함수 객체가 되기 때문이다.

A.10.5 함수 객체의 생성과 이름 바인딩을 추적하기

이번에는 실행 결과가 아니라 객체 관계 자체를 분석해 보자.

앞에서 작성한 코드를 개념적으로 풀어 쓰면 다음과 같다.

```
decorator_A = layer("A")
decorator_B = layer("B")
```

```
def calculate(x):
    return x * 2


original = calculate

inner = decorator_B(original)
outer = decorator_A(inner)

calculate = outer
```

이 코드는 핵심적인 객체 변환 관계를 나타낸 것이다.

실제 데코레이터 구문의 모든 평가·바인딩 세부사항을 그대로 재현한 것은 아니다.

이제 각 이름이 참조하는 객체를 정리해 보자.

이름	참조 객체	역할
`decorator_A`	A 설정을 유지하는 데코레이터 함수	바깥쪽 wrapper 생성
`decorator_B`	B 설정을 유지하는 데코레이터 함수	안쪽 wrapper 생성
`original`	원본 함수 객체	실제 계산 수행
`inner`	B의 wrapper 함수 객체	원본 함수 호출 중개
`outer`	A의 wrapper 함수 객체	B의 wrapper 호출 중개
`calculate`	`outer`와 동일한 객체	최종 공개 이름

이 구조에서 새로운 함수 객체는 여러 개 생성되지만 원본 함수 객체가 수정되는 것은 아니다.

데코레이터가 각각 새로운 wrapper를 생성하고 반환하며, 최종 이름이 가장 바깥쪽 객체에 바인딩되는 것이다.

따라서 다음과 같은 관계가 성립한다.

```
calculate is outer  # True
outer is inner       # False
inner is original    # False
```

데코레이터가 wrapper 대신 원본 함수를 그대로 반환한다면 이러한 객체 수와 동일성 관계는 달라질 수 있다.

즉, 데코레이터를 여러 번 적용했다는 사실만으로 반드시 그 수만큼 새로운 함수 객체가 생성된다고 단정할 수는 없다.

A.10.6 `__wrapped__` 체인을 통해 중첩 구조 확인하기

A.8에서 학습한 `functools.wraps`를 사용하면 원본 함수 참조가 체인 형태로 유지된다.

앞의 `layer`와 `calculate`를 그대로 사용한다고 하자.

```
import inspect

outer = calculate
inner = outer.__wrapped__
original = inner.__wrapped__

print(outer is inner)
print(inner is original)

print(inspect.unwrap(calculate) is original)

print(outer.__name__)
print(inner.__name__)
print(original.__name__)
```

예상 출력:

```
False
False
True
calculate
calculate
calculate
```

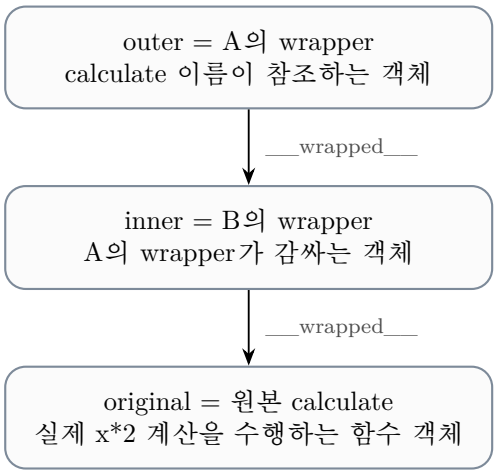
여기서 흥미로운 점이 있다.

세 함수 객체는 모두 다른 객체인데 `__name__`은 같다.

왜냐하면 각 wrapper에 `@wraps(func)`가 적용되어 원본 쪽 함수의 메타데이터가 설정되었기 때문이다.

따라서 `__name__`만으로는 어떤 wrapper인지 구별할 수 없다.

반면 `__wrapped__`는 각 wrapper가 어떤 객체로 감싸도록 설정되었는지 추적할 수 있게 한다.



__wrapped__는 각 wrapper가 감싸는 객체를 가리킨다. 실제 호출도 이 경로를 따르는지는 wrapper의 본문에 달려 있다.

`inspect.signature()`의 동작

다음 코드를 살펴보자.

```
print(inspect.signature(calculate))

print(
    inspect.signature(
        calculate,
        follow_wrapped=False
    )
)
```

예상 출력:

```
(x)
(*args, **kwargs)
```

첫 번째 결과는 `__wrapped__` 체인을 따라 원본 함수의 시그니처를 조회한 결과다.

두 번째 결과는 바깥쪽 wrapper 자체의 시그니처를 조회한 결과이다.

따라서 여러 데코레이터가 중첩되어 있어도 `functools.wraps`를 적절히 사용하면 외부 도구가 원본 함수의 인터페이스를 분석할 수 있다.

다만 중간 데코레이터가 `__wrapped__`를 설정하지 않으면 체인이 끊길 수

있다.

그리고 `__wrapped__` 체인은 메타데이터에 관한 관계이므로, wrapper가 실제로 `func()`를 호출하는지까지 보장하지는 않는다.

A.10.7 `__closure__`로 실제 함수 참조 관계 검증하기

이번에는 A.4와 A.5에서 학습한 클로저 셀을 활용해 본다.

앞의 `layer` 구현에서 wrapper는 다음 두 자유 변수를 참조한다.

```
label
func
```

따라서 다음과 같이 클로저 셀을 확인할 수 있다.

```
def closure_map(func):
    return dict(
        zip(
            func.__code__.co_freevars,
            func.__closure__ or ()
        )
    )


outer_cells = closure_map(calculate)
inner_cells = closure_map(calculate.__wrapped__)

print(outer_cells["label"].cell_contents)
print(inner_cells["label"].cell_contents)

print(
    outer_cells["func"].cell_contents
    is calculate.__wrapped__
)

print(
    inner_cells["func"].cell_contents
    is calculate.__wrapped__.__wrapped__
)
```

예상 출력:

```
A
B
```

```
True
True
```

이 결과를 해석해 보자.

A의 wrapper에 저장된 `func`는 B의 wrapper 함수 객체를 참조한다.

반면 B의 wrapper에 저장된 `func`는 원본 `calculate` 함수 객체를 참조한다.

따라서 두 wrapper는 같은 이름 `func`를 자유 변수로 사용하지만 서로 다른 클로저 셀에 접근한다.

함수 객체	label 셀의 내용	func 셀의 내용
A의 wrapper	"A"	B의 wrapper
B의 wrapper	"B"	원본 calculate

여기서 클로저의 핵심 원리가 다시 나타난다.

자유 변수의 이름이 같다는 사실은 동일한 바인딩을 공유한다는 뜻이 아니다.

각 wrapper가 어느 데코레이터 호출에서 생성되었는지에 따라 클로저 셀이 달라진다.

> 한 가지 더: 서로 다른 함수 객체가 같은 코드 객체를 사용할 수 있다

앞의 `layer` 구현에서는 A와 B의 wrapper가 동일한 `def wrapper` 정의문에서 생성된다.

따라서 다음 결과를 확인할 수 있다.

```
outer = calculate
inner = calculate.__wrapped__

print(outer is inner)
print(outer.__code__ is inner.__code__)
print(outer.__closure__ is inner.__closure__)
```

예상 출력:

```
False
True
False
```

이 결과는 중요하다.

A의 wrapper와 B의 wrapper는 서로 다른 함수 객체다.

하지만 같은 함수 정의문의 코드 객체를 공유할 수 있다.

반면 클로저 환경은 서로 다르다.

즉, Python의 함수 객체를 분석할 때 다음 세 요소를 구별해야 한다.

- 함수 객체의 동일성
- 실행할 코드 객체의 동일성
- 함수 객체에 연결된 클로저 환경의 동일성

같은 코드 객체를 실행하더라도 서로 다른 클로저를 참조하면 동작이 달라질 수 있다.

이것이 함수 객체와 코드 객체를 구별해야 하는 이유 중 하나다.

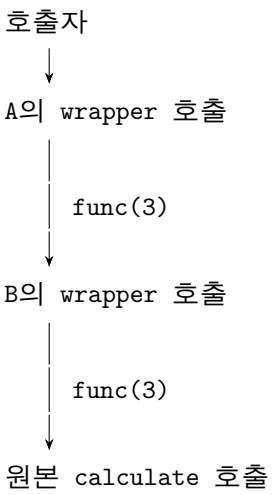
A.10.8 호출 프레임과 반환 순서

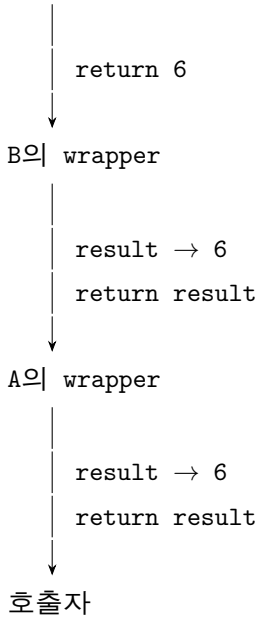
이제 실행 환경의 관점에서 중첩 호출을 분석해 보자.

다음 호출을 생각해 보자.

```
calculate(3)
```

실제 호출 경로는 다음과 같다.





여기서 wrapper마다 별도의 함수 호출이 발생하므로 각 호출은 자신의 지역 실행 환경을 가진다.

예를 들어 A의 wrapper에도 `args`, `kwargs`, `result`라는 지역 이름이 존재하고, B의 wrapper에도 같은 이름이 존재한다.

하지만 이 이름들이 하나의 네임스페이스를 공유하는 것은 아니다.

A의 `result`와 B의 `result`는 서로 다른 함수 호출의 지역 바인딩이다.

두 이름이 같은 정수 객체 6을 참조할 수 있을 뿐이다.

따라서 다음 원리가 성립한다.

중첩된 wrapper가 같은 반환 객체를 전달하더라도

각 wrapper의 지역 이름은 독립적이다.

그리고 `return result`는 객체를 자동으로 복사하지 않는다.

각 wrapper가 반환 객체를 변경하거나 교체하지 않는다면 원본 함수가 반환한 객체가 그대로 호출자에게 전달될 수 있다.

A.10.9 데코레이터의 적용 순서를 바꾸면 결과도 달라질까?

지금까지 살펴본 데코레이터는 원본 함수의 반환값을 그대로 전달했다.

그렇다면 wrapper가 반환값을 변경하는 경우에는 어떻게 될까?

다음 두 데코레이터를 정의해 보자.

```
from functools import wraps

def add_one(func):

    @wraps(func)
    def wrapper(x):
        return func(x) + 1

    return wrapper


def double(func):

    @wraps(func)
    def wrapper(x):
        return func(x) * 2

    return wrapper
```

이제 적용 순서를 다르게 해 보자.

```
@double
@add_one
def first(x):
    return x


@add_one
@double
def second(x):
    return x


print(first(3))
print(second(3))
```

예상 출력:

```
8
```

7

왜 서로 다른 결과가 발생할까?

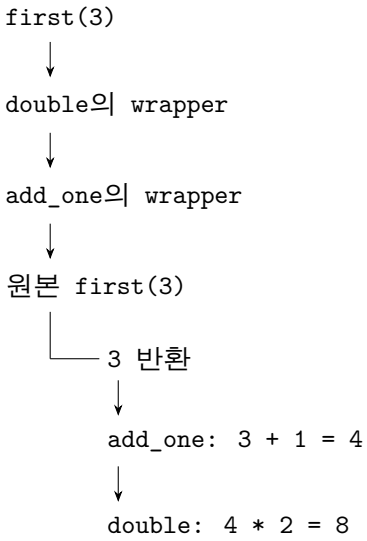
첫 번째 함수

```
@double
@add_one
def first(x):
    return x
```

이 코드는 다음과 대응한다.

```
first = double(add_one(first))
```

실제 호출은 다음 순서로 진행된다.



따라서 8이 반환된다.

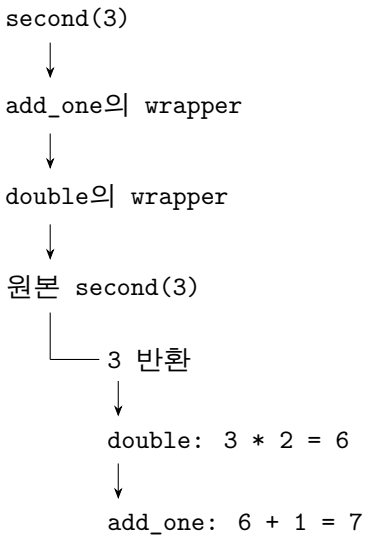
두 번째 함수

```
@add_one
@double
def second(x):
    return x
```

이 코드는 다음과 대응한다.

```
second = add_one(double(second))
```

실제 호출은 다음과 같다.



따라서 7이 반환된다.

이 차이는 수학적인 함수 합성(Function Composition)과 연결된다.

원본 함수를 f, 반환값에 1을 더하는 변환을 P, 두 배로 만드는 변환을 D라고 하자.

그러면 첫 번째 경우는 다음과 같다.

$$D(P(f))(x) = 2(f(x) + 1)$$

두 번째 경우는 다음과 같다.

$$P(D(f))(x) = 2f(x) + 1$$

일반적으로 두 결과는 같지 않다.

즉, 데코레이터의 적용 순서는 교환법칙을 만족하지 않는다.

서로 다른 데코레이터를 적용하는 순서를 바꾸면 호출 구조뿐 아니라 최종 반환값도 달라질 수 있다.

따라서 여러 데코레이터를 사용할 때는 각 데코레이터가 인수, 반환값, 상태, 예외 중 무엇을 변경하는지 확인해야 한다.

A.10.10 예외가 발생하면 wrapper는 어떤 순서로 종료될까?

A.7에서는 예외가 발생하면 원본 함수의 정상적인 반환 경로가 중단된다는 사실을 학습했다.

여러 wrapper가 중첩된 경우에도 같은 규칙이 적용된다.

다음 코드를 살펴보자.

```
from functools import wraps

def layer(label):

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):
            print(label, "enter")

            try:
                return func(*args, **kwargs)

            finally:
                print(label, "finally")

        return wrapper

    return decorator

@layer("A")
@layer("B")
def fail():
    print("body")
    raise ValueError("failed")

try:
    fail()

except ValueError:
    print("caught")
```

예상 출력:

```
A enter
B enter
body
B finally
A finally
caught
```

이 실행 순서를 분석해 보자.

먼저 A의 wrapper가 호출된다.

A의 wrapper는 `"A enter"`를 출력하고 B의 wrapper를 호출한다.

B의 wrapper는 `"B enter"`를 출력하고 원본 `fail` 함수를 호출한다.

원본 함수는 `"body"`를 출력한 다음 `ValueError`를 발생시킨다.

여기서부터는 정상적인 반환이 아니라 예외 전파가 시작된다.

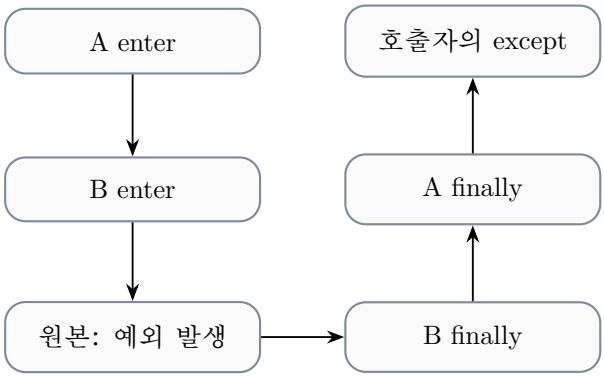
B의 wrapper는 `try` 블록에서 예외를 받지만 해당 예외를 처리하는 `except` 블록이 없다.

다만 `finally`가 존재하므로 `"B finally"`를 출력한다.

그다음 예외가 A의 wrapper로 전파된다.

A의 wrapper 역시 `finally`를 실행하여 `"A finally"`를 출력한다.

마지막으로 호출자의 `except ValueError` 블록이 예외를 처리한다.



예외 발생 시 실행 경로
진입은 안쪽으로, 예외 전파는 바깥쪽으로

여기서 중요한 점은 다음이다.

예외가 발생하면 `wrapper`의 정상적인 반환 처리는 생략될 수 있지만,

`finally`는 예외가 전파되는 과정에서 실행된다.

다만 `finally` 자체에서 새로운 예외가 발생하거나 `return` 등으로 제어 흐름을 변경하면 기존 예외의 전파 방식도 달라질 수 있다.

따라서 여러 데코레이터의 예외 처리 구조를 분석할 때는 각 wrapper가 예외를 처리하는지, 다시 발생시키는지, 다른 예외로 변환하는지 구별해야 한다.

A.10.11 실전 응용 - 로깅과 재시도의 적용 순서

이제 A.9에서 학습한 재시도 데코레이터와 로깅 데코레이터를 결합해 보자.

이 예제는 데코레이터 순서가 단순한 문법 문제가 아니라 실제 프로그램의 관찰 가능한 동작을 결정한다는 사실을 보여준다.

```
from functools import wraps

def log(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("LOG")
        return func(*args, **kwargs)
```

```
    return wrapper


def retry(max_attempts):

    def decorator(func):

        @wraps(func)
        def wrapper(*args, **kwargs):

            for attempt in range(max_attempts):

                try:
                    return func(*args, **kwargs)

                except ValueError:
                    if attempt == max_attempts - 1:
                        raise

        return wrapper

    return decorator
```

이제 같은 기능의 함수를 두 가지 순서로 데코레이션 해보자.

```
state_a = {"count": 0}


@log
@retry(3)
def task_a():
    state_a["count"] += 1

    if state_a["count"] < 3:
        raise ValueError

    return "Success"


state_b = {"count": 0}


@retry(3)
@log
def task_b():
    state_b["count"] += 1

    if state_b["count"] < 3:
```

```
        raise ValueError

    return "Success"


print("A:", task_a())
print("B:", task_b())
```

예상 출력:

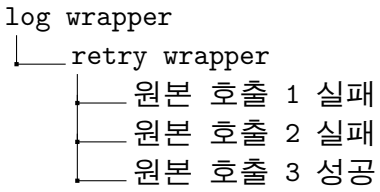
```
LOG
A: Success
LOG
LOG
LOG
B: Success
```

왜 A에서는 로그가 한 번만 출력되고 B에서는 세 번 출력될까?

사례 A - 로깅이 바깥쪽

```
@log
@retry(3)
def task_a():
    ...
```

호출 구조는 다음과 같다.



로깅 wrapper는 한 번 호출된다.

그 내부에서 재시도 wrapper가 세 번의 원본 함수 호출을 수행하는 것이다.

따라서 "LOG"는 한 번만 출력된다.

사례 B - 재시도가 바깥쪽

```
@retry(3)
```

```
@log
def task_b():
    ...
```

호출 구조는 다음과 같다.

```
retry wrapper
 ├── log wrapper → 원본 호출 1 실패
 ├── log wrapper → 원본 호출 2 실패
 └── log wrapper → 원본 호출 3 성공
```

이번에는 재시도 wrapper가 로깅 wrapper를 세 번 호출한다.

따라서 "LOG"도 세 번 출력된다.

이 차이는 실전에서 중요하다.

목적	적합한 구조
사용자 요청 한 건당 로그 한 번	로깅을 재시도 바깥쪽에 배치
재시도 시도마다 로그 기록	로깅을 재시도 안쪽에 배치

물론 실제 로깅 구현에서 무엇을 기록하는지에 따라 세부 설계는 달라질 수 있다.

핵심은 데코레이터를 어느 위치에 배치하는지가 기능의 적용 범위를 결정한다는 사실이다.

이 원리는 로깅뿐 아니라 캐싱, 인증, 트랜잭션, 성능 측정 등에도 적용된다.

A.10.12 모든 데코레이터가 wrapper를 생성하는 것은 아니다

지금까지는 모든 데코레이터가 새로운 wrapper를 반환하는 구조를 살펴봤다.

하지만 A.6에서 학습했듯 데코레이터는 원본 함수 객체를 그대로 반환할 수도 있다.

대표적인 사례가 등록형 데코레이터(Registration Decorator)다.

다음 코드를 살펴보자.

```
from functools import wraps
```

```
registry = {}

def register(func):
    registry[func.__name__] = func
    return func


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE")
        return func(*args, **kwargs)

    return wrapper
```

이제 적용 순서를 달리해 보자.

```
@register
@trace
def first():
    return 10


@trace
@register
def second():
    return 20


print(registry["first"] is first)

print(registry["second"] is second)

print(
    registry["second"]
    is second.__wrapped__
)
```

예상 출력:

```
True
False
True
```

왜 이런 차이가 발생할까?

첫 번째 함수는 다음과 같이 반환된다.

```
first = register(trace(first))
```

`trace(first)`가 먼저 실행되어 wrapper 함수 객체를 반환한다.

그다음 `register`가 그 wrapper 객체를 전달받아 등록한다.

`register`는 전달받은 객체를 그대로 반환하므로 최종 이름도 `first`도 등록된 wrapper를 참조한다.

따라서 다음 비교는 `True`다.

```
registry["first"] is first
```

반면 두 번째 함수는 다음과 같다.

```
second = trace(register(second))
```

이번에는 `register`가 원본 함수 객체를 먼저 등록한다.

그다음 `trace`가 새로운 wrapper를 생성해 반환한다.

따라서 레지스트리에는 원본 함수 객체가 저장되어 있지만 최종 이름 `second`는 wrapper를 참조한다.

그래서 다음 비교가 `False`인 것이다.

```
registry["second"] is second
```

이 예제는 중요한 사실을 보여준다.

데코레이터의 적용 순서는 어떤 객체가 외부 시스템에 전달되거나 등록되는지도 결정할 수 있다.

이 내용은 `A.12`에서 FastAPI와 등록형 데코레이터를 분석할 때 다시 연결할 것이다.

A.10.13 데코레이터 적용 중 예외가 발생하면 이름 바인딩은 어떻게 될까?

이제 데코레이터가 적용되는 도중 오류가 발생하는 경계 조건을 살펴본다.

```
def greet():
    return "Old"


def reject(func):
    print("reject called")
    raise ValueError("Decoration failed")


try:
    @reject
    def greet():
        return "New"

except ValueError:
    print("caught")


print(greet())
```

예상 출력:

```
reject called
caught
Old
```

왜 "New"가 아니라 "Old"가 출력될까?

두 번째 def greet() 문장이 실행될 때 새로운 원본 함수 객체가 생성된다.

그 객체가 reject에 전달된다.

하지만 reject가 ValueError를 발생시키므로 데코레이터 적용이 정상적으로 완료되지 않는다.

따라서 새로운 반환 객체를 이름 greet에 바인딩하는 단계에 도달하지 못한다.

기존 모듈 이름 greet은 여전히 이전 함수 객체를 참조한다.

즉, 함수 정의문이 실행되는 과정에서 데코레이터 적용이 실패하면 새로운 함수 객체의 최종 이름 바인딩이 이루어지지 않을 수 있다.

이것은 데코레이터가 기존 함수를 제자리에서 수정하는 기능이 아니라, 최종 반환 객체를 이름에 바인딩하는 구조라는 사실을 다시 보여준다.

단, 데코레이터가 예외를 발생시키기 전에 수행한 다른 부작용까지 자동으로

취소되는 것은 아니다.

A.10.14 데코레이터 평가·적용·호출을 구별하는 판단 기준

지금까지의 내용을 하나의 표로 정리해 본다.

질문	확인할 대상	판단 기준
어떤 데코레이터 표현식이 먼저 실행되는가?	`@A()`, `@B()`	위에서 아래로 평가
어떤 데코레이터가 원본 함수를 먼저 전달받는가?	`A(B(func))`	아래쪽 `B`부터 적용
최종 이름은 어떤 객체를 참조하는가?	가장 바깥쪽 적용 결과	마지막 데코레이터의 반환 객체
어떤 `wrapper`가 먼저 호출되는가?	최종 이름이 참조하는 객체	바깥쪽 `wrapper`부터
원본 함수는 언제 호출되는가?	각 `wrapper`의 실제 구현	안쪽 `wrapper`가 원본을 호출할 때
반환값은 어떤 순서로 전달되는가?	중첩된 호출의 반환	안쪽에서 바깥쪽으로
예외는 어떤 순서로 전파되는가?	`try`, `except`, `finally`	호출 스택을 따라 바깥쪽으로
원본 함수 객체는 어디에 있는가?	`__wrapped__`, 클로저 등	실제 데코레이터 구현에 따라 다름
적용 순서를 바꿔도 결과가 같은가?	인수·반환·상태·예외 처리	일반적으로 보장되지 않음

여기서 특히 주의해야 할 점은 데코레이터 적용 순서와 wrapper 호출 순서를 같은 의미로 사용하지 않는 것이다.

데코레이터를 적용하는 과정에서는 함수 객체가 바깥쪽으로 쌓이고, 실제 호출에서는 그 바깥쪽부터 실행이 시작된다.

A.10.15 기존 Python 객체 모델과의 연결

이번 `A.10`은 데코레이터만의 독립적인 규칙을 학습한 것이 아니다.

지금까지 학습한 Python 객체 모델을 하나의 구조로 결합한 학습이다.

함수 객체와 이름 바인딩 - A.1

데코레이터가 반환한 객체가 최종 함수 이름에 바인딩된다.

이때 원본 함수 객체와 wrapper 함수 객체의 동일성은 별개다.

렉시컬 스코프와 LEGB - A.2, A.3

각 wrapper는 자신이 정의된 렉시컬 환경을 기준으로 외부 이름을 탐색한다.

호출자가 어떤 위치에서 함수를 실행하는지는 클로저의 자유 변수 탐색 경로를 변경하지 않는다.

클로저와 셀 객체 - A.4, A.5

각 wrapper는 외부 함수 객체나 설정값을 클로저 셀을 통해 유지할 수 있다.

같은 코드 객체를 공유하는 wrapper라도 서로 다른 클로저를 사용하면 다른 함수 객체를 호출할 수 있다.

데코레이터 변환식 - A.6

여러 데코레이터의 적용은 중첩된 함수 호출 형태로 표현할 수 있다.

```
func = A(B(func))
```

Wrapper의 인수 전달과 반환 - A.7

각 wrapper는 독립적으로 인수를 전달하고 반환값을 처리한다.

따라서 여러 wrapper를 거치면서 반환값이 변경되거나 예외가 처리될 수 있다.

functools.wraps와 메타데이터 - A.8

`__wrapped__` 체인을 이용해 원본 함수 객체를 추적할 수 있다.

하지만 이 체인은 실제 호출을 강제하는 장치가 아니다.

데코레이터 팩토리 - A.9

`@A()`와 `@B()`는 먼저 각각 평가되어 데코레이터 객체를 준비한다.

그다음 아래쪽부터 원본 함수 객체에 적용된다.

결국 A.10은 이러한 개념들을 하나의 실행 흐름으로 연결하는 것이다.

A.10.16 핵심 정리

이번 학습에서 반드시 구별해야 하는 것은 네 가지다.

첫째, 데코레이터 표현식의 평가 순서

```
@A()
@B()
def func():
    ...
```

`A()`가 먼저 평가되고 그다음 `B()`가 평가된다.

둘째, 데코레이터의 적용 순서

원본 함수 객체가 생성된 다음 B가 먼저 적용되고 A가 나중에 적용된다.

```
func = A_decorator(B_decorator(original))
```

셋째, 실제 wrapper 호출 순서

최종 이름이 A의 wrapper를 참조한다면 A의 wrapper가 먼저 호출된다.

`A wrapper` → `B wrapper` → 원본 함수

넷째, 정상 반환과 예외 전파 순서

정상적인 반환은 원본 함수에서 시작해 B의 wrapper, A의 wrapper 순서로 전달된다.

예외 역시 처리되지 않는다면 호출 스택을 따라 바깥쪽으로 전파된다.

이 네 가지를 이해하면 여러 데코레이터가 적용된 코드를 볼 때 단순히 출력 순서를 암기하는 것이 아니라, 각 함수 객체가 어떤 객체를 참조하고 어떤 시점에 호출되는지를 근거로 실행 과정을 예측할 수 있다.

A.11 클래스·메서드 바인딩과 데코레이터

Method Binding · Descriptor Protocol · classmethod · staticmethod · property · Decorator Stacking

A.11.1 이전 학습과의 연결

A.10에서는 여러 데코레이터를 다음과 같은 함수 객체 변환 과정으로 분석했다.

```
@first
@second
def calculate(x):
    return x * 2
```

개념적으로 다음과 대응한다.

```
calculate = first(second(calculate))
```

이 과정에서 원본 함수 객체가 생성되고, 데코레이터가 반환한 객체가 최종적으로 calculate라는 이름에 바인딩된다.

그런데 함수가 클래스 내부에 정의되면 새로운 문제가 등장한다.

```
class Person:

    def introduce(self):
        return "Hello"
```

이때 introduce는 어떤 객체일까?

일반적인 함수 객체일까, 아니면 처음부터 인스턴스 메서드 객체일까?

그리고 다음 두 표현식은 왜 다르게 동작할까?

```
Person.introduce
Person().introduce
```

이 차이를 설명하려면 데코레이터뿐 아니라 속성 탐색(Attribute Lookup)과 디스크립터 프로토콜(Descriptor Protocol)을 함께 이해해야 한다.

이번 학습의 핵심은 다음과 같다.

> 데코레이터는 클래스 네임스페이스에 저장될 객체를 결정하고,
>
> 디스크립터는 객체가 속성 조회 과정에서 어떻게 바인딩되는지 결정한다.

두 과정은 서로 다른 시점에 발생한다.

A.11.2 클래스 내부의 def는 무엇을 생성할까?

먼저 데코레이터가 없는 클래스를 살펴보자.

```
class Person:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        return f"Hello, {self.name}"

p = Person("Kim")
```

클래스 정의문이 실행될 때 클래스 본문도 실행된다.

이 과정에서 `def introduce` 문장은 일반적인 Python 함수 객체를 생성하고, `introduce`라는 이름을 클래스 생성에 사용되는 네임스페이스에 바인딩한다.

클래스 객체가 생성된 뒤에는 다음과 같이 확인할 수 있다.

```
print(type(Person.__dict__["introduce"]))

print(
    Person.__dict__["introduce"]
    is Person.introduce
)
```

예상 출력:

```
<class 'function'>
True
```

여기서 첫 번째 결과가 중요하다.

`Person` 클래스의 `__dict__`에 저장된 것은 바운드 메서드가 아니라 함수 객체(Function Object)다.

`Person.__dict__`는 클래스 네임스페이스를 보여주는 `mappingproxy` 객체이고, 그 안에서 `"introduce"`라는 키가 함수 객체를 참조하고 있다.

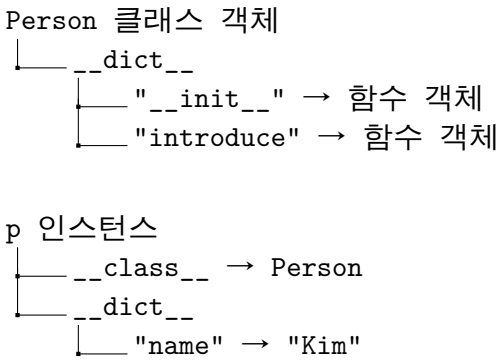
반면 `p.__dict__`에는 다음과 같은 인스턴스 상태가 저장된다.

```
print(p.__dict__)
```

예상 출력:

```
{'name': 'Kim'}
```

즉, 클래스의 함수와 인스턴스의 데이터는 서로 다른 네임스페이스에 존재한다.



그런데 `p.introduce`를 조회하면 단순히 클래스에 저장된 함수 객체를 그대로 반환하지 않는다.

왜 그럴까?

일반적인 사용자 정의 함수 객체가 `__get__`을 제공하는 비데이터 디스크립터(Non-data Descriptor)이기 때문이다.

A.11.3 함수 객체가 바운드 메서드가 되는 과정

다음 코드를 살펴보자.

```
class Person:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        return f"Hello, {self.name}"


p = Person("Kim")

func = Person.__dict__["introduce"]
method = p.introduce
```

```
print(type(func))
print(type(method))

print(method.__self__ is p)
print(method.__func__ is func)

print(method())
```

예상 출력:

```
<class 'function'>
<class 'method'>
True
True
Hello, Kim
```

이 결과를 순서대로 분석해 보자.

1. 클래스 네임스페이스에서 함수 객체 탐색

```
func = Person.__dict__["introduce"]
```

func는 원본 함수 객체를 참조한다.

2. 인스턴스에서 속성 조회

```
method = p.introduce
```

일반적인 인스턴스 속성 조회에서는 클래스의 MRO를 따라 속성을 탐색하고, 발견된 객체가 디스크립터라면 관련 규칙에 따라 __get__을 호출할 수 있다.

여기서 함수 객체의 __get__이 사용된다.

핵심 동작은 다음과 같이 표현할 수 있다.

```
method = func.__get__(p, Person)
```

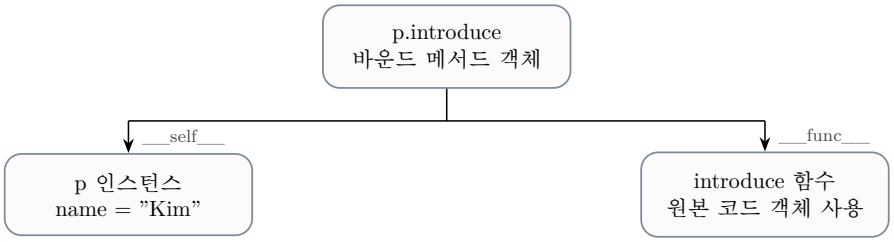
그 결과 바운드 메서드 객체가 반환된다.

3. 바운드 메서드의 내부 참조

바운드 메서드는 다음 두 객체를 참조한다.

속성	참조 대상
`method.__self__`	바인딩된 `p` 인스턴스
`method.__func__`	원본 `introduce` 함수 객체

따라서 다음과 같은 구조가 형성된다.



바운드 메서드는 바인딩된 인스턴스와 함수 객체를 연결하는 별도의 객체다.

4. 바운드 메서드 호출

```
method()
```

바운드 메서드는 자신이 참조하는 인스턴스를 원본 함수의 첫 번째 인수로 전달한다.

따라서 일반적인 경우 다음 두 호출은 같은 함수 실행을 수행한다.

```
p.introduce()
Person.introduce(p)
```

여기서 `self`는 특별한 키워드가 아니다.

함수 정의에서 사용한 첫 번째 매개변수 이름일 뿐이다.

인스턴스를 자동 전달하는 기능은 `self`라는 이름에서 발생하는 것이 아니라, 함수 디스크립터가 만들어 낸 바운드 메서드의 호출 규칙에서 발생한다.

A.11.4 세 종류의 메서드는 무엇이 다른가?

다음 클래스를 보자.

```
class Example:

    def instance_method(self):
        return self

    @classmethod
    def class_method(cls):
        return cls

    @staticmethod
    def static_method():
        return "static"


obj = Example()
```

세 메서드 모두 def 문장을 통해 함수 객체를 생성하지만, 데코레이터 적용 이후 클래스 네임스페이스에 저장되는 객체가 달라진다.

```
print(type(Example.__dict__["instance_method"]))
print(type(Example.__dict__["class_method"]))
print(type(Example.__dict__["static_method"]))
```

예상 출력:

```
<class 'function'>
<class 'classmethod'>
<class 'staticmethod'>
```

그리고 클래스나 인스턴스를 통해 속성을 조회하면 다음과 같은 결과를 얻는다.

```
print(type(Example.instance_method))
print(type(obj.instance_method))

print(type(Example.class_method))
print(type(obj.class_method))

print(type(Example.static_method))
print(type(obj.static_method))
```

예상 출력:

```
<class 'function'>
<class 'method'>
<class 'method'>
```

```
<class 'method'>
<class 'function'>
<class 'function'>
```

이 차이를 표로 정리하면 다음과 같다.

구분	클래스에 저장된 객체	클래스 접근 결과	인스턴스 접근 결과
일반 인스턴스 메서드	함수 객체	함수 객체	인스턴스 바운드 메서드
`@classmethod`	`classmethod` 디스크립터	클래스 바운드 메서드	클래스 바운드 메서드
`@staticmethod`	`staticmethod` 디스크립터	함수 객체	함수 객체

이 표가 이번 학습 전체를 관통하는 기준이다.

`classmethod`와 `staticmethod`의 차이를 단순히 `self`와 `cls`의 차이로 이해하면 불완전하다.

실제 차이는 클래스 네임스페이스에 저장된 디스크립터가 속성 조회 과정에서 어떤 객체를 반환하느냐에 있다.

A.11.5 `classmethod` - 함수 객체를 클래스에 바인딩하기

먼저 클래스 메서드부터 분석해 보자.

```
class Animal:

    @classmethod
    def identify(cls):
        return cls.__name__


a = Animal()
```

이 코드에서 다음 부분은

```
@classmethod
def identify(cls):
    return cls.__name__
```

개념적으로 다음 변환식과 대응한다.

```
def identify(cls):
    return cls.__name__

identify = classmethod(identify)
```

classmethod는 원본 함수 객체를 받아 디스크립터 객체를 생성한다.

따라서 클래스 네임스페이스에는 일반 함수 객체가 아니라 classmethod 객체가 저장된다.

```
raw = Animal.__dict__["identify"]

print(type(raw))
print(type(raw.__func__))
```

예상 출력:

```
<class 'classmethod'>
<class 'function'>
```

여기서 raw.__func__는 classmethod가 감싸고 있는 원본 함수 객체를 참조한다.

이제 클래스를 통해 속성을 조회해 보자.

```
method = Animal.identify

print(type(method))
print(method.__self__ is Animal)
print(method.__func__ is raw.__func__)
print(method())
```

예상 출력:

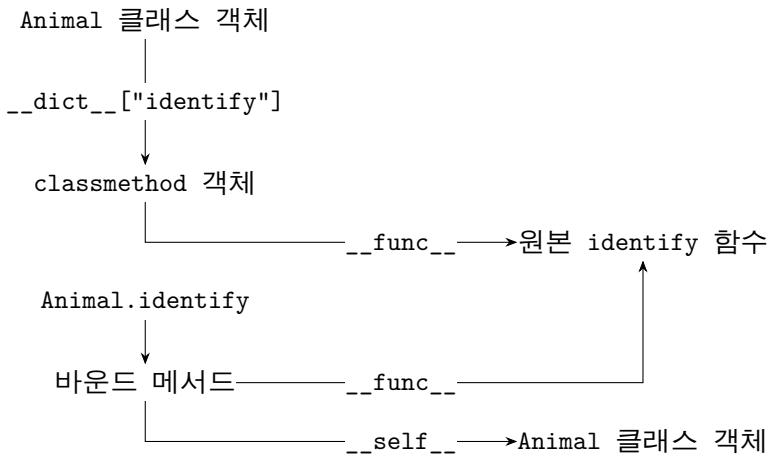
```
<class 'method'>
True
True
Animal
```

왜 이런 결과가 발생할까?

Animal.identify를 조회하면 클래스 네임스페이스에서 classmethod 디스크립터를 찾게 된다.

그 디스크립터의 `__get__`이 호출되면서 원본 함수 객체를 `Animal` 클래스에 바인딩한 메서드가 반환된다.

따라서 다음과 같은 관계가 형성된다.



이제 다음 호출을 생각해 보자.

```
Animal.identify()
```

바운드 메서드가 `Animal`을 첫 번째 인수로 전달하므로 원본 함수에서는 다음 바인딩이 형성된다.

$$\texttt{cls} \rightarrow \texttt{Animal 클래스 객체}$$

즉, 개념적으로 다음과 같은 호출이다.

```
raw.__func__(Animal)
```

> 인스턴스를 통해 조회한다면?

```
method = a.identify

print(method.__self__ is Animal)
print(method.__self__ is a)
```

예상 출력:

```
True
```

```
False
```

인스턴스를 통해 접근하더라도 `classmethod`는 인스턴스가 아니라 클래스에 바인딩된다.

따라서 클래스 메서드의 핵심은 첫 번째 매개변수 이름이 `cls`라는 사실이 아니라, 디스크립터가 클래스 객체를 바인딩한다는 사실이다.

A.11.6 상속과 `classmethod` - MRO와 동적 클래스 바인딩

이제 이전에 학습한 MRO와 연결해 보자.

```
class Parent:

    @classmethod
    def identify(cls):
        return cls.__name__


class Child(Parent):
    pass


print(Parent.identify())
print(Child.identify())

print(Parent.identify.__self__ is Parent)
print(Child.identify.__self__ is Child)

print(
    Parent.identify.__func__
    is Child.identify.__func__
)
```

예상 출력:

```
Parent
Child
True
True
True
```

여기서 중요한 질문이 있다.

`Child`는 `identify`를 정의하지 않았는데 왜 `cls`가 `Parent`가 아니라 `Child`를

참조할까?

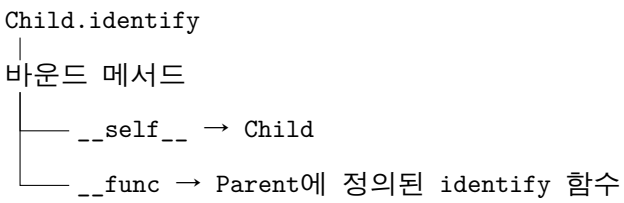
속성 탐색과 메서드 바인딩을 구별하면 이해할 수 있다.

먼저 `Child.identify`라는 속성을 조회할 때 MRO를 따라

`Parent`에서 `identify` 디스크립터를 찾는다.

하지만 그 디스크립터가 반환하는 메서드는 실제 접근한 클래스인 `Child`에 바인딩된다.

따라서 다음 관계가 형성된다.



즉, 함수를 어디에서 찾았는지와 그 함수가 어느 클래스에 바인딩되는지는 별개의 문제다.

이것이 클래스 메서드를 대체 생성자(Alternative Constructor)로 사용할 수 있는 중요한 이유다.

```
class Model:

    def __init__(self, hidden_size):
        self.hidden_size = hidden_size

    @classmethod
    def from_default(cls):
        return cls(128)


class TransformerModel(Model):
    pass


model = TransformerModel.from_default()

print(type(model).__name__)
print(model.hidden_size)
```

예상 출력:

```
TransformerModel
128
```

`from_default`는 `Model`에서 정의되었지만

`TransformerModel.from_default()`로 호출했으므로 `cls`는

`TransformerModel`을 참조한다.

따라서 다음 표현식은

```
cls(128)
```

`TransformerModel(128)`을 호출하는 효과를 가진다.

다만 하위 클래스가 생성자 인수 구조를 변경했다면 `cls(128)`이 반드시 성공하는 것은 아니다.

A.11.7 `staticmethod` - 자동 바인딩을 수행하지 않는 디스크립터

이번에는 정적 메서드를 살펴보자.

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b


obj = Calculator()
```

클래스 본문의 다음 부분은

```
@staticmethod
def add(a, b):
    return a + b
```

개념적으로 다음과 대응한다.

```
def add(a, b):
    return a + b
```

```
add = staticmethod(add)
```

클래스 네임스페이스에는 staticmethod 객체가 저장된다.

```
raw = Calculator.__dict__["add"]

print(type(raw))
print(type(raw.__func__))
```

예상 출력:

```
<class 'staticmethod'>
<class 'function'>
```

하지만 클래스나 인스턴스를 통해 속성을 조회하면 감싸고 있던 함수 객체가 반환된다.

```
print(Calculator.add is raw.__func__)
print(obj.add is raw.__func__)
```

예상 출력:

```
True
True
```

이것은 staticmethod가 속성 조회 시 원본 함수 객체를 반환하고, 인스턴스나 클래스를 자동 인수로 바인딩하지 않기 때문이다.

따라서 다음 두 호출은 모두 정상적으로 실행된다.

```
print(Calculator.add(10, 20))
print(obj.add(10, 20))
```

예상 출력:

```
30
30
```

여기서 obj.add(10, 20)은 일반적인 인스턴스 메서드 호출과 다르다.

인스턴스 obj가 첫 번째 인수로 자동 전달되지 않는다.

결국 staticmethod의 정확한 의미는 다음과 같다.

클래스 네임스페이스에 함수를 저장하되, 일반적인 인스턴스 메서드 바인딩을 적용하지 않도록 하는 디스크립터다.

A.11.8 property - 속성 조회가 함수 호출을 유발하는 구조

이번에는 property를 살펴보자.

```
class Account:

    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance


account = Account(100)
```

이 코드에서

```
@property
def balance(self):
    return self._balance
```

는 개념적으로 다음과 대응한다.

```
def balance(self):
    return self._balance

balance = property(balance)
```

따라서 클래스 네임스페이스에는 함수 객체가 아니라 property 객체가 저장된다.

```
raw = Account.__dict__["balance"]

print(type(raw))
print(type(raw.fget))
```

예상 출력:

```
<class 'property'>
```

```
<class 'function'>
```

`raw.fget`은 getter 함수 객체를 참조한다.

이제 다음 표현식을 평가한다고 하자.

```
account.balance
```

일반적인 함수 디스크립터였다면 바운드 메서드가 반환되었겠지만,

`property`는 다르게 동작한다.

`property.__get__`이 getter 함수를 호출하고, 그 함수의 반환값을 속성 조회 결과로 반환한다.

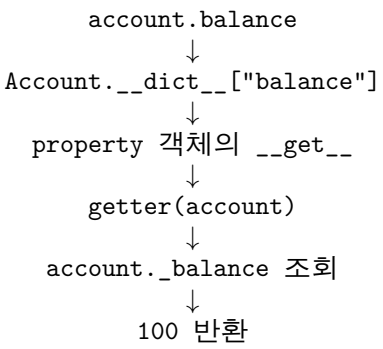
따라서 다음 결과를 얻는다.

```
print(account.balance)
```

예상 출력:

```
100
```

핵심 실행 흐름은 다음과 같다.



`property`는 데이터 디스크립터다

속성 탐색 우선순위를 생각해보자.

일반적인 인스턴스 속성 조회에서는 데이터 디스크립터가 인스턴스의

`__dict__`보다 우선한다.

property는 setter가 정의되지 않았더라도 데이터 디스크립터에 해당한다.

다음 코드로 확인할 수 있다.

```
account.__dict__["balance"] = 999

print(account.__dict__["balance"])
print(account.balance)
```

예상 출력:

```
999
100
```

인스턴스 딕셔너리에 "balance"라는 키가 존재하지만

account.balance를 조회하면 property가 우선 적용된다.

따라서 getter가 실행되어 self._balance의 값인 100이 반환되는 것이다.

> @balance.setter는 무엇을 반환할까?

```
class Account:

    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("잔액은 음수일 수 없습니다.")

        self._balance = value
```

여기서 @balance.setter는 기존 property 객체에 setter를 제자리에서 추가하는 것과는 다르다.

property.setter()는 지정된 setter를 포함하는 새로운 property 객체를 반환한다.

그 반환 객체가 다시 클래스 네임스페이스의 balance 이름에 바인딩된다.

최종 객체를 확인해 보자.

```
descriptor = Account.__dict__["balance"]

print(type(descriptor.fget))
print(type(descriptor.fset))
```

예상 출력:

```
<class 'function'>
<class 'function'>
```

이제 다음 표현식은

```
account.balance = 200
```

일반적인 인스턴스 딕셔너리 할당이 아니라 `property.__set__`을 통한 setter 호출로 처리된다.

따라서 `property`는 함수 호출을 속성 읽기·쓰기 연산에 연결하는 디스크립터라고 이해하면 된다.

A.11.9 사용자 정의 데코레이터를 인스턴스 메서드에 적용하기

이제 이번 학습의 핵심으로 들어가 본다.

일반적인 함수 데코레이터를 인스턴스 메서드에 적용하면 어떤 일이 발생할까?

```
from functools import wraps


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE:", func.__name__)
        return func(*args, **kwargs)

    return wrapper


class Calculator:
```

```
    def __init__(self, base):
        self.base = base

    @trace
    def add(self, value):
        return self.base + value


obj = Calculator(10)

print(obj.add(5))
```

예상 출력:

```
TRACE: add
15
```

이 코드는 세 단계로 분석해야 한다.

1. 클래스 본문에서 데코레이터 적용

먼저 원본 `add` 함수 객체가 생성된다.

그다음 `trace`에 원본 함수 객체가 전달된다.

```
add = trace(add)
```

`trace`는 wrapper 함수 객체를 생성하고 반환한다.

따라서 클래스 네임스페이스에 저장되는 객체는 원본 `add`가 아니라 wrapper다.

2. 인스턴스에서 `add` 조회

```
obj.add
```

클래스 네임스페이스에서 발견되는 객체는 wrapper 함수 객체다.

그런데 wrapper 역시 일반적인 Python 함수 객체이므로 `__get__`을 제공한다.

따라서 인스턴스 속성 조회에서 wrapper가 `obj`에 바인딩된다.

즉, `obj.add`는 wrapper를 감싸는 바운드 메서드 객체다.

3. 바운드 메서드 호출

```
obj.add(5)
```

바운듣 메서드는 다음과 같은 호출을 수행한다.

```
wrapper(obj, 5)
```

따라서 wrapper 내부에서 다음 바인딩이 형성된다.

```
args = (obj, 5)
kwargs = {}
```

이후 wrapper는 클로저에 저장된 원본 add 함수 객체를 호출한다.

```
func(*args, **kwargs)
```

결국 원본 함수는 다음 인수를 전달받아 실행된다.

```
original_add(obj, 5)
```

원본 함수에서 self는 obj를 참조하고 value는 5를 참조한다.

따라서 obj.base + value의 결과인 15를 반환한다.

바인딩 구조를 검증해 보자

```
bound = obj.add

wrapper_function = Calculator.__dict__["add"]
original_function = wrapper_function.__wrapped__

print(bound.__self__ is obj)
print(bound.__func__ is wrapper_function)
print(wrapper_function is original_function)
```

예상 출력:

```
True
True
False
```

이 결과는 다음 사실을 보여준다.

> 인스턴스에 바인딩되는 함수는 원본 add가 아니라
>
> 데코레이터가 반환한 wrapper 함수 객체다.

그리고 wrapper가 원본 함수에 self를 포함한 인수들을 전달하는 것이다.

A.11.10 @classmethod와 사용자 정의 데코레이터의 적용 순서

A.10에서 학습한 데코레이터 적용 순서는 클래스 메서드에서도 그대로 성립한다.

그런데 classmethod가 반환하는 객체는 일반 함수 객체가 아니라 디스크립터이기 때문에, 적용 순서를 바꾸면 결과가 크게 달라질 수 있다.

먼저 다음 코드를 보자.

```
from functools import wraps


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE:", func.__name__)
        return func(*args, **kwargs)

    return wrapper


class Service:

    def __init__(self, value):
        self.value = value

    @classmethod
    @trace
    def create(cls, value):
        return cls(value)


service = Service.create(10)

print(type(service).__name__)
print(service.value)
```

예상 출력:

```
TRACE: create
Service
10
```

여기서 데코레이터는 아래에서 위로 적용된다.

따라서 다음 코드와 대응한다.

```
create = classmethod(trace(create))
```

이 과정을 객체 생성 순서로 분석해 보자.

첫 번째 단계

원본 create 함수 개체가 생성된다.

두 번째 단계

trace가 원본 함수 객체를 전달받아 wrapper 함수 객체를 생성한다.

세 번째 단계

classmethod가 wrapper 함수 객체를 감싸는 디스크립터를 생성한다.

네 번째 단계

클래스 네임스페이스의 create 이름에 classmethod 객체가 저장된다.

이제 다음 표현식이 평가된다.

```
Service.create
```

classmethod의 __get__이 wrapper를 Service 클래스에 바인딩한 메서드를 반환한다.

따라서 다음 호출은

```
Service.create(10)
```

개념적으로 다음과 같다.

```
wrapper(Service, 10)
```

Wrapper는 클로저에 저장된 원본 함수 객체를 호출한다.

```
func(Service, 10)
```

결국 원본 `create`의 `cls`는 `Service` 클래스 객체를 참조하게 된다.

내부 구조 검증

```
descriptor = Service.__dict__["create"]

bound = Service.create

wrapper_function = descriptor.__func__
original_function = wrapper_function.__wrapped__

print(type(descriptor))

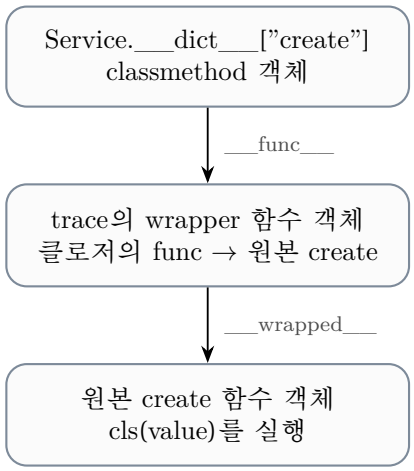
print(bound.__self__ is Service)
print(bound.__func__ is wrapper_function)

print(wrapper_function is original_function)
```

예상 출력:

```
<class 'classmethod'>
True
True
False
```

이 관계를 그림으로 나타내면 다음과 같다.



Service.create → wrapper가 클래스에 바인딩된 메서드

여기서 `classmethod`는 원본 함수를 직접 감싸는 것이 아니라, `trace`가 반환한 wrapper를 감싸고 있다는 점이 중요하다.

A.11.11 `@trace @classmethod`로 순서를 바꾸면?

이번에는 데코레이터의 순서를 반대로 작성해 보자.

```
class Service:

    @trace
    @classmethod
    def create(cls, value):
        return cls(value)
```

이 코드는 다음과 대응한다.

```
create = trace(classmethod(create))
```

앞의 예제와 결정적인 차이가 있다.

이번에는 `trace`에 전달되는 객체가 일반 함수가 아니라 `classmethod` 객체다.

`trace`는 이를 `func`라는 이름으로 참조하고 ㅈrapper를 생성한다.

최종적으로 클래스 네임스페이스에 저장되는 객체는 `classmethod`가 아니라

일반 wrapper 함수 객체다.

따라서 Service.create를 조회할 때 classmethod의 클래스 바인딩이 적용되지 않는다.

게다가 wrapper 내부에서 다음 코드를 실행하면

```
return func(*args, **kwargs)
```

func가 참조하는 classmethod 객체를 직접 호출하려고 한다.

일반적인 classmethod 객체는 직접 호출할 수 없으므로 TypeError가 발생한다.

두 구조를 비교해 보자.

클래스 바인딩 유지

```
@classmethod
@trace
def create(cls):
    ...
```

- 최종 저장 객체 : classmethod(wrapper)
- 속성 조회 : wrapper를 클래스에 바인딩

일반적인 trace에서는 문제 발생

```
@trace
@classmethod
def create(cls):
    ...
```

- 최종 저장 객체 : 일반 wrapper 함수
- 속성 조회 : 클래스 메서드 바인딩이 유지되지 않음

따라서 일반적인 함수 ㅈrapper 데코레이터를 클래스 메서드에 적용할 때는 보통 다음 구조가 적절하다.

```
@classmethod
```

```
@trace
def create(cls):
    ...
```

다만 모든 데코레이터가 반드시 이 순서를 요구하는 것은 아니다.

classmethod 디스크립터를 인식하고 별도로 처리하도록 설계한 데코레이터라면 다른 구조도 가능하다.

판단 기준은 데코레이터가 어떤 타입의 객체를 전달받고 어떤 타입의 객체를 반환하는가이다.

A.11.12 staticmethod와 데코레이터의 적용 순서

정적 메서드에도 같은 원리를 적용할 수 있다.

```
class Calculator:

    @staticmethod
    @trace
    def multiply(a, b):
        return a * b


obj = Calculator()

print(Calculator.multiply(3, 4))
print(obj.multiply(3, 4))
```

예상 출력:

```
TRACE: multiply
12
TRACE: multiply
12
```

이 구조는 다음과 대응한다.

```
multiply = staticmethod(trace(multiply))
```

먼저 원본 함수에 trace가 적용되어 wrapper가 생성되고,

그 wrapper를 staticmethod가 감싼다.

따라서 클래스나 인스턴스를 통해 속성을 조회해도 wrapper 함수 객체가 그대로 반환된다.

인스턴스가 첫 번째 인수로 자동 전달되지 않는 것이다.

순서를 반대로 작성하면?

```
class Calculator:

    @trace
    @staticmethod
    def multiply(a, b):
        return a * b
```

이번에는 다음과 대응한다.

```
multiply = trace(staticmethod(multiply))
```

`trace`가 `staticmethod` 객체를 전달받아 일반 함수 wrapper를 반환한다.

여기서 `classmethod`와는 차이가 있다.

Python 3.10부터 `staticmethod` 객체 자체도 직접 호출할 수 있다.

따라서 다음과 같은 클래스 접근 호출은 동작할 수 있다.

```
Calculator.multiply(3, 4)
```

하지만 인스턴스 접근은 문제가 달라진다.

```
obj = Calculator()

obj.multiply(3, 4)
```

최종 클래스 속성에 저장된 것은 일반 wrapper 함수 객체이므로 인스턴스 바인딩이 발생한다.

Wrapper는 `obj`까지 포함하여 원본 쪽으로 인수를 전달하려고 한다.

결국 원본 `multiply(a, b)`에 세 개의 위치 인수가 전달되어 `TypeError`가 발생할 수 있다.

즉, `staticmethod` 객체가 호출 가능하다는 사실과 정적 메서드의 바인딩 규칙이 보존된다는 사실은 별개다.

이 경우에도 일반적으로 다음 순서가 의도에 맞다.

```
@staticmethod
@trace
def multiply(a, b):
    ...
```

A.11.13 property와 사용자 정의 데코레이터

property에도 함수 wrapper를 적용할 수 있다.

```
from functools import wraps

def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE:", func.__name__)
        return func(*args, **kwargs)

    return wrapper


class Account:

    def __init__(self, balance):
        self._balance = balance

    @property
    @trace
    def balance(self):
        return self._balance


account = Account(100)

print(account.balance)
```

예상 출력:

```
TRACE: balance
100
```

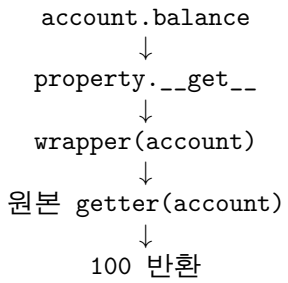
이 구조는 다음과 대응한다.

```
balance = property(trace(balance))
```

먼저 원본 getter 함수에 `trace`가 적용되어 wrapper를 생성한다.

그다음 `property`가 wrapper를 getter로 저장한다.

따라서 `account.balance`를 조회하면 다음 호출 경로가 형성된다.



여기서 wrapper는 인스턴스에 바인딩된 메서드로 조회되는 것이 아니다.

`property`가 getter로 저장된 wrapper 함수 객체를 인스턴스 인수와 함께 직접 호출하는 구조다.

반대로 다음과 같이 작성하면

```
@trace
@property
def balance(self):
    ...
```

`trace`가 `property` 객체를 전달받게 된다.

일반적인 `trace`는 전달받은 객체를 함수처럼 호출하도록 구현되어 있으므로 정상적인 getter 동작을 보장하지 못한다.

더구나 최종적으로 클래스 네임스페이스에 일반 wrapper 함수가 저장되므로

`property`의 데이터 디스크립터 동작도 유지되지 않는다.

여기서도 데코레이터의 적용 순서가 최종 클래스 속성의 종류를 결정한다는 원리가 적용된다.

A.11.14 호출 가능한 객체와 메서드 바인딩은 같은 개념일까?

이번에는 한 단계 더 깊이 들어가 보자.

A.9에서 클래스 인스턴스를 데코레이터로 사용할 수 있다는 사실을 학습했다.

그렇다면 함수 wrapper 대신 호출 가능한 클래스 인스턴스를 반환하면 메서드 바인딩도 자동으로 이루어질까?

다음 코드를 보자.

```
class Counter:

    def __init__(self, func):
        self.func = func
        self.calls = 0

    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)


class Example:

    @Counter
    def calculate(self, x):
        return x * 2


obj = Example()
```

클래스 본문에서는 개념적으로 다음 변환이 수행된다.

```
calculate = Counter(calculate)
```

따라서 클래스 네임스페이스에 저장되는 객체는 함수 객체가 아니라 Counter 인스턴스다.

```
print(type(Example.__dict__["calculate"]))

print(
    obj.calculate
    is Example.__dict__["calculate"]
)
```

예상 출력:

```
<class '__main__.Counter'>
True
```

두 번째 결과에 주목해 보자.

Counter는 __call__을 구현했으므로 호출 가능한 객체다.

하지만 __get__은 구현하지 않았다.

따라서 일반적인 함수 객체처럼 인스턴스 바인딩을 수행하지 않는다.

이제 다음 호출을 생각해 보자.

```
obj.calculate(10)
```

obj.calculate는 클래스에 저장된 Counter 인스턴스 자체를 반환한다.

그 객체의 __call__이 실행되고 다음 코드에 도달한다.

```
return self.func(*args, **kwargs)
```

여기서 self.func은 원본 calculate 함수 객체를 참조한다.

원본 함수는 두 개의 인수를 요구한다.

```
def calculate(self, x):
    ...
```

하지만 전달된 인수는 10 하나뿐이다.

따라서 TypeError가 발생한다.

왜 일반 함수 wrapper는 정상적으로 동작했을까?

두 객체의 프로토콜이 다르기 때문이다.

구분	일반 함수 wrapper	위 예제의 Counter 인스턴스
호출 가능 여부	가능	가능
__call__	제공	제공
__get__	제공	제공하지 않음
인스턴스 접근 결과	바운드 메서드	Counter 객체 자체
인스턴스 자동 전달	수행	수행하지 않음

이것은 중요한 객체 모델의 구분이다.

호출 가능한 객체(Callable)라는 사실만으로 디스크립터 바인딩까지 제공되는 것은 아니다.

`__call__`은 호출 프로토콜에 관여하고, `__get__`은 속성 조회 및 바인딩 프로토콜에 관여한다.

A.11.15 클래스 기반 데코레이터에 `__get__` 구현하기

그렇다면 앞의 `Counter`를 어떻게 수정해야 할까?

가장 직접적인 방법은 `__get__`을 구현하는 것이다.

```
class Counter:

    def __init__(self, func):
        self.func = func
        self.calls = 0

    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)

    def __get__(self, instance, owner=None):

        if instance is None:
            return self

        def bound(*args, **kwargs):
            return self(instance, *args, **kwargs)

        return bound


class Example:

    @Counter
    def calculate(self, x):
        return x * 2


a = Example()
b = Example()

print(a.calculate(10))
print(b.calculate(20))
```

```
print(Example.__dict__["calculate"].calls)
```

예상 출력:

```
20
40
2
```

이번에는 왜 정상적으로 동작할까?

a.calculate를 조회하면 Counter 인스턴스의 __get__이 호출된다.

```
counter.__get__(a, Example)
```

이 함수는 bound라는 새로운 함수 객체를 생성하고 반환한다.

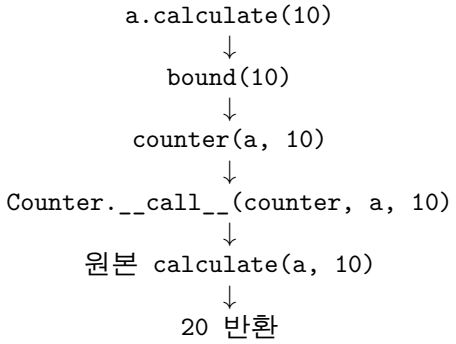
bound는 클로저를 통해 instance와 self를 참조한다.

여기서 instance는 a를, self는 Counter 인스턴스를 참조한다.

따라서 다음 호출은

```
a.calculate(10)
```

다음 순서로 실행된다.



여기서 한 가지 주의해야 한다.

이 구현이 반환한 bound는 일반적인 method 타입의 바운드 메서드가 아니다.

인스턴스 참조를 클로저로 유지하는 일반 함수 객체다.

따라서 일반적인 바운드 메서드의 `__self__`, `__func__` 구조와는 완전히 같지는 않다.

> 호출 횟수는 어디에 저장될까?

```
print(Example.__dict__["calculate"].calls)
```

결과는 2다.

a와 b는 서로 다른 인스턴스지만, 클래스 네임스페이스에 저장된 하나의 Counter 객체를 공유하기 때문이다.

따라서 이 예제에서 calls는 인스턴스별 호출 횟수가 아니라 데코레이터 객체의 누적 호출 횟수다.

이 차이는 실전에서 상태를 가진 클래스 기반 데코레이터를 설계할 때 중요하다.

A.11.16 디스크립터와 인스턴스 네임스페이스의 탐색 우선순위

이번에는 A.11의 내용을 기존 속성 탐색 규칙과 연결해 본다.

일반적인 `object.__getattribute__` 기반의 인스턴스 속성 탐색에서는 다음 우선순위가 적용된다.

우선순위	탐색 대상
1	클래스의 MRO에서 찾은 데이터 디스크립터
2	인스턴스의 `__dict__`
3	클래스의 MRO에서 찾은 비데이터 디스크립터 또는 일반 속성
4	일반 탐색 실패 시 적용될 수 있는 `__getattr__`

일반적인 함수 객체는 비데이터 디스크립터다.

따라서 인스턴스의 `__dict__`에 같은 이름의 속성이 있으면 인스턴스 속성이 우선할 수 있다.

```
class Example:
```

```
    def method(self):
        return "class method"


obj = Example()

obj.method = lambda: "instance attribute"

print(obj.method())
print(Example.method(obj))
```

예상 출력:

```
instance attribute
class method
```

왜 이런 결과가 발생할까?

`obj.method`를 조회할 때 인스턴스 딕셔너리에 `"method"`가 존재하므로 그 객체가 반환된다.

이때 저장된 `lambda` 함수는 인스턴스 속성으로 직접 조회되는 것이므로 클래스 함수 디스크립터의 바인딩이 적용되지 않는다.

반면 `Example.method`는 클래스 네임스페이스에 저장된 원본 함수 객체를 조회한다.

따라서 `Example.method(obj)`는 원본 함수를 명시적으로 호출하는 형태다.

이것은 일반적인 함수 객체가 비데이터 디스크립터라는 사실과 연결된다.

반면 `property`는 데이터 디스크립터이므로 인스턴스 딕셔너리보다 우선한다.

즉, 데코레이터가 반환하는 객체가 데이터 디스크립터인지, 비데이터 디스크립터인지, 일반 객체인지에 따라 속성 탐색 결과가 달라질 수 있다.

A.11.17 `inspect.signature()`와 바운드 메서드

A.8에서 학습한 함수 메타데이터를 메서드 바인딩과 연결해 보자.

```
import inspect
from functools import wraps
```

```
def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


class Calculator:

    @trace
    def add(self, x: int, y: int = 1) -> int:
        return x + y


obj = Calculator()

print(inspect.signature(Calculator.add))
print(inspect.signature(obj.add))
```

예상 출력:

```
(self, x: int, y: int = 1) -> int
(x: int, y: int = 1) -> int
```

왜 결과가 다를까?

`Calculator.add`는 클래스에서 조회한 wrapper 함수 객체다.

`@wraps(func)`가 설정한 `__wrapped__`를 따라 원본 함수의 시그니처를 확인할 수 있다.

따라서 첫 번째 결과에는 `self`가 포함된다.

반면 `obj.add`는 인스턴스에 바인딩된 메서드 객체다.

`inspect.signature()`는 바운드 메서드의 바인딩 상태를 고려해 이미 바인딩된 첫 번째 인수를 호출자에게 요구되는 인수 목록에서 제외한다.

여기서 서로 다른 세 메커니즘이 결합된다.

메커니즘	역할
`functools.wraps`	메타데이터와 `__wrapped__` 참조 설정
함수 디스크립터 `__get__`	인스턴스 바운드 메서드 생성
`inspect.signature`	메타데이터와 바인딩 상태를 분석하여 시그니처 구성

이 셋을 같은 기능으로 이해하면 안 된다.

특히 `wraps`가 `self`를 자동으로 전달하는 것은 아니다.

실제 바인딩은 디스크립터 프로토콜이 담당한다.

A.11.18 PyTorch `nn.Module`과의 연결

이제 지금까지 학습한 객체 모델을 PyTorch에 연결해 보자.

```
import torch
from torch import nn
from functools import wraps


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("FORWARD")
        return func(*args, **kwargs)

    return wrapper


class Model(nn.Module):

    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(2, 1)

    @trace
    def forward(self, x):
        return self.linear(x)


model = Model()
```

```
x = torch.tensor([[1.0, 2.0]])

y = model(x)

print(y.shape)
```

예상 출력:

```
FORWARD
torch.Size([1, 1])
```

이 코드는 두 가지 호출 메커니즘을 결합한다.

먼저 Model 클래스 네임스페이스에는 trace가 반환한 wrapper 함수 객체가 forward라는 이름으로 저장된다.

따라서 model.forward를 조회하면 wrapper가 model 인스턴스에 바인딩된다.

이것은 일반적인 Python 디스크립터 프로토콜의 동작이다.

하지만 다음 표현식은

```
model(x)
```

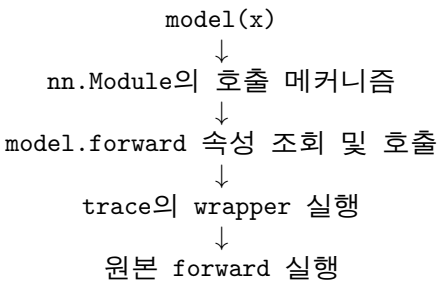
단순히 model.forward(x)로 치환되는 것은 아니다.

nn.Module의 호출 메커니즘이 관여하며, 필요한 경우 등록된 훅(Hook) 등의 처리를 수행한다.

그 과정에서 forward가 호출되면 wrapper가 실행되고,

wrapper는 원본 forward를 호출한다.

따라서 다음 네 단계를 구별해야 한다.



↓
self.linear(x)
↓
Tensor 반환

이 예제에서 @trace는 nn.Module.__call__을 대체하는 것이 아니다.

클래스의 forward 속성에 저장되는 함수 객체를 wrapper로 변경하는 것이다.

그리고 PyTorch는 Python의 일반적인 객체 모델 위에 자체적인 모듈 호출 및 훅 처리 기능을 추가한 것이다.

A.11.19 전체 개념 비교

이번 학습에서 다룬 객체들을 하나의 표로 정리해 보자.

객체	클래스 속성에 저장된 형태	속성 조회 시 동작	핵심 메커니즘
일반 함수	function	인스턴스 접근 시 바운드 메서드 생성	함수의 __get__
classmethod	classmethod 객체	클래스 바운드 메서드 생성	classmethod.__get__
staticmethod	staticmethod 객체	감싼 함수 객체 반환	staticmethod.__get__
property	property 객체	getter 실행 또는 setter 처리	property.__get__, __set__
일반 함수 wrapper	function	인스턴스 접근 시 wrapper 바인딩	함수의 __get__
__call__만 가진 데코레이터 인스턴스	사용자 정의 객체	자동 메서드 바인딩 없음	호출 프로토콜만 제공
__get__도 가진 데코레이터 인스턴스	사용자 정의 디스크립터	구현에 따라 바인딩 수행	사용자 정의 __get__

이 표에서 가장 중요한 구분은 호출 가능 여부와 디스크립터 여부가 서로 다른 성질이라는 점이다.

어떤 객체가 호출 가능하더라도 메서드 바인딩을 수행하지 않을 수 있다.

반대로 property처럼 일반적인 함수 호출 방식으로 직접 사용하지 않는 객체도 속성 조회에 개입할 수 있다.

A.11.20 핵심 정리와 판단 기준

이번 학습의 전체 실행 구조를 정리해 보자.

클래스 메서드에 데코레이터가 적용되는 전체 흐름

1. 클래스 본문 실행
   def 문장이 원본 함수 객체를 생성한다.

2. 데코레이터 적용
   데코레이터가 원본 함수 또는 다른 데코레이터의 반환 객체를 전달받아 새로운 객체를 반환할 수 있다.

3. 클래스 네임스페이스 바인딩
   최종 반환 객체가 클래스 속성 이름에 바인딩된다.

4. 클래스 또는 인스턴스 속성 조회
   속성 탐색 규칙에 따라 객체를 찾고, 디스크립터라면 __get__ 등의 동작이 적용될 수 있다.

5. 호출 또는 속성값 사용
   반환된 객체가 바운드 메서드인지, 일반 함수인지, 계산된 속성값인지에 따라 이후 동작이 달라진다.

실제 코드를 분석할 때는 다음 다섯 가지를 판단하면 된다.

1. 클래스 네임스페이스에 저장된 객체 타입이 무엇인가?
   일반 함수인지, wrapper인지, classmethod인지, property인지에 따라 속성 조회가 달라진다.

2. 그 객체는 어떤 디스크립터 메서드를 제공하는가?
   __get__, __set__, __delete__의 존재 여부가 속성 탐색과 바인딩에 영향을 준다.

3. 바운드 메서드가 반환되었다면 __self__와 __func__는 무엇을 참조하는가?
   이를 확인하면 어떤 객체가 자동 인수로 전달되는지 추적할 수 있다.

4. wrapper는 클로저를 통해 어떤 객체를 참조하는가?
   원본 함수인지, 다른 wrapper인지, 디스크립터 객체인지 구별해야 한다.

5. 데코레이터 적용 순서를 바꾸면 클래스에 저장되는 객체도 달라지는가?
   객체가 달라진다면 이후 속성 조회와 호출 규칙도 달라질 수 있다.

이번 학습의 핵심 결론은 다음과 같다.

데코레이터는 함수 정의 과정에서 객체를 변환하고,

디스크립터는 이후 속성 조회 과정에서 그 객체의 접근 결과를 결정한다.

이 두 메커니즘을 분리하면 인스턴스 메서드, 클래스 메서드, 정적 메서드, 프로퍼티, 사용자 정의 클래스 기반 데코레이터를 모두 같은 Python 객체 모델로 설명할 수 있다.

A.12 등록형 데코레이터와 FastAPI

Registration Decorator · Decorator Factory · Route Registry · Endpoint · Dependency Injection · Function Introspection

A.12.1 이전 학습과의 연결

A.11에서는 클래스 내부에 정의된 함수에 데코레이터를 적용할 때 두 가지 메커니즘을 구별했다.

첫째, 데코레이터 적용은 함수 정의문이 실행되는 과정에서 원본 함수 객체를 다른 객체로 변환할 수 있다.

둘째, 디스크립터 바인딩은 이후 클래스나 인스턴스를 통해 속성을 조회할 때 그 객체의 접근 결과를 결정한다.

이번에는 데코레이터의 또 다른 활용 방식인 등록형 데코레이터(Registration Decorator)를 학습할 것이다.

지금까지 우리가 주로 사용한 데코레이터는 다음과 같은 구조였다.

```
@trace
def calculate(x):
    return x * 2
```

trace는 원본 함수 객체를 전달받아 새로운 wrapper 함수 객체를 반환하였다.

그런데 다음 코드를 살펴보자.

```
@app.get("/items/{item_id}")
def get_item(item_id: int):
```

```
    return {"item_id": item_id}
```

이것은 FastAPI에서 사용하는 경로 처리 데코레이터(Path Operation Decorator)다.

겉으로 보면 지금까지 학습한 데코레이터와 같지만, 목적이 다르다.

`@app.get(...)`은 원본 함수에 로깅이나 성능 측정 기능을 덧붙이기 위해 wrapper를 생성하는 것이 아니다.

특정 HTTP 요청을 처리할 함수 객체를 FastAPI 애플리케이션에 등록하는 것이 주된 목적이다.

이 차이를 이해하면 데코레이터를 단순한 함수 변환 도구가 아니라, 프로그램의 구성 정보를 선언하고 연결하는 수단으로 바라볼 수 있다.

A.12.2 이번 학습의 핵심 질문

이번 학습에서는 다음 질문을 해결할 것이다.

1. 등록형 데코레이터는 일반적인 wrapper 데코레이터와 무엇이 다를까?
2. `@app.get("/tiems")`에서 `app.get(...)`은 어떤 객체를 반환할까?
3. 함수 객체는 언제 생성되고, 언제 FastAPI에 등록될까?
4. 등록 이후 함수 이름과 FastAPI 내부의 엔드포인트 참조는 어떤 관계일까?
5. HTTP 요청이 들어오면 Python 함수 객체는 어떤 과정을 거쳐 호출될까?
6. FastAPI는 함수의 매개변수와 타입 힌트를 어떻게 분석할까?
7. `@app.get`과 `@trace`의 적용 순서를 바꾸면 어떤 차이가 발생할까?
8. 동기 함수와 비동기 함수에 데코레이터를 적용할 때 무엇을 주의해야 할까?

핵심은 등록 시점(Registration Time)과 요청 처리 시점(Request Time)을 명확히 구별하는 것이다.

A.12.3 등록형 데코레이터의 정확한 정의

등록형 데코레이터는 전달받은 객체를 레지스트리(Registry)와 같은 외부 관리 구조에 등록하는 부작용을 수행하는 데코레이터다.

등록 과정이 끝나면 원본 객체를 그대로 반환할 수도 있고, 다른 객체를 반환할 수도 있다.

따라서 등록형 데코레이터의 본질은 wrapper를 생성하지 않는다는 데 있지 않다.

전달받은 객체와 외부 관리 구조 사이에 등록 관계를 형성한다는 데 있다.

다음 두 구현을 비교해 보자.

Wrapper 데코레이터

```
def trace(func):
    def wrapper(*args, **kwargs):
        print("TRACE")
        return func(*args, **kwargs)

    return wrapper
```

함수 호출을 중개하는 새 함수 객체를 반환한다.

등록형 데코레이터

```
registry = {}

def register(func):
    registry[func.__name__] = func
    return func
```

함수 객체를 외부 자료구조에 저장하고 원본을 반환한다.

두 번째 예제에서 중요한 문장은 다음이다.

```
registry[func.__name__] = func
```

이 문장은 새로운 wrapper를 생성하지 않는다.

대신 `registry` 딕셔너리에 함수 객체에 대한 참조를 저장한다.

그리고 다음 문장은

```
return func
```

원본 함수 객체를 그대로 반환한다.

따라서 데코레이션 이후 함수 이름이 참조하는 객체의 동일성은 유지된다.

하지만 외부 레지스트리에는 새로운 등록 정보가 추가된다.

이것이 등록형 데코레이터의 가장 기본적인 형태다.

A.12.4 가장 단순한 등록형 데코레이터 구현

먼저 FastAPI없이 Python만으로 등록형 데코레이터를 만들어 보자.

```
registry = {}


def register(func):
    print("register:", func.__name__)

    registry[func.__name__] = func

    return func


@register
def greet(name):
    print("Hello", name)
    return name.upper()


print("definition complete")

print(registry["greet"] is greet)

result = registry["greet"]("Kim")

print("result:", result)
```

예상 출력:

```
register: greet
definition complete
True
Hello Kim
result: KIM
```

이 결과를 객체 생성과 바인딩 순서에 따라 분석해 보자.

1. 함수 정의문 실행

Python이 다음 함수 정의문에 도달한다.

```
@register
def greet(name):
    print("Hello", name)
    return name.upper()
```

원본 greet 함수 객체가 생성된다.

여기서 중요한 점은 함수 객체가 생성되었다고 함수 본문이 실행되는 것은 아니라는 사실이다.

따라서 아직 "Hello Kim"은 출력되지 않는다.

2. 데코레이터 적용

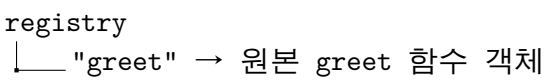
생성된 함수 객체가 register에 전달된다.

```
register(original_greet)
```

register 내부에서는 다음 문장이 실행된다.

```
registry[func.__name__] = func
```

func.__name__은 "greet"이므로 레지스트리에 다음 관계가 형성된다.



그리고 register는 같은 함수 객체를 반환한다.

3. 최종 이름 바인딩

데코레이터의 반환값이 모듈 네임스페이스의 greet 이름에 바인딩된다.

따라서 다음 두 표현식은 동일한 함수 객체를 참조한다.

```
greet
registry["greet"]
```

이것이 다음 결과가 True인 이유다.

```
registry["greet"] is greet
```

4. 레지스트리를 통한 호출

마지막으로 다음 표현식이 실행된다.

```
registry["greet"]("Kim")
```

딕셔너리 조회 결과는 함수 객체다.

따라서 일반적인 함수 호출 규칙이 적용된다.

이때 비로소 원본 함수 본문이 실행되고 `"Hello Kim"`이 출력된다.

여기서 발생한 상태 변화

이 예제에서는 원본 함수 객체를 수정한 것이 아니다.

변경된 것은 외부 딕셔너리의 상태다.

대상	실행 전	실행 후
원본 `greet` 함수 객체	생성 전	생성됨
`registry` 딕셔너리	빈 딕셔너리	`"greet"` → 함수 객체
모듈 이름 `greet`	정의 전	원본 함수 객체 참조
원본 함수 본문	미실행	레지스트리에서 호출할 때 실행

이것이 wrapper 데코레이터와 등록형 데코레이터를 구별하는 첫 번째 기준이다.

A.12.5 등록형 데코레이터에도 인수를 전달할 수 있다

A.9에서는 데코레이터 팩토리를 학습했다.

등록형 데코레이터도 같은 구조로 확장할 수 있다.

```
registry = {}


def register(name):
    print("factory:", name)
```

```
    def decorator(func):
        print("register:", func.__name__)

        registry[name] = func

        return func

    return decorator


@register("hello")
def greet(name):
    return f"Hello, {name}"


print("definition complete")

print(registry["hello"] is greet)
print(registry["hello"]("Kim"))
```

예상 출력:

```
factory: hello
register: greet
definition complete
True
Hello, Kim
```

이제 데코레이터의 실행 과정을 세 단계로 구분할 수 있다.

데코레이터 팩토리 → 등록 → 호출

1. 팩토리 호출
   `register("hello")`가 실행되어 `"hello"`를 클로저로 유지하는 `decorator` 함수 객체를 반환한다.

2. 등록 수행
   원본 `greet` 함수 객체가 생성된 뒤 `decorator(greet)`이 호출되고, `registry["hello"]`에 함수 객체가 저장된다.

3. 실제 함수 호출
   이후 `registry["hello"]("Kim")`을 실행할 때 비로소 원본 함수 본문이 실행된다.

여기서 `register("hello")`의 반환값은 데코레이터 함수 객체다.

반면 그 데코레이터의 반환값은 원본 `greet` 함수 객체다.

따라서 다음 두 return은 의미가 다르다.

```
return decorator
```

이것은 설정이 완료된 데코레이터 함수 객체를 반환하는 단계다.

```
return func
```

이것은 등록된 원본 함수 객체를 반환하는 단계다.

이 구분을 이해했다면 FastAPI의 @app.get(...) 구조도 같은 원리로 분석할 수 있다.

A.12.6 FastAPI의 @app.get(...)은 어떤 구조일까?

이제 실제 FastAPI를 살펴보자.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}
```

여기서 가장 먼저 구별해야 할 객체는 세 가지다.

표현식	객체의 역할
app	FastAPI 애플리케이션 인스턴스
app.get	app에 바인딩된 메서드
app.get("/items/{item_id}")	경로 등록을 수행할 데코레이터 함수

A.11에서 학습한 메서드 바인딩을 연결해 보자.

```
print(app.get.__self__ is app)
```

예상 출력:

```
True
```

`app.get`은 FastAPI 클래스에 정의된 메서드가 `app` 인스턴스에 바인딩된 결과다.

따라서 다음 표현식은

```
app.get("/items/{item_id}")
```

FastAPI 인스턴스의 `get` 메서드를 호출하는 것이다.

그 호출 결과로 데코레이터 함수 객체가 반환된다.

여기서 `get` 메서드 호출과 엔드포인트 함수 호출은 완전히 다른 사건이다.

`app.get(...)`을 실행한다고 `get_item` 함수의 본문이 실행되는 것은 아니다.

실제 변환식

다음 코드는

```
@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}
```

개념적으로 다음과 대응한다.

```
decorator = app.get("/items/{item_id}")


def get_item(item_id: int):
    return {"item_id": item_id}


get_item = decorator(get_item)
```

FastAPI의 일반적인 경로 등록 데코레이터는 전달받은 엔드포인트 함수 객체를 라우터에 등록한 뒤 그 함수 객체를 그대로 반환한다.

따라서 데코레이션 과정에서 별도의 사용자 정의 wrapper가 반드시 생성되는 것은 아니다.

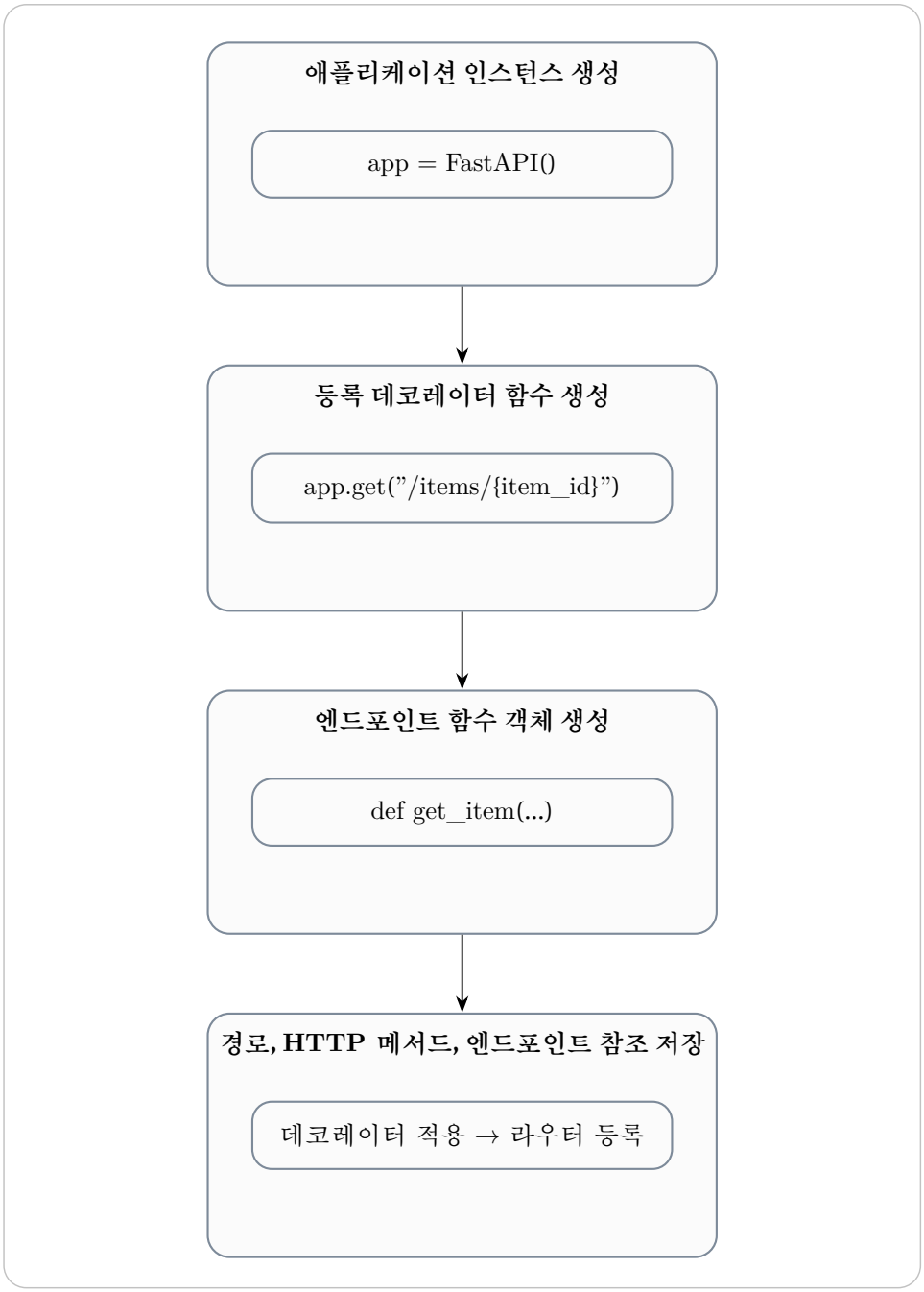
다만 FastAPI는 등록된 엔드포인트를 HTTP 요청으로 실행하기 위한 내부 요청 처리 함수를 별도로 구성한다.

등록 데코레이터가 원본 함수를 반환한다는 사실과 프레임워크 내부에 요청 처리 계층이 존재한다는 사실은 양립한다.

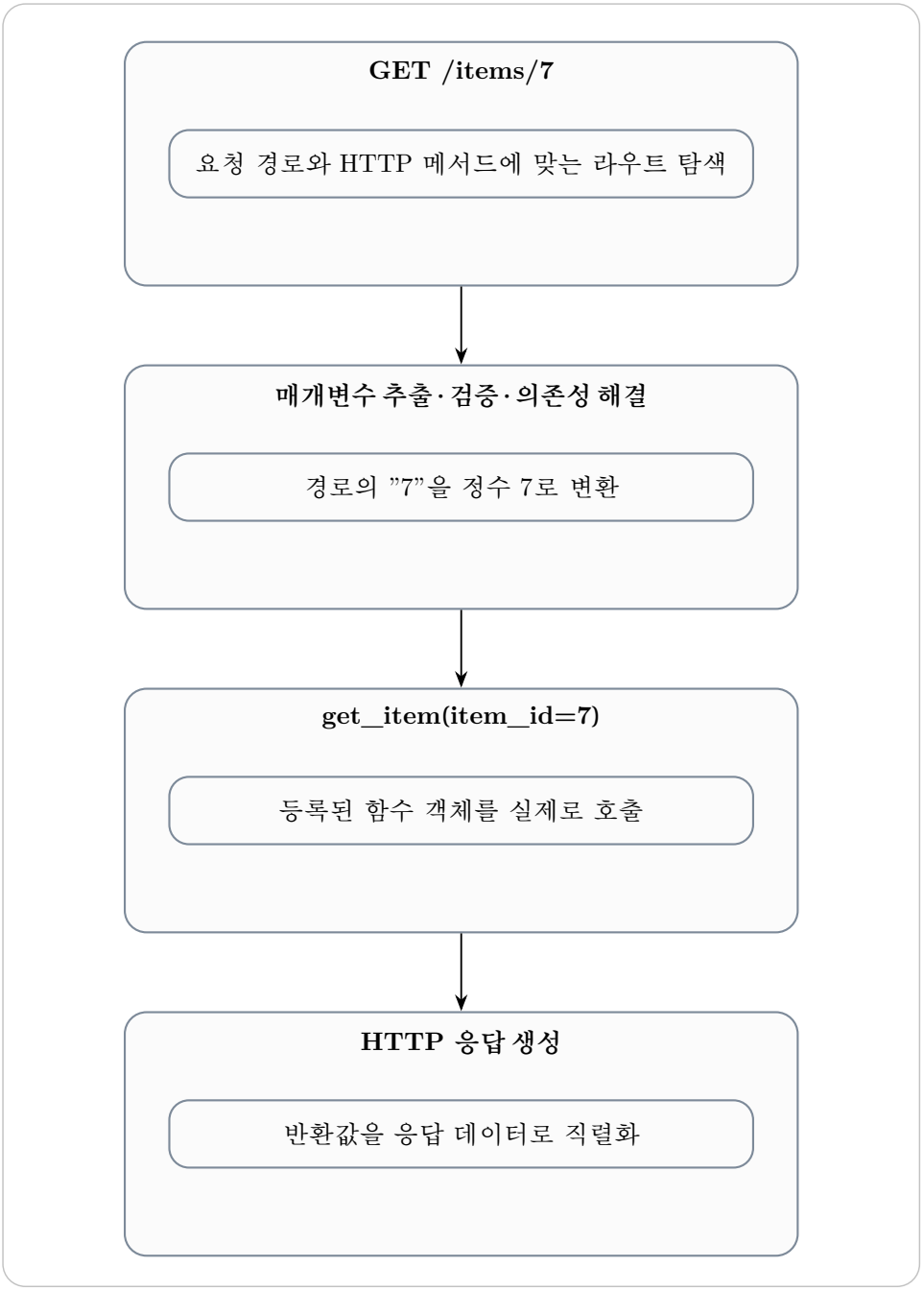
A.12.7 등록 시점과 HTTP 요청 처리 시점

FastAPI의 실행 과정을 두 시점으로 나누어 보자.

```
1. 등록 시점 - Python 모듈 실행
```



2. 요청 처리 시점 - HTTP GET 요청 도착



여기서 중요한 것은 등록과 호출 사이에 시간적 간격이 존재한다는 사실이다.

보통 Python 모듈이 실행되어 경로가 등록된 뒤, 나중에 HTTP 요청이 도착했을 때 엔드포인트가 호출된다.

같은 애플리케이션 프로세스에서 요청이 여러 번 발생한다고 해서 매번 함수 정의문이나 `@app.get(...)`이 다시 실행되는 것은 아니다.

A.12.8 실제 FastAPI의 등록 객체를 확인하기

이번에는 FastAPI 내부에 어떤 객체가 저장되는지 확인해 보자.

다음 예제는 `fastapi`와 `httpx`가 설치된 환경에서 독립적으로 실행할 수 있다.

`TestClient`는 별도의 웹 서버를 실행하지 않고 애플리케이션에 테스트 요청을 전달하는 도구다.

```
from fastapi import FastAPI
from fastapi.routing import APIRoute
from fastapi.testclient import TestClient

app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: int):
    print("endpoint called:", item_id)
    return {"item_id": item_id}

route = next(
    route
    for route in app.routes
    if isinstance(route, APIRoute)
    and route.path == "/items/{item_id}"
)

print(route.endpoint is get_item)

client = TestClient(app)

response = client.get("/items/7")

print(response.status_code)
print(response.json())

invalid = client.get("/items/abc")

print(invalid.status_code)
```

예상 출력:

```
True
endpoint called: 7
200
{'item_id': 7}
422
```

이 예제에는 서로 다른 두 종류의 함수 호출이 등장한다.

1. 등록된 엔드포인트의 동일성

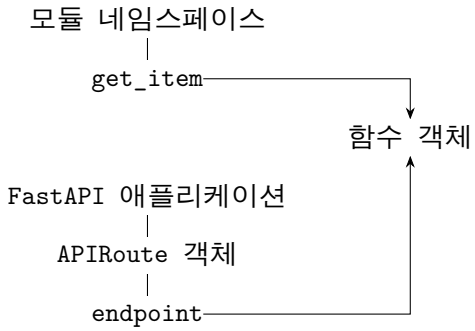
```
route.endpoint is get_item
```

결과는 `True`다.

`route`는 FastAPI의 `APIRoute` 객체이고, `route.endpoint`는 등록된 엔드포인트 함수 객체를 참조한다.

현재 예제에서는 `@app.get(...)`이 원본 함수 객체를 반환했으므로 모듈 이름 `get_item`도 같은 객체를 참조한다.

따라서 다음 구조가 형성된다.



여기서 함수 객체가 복사된 것은 아니다.

서로 다른 두 위치에서 동일한 함수 객체를 참조하는 것이다.

2. 실제 HTTP 요청

```
response = client.get("/items/7")
```

이 코드는 등록된 함수 객체를 직접 호출하는 코드가 아니다.

HTTP 요청을 모사하여 FastAPI 애플리케이션의 요청 처리 경로로 전달한다.

FastAPI는 요청 경로에서 "7"을 추출하고, 엔드포인트의 타입 힌트를 근거로 정수 7을 준비한다.

그다음 엔드포인트 함수를 호출한다.

이때 비로소 다음 출력이 발생한다.

```
endpoint called: 7
```

반면 다음 요청은

```
client.get("/items/abc")
```

"abc"를 정수로 변환할 수 없으므로 요청 매개변수 검증 단계에서 실패한다.

따라서 HTTP 상태 코드 422가 반환되고, 해당 요청에서는 get_item 함수 본문이 실행되지 않는다.

이것이 함수 객체의 등록과 함수 본문의 실행이 서로 다른 시점에 발생한다는 사실을 보여주는 예제다.

A.12.9 직접 함수 호출과 HTTP 요청은 같지 않다

앞의 예제에서 다음 두 표현식을 비교해 보자.

```
get_item("7")
```

```
client.get("/items/7")
```

첫 번째는 일반적인 Python 함수 호출이다.

두 번째는 FastAPI의 HTTP 요청 처리 과정이다.

따라서 동작이 다르다.

```
print(get_item("7"))
```

예상 출력:

```
endpoint called: 7
```

```
{'item_id': '7'}
```

여기서는 item_id에 문자열 "7"이 그대로 전달된다.

Python의 타입 힌트 item_id: int 자체는 일반적인 함수 호출에서 자동 형변환을 수행하지 않는다.

반면 HTTP 요청에서는 FastAPI가 엔드포인트 시그니처를 분석하여 입력값을 검증하고 변환한다.

구분	직접 함수 호출	FastAPI HTTP 요청
함수 호출 주체	Python 코드	FastAPI 요청 처리 계층
경로 매개변수 추출	없음	있음
타입 힌트 기반 입력 검증	자동 수행되지 않음	수행
의존성 주입	자동 수행되지 않음	수행
HTTP 응답 생성	없음	수행
원본 함수 본문	직접 실행	검증·의존성 처리 후 실행

여기서 중요한 점은 FastAPI가 Python 함수 호출 규칙을 변경한 것이 아니라는 사실이다.

FastAPI는 함수를 호출하기 전에 HTTP 요청을 분석하고, 필요한 인수들을 구성하는 별도의 계층을 제공하는 것이다.

A.12.10 FastAPI는 함수 시그니처를 어떻게 활용할까?

이제 A.8에서 학습한 inspect.signature()와 연결해 보자.

FastAPI는 등록된 함수 매개변수, 타입 힌트, 기본값 등을 분석하여 HTTP 요청에서 어떤 값을 추출하고 어떤 의존성을 해결해야 하는지 결정한다.

다음 예제를 보자.

```python
import inspect
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

app = FastAPI()
```

```
def get_multiplier() -> int:
    return 3

@app.get("/scores/{score}")
def calculate_score(
    score: int,
    offset: int = 0,
    multiplier: int = Depends(get_multiplier),
):
    return {
        "result": (score + offset) * multiplier
    }

signature = inspect.signature(calculate_score)

print(signature.parameters["score"].annotation)
print(signature.parameters["offset"].default)

print(
    type(
        signature.parameters["multiplier"].default
    ).__name__
)

client = TestClient(app)

response = client.get("/scores/4?offset=2")

print(response.json())
```

예상 출력:

```
<class 'int'>
0
Depends
{'result': 18}
```

이 코드에서 세 매개변수는 서로 다른 방식으로 처리된다.

매개변수	선언	FastAPI에서의 역할
score	score: int	URL 경로에서 추출
offset	offset: int = 0	쿼리 매개변수에서 추출, 생략 시 기본값 사용
multiplier	multiplier: int = Depends(get_multiplier)	의존성 함수의 반환값을 전달

요청은 다음과 같다.

```
GET /scores/4?offset=2
```

FastAPI는 등록된 경로와 함수 시그니처를 근거로 인수를 준비한다.

```
score = 4
offset = 2
multiplier = 3
```

여기서 multiplier의 값은 get_multiplier()를 호출하여 얻는다.

따라서 엔드포인트 호출은 개념적으로 다음과 같다.

```
calculate_score(
    score=4,
    offset=2,
    multiplier=3,
)
```

그 결과는 다음과 같다.

$$(4 + 2) \times 3 = 18$$

Depends(...)는 실제 의존성의 반환값이 아니다

여기서 중요한 점이 있다.

다음 선언을 보자.

```
multiplier: int = Depends(get_multiplier)
```

일반적인 Python 함수 정의 관점에서는 Depends(get_multiplier)의 호출

결과인 Depends 객체가 기본값으로 저장된다.

이 객체 자체가 정수 3인 것은 아니다.

FastAPI는 이 기본값을 의존성 해결을 위한 선언 정보로 해석한다.

따라서 다음처럼 직접 호출하면

```
calculate_score(4, offset=2)
```

FastAPI의 의존성 주입 과정이 실행되지 않는다.

multiplier에는 기본값인 Depends 객체가 바인딩되므로, 이 예제에서는 정수와 Depends 객체를 곱하려는 연산때문에 TypeError가 발생한다.

반면 다음 직접 호출은 가능하다.

```
calculate_score(
    4,
    offset=2,
    multiplier=3,
)
```

이처럼 FastAPI의 의존성 주입은 Python의 함수 호출 규칙 자체가 아니라 프레임워크가 함수 호출 전에 수행하는 별도의 인수 구성 과정이다.

A.12.11 @app.get과 @trace의 적용 순서를 바꾸면?

이번 학습에서 가장 중요한 실전 문제다.

A.10에서는 여러 데코레이터의 적용 순서가 최종 객체 구조를 결정한다는 사실을 학습했다.

FastAPI에서도 같은 원리가 적용된다.

먼저 일반적인 로깅 데코레이터를 정의해 보자.

```
from functools import wraps

from fastapi import FastAPI
from fastapi.routing import APIRoute
from fastapi.testclient import TestClient
```

```
app = FastAPI()


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE:", func.__name__)
        return func(*args, **kwargs)

    return wrapper
```

이제 적용 순서를 다르게 하여 두 엔드포인트를 등록해 보자.

```
@app.get("/logged")
@trace
def logged():
    return {"name": "logged"}


@trace
@app.get("/unlogged")
def unlogged():
    return {"name": "unlogged"}
```

겉으로는 두 함수 모두 `@app.get`과 `@trace`를 사용했다.

하지만 FastAPI에 등록되는 함수 객체는 다르다.

첫 번째 - `@app.get`이 바깥쪽

```
@app.get("/logged")
@trace
def logged():
    ...
```

데코레이터의 적용 순서는 아래에서 위다.

따라서 개념적으로 다음과 같다.

```
logged = app.get("/logged")(trace(original_logged))
```

실행 과정을 추적해 보자.

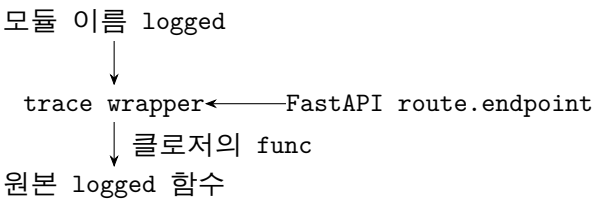
먼저 원본 `logged` 함수 객체가 생성된다.

그다음 trace가 원본 함수를 전달받아 wrapper를 반환한다.

이제 app.get("/logged")가 반환한 등록 데코레이터가 wrapper 함수 객체를 엔드포인트로 등록한다.

마지막으로 wrapper 함수 객체가 모듈 이름 logged에 바인딩된다.

따라서 다음 관계가 성립한다.



HTTP 요청이 들어오면 FastAPI가 wrapper를 호출한다.

Wrapper는 "TRACE: logged"를 출력한 뒤 원본 함수를 호출한다.

두 번째 - @trace가 바깥쪽

```
@trace
@app.get("/unlogged")
def unlogged():
    ...
```

이번에는 다음과 대응한다.

```
unlogged = trace(
    app.get("/unlogged")(original_unlogged)
)
```

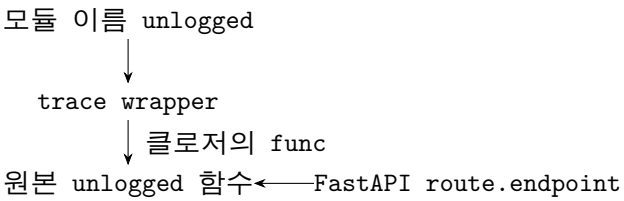
먼저 원본 함수 객체가 생성된다.

그다음 @app.get이 원본 함수 객체를 FastAPI에 등록하고 그대로 반환한다.

이후 trace가 그 원본 함수 객체를 감싸 새로운 wrapper를 반환한다.

마지막으로 모듈 이름 unlogged가 wrapper에 바인딩된다.

따라서 구조는 다음과 같다.



이번에는 FastAPI가 원본 함수 객체를 등록했다.

따라서 HTTP 요청을 처리할 때 wrapper를 거치지 않는다.

하지만 Python 코드에서 `unlogged()`를 직접 호출하면 wrapper가 실행된다.

실제 객체 동일성 검증

앞의 코드에 이어 다음을 실행해 보자.

```
def endpoint_for(path):
    return next(
        route.endpoint
        for route in app.routes
        if isinstance(route, APIRoute)
        and route.path == path
    )


print(endpoint_for("/logged") is logged)

print(endpoint_for("/unlogged") is unlogged)

print(
    endpoint_for("/unlogged")
    is unlogged.__wrapped__
)


client = TestClient(app)

client.get("/logged")
client.get("/unlogged")

unlogged()
```

예상 출력:

```
True
False
True
```

```
TRACE: logged
TRACE: unlogged
```

출력에서 "TRACE: logged"는 HTTP 요청에 의해 발생한다.

반면 "TRACE: unlogged"는 마지막의 직접 함수 호출에 의해 발생한다.

client.get("/unlogged") 자체는 로그를 출력하지 않는다.

이 결과는 단순한 실행 순서 차이가 아니다.

FastAPI가 참조하는 함수 객체와 모듈 이름이 참조하는 함수 객체가 서로 달라진 결과다.

두 순서의 비교

구분	@app.get 바깥쪽	@trace 바깥쪽
등록되는 함수 객체	trace wrapper	원본 함수
최종 함수 이름	trace wrapper	trace wrapper
HTTP 요청에서 trace 실행	실행됨	실행되지 않음
직접 함수 호출에서 trace 실행	실행됨	실행됨
route.endpoint is 함수이름	True	False

이것이 A.10에서 학습한 데코레이터 적용 순서가 실제 웹 프레임워크의 동작에 영향을 주는 대표적인 사례다.

A.12.12 functools.wraps가 FastAPI에서 중요한 이유

앞의 trace 구현에는 다음 코드가 있었다.

```
@wraps(func)
def wrapper(*args, **kwargs):
    ...
```

왜 @wraps(func)를 사용했을까?

A.8에서 학습했듯 functools.wraps는 wrapper의 메타데이터를 설정하고 __wrapped__를 통해 원본 함수 객체를 추적할 수 있도록 한다.

FastAPI는 엔드포인트의 시그니처를 분석하므로 이 정보가 중요하다.

다음 코드를 살펴보자.

```
import inspect
from functools import wraps


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@trace
def calculate(x: int, y: int = 1) -> int:
    return x + y


print(inspect.signature(calculate))

print(
    inspect.signature(
        calculate,
        follow_wrapped=False
    )
)
```

예상 출력:

```
(x: int, y: int = 1) -> int
(*args, **kwargs)
```

두 결과가 다른 이유는 다음과 같다.

`inspect.signature(calculate)`는 기본적으로 `__wrapped__` 체인을 따라 원본 함수의 시그니처를 분석한다.

반면 `follow_wrapped=False`를 지정하면 실제 wrapper 함수 객체의 시그니처를 확인한다.

이 차이가 FastAPI에서 중요하다.

> `@wraps`가 없으면 어떤 문제가 발생할까?

다음 코드를 살펴보자.

```
from fastapi import FastAPI
from fastapi.testclient import TestClient


app = FastAPI()


def bad_trace(func):

    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper


@app.get("/broken/{item_id}")
@bad_trace
def broken(item_id: int):
    return {"item_id": item_id}


client = TestClient(app)

response = client.get("/broken/5")

print(response.status_code)
```

예상 출력:

```
422
```

왜 정상적으로 실행되지 않을까?

FastAPI에 등록된 함수 객체는 `bad_trace`가 반환한 wrapper다.

그런데 이 wrapper의 시그니처는 다음과 같다.

```
(*args, **kwargs)
```

`__wrapped__` 참조도 설정되어 있지 않다.

따라서 FastAPI는 원본 함수의 `item_id: int` 매개변수를 정상적으로 추적하지 못하고, wrapper의 `args`, `kwargs`를 요청 매개변수로 분석할 수 있다.

이 예제에서는 필요한 쿼리 매개변수가 제공되지 않았다고 판단하여 422 응

답이 발생한다.

`@wraps(func)`를 사용하면 원본 함수의 시그니처를 추적할 수 있으므로 이런 문제를 방지하는 데 도움이 된다.

하지만 한 가지 주의해야 한다.

`@wraps`는 메타데이터를 설정하는 기능이지, wrapper의 실제 호출 동작을 원본 함수와 동일하게 만들어 주는 기능은 아니다.

예를 들어 원본 함수의 시그니처를 노출하더라도 wrapper가 전달받은 인수를 올바르게 처리하지 않으면 실행 시 오류가 발생할 수 있다.

따라서 FastAPI용 데코레이터는 메타데이터와 실제 인수 전달 규칙을 모두 고려해야 한다.

A.12.13 비동기 엔드포인트와 데코레이터

이제 FastAPI에서 특히 중요한 문제를 살펴보자.

FastAPI는 다음 두 종류의 엔드포인트를 지원한다.

```
def sync_endpoint():
    ...

async def async_endpoint():
    ...
```

두 함수는 호출 결과와 실행 방식이 다르다.

일반적인 `def` 함수는 호출하면 본문을 실행하고 반환값을 돌려준다.

반면 `async def`로 정의한 코루틴 함수(Coroutine Function)를 호출하면 코루틴 객체가 반환된다.

함수 본문의 실행을 진행하려면 해당 코루틴을 `await`하는 등의 과정이 필요하다.

이 차이는 데코레이터에도 영향을 준다.

> 비동기 함수를 동기 wrapper로 감싸면?

다음 코드를 살펴보자.

```
from functools import wraps
import inspect


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("TRACE")
        return func(*args, **kwargs)

    return wrapper


@trace
async def calculate(x):
    return x * 2


print(inspect.iscoroutinefunction(calculate))
```

예상 출력:

```
False
```

왜 `False`일까?

원본 `calculate`는 코루틴 함수다.

하지만 데코레이션 이후 `calculate` 이름은 일반적인 `def wrapper` 함수 객체를 참조한다.

`@wraps(func)`가 `__wrapped__`를 설정했다고 해서 wrapper 자체가 `async def` 함수로 변환되는 것은 아니다.

이제 다음 표현식을 생각해 보자.

```
result = calculate(10)
```

일반 wrapper가 실행되면서 원본 코루틴 함수를 호출한다.

원본 함수 호출 결과는 코루틴 객체다.

Wrapper는 그 코루틴 객체를 그대로 반환한다.

따라서 `result`는 정수 20이 아니라 코루틴 객체다.

이 코루틴을 실행하여 결과를 얻으려면 별도로 `await` 해야 한다.

FastAPI에서는 왜 문제가 될까?

FastAPI는 엔드포인트가 동기 함수인지 비동기 함수인지에 따라 실행 방식을 달리한다.

일반적인 동기 엔드포인트는 스레드풀에서 실행하고, 비동기 엔드포인트는 비동기 실행 경로에서 `await`한다.

그런데 코루틴 함수를 동기 wrapper로 감싸면 FastAPI가 최종 등록된 wrapper를 동기 함수로 판단할 수 있다.

이 경우 wrapper 호출 결과로 코루틴 객체가 반환되지만, 그 객체가 정상적으로 `await`되지 않은 채 응답 직렬화 단계에 전달되어 오류가 발생할 수 있다.

따라서 `@wraps`만 사용했다고 해서 비동기 엔드포인트에 안전한 데코레이터가 되는 것은 아니다.

동기·비동기 함수를 구별하는 데코레이터

다음처럼 두 종류의 wrapper를 구별할 수 있다.

```
import inspect
from functools import wraps

from fastapi import FastAPI
from fastapi.testclient import TestClient


def trace(func):

    if inspect.iscoroutinefunction(func):

        @wraps(func)
        async def async_wrapper(*args, **kwargs):
            print("TRACE async:", func.__name__)

            result = await func(*args, **kwargs)

            return result

        return async_wrapper

    @wraps(func)
    def sync_wrapper(*args, **kwargs):
        print("TRACE sync:", func.__name__)
```

```
        result = func(*args, **kwargs)

        return result

    return sync_wrapper


app = FastAPI()


@app.get("/sync")
@trace
def sync_endpoint():
    return {"mode": "sync"}


@app.get("/async")
@trace
async def async_endpoint():
    return {"mode": "async"}


client = TestClient(app)

print(client.get("/sync").json())
print(client.get("/async").json())
```

여기서 두 wrapper는 서로 다른 함수 객체다.

동기 엔드포인트에 적용된 wrapper는 일반 함수 객체이고, 비도익 엔드포인트에 적용된 wrapper는 코루틴 함수 객체다.

따라서 FastAPI는 각 엔드포인트의 실행 방식을 적절하게 구별할 수 있다.

다만 이 구현은 직접 전달받은 일반적인 동기 함수와 코루틴 함수를 처리하는 예제다.

이미 다른 데코레이터로 감싸진 함수, 비동기 제네레이터, 복잡한 호출 가능 객체까지 모두 처리하는 범용 구현은 아니다.

여기서 핵심은 다음과 같다.

함수 객체의 메타데이터, 실제 호출 결과,

동기·비동기 실행 방식은 각각 구별해서 분석해야 한다.

A.12.14 APIRouter - 등록 정보를 모듈 단위로 분리하기

FastAPI 애플리케이션이 커지면 모든 엔드포인트를 하나의 파일에 정의하기 어려워진다.

이때 APIRouter를 사용할 수 있다.

```
from fastapi import FastAPI, APIRouter
from fastapi.testclient import TestClient


router = APIRouter(prefix="/v1")


@router.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}


app = FastAPI()

app.include_router(router)


client = TestClient(app)

response = client.get("/v1/items/10")

print(response.status_code)
print(response.json())
```

예상 출력:

```
200
{'item_id': 10}
```

이 구조에서 다음 두 과정은 구별해야 한다.

첫째, @router.get(...)에 의한 등록

```
@router.get("/items/{item_id}")
```

이 데코레이터는 엔드포인트 함수를 router에 등록한다.

둘째, app.include_router(router)에 의한 애플리케이션 구성

```
app.include_router(router)
```

이 호출은 해당 라우터에 정의된 경로들을 FastAPI 애플리케이션에 포함시킨다.

따라서 전체 경로는 다음과 같이 구성된다.

```
APIRouter prefix "/v1"
          +
등록된 경로 "/items/item_id"
          ↓
"/v1/items/item_id"
```

이 구조를 객체 참조 관점에서 생각해 보자.

`router`와 `app`은 서로 다른 객체다.

`@router.get(...)`은 먼저 `router`에 경로 정보를 등록한다.

그다음 `app.include_router(router)`가 라우터의 등록 정보를 애플리케이션에 반영한다.

FastAPI의 일반적인 `include_router` 구현에서는 포함 시점에 라우터의 경로들을 애플리케이션 쪽 라우터에 등록하는 방식으로 처리한다.

따라서 이미 `include_router`를 실행한 뒤 원래 `router`에 새로운 경로를 추가했다고 해서 그 경로가 기존 애플리케이션에 자동 반영된다고 가정하면 안 된다.

이것이 모듈을 나누어 FastAPI 애플리케이션을 구성할 때 등록 시점을 고려해야 하는 이유다.

A.12.15 하나의 함수 객체를 여러 경로에 등록할 수 있을까?

가능하다.

다음 코드를 살펴보자.

```
from fastapi import FastAPI
from fastapi.routing import APIRoute


app = FastAPI()
```

```python
@app.get("/first")
@app.get("/second")
def greet():
    return {"message": "Hello"}


routes = [
    route
    for route in app.routes
    if isinstance(route, APIRoute)
]

for route in routes:
    print(route.path, route.endpoint is greet)
```

예상 출력:

```
/second True
/first True
```

왜 /second가 먼저 등록될까?

A.10에서 학습한 데코레이터 적용 순서 때문이다.

데코레이터 표현식은 위에서 아래로 평가되지만, 함수 객체에 대한 적용은 아래에서 위로 진행된다.

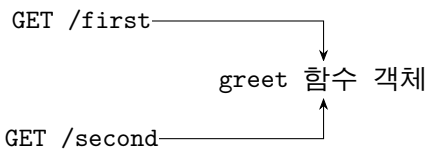
따라서 개념적으로 다음과 같다.

```python
greet = app.get("/first")(
    app.get("/second")(greet)
)
```

먼저 /second가 등록되고, 그다음 /first가 등록된다.

그리고 두 등록 데코레이터 모두 원본 함수 객체를 그대로 반환한다.

따라서 두 라우트의 endpoint는 동일한 함수 객체를 참조한다.



이 구조는 하나의 함수를 여러 URL에 연결할 때 활용할 수 있다.

다만 라우트 등록 순서는 실제 요청 매칭에도 영향을 줄 수 있다.

예를 들어 고정 경로와 동적 경로가 서로 겹치는 경우에는 어떤 경로를 먼저 등록했는지 고려해야 한다.

A.12.16 등록 이후 함수 이름을 재바인딩하면 어떻게 될까?

등록형 데코레이터의 가장 중요한 특징 중 하나는 외부 레지스트리가 함수 객체에 대한 참조를 보유한다는 점이다.

다음 코드를 보자.

```
from fastapi import FastAPI
from fastapi.routing import APIRoute


app = FastAPI()


@app.get("/hello")
def greet():
    return {"message": "Original"}


route = next(
    route
    for route in app.routes
    if isinstance(route, APIRoute)
    and route.path == "/hello"
)


registered = route.endpoint


def replacement():
    return {"message": "Replacement"}


greet = replacement


print(route.endpoint is registered)
print(route.endpoint is greet)
```

```
print(route.endpoint())
print(greet())
```

예상 출력:

```
True
False
{'message': 'Original'}
{'message': 'Replacement'}
```

왜 이런 결과가 발생할까?

처음 `@app.get("/hello")`가 적용될 때 FastAPI는 원본 `greet` 함수 객체를 등록한다.

그 이후 다음 문장이 실행된다.

```
greet = replacement
```

이 문장은 모듈 이름 `greet`을 다른 함수 객체에 재바인딩한다.

하지만 이미 등록된 `route.endpoint`의 참조는 변경되지 않는다.

따라서 `FastAPI`가 등록한 엔드포인트와 모듈 이름 `greet`이 서로 다른 객체를 참조하게 된다.

이것은 `A.1`에서 학습한 이름 바인딩과 객체 동일성의 원리가 그대로 적용된 결과다.

여기서 한 가지 예외적인 상황도 고려해야 한다.

등록된 원본 함수의 본문에서 모듈 전역 이름을 다시 조회한다면, 그 전역 이름의 재바인딩이 함수 실행에 영향을 줄 수 있다.

즉, 등록된 함수 객체의 동일성이 유지된다는 사실이 함수가 사용하는 모든 외부 상태까지 고정된다는 뜻은 아니다.

A.12.17 등록형 데코레이터의 경계 조건

등록형 데코레이터는 함수 정의 시점에 외부 상태를 변경하므로 몇 가지 경계 조건이 존재한다.

상황	발생할 수 있는 결과	판단 기준
엔드포인트가 정의된 모듈이 실행되지 않음	등록 데코레이터도 실행되지 않음	모듈 실행 여부
같은 경로와 HTTP 메서드를 중복 등록	중복 라우트가 생성될 수 있음	경로 등록 상태와 매칭 순서
등록 후 함수 이름 재바인딩	등록된 엔드포인트 참조는 그대로 유지	객체 동일성
등록 전에 wrapper 적용	wrapper가 엔드포인트로 등록됨	데코레이터 적용 순서
등록 후 wrapper 적용	원본 함수가 등록될 수 있음	데코레이터 적용 순서
@wraps 없이 wrapper 등록	시그니처 분석이 잘못될 수 있음	__wrapped__와 실제 시그니처
비동기 함수를 동기 wrapper로 감쌈	코루틴 객체가 정상적으로 처리되지 않을 수 있음	실제 wrapper의 실행 방식

특히 Python 모듈을 import하면 해당 모듈의 최상위 코드가 실행될 수 있다는 사실을 기억해야 한다.

엔드포인트 정의문도 모듈 최상위 코드에 있다면 그 과정에서 실행된다.

따라서 FastAPI의 경로 등록은 일반적인 Python 모듈 실행 및 함수 정의 규칙 위에서 이루어지는 작업이다.

A.12.18 지금까지의 Python 개념과 연결하기

이번 학습은 새로운 문법을 학습했다기보다 이미 학습한 객체 모델을 실제 프레임워크에 적용하였다.

이전 개념	FastAPI에서의 연결
함수 객체와 이름 바인딩 - A.1	엔드포인트 함수 객체 생성과 최종 이름 바인딩
렉시컬 스코프 - A.2~A.3	등록 데코레이터가 경로 설정을 참조
클로저 - A.4~A.5	app.get(...)이 반환하는 데코레이터의 설정 유지
데코레이터 변환식 - A.6	func = decorator(func)
Wrapper와 반환 - A.7	HTTP 처리에 wrapper를 적용하는 경우
functools.wraps - A.8	엔드포인트 시그니처 추적
데코레이터 팩토리 - A.9	@app.get("/path")
다중 데코레이터 순서 - A.10	등록되는 함수 객체의 결정
메서드 바인딩 - A.11	app.get이 FastAPI 인스턴스에 바인딩된 메서드
함수 호출과 인수 전달	요청 데이터로부터 엔드포인트 호출 인수 구성

여기서 FastAPI가 Python의 기본 규칙에 추가한 기능도 구별해야 한다.

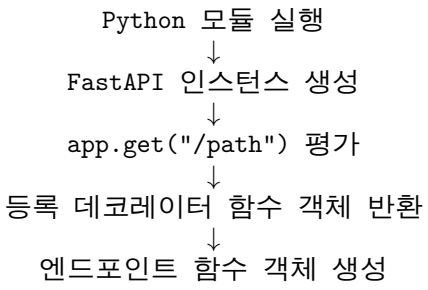
Python 자체는 함수 객체를 생성하고 호출하는 기능을 제공한다.

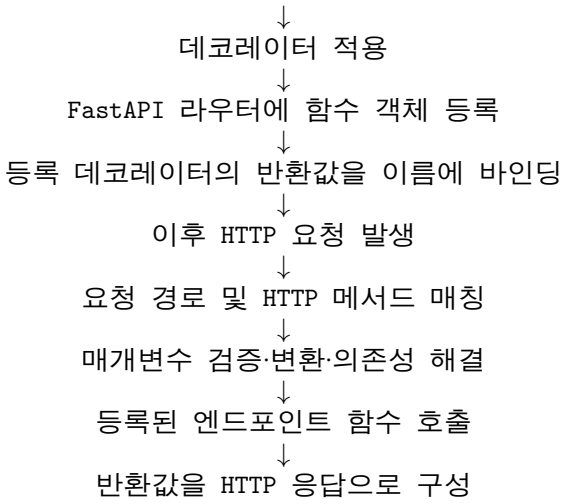
FastAPI는 그 함수 객체를 HTTP 경로에 등록하고, 요청을 분석하고, 의존성을 해결하고, 응답을 생성하는 기능을 제공한다.

즉, FastAPI는 Python의 함수 객체 모델을 변경하는 것이 아니라, 그 모델을 이용하여 HTTP 요청 처리 시스템을 구성하는 프레임워크다.

A.12.19 핵심 정리와 판단 기준

이번 학습의 전체 실행 구조를 정리해 보자.





이 구조를 분석할 때는 다음 다섯 가지를 확인하면 된다.

첫째, @app.get(...)은 언제 평가되는가?

함수 정의문이 실행되는 과정에서 평가된다.

HTTP 요청이 들어올 때마다 평가되는 것이 아니다.

둘째, 무엇이 FastAPI에 등록되는가?

데코레이터가 적용되는 시점에 전달받은 함수 객체가 등록된다.

다른 데코레이터의 적용 순서에 따라 원본 함수일 수도 있고 wrapper일 수도 있다.

셋째, 등록 이후 함수 이름과 엔드포인트 참조는 같은 객체인가?

기본적인 @app.get만 사용하면 동일하지만, 추가 데코레이터나 이름 재바인딩이 있으면 달라질 수 있다.

넷째, HTTP 요청에서 누가 함수를 호출하는가?

FastAPI의 요청 처리 계층이 매개변수를 준비하고 엔드포인트를 호출한다.

따라서 직접 함수 호출과 동작이 다를 수 있다.

다섯째, wrapper의 실제 시그니처와 실행 방식은 무엇인가?

@wraps가 메타데이터를 제공하더라도 인수 전달과 동기·비동기 호출 방식까지 자동으로 보장하지는 않는다.

이번 단원의 가장 중요한 결론은 다음과 같다.

등록형 데코레이터는 함수 객체를 다른 객체로 변환하는 것보다, 함수 객체와 외부 실행 시스템 사이에 참조 관계를 형성하는 데 목적이 있다.

FastAPI의 `@app.get(...)`은 이 원리를 HTTP 라우팅에 적용한 대표적인 사례다.

A.13 실전 응용 · 예외 · 통합 정리

Function Object · Name Binding · Closure · Decorator · Descriptor · Registration · Exception Handling · Async

A.13.1 이번 학습의 목표

`A.1`부터 `A.12`까지 Python의 함수 객체와 데코레이터를 여러 관점에서 분석하였다.

처음에는 함수 저으이와 호출의 차이에서 출발했다.

```
def calculate(x):
    return x * 2
```

이후 중첩 함수와 클로저를 학습했고, 함수 객체를 다른 함수에 전달하여 새로운 함수를 반환하는 데코레이터를 다뤘다.

```
@trace
def calculate(x):
    return x * 2
```

`A.11`에서는 클래스 네임스페이스에 저장된 함수 객체가 디스크립터 프로토콜을 통해 바운드 메서드로 변환되는 과정을 분석했다.

마지막으로 `A.12`에서는 FastAPI의 등록형 데코레이터를 통해 함수 객체가 외부 시스템에 등록되고, 나중에 HTTP 요청을 통해 호출되는 구조까지 살펴보았다.

이제 남은 과제는 이 개념들을 하나의 일관된 Python 실행 모델로 통합하는 것이다.

이번 학습에서는 새로운 문법을 많이 추가하지 않을 것이다.

대신 지금까지 배운 개념이 실제 프로그램에서 어떻게 상호작용하는지를 추적할 것이다.

핵심 질문은 다음과 같다.

> 여러 데코레이터가 적용된 클래스 메서드를 호출할 때,
>
> 어떤 객체가 생성되고, 어떤 이름이 어디에 바인딩되며,
>
> 어떤 디스크립터와 클로저를 거쳐 최종 함수가 실행될까?

그리고 정상적인 호출뿐 아니라 예외 발생, 상태 변경, 비동기 함수, 등록형 데코레이터의 경계 조건까지 분석할 것이다.

A.13.2 통합 분석을 위한 예제

이번에는 다음 기능을 가진 작은 시스템을 구현해 보자.

- `register`: 함수 객체를 외부 레지스트리에 등록하는 데코레이터 팩토리
- `monitor`: 함수 호출 횟수를 기록하는 데코레이터 팩토리
- `classmethod`: 함수를 클래스 객체에 바인딩하는 디스크립터
- `Base`, `Derived`: 상속과 MRO를 확인하기 위한 클래스

이 예제 하나로 A.1부터 A.12까지의 핵심 개념 대부분을 연결할 수 있다.

전체 코드

```
from functools import wraps


registry = {}


def register(key):
    print("evaluate register", key)

    def decorator(func):
        print("apply register", key)
```

```
        registry[key] = func

        return func

    return decorator


def monitor(tag):
    print("evaluate monitor", tag)

    def decorator(func):
        print("apply monitor", tag)

        calls = 0

        @wraps(func)
        def wrapper(*args, **kwargs):
            nonlocal calls
            calls += 1

            print("enter", tag, calls)

            try:
                return func(*args, **kwargs)
            finally:
                print("leave", tag)

        wrapper.count = lambda: calls

        return wrapper

    return decorator


class Base:
    scale = 10

    @classmethod
    @register("predict")
    @monitor("predict")
    def predict(cls, value):
        print("body", cls.__name__, value)
        return cls.scale * value


class Derived(Base):
    scale = 20


print("ready")
```

```
print(
    "identity",
    registry["predict"]
    is Base.__dict__["predict"].__func__
)

print(
    "binding",
    Derived.predict.__self__ is Derived
)

print("result", Derived.predict(3))

try:
    Derived.predict(None)
except TypeError:
    print("caught TypeError")

print("calls", registry["predict"].count())
```

실행 결과

다음 출력은 Python에서 실행하여 확인할 수 있는 결과다.

```
evaluate register predict
evaluate monitor predict
apply monitor predict
apply register predict
ready
identity True
binding True
enter predict 1
body Derived 3
leave predict
result 60
enter predict 2
body Derived None
leave predict
caught TypeError
calls 2
```

여기서 우리가 설명해야 할 현상은 여섯 가지다.

왜 `evaluate register`가 `evaluate monitor`보다 먼저 출력될까?

왜 적용 순서는 반대로 `apply monitor`가 먼저일까?

왜 레지스트리에 저장된 함수와 classmethod.__func__가 동일할까?

왜 Base에 정의된 함수의 cls가 Derived를 참조할까?

왜 Derived.predict(None)에서 예외가 발생해도 leave가 출력될까?

왜 정상 호출은 한 번인데 호출 횟수는 2일까?

A.13.3 첫 번째 단계 - 데코레이터 표현식 평가

가장 먼저 다음 부분을 살펴보자.

```
class Base:
    scale = 10

    @classmethod
    @register("predict")
    @monitor("predict")
    def predict(cls, value):
        ...
```

Python은 클래스 본문을 실행하는 과정에서 predict 함수 정의문을 처리한다.

이때 데코레이터 표현식은 위에서 아래로 평가된다.

따라서 다음 순서가 성립한다.

1. classmethod 평가
2. register("predict") 호출
3. monitor("predict") 호출
4. 원본 predict 함수 객체 생성

classmethod는 이미 존재하는 내장 객체를 참조하므로 별도의 출력이 없다.

반면 register("predict")는 팩토리 함수를 호출하므로 다음 출력이 발생한다.

```
evaluate register predict
```

그리고 monitor("predict")를 호출하면서 다음 출력이 발생한다.

```
evaluate monitor predict
```

여기서 중요한 사실이 있다.

아직 원본 `predict` 함수 본문은 실행되지 않았다.

`register`와 `monitor`는 데코레이터 팩토리로서 실행되었지만, `predict`는 호출된 것이 아니라 함수 객체로 생성되는 과정에 있다.

따라서 `body`는 출력되지 않는다.

이전 단원과 연결하면 다음과 같다.

> `A.1`: 함수 객체와 함수 호출은 다르다.
>
> `A.2`: 중첩 함수는 함수 객체를 반환할 수 있다.
>
> `A.9`: 데코레이터 팩토리는 데코레이터 함수 객체를 반환한다.
>
> `A.10`: 여러 데코레이터 표현식은 위에서 아래로 평가된다.

A.13.4 두 번째 단계 - 데코레이터 적용과 클래스 네임스페이스

데코레이터 표현식의 평가가 끝나면 원본 `predict` 함수 객체가 생성되고, 데코레이터가 아래에서 위로 적용된다.

개념적으로 다음과 대응한다.

```
register_decorator = register("predict")
monitor_decorator = monitor("predict")

original = predict

wrapper = monitor_decorator(original)

registered = register_decorator(wrapper)

descriptor = classmethod(registered)

predict = descriptor
```

이 코드는 객체 관계를 이해하기 위한 개념적 변환식이다.

실제 클래스 정의문의 내부 실행 절차를 문장 단위로 완전히 재현한 것은 아니다.

1. monitor 적용

먼저 다음 호출이 수행된다.

```
wrapper = monitor_decorator(original)
```

monitor_decorator는 calls = 0을 초기화하고 새로운 wrapper 함수 객체를 생성한다.

이 wrapper는 클로저를 토앻 다음 객체를 참조할 수 있다.

```
calls → 정수 객체 0
func  → 원본 predict 함수 객체
tag   → 문자열 "predict"
```

그리고 wrapper 함수 객체를 반환한다.

이 과정에서 다음 출력이 발생한다.

```
apply monitor predict
```

2. register ᄌ거용

다음 호출이 수행된다.

```
registered = register_decorator(wrapper)
```

이때 register에 전달되는 것은 원본 함수가 아니다.

이미 monitor가 반환한 wrapper 함수 객체다.

따라서 다음 문장은

```
registry[key] = func
```

개념적으로 다음과 같다.

```
registry["predict"] = wrapper
```

그리고 register는 동일한 wrapper 객체를 반환한다.

따라서 다음 출력이 발생한다.

apply register predict

3. classmethod 적용

마지막으로 다음 호출이 수행된다.

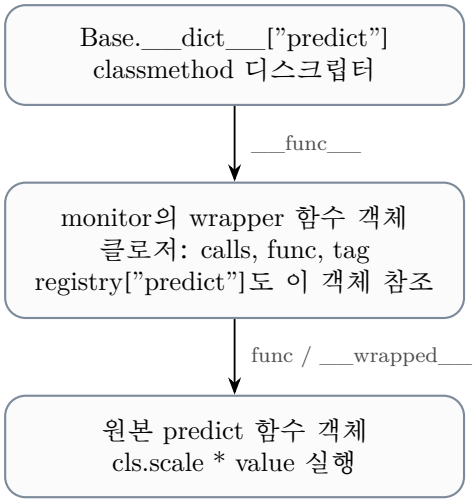
```
descriptor = classmethod(wrapper)
```

이 호출은 wrapper 함수 객체를 감싸는 classmethod 디스크립터를 생성한다.

따라서 Base 클래스 네임스페이스에는 원본 함수도, wrapper 함수도 직접 저장되지 않는다.

최종적으로 classmethod 객체가 저장된다.

전체 구조는 다음과 같다.



등록·wrapper·디스크립터는 서로 다른 객체 관계

이 구조를 정확히 이해하면 A.11과 A.12가 하나로 연결된다.

- monitor는 wrapper를 생성한다.
- register는 wrapper를 외부 레지스트리에 등록한다.
- classmethod는 wrapper의 클래스 바인딩을 담당한다.

세 데코레이터의 역할은 서로 다르다.

A.13.5 세 번째 단계 - 속성 탐색과 클래스 메서드 바인딩

클래스 정의가 완료되었으므로 이제 다음 표현식을 평가할 수 있다.

```
Derived.predict
```

Derived는 predict를 직접 정의하지 않았다.

따라서 클래스 속성 탐색은 MRO를 따라 Base에서 predict를 찾는다.

```
print(Derived.__mro__)
```

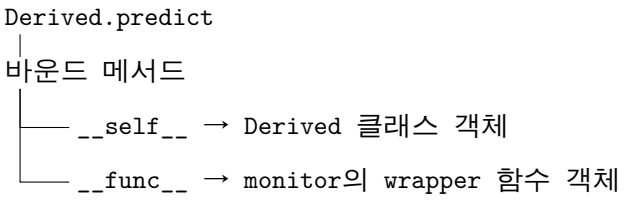
예상 출력:

```
(<class '__main__.Derived'>, <class '__main__.Base'>, <class 'object'>)
```

Base에서 발견되는 객체는 classmethod 디스크립터다.

이 디스크립터는 자신이 감싸고 있는 wrapper 함수 객체를 접근한 클래스인 Derived에 바인딩한다.

따라서 다음과 같은 바운드 메서드가 만들어진다.



이 관계는 다음 코드로 검증할 수 있다.

```
method = Derived.predict

print(method.__self__ is Derived)

print(
    method.__func__
    is registry["predict"]
)
```

예상 출력:

```
True
True
```

여기서 중요한 점은 두 가지다.

첫째, `predict`를 정의한 클래스와 실제 바인딩되는 클래스는 다를 수 있다.

`predict`는 `Base`에서 정의되었지만,

`Derived.predict`를 조회했으므로 `__self__`는 `Derived`를 참조하다.

둘째, 바인딩되는 함수는 원본 `predict`가 아니라 `monitor`가 반환한 wrapper다.

`classmethod`는 자신이 전달받은 wrapper를 클래스에 바인딩할 뿐, 그 wrapper가 클로저로 참조하는 원본 함수까지 별도로 바인딩하지는 않는다.

원본 함수는 나중에 wrapper 내부에서 호출된다.

A.13.6 네 번째 단계 - 실제 함수 호출과 반환

이제 다음 표현식을 실행해 보자.

```
Derived.predict(3)
```

이 호출은 개념적으로 다음과 같다.

```
wrapper(Derived, 3)
```

바운드 메서드가 `Derived` 클래스 객체를 첫 번째 인수로 전달하기 때문이다.

1. Wrapper 호출 프레임 생성

Wrapper의 매개변수는 다음과 같다.

```
def wrapper(*args, **kwargs):
    ...
```

따라서 다음 바인딩이 형성된다.

```
args = (Derived, 3)
```

```
kwargs = {}
```

여기서 `args`는 튜플 객체이고, `kwargs`는 딕셔너리 객체다.

`Derived` 클래스 객체나 정수 3이 복사되어 새로운 객체가 만들어진다는 뜻은 아니다.

이 객체들에 대한 참조가 인수 전달 구조에 포함되는 것이다.

2. 클로저 상태 변경

다음 코드가 실행된다.

```
nonlocal calls
calls += 1
```

`calls`는 wrapper 호출 프레임의 지역 변수가 아니다.

바깥쪽 `decorator` 함수의 실행에서 생성된 클로저 셀에 연결된 이름이다.

처음에는 `calls`가 정수 객체 0을 참조했다.

`calls += 1`이 실행되면 새로운 정수 결과 객체에 `calls`가 재바인딩된다.

호출 전

`calls` 셀 → 0

호출 후

`calls` 셀 → 1

정수 객체 자체를 변경하는 것이 아니다.

이후 다음 문장이 실행된다.

```
print("enter", tag, calls)
```

따라서 다음 출력이 발생한다.

```
enter predict 1
```

3. 원본 함수 호출

다음 문장이 실행된다.

```
return func(*args, **kwargs)
```

func은 클로저를 통해 원본 predict 함수 객체를 참조한다.

따라서 다음과 같은 호출이 수행된다.

```
original_predict(Derived, 3)
```

원본 함수의 매개변수 바인딩은 다음과 같다.

```
cls = Derived
value = 3
```

이제 원본 함수 본문이 실행된다.

```
print("body", cls.__name__, value)

return cls.scale * value
```

먼저 다음 출력이 발생한다.

```
body Derived 3
```

그리고 다음 표현식이 평가된다.

```
cls.scale
```

cls는 Derived 클래스 객체를 참조한다.

Dervied의 클래스 네임스페이스에는 scale = 20이 정의되어 있으므로 그 값을 사용한다.

따라서 계산 결과는 다음과 같다.

$$20 \times 3 = 60$$

원본 함수는 정수 객체 60을 반환한다.

4. finally 실행과 반환

Wrapper의 코드를 다시 보자.

```
try:
    return func(*args, **kwargs)
finally:
    print("leave", tag)
```

return 문장이 실행되더라도 함수가 즉시 종료되는 것은 아니다.

try블록에서 반환값이 결정된 뒤, 해당 finally 블록이 실행된다.

따라서 다음 출력이 발생한다.

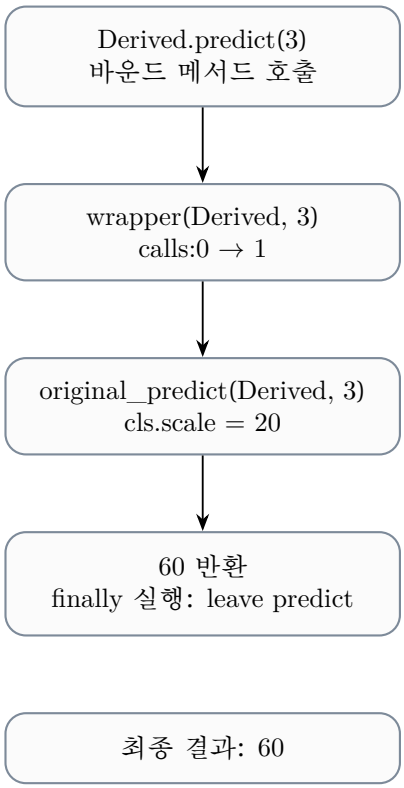
```
leave predict
```

이후 반환값 60이 호출자에게 전달된다.

마지막으로 바깥쪽 print가 실행되면서 다음 출력이 발생한다.

```
result 60
```

전체 호출 흐름



Derived.predict(3)의 호출 및 반환 경로

메서드 바인딩은 wrapper 호출 이전에, 클로저 탐색은 wrapper 실행 중에 발생한다.

이 과정에서 우리가 학습한 객체 모델이 모두 연결된다.

`__get__`: 클래스 바인딩

`__self__`: 바인딩된 `Derived`

`__func__`: `wrapper` 함수 객체

`*args, **kwargs`: 인수 전달

`__closure__`: 원본 함수와 상태 참조

`nonlocal`: 클로저 셀의 이름 재바인딩

`return`: 원본 함수의 반환값 전달

`finally`: 반환 전 정리 코드 실행

A.13.7 다섯 번째 단계 - 예외가 발생하면 무엇이 달라질까?

이제 다음 코드를 실행한다고 하자.

```
try:
    Derived.predict(None)
except TypeError:
    print("caught TypeError")
```

앞의 정상 호출과 같은 경로를 따라 실행되지만, 원본 함수의 연산에서 예외가 발생한다.

1. Wrapper 실행

erived.predict(None)을 조회하고 호출하면 wrapper는 다음 인수를 전달받는다.

```
args = (Derived, None)
```

이후 `calls += 1`이 실행된다.

첫 번째 호출에서 `calls`가 1이 되었으므로 이번에는 2가 된다.

```
enter predict 2
```

2. 원본 함수 실행

원본 함수는 다음 바인딩으로 실행된다.

```
cls = Derived
value = None
```

따라서 다음 출력이 발생한다.

```
body Derived None
```

이후 다음 표현식이 평가된다.

```
cls.scale * value
```

이는 다음과 같다.

```
20 * None
```

정수와 None 사이에는 이 곱셈을 수행하는 연산이 정의되어 있지 않으므로 TypeError가 발생한다.

3. 예외 전파와 finally

원본 함수가 정상적으로 반환하지 못했으므로 wrapper의 return도 정상적으로 완료되지 못한다.

하지만 finally 블록은 실행된다.

```
leave predict
```

Wrapper는 예외를 처리하는 except 블록을 갖고 있지 않으므로 TypeError는 호출자에게 전파된다.

바깥쪽 except TypeError가 이를 처리하고 다음 내용을 출력한다.

```
caught TypeError
```

4. 클로저의 상태는 어떻게 될까?

여기서 중요한 점이 있다.

함수 호출은 실패했지만 다음 코드는 이미 실행되었다.

```
calls += 1
```

따라서 클로저 상태는 이전 값으로 자동 복구되지 않는다.

```
print(registry["predict"].count())
```

결과는 다음과 같다.

```
2
```

이것은 일반적인 Python 예외 처리의 중요한 성질이다.

> 예외가 발생했다고 해서 예외 이전에 수행된 모든 상태 변경이 자동으로 취소되는 것은 아니다.

이번 예제의 calls는 성공 횟수가 아니라 wrapper 진입 횟수를 나타낸다.

성공한 호출만 세고 싶다면 상태를 변경하는 위치를 바꿔야 한다.

예를 들어 다음과 같이 설계할 수 있다.

```
result = func(*args, **kwargs)
calls += 1
return result
```

이 경우 원본 함수가 정상적으로 반환한 뒤에만 `calls`를 증가시킨다.

다만 wrapper에서 이후 수행하는 처리까지 모두 성공해야 성공으로 볼 것인지에 따라 더 세밀한 설계가 필요할 수 있다.

A.13.8 클로저 셀과 함수 메타데이터를 직접 검증하기

이제 앞의 통합 예제를 객체 내부 구조 관점에서 검증해 보자.

먼저 클래스 네임스페이스에 저장된 `classmethod` 객체를 가져온다.

```
descriptor = Base.__dict__["predict"]
```

그 객체가 감싸고 있는 wrapper를 가져온다.

```
wrapper = descriptor.__func__
```

그리고 `functools.wraps`가 설정한 `__wrapped__`를 이용하여 원본 함수 객체를 확인한다.

```
original = wrapper.__wrapped__
```

다음 코드를 실행해 보자.

```
print(type(descriptor).__name__)
print(type(wrapper).__name__)

print(wrapper is registry["predict"])

print(
    original is wrapper.__wrapped__
)
```

예상 출력:

```
classmethod
function
True
True
```

__closure__와 __wrapped__는 다르다

__wrapped__는 functools.wraps가 설정한 함수 메타데이터 참조다.

반면 __closure__는 함수 객체가 참조하는 클로저 셀들의 튜플이다.

두 구조는 서로 다른 목적을 가진다.

```
def closure_map(func):
    names = func.__code__.co_freevars
    cells = func.__closure__ or ()

    return dict(zip(names, cells))


cells = closure_map(wrapper)

print(wrapper.__code__.co_freevars)

print(cells["calls"].cell_contents)

print(
    cells["func"].cell_contents
    is original
)
```

앞의 통합 예제를 모두 실행한 뒤라면 결과는 다음과 같다.

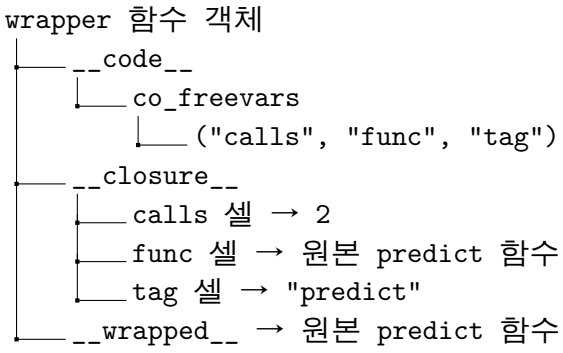
```
('calls', 'func', 'tag')
2
True
```

여기서 co_freevars는 함수 코드 객체가 사용하는 자유 변수 이름들을 보여준다.

__closure__는 그 이름에 대응하는 셀 객체들을 제공한다.

따라서 closure_map을 이용하면 이름과 셀을 연결해서 확인할 수 있다.

이 예제에서는 다음 관계가 형성된다.



func 셀과 __wrapped__는 이 예제에서 같은 원본 함수 객체를 참조한다.

하지만 둘이 같은 개념인 것은 아니다.

func 셀은 wrapper가 실행될 때 원본 함수를 호출하기 위해 사용하고,

__wrapped__는 도구가 원본 함수를 추적할 수 있도록 제공되는 메타데이터다.

> wrapper.count는 어떻게 같은 상태를 읽을까?

다음 문장을 기억해 보자.

```
wrapper.count = lambda: calls
```

이 코드는 wrapper 함수 객체의 __dict__에 "count" 라는 속성을 추가한다.

그 속성값은 lambda로 생성한 별도의 함수 객체다.

그런데 이 함수 역시 같은 calls 셀을 참조한다.

다음과 같이 확인할 수 있다.

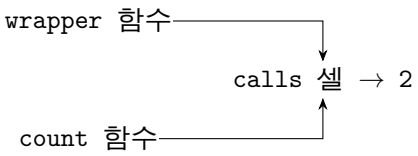
```
wrapper_cells = closure_map(wrapper)
count_cells = closure_map(wrapper.count)

print(
    wrapper_cells["calls"]
    is count_cells["calls"]
)
```

예상 출력:

```
True
```

즉, 두 함수 객체는 서로 다른 코드와 호출 동작을 갖지만 같은 클로저 셀을 공유한다.



따라서 wrapper에서 `nonlocal calls`로 상태를 변경하면

`wrapper.count()`가 그 변경된 값을 읽을 수 있다.

여기서도 핵심은 함수 객체의 동일성이 아니라 공유하는 클로저 셀의 동일성이다.

A.13.9 `inspect.signature()`는 무엇을 보여줄까?

A.8과 A.11의 내용을 연결해 보자.

앞의 통합 예제에 다음 코드를 추가한다.

```
import inspect


print(inspect.signature(wrapper))

print(
    inspect.signature(
        wrapper,
        follow_wrapped=False
    )
)

print(inspect.signature(Derived.predict))
```

예상 출력:

```
(cls, value)
(*args, **kwargs)
(value)
```

세 결과가 다른 이유를 분석해 보자.

표현식	결과	원인
`inspect.signature(wrapper)`	`(cls, value)`	`__wrapped__`를 따라 원본 함수의 시그니처 분석
`inspect.signature(wrapper, follow_wrapped=False)`	`(*args, **kwargs)`	실제 `wrapper`의 매개변수 구조 확인
`inspect.signature(Derived.predict)`	`(value)`	원본 시그니처와 클래스 바인딩 상태를 함께 고려

여기서 중요한 구별이 있다.

`@wraps`가 실제 wrapper의 매개변수를 `(cls, value)`로 변경한 것은 아니다.

실제 wrapper의 매개변수는 여전히 `*args, **kwargs`다.

`inspect.signature`가 원본 함수의 메타데이터를 추적하여 외부에 보여주는 시그니처를 구성하는 것이다.

그리고 `Derived.predict`는 클래스에 바인딩된 메서드이므로 호출자가 직접 전달할 필요가 없는 `cls`가 표시되지 않는다.

따라서 실제 함수 호출 규칙, 함수 메타데이터, 바운드 메서드의 시그니처 표현은 구별해야 한다.

A.13.10 레지스트리에 저장된 함수는 자동으로 클래스에 바인딩될까?

앞의 통합 예제에서 다음 관계가 성립했다.

```
registry["predict"] is Base.__dict__["predict"].__func__
```

두 표현식은 동일한 wrapper 함수 객체를 참조한다.

그렇다면 다음 두 호출도 동일할까?

```
Derived.predict(3)

registry["predict"](3)
```

그렇지 않다.

첫 번째는 classmethod 디스크립터를 통해 생성된 바운드 메서드의 호출이다.

따라서 Derived 클래스 객체가 자동으로 첫 번재 인수에 전달된다.

반면 두 번째는 딕셔너리에서 꺼낸 일반 함수 객체를 직접 호출하는 것이다.

딕셔너리 조회는 classmethod 바인딩을 수행하지 않는다.

따라서 wrapper에 전달되는 인수는 3 하나뿐이다.

Wrapper가 원본 함수에 그 인수를 전달하면

원본 predict(cls, value)에서 value에

해당하는 인수가 부족하여 TypeError 가 발생한다.

레지스트리를 통해 정상적으로 호출하려면 다음처럼 클래스를 명시적으로 전달해야 한다.

```
registry["predict"](Derived, 3)
```

이 호출에서는 wrapper가 다음 인수를 전달 받는다.

```
args = (Derived, 3)
```

따라서 원본 함수도 정상적으로 실행할 수 있다.

이 차이는 중요한 판단 기준이다.

같은 함수 객체를 참조하더라도 어떤 경로로 접근했는지에 따라 디스크립터 바인딩이 적용될 수도 있고 적용되지 않을 수도 있다.

A.11에서 학습한 속성 탐색과 A.12에서 학습한 등록형 데코레이터가 만나는 지점이다.

A.13.11 예외 처리에서 주의해야 할 return과 finally

앞의 monitor는 다음과 같이 구현했다.

```
try:
    return func(*args, **kwargs)
finally:
    print("leave", tag)
```

이 구조는 원본 함수가 정상 반환하든 예외를 발생시키든 정리 코드를 실행한다.

하지만 `finally` 블록에서 직접 `return`을 수행하면 의미가 달라진다.

```
from functools import wraps


def unsafe(func):

    @wraps(func)
    def wrapper(*args, **kwargs):

        try:
            return func(*args, **kwargs)
        finally:
            return "forced"

    return wrapper


@unsafe
def fail():
    raise ValueError("failed")


print(fail())
```

예상 출력:

```
forced
```

원본 함수에서 `ValueError`가 발생했지만 호출자에게 전달되지 않는다.

왜 그럴까?

`finally` 블록의 `return`이 이전에 진행 중이던 예외 전파를 중단하고 새로운 반환값을 결정했기 때문이다.

따라서 wrapper를 구현할 때는 다음 세 가지를 구별해야 한다.

구현	정상 반환	예외 발생
`return func(...)`	원본 반환값 전달	예외 전파
`try: return func(...)` + `finally`에서 로깅	정리 코드 실행 후 반환	정리 코드 실행 후 예외 전파
`finally`에서 직접 `return`	`finally`의 값으로 반환	예외가 억제될 수 있음

여기서 두 번째 경우에도 `finally` 내부에서 새로운 예외가 발생하면 기존 예외를 대체하는 등 결과가 달라질 수 있다.

따라서 로깅이나 리소스 정리를 위한 데코레이터에서는 예외 전파 규칙을 명확하게 설계해야 한다.

A.13.12 비동기 함수와 제네레이터는 호출 시점이 다르다

지금까지의 통합 예제는 일반적인 동기 함수를 대상으로 했다.

하지만 데코레이터를 비동기 함수나 제네레이터 함수에 적용할 때는 호출 결과와 실행 시점을 추가로 고려해야 한다.

비동기 함수

다음 코드를 보자.

```
from functools import wraps
import inspect


def trace(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        print("wrapper")
        return func(*args, **kwargs)

    return wrapper


@trace
async def calculate(x):
    print("body")
    return x * 2
```

```
print(inspect.iscoroutinefunction(calculate))
```

예상 출력:

```
False
```

원본 `calculate`는 코루틴 함수지만, 데코레이터가 반환한 객체는 일반적인 동기 wrapper 함수다.

`@wraps`는 wrapper를 코루틴 함수로 변환하지 않는다.

따라서 `calculate(5)`를 호출하면 동기 wrapper가 실행되고, 원본 코루틴 함수의 호출 결과인 코루틴 객체가 반환된다.

코루틴 본문은 그 객체가 실제로 실행될 때 진행된다.

FastAPI같은 프레임워크에서는 최종 등록된 함수가 동기 함수인지 비동기 함수인지가 실행 방식에 영향을 주기 때문에 중요하다.

일반적인 코루틴 함수와 동기 함수를 모두 지원하려면 다음처럼 wrapper를 구분할 수 있다.

```
import asyncio
import inspect
from functools import wraps


def trace(func):

    if inspect.iscoroutinefunction(func):

        @wraps(func)
        async def async_wrapper(*args, **kwargs):
            print("enter async")
            return await func(*args, **kwargs)

        return async_wrapper

    @wraps(func)
    def sync_wrapper(*args, **kwargs):
        print("enter sync")
        return func(*args, **kwargs)

    return sync_wrapper


@trace
```

```
async def calculate(x):
    return x * 2


print(inspect.iscoroutinefunction(calculate))
print(asyncio.run(calculate(5)))
```

일반 Python 스크립트에서의 예상 출력은 다음과 같다.

```
True
enter async
10
```

이 코드를 이미 이벤트 루프가 실행 중인 Jupyter 환경에서 실행한다면

`asyncio.run()` 대신 다음과 같이 호출해야 한다.

```
result = await calculate(5)
print(result)
```

제네레이터 함수

제네레이터 함수도 비슷하게 실행 시점을 구별해야 한다.

```
def trace(func):

    def wrapper(*args, **kwargs):
        print("wrapper called")
        return func(*args, **kwargs)

    return wrapper


@trace
def numbers():
    print("generator body")
    yield 10
    yield 20


iterator = numbers()

print(next(iterator))
```

예상 출력:

```
wrapper called
generator body
10
```

왜 출력 순서가 이렇게 될까?

`numbers()`는 실제로 일반 wrapper 함수 객체를 호출한다.

따라서 `"wrapper called"`가 먼저 출력된다.

그다음 wrapper가 원본 제네레이터 함수를 호출하여 제네레이터 객체를 반환한다.

하지만 이 시점에는 원본 제너레이터 본문이 아직 실행되지 않았다.

`next(iterator)`가 호출될 때 비로소 제너레이터 본문이 실행을 시작한다.

따라서 함수 객체의 종류뿐 아니라 데코레이터가 반환한 객체의 종류와 실제 호출 결과도 구별해야 한다.

A.13.13 실전 데코레이터 설계에서 발생하는 경계 조건

지금가지의 내용을 바탕으로 실전에서 자주 발생하는 문제를 정리해 보자.

문제 상황	발생 원인	설계 시 확인할 사항
원본 함수의 반환값이 사라짐	wrapper에서 return 누락	return func(*args, **kwargs)가 필요한가?
원본 함수가 너무 일찍 실행됨	func 대신 func()를 데코레이션 시점에 호출	함수 객체와 호출 결과를 구별했는가?
여러 호출의 상태가 공유됨	wrapper들이 같은 클로저 셀 참조	상태를 호출별·인스턴스별·함수별 중 어디에 저장할 것인가?
예외가 사라짐	except 또는 finally에서 예외 억제	예외를 전파할 것인가, 변환할 것인가?
클래스 메서드 바인딩 실패	데코레이터가 classmethod 객체를 직접 감쌈	최종 클래스 속성의 타입은 무엇인가?
클래스 기반 데코레이터에서 self 누락	반환 객체가 __get__을 제공하지 않음	호출 가능성과 디스크립터 여부를 구별했는가?
FastAPI에서 매개변수 검증 실패	wrapper 시그니처와 메타데이터 문제	__wrapped__, 실제 인수 전달이 적절한가?
비동기 엔드포인트 실행 오류	코루틴 함수를 동기 wrapper로 감쌈	최종 함수가 코루틴 함수인가?
등록된 함수와 이름이 달라짐	등록 이후 다른 wrapper 적용 또는 이름 재바인딩	레지스트리는 어느 객체를 참조하는가?
호출 횟수 집계가 부정확함	공유 상태 또는 동시 실행	상태 변경의 원자성·동기화가 필요한가?

공유 상태와 동시 실행

앞의 monitor에서 calls는 클로저 셀을 통해 유지된다.

이 상태는 동일한 wrapper를 호출하는 모든 호출자가 공유한다.

예를 들어 Base.preidct(...)와 Derived.predict(...)는 같은 wrapper 함수 객체를 사용하므로 호출 횟수도 공유한다.

그런데 여러 스레드에서 동시에 호출하는 환경이라면 다음 연산의 동기화까지 고려해야 한다.

```
calls += 1
```

이 표현식을 모든 Python 구현과 실행 환경에서 안전한 원자적 카운터 연산이라고 가정하면 안 된다.

정확한 동시 호출 횟수 집계가 필요하다면 잠김(Lock) 등 적절한 동기화 수단

을 사용해야 한다.

또한 호출 횟수가 인스턴스별로 분리되어야 한다면 하나의 공유 클로저 셀에 저장하는 방식이 요구사항에 맞지 않을 수 있다.

결국 클로저는 상태를 유지할 수 있는 메커니즘을 제공하지만, 그 상태의 공유 범위와 동시성 정책은 프로그램 설계자가 결정해야 한다.

A.13.14 데코레이터를 분석하는 최종 절차

이제 처음 보는 데코레이터 코드도 다음 순서로 분석하면 된다.

Python 데코레이터 실행 분석 절차
1. 정의가 실행되는 위치를 확인한다.
   모듈 본문인지, 클래스 본문인지, 함수 내부인지 확인한다.
   그 위치에 따라 이름이 바인딩되는 네임스페이스가 달라진다.

2. 데코레이터 표현식의 평가 순서를 추적한다.
   데코레이터 팩토리가 있다면 언제 호출되고 무엇을 반환하는지 확인한다.

3. 원본 함수 객체의 생성과 데코레이터 적용을 구별한다.
   함수 본문이 실행된 것인지, 함수 객체만 생성된 것인지 구별해야 한다.

4. 각 데코레이터의 입력 객체와 반환 객체를 기록한다.
   함수 객체, wrapper, 디스크립터, 호출 가능한 인스턴스 중 무엇이
   전달되고 반환되는지 확인한다.

5. 최종 이름 바인딩과 외부 등록 참조를 확인한다.
   함수 이름이 어떤 객체를 참조하는지, 레지스트리는 어떤 객체를
   참조하는지 구별한다.

6. 속성 조회와 메서드 바인딩을 분석한다.
   클래스 또는 인스턴스를 통해 접근한다면 __get__, __self__,
   __func__를 확인한다.

7. 실제 호출에서 인수와 클로저를 추적한다.
   args, kwargs, 자유 변수, 클로저 셀, nonlocal에 의한
   상태 변화를 분석한다.

8. 반환·예외·최종 상태를 확인한다.
   반환값이 그대로 전달되는지, 예외가 전파되는지, 상태 변경이 남는지,
   비동기 실행이 필요한지 확인한다.

이 절차는 데코레이터뿐 아니라 콜백(Callback), 이벤트 핸들러(Event Handler), 의존성 주입(Dependency Injection), 클래스 기반 프레임워크의 실행 흐름을 분석할 때도 활용할 수 있다.

A.13.15 전체 개념 통합

지금까지 학습한 내용을 객체 모델 관점에서 다시 연결해 보자.

단원	핵심 개념	최종적으로 설명할 수 있어야 하는 것
A.1	함수 객체와 이름 바인딩	func와 func()의 차이
A.2	중첩 함수와 렉시컬 스코프	함수가 정의된 위치와 외부 이름 참조
A.3	LEGB	이름 탐색이 이루어지는 네임스페이스
A.4	클로저와 자유 변수	외부 함수 종료 후에도 참조가 유지되는 이유
A.5	nonlocal, __closure__	클로저 셀을 통한 상태 재바인딩
A.6	데코레이터 변환식	@decorator의 함수 객체 변환
A.7	Wrapper와 인수 전달	호출 중개, 반환값, 예외 처리
A.8	functools.wraps	메타데이터와 실제 함수 동작의 구별
A.9	데코레이터 팩토리	설정값을 클로저로 유지하는 구조
A.10	다중 데코레이터	평가·적용·호출 순서의 차이
A.11	메서드 바인딩과 디스크립터	클래스 속성 조회와 바운드 메서드 생성
A.12	등록형 데코레이터	함수 객체와 외부 레지스트리의 참조 관계
A.13	실행 모델 통합	전체 객체 생성·바인딩·탐색·호출·반환·상태 변화 추적

이번 학습 범위를 하나의 문장으로 정리하면 다음과 같다.

데코레이터는 함수 객체를 전달받아 어떤 처리를 수행하고b객체를 반환하는 Python의 일반적인 호출 구조이며, 그 결과는 이름 바인딩·클로저·디스크립터·외부 레지스트리·실제 호출 환경에 따라 서로 다른 실행동작으로 연결된다.

이 원리를 이해했다면 새로운 데코레이터를 만났을 때 단순히 문법을 암기할 필요가 없다.

어떤 객체가 어디에 저장되어 있고, 어떤 시점에 조회되고 호출되는지를 추적하면 동작을 설명할 수 있기 때문이다.